

数据结构与算法综合实践

胡燕 钟欣 袁景凌主编

内容简介

能够熟练地选择和设计各种数据结构和算法，是区分一个程序设计人员水平高低的一个重要标志。本书强调对学生离散数学，高级语言程序设计，数据结构，算法设计与分析课程群的知识点综合运用能力的全面综合训练。书中采用 C++ 进行编程实践，所有应用程序均在 Microsoft Visual Studio 2010 集成开发环境下编译通过。

本书结合企业实用软件开发技术，对栈、队列、串、二叉树、图等重要数据结构和算法中知识单元，结合实际案例和应用环境进行了分析和设计，并在重点训练每一个知识单元编程实践的基础上，结合 C++、MFC Dialog 可视化界面和 GDI 绘图等实用开发技术，通过一个游戏项目贯穿数据结构与算法核心知识，采用迭代开发思路进行每一个功能开发，既应用线性结构知识进行案例编程实践，又应用非线性结构知识进行案例重构，完整训练核心知识单元在企业真实项目中应用。

全书概念清楚，重点突出，强调实战训练，各环节可操作性强，并配有针对性的实训项目，便于讲授和学习。

本书既可作为独立开设的《数据结构与算法综合实验》相关实践课程的教材，也可作为数据结构及相关理论课程的实验教材，同时适合作为高等院校《数据结构与算法》实践教学参考用书，也可供有志于从事计算机工程与应用的广大科技工作者参考。

前言

数据结构是计算机专业的必修主干课程之一，是一门实践性很强的课程。为了帮助读者学会如何把现实世界的问题转化为计算机内部的表示和处理，我们为几种最常用的数据结构和算法精心设计了实验项目。这些实验项目全部按企业项目开发思路进行分析设计、编码实现和测试。

除此之外，我们还设计了 2 套业务相同但数据结构不同的实践案例，让读者能够深入理解数据结构不同而产生的算法变化，并且结合 C++、MFC Dialog 可视化界面和 GDI 绘图等实用开发技术，提高数据结构和算法应用实践能力。在编写过程中，引导读者如何理解数据结构和算法中知识单元与项目需求和技术对接，并采用迭代开发思路进行每一个功能开发。

全书分为技术专题和综合实践两部分，共 12 章：

第 1 章是概述，主要包括本教材的实践目标和内容安排，并介绍了应用程序的集成开发环境。

第 2 章至第 9 章是技术专题部分，针对栈、队列、串、树、图均设计了专题实训项目，针对每个实训项目，均以软件开发形式进行开发，涉及实验目标、实验任务、分析和设计、编程实现、调试和运行等内容。

第 10 章至第 11 章是综合实践，针对线性结构和非线性结构，分别设计了 2 套不同结构的“连连看游戏”实践案例，完整实践数据结构和算法核心知识，阐释了不同的数据结构是如何在真实企业项目中的应用，并通过“软件迭代开发”详细讲解实践项目开发过程。

第 12 章是附录，提供了本书参考资料、C++编码规范、MFC 应用程序等内容。

全书的编写由胡燕、钟欣、袁景凌、李晓红、夏红霞、饶文碧、马庆槐合作完成，是集体智慧的结晶，全书由胡燕统一定稿。

在本教材编写过程中，软酷网为该书提供实践参考资料，其中，马庆槐技术总监为我们提供了项目资源和企业项目实施过程资料，闫茂源和郑婕项目经理负责项目的开发和测试，张璇和张欢完成部分图表绘制及文档排版工作，本书的出版还得到了武汉理工大学出版社的大力支持，作者在此一并致以衷心的感谢！

限于编者的水平和时间，本教材难免存在纰漏和不足之处，敬请读者批评指正。

作者

2015 年 8 月

目录

内容简介.....	2
前言.....	3
目录.....	4
1 概述.....	9
1.1 实践目标.....	9
1.2 内容安排.....	9
1.3 开发环境.....	11
第一部分 技术专题	13
2 栈与表达式求值实践	13
2.1 实验目标.....	13
2.2 实验任务.....	13
2.3 分析和设计.....	14
2.3.1 表达式组成.....	14
2.3.2 核心算法.....	14
2.3.3 栈(stack).....	16
2.4 编码实现.....	21
2.4.1 创建工程.....	21
2.4.2 定义栈 (链表).....	23
2.4.3 表达式解析.....	24
2.4.4 比较操作符.....	28
2.4.5 表达式计算.....	30
2.4.6 最终结果.....	32
2.5 调试和运行.....	33
习题.....	34
3 队列与停车场管理程序实践	35
3.1 实验目标.....	35
3.2 实验任务.....	35
3.3 分析和设计.....	37
3.3.1 数据结构.....	37
3.3.2 业务流程.....	39
3.3.3 技术分析.....	41
3.4 编码实现.....	47
3.4.1 创建工程.....	47
3.4.2 定义数据结构.....	50

3.4.3 车辆入场.....	52
3.4.4 车辆离开.....	54
3.4.5 查看停车场.....	56
3.5 调试和运行.....	58
习题.....	60
4 串与字符串替换程序实践	61
4.1 实验目标.....	61
4.2 实验任务.....	61
4.3 分析和设计.....	62
4.3.1 数据结构.....	62
4.3.2 业务流程.....	63
4.3.3 定位子串在主串的位置.....	64
4.3.4 字符串的替换.....	64
4.4 编码实现.....	64
4.4.1 创建工程.....	65
4.4.2 定义串的存储结构.....	65
4.4.3 串的基本操作.....	65
4.4.4 主函数.....	69
4.4.5 扩展优化.....	71
4.5 调试和运行.....	71
习题.....	73
5 二叉树与赫夫曼图片压缩实践	74
5.1 实验目标.....	74
5.2 实验任务.....	74
5.3 分析和设计.....	78
5.3.1 Huffman 树	79
5.3.2 Huffman 编码	82
5.3.3 压缩过程.....	84
5.3.4 压缩与解压.....	86
5.3.5 文件操作.....	87
5.4 编码实现.....	87
5.4.1 创建工程.....	87
5.4.2 读取原文件.....	89
5.4.3 生成 Huffman 树	90
5.4.4 生成 Huffman 编码	92
5.4.5 压缩原文件.....	94
5.4.6 保存压缩文件.....	98
5.4.7 扩展功能.....	100
5.5 调试和运行.....	100
习题.....	102
6 图与景区信息管理系统实践	103

6.1 实验目标.....	103
6.2 实验任务.....	103
6.3 分析和设计.....	107
6.3.1 程序设计.....	107
6.3.2 界面设计.....	108
6.3.3 算法设计.....	108
6.3.4 数据结构设计.....	109
6.4 编码实现.....	110
6.4.1 创建工程.....	110
6.4.2 定义图.....	112
6.4.3 查询景点信息.....	114
6.5 调试和运行.....	115
习题.....	117
7 深度优先搜索与旅游景点导航实践	118
7.1 实验目标.....	118
7.2 实验任务.....	118
7.3 分析和设计.....	118
7.4 编码实现.....	120
7.4.1 导入工程.....	121
7.4.2 遍历景区景点图(一条路线).....	121
7.4.3 优化遍历算法(多条路线).....	122
7.5 调试和运行.....	123
习题.....	125
8 Dijkstra 算法与搜索最短路径实践	126
8.1 实验目标.....	126
8.2 实验任务.....	126
8.3 分析和设计.....	126
8.4 编码实现.....	127
8.4.1 导入工程.....	128
8.4.2 搜索最短路径.....	128
8.4.3 查询最短路径.....	128
8.5 调试和运行.....	129
习题.....	130
9 最小生成树与铺设电路规划实践	131
9.1 实验目标.....	131
9.2 实验任务.....	131
9.3 分析和设计.....	131
9.4 编码实现.....	134
9.4.1 导入工程.....	134
9.4.2 构建最小生成树.....	134
9.4.3 查询铺设电路规划图.....	134

9.5 调试和运行.....	135
习题.....	136
第二部分 综合实践	137
10 线性结构与连连看游戏综合实践	137
10.1 实践目标.....	137
10.2 需求分析.....	137
10.2.1 项目简介.....	137
10.2.2 游戏规则.....	138
10.2.3 功能简介.....	140
10.3 系统设计.....	142
10.3.1 界面设计.....	142
10.3.2 程序结构设计	144
10.3.3 数据结构设计	145
10.3.4 算法设计.....	145
10.4 创建工程.....	150
10.4.1 功能需求.....	150
10.4.2 设计思路.....	151
10.4.3 编码实现.....	151
10.5 主界面.....	157
10.5.1 功能需求.....	157
10.5.2 设计思路.....	158
10.5.3 编码实现.....	159
10.6 开始游戏.....	162
10.6.1 功能需求.....	163
10.6.2 设计思路.....	165
10.6.3 编码实现.....	168
10.7 消子判断.....	178
10.7.1 功能需求.....	178
10.7.2 设计思路.....	180
10.7.3 编码实现.....	182
10.8 判断胜负.....	191
10.8.1 功能需求.....	192
10.8.2 设计思路.....	192
10.8.3 编码实现.....	193
11 非线性结构与连连看游戏综合实践	195
11.1 实践目标.....	195
11.2 需求分析.....	195
11.3 系统设计.....	197
11.3.1 程序结构设计	197
11.3.2 数据结构设计	197
11.3.3 开局与重排算法.....	199

11.3.4	消子判断算法	200
11.3.5	消子算法	201
11.3.6	胜负判断算法	201
11.4	创建工程	201
11.4.1	功能需求	201
11.4.2	设计思路	203
11.4.3	编码实现	203
11.5	主界面	209
11.5.1	功能需求	209
11.5.2	设计思路	210
11.5.3	编码实现	211
11.6	开始游戏	214
11.6.1	功能需求	215
11.6.2	设计思路	215
11.6.3	编码实现	217
11.7	相邻同色消子	224
11.7.1	功能需求	224
11.7.2	设计思路	225
11.7.3	编码实现	226
11.8	消子判断	229
11.8.1	功能需求	230
11.8.2	设计思路	230
11.8.3	编码实现	230
11.9	判断胜负	233
11.9.1	功能需求	234
11.9.2	设计思路	234
11.9.3	编码实现	235
11.10	扩展功能	236
11.10.1	提示	236
11.10.2	重排	236
12	附录	237
12.1	C++编码规范	237
12.2	MFC 应用程序介绍	240

1 概述

1.1 实践目标

通过学习、开发本教材中提供的实践项目，达到以下目标：

- 1、理解 Microsoft Visual Studio 2010 工具，掌握 C++编程和 MFC Dialog 应用程序。
 - 2、掌握栈、队列、串、树和二叉树、图等重要数据结构应用和编程实践。
 - 3、理解企业软件开发过程，理解系统需求分析和设计，应用迭代开发思路进行项目开发。
- 养成良好的编码习惯和培养软件工程化思维，综合应用 C++程序设计、线性结构、图等知识，开发“连连看游戏”项目，达到掌握和应用数据结构核心知识。

1.2 内容安排

本教材的内容主要分为专题编程和综合实践 2 大部分，先通过技术专题训练数据结构和算法中知识单元内容，再通过综合实践，训练各知识单元综合解决企业项目应用。

第一部分技术专题采用控制台应用程序，聚焦数据结构中知识单元训练，第二部分综合实践部分采用桌面应用程序，聚焦为整个数据结构核心知识训练。

1、技术专题部分

针对栈、队列、串、树和二叉树、图、深度优先搜索、Dijkstra 算法、最小生成树等数据结构和算法，安排 8 个技术专题，分别训练相关知识。

编号	阶段	实验	技术专题	训练知识点
1	技术专题	栈	表达式求值(栈)	1、栈的定义与实现 2、栈的基本操作 3、链式栈与顺序栈
2		队列	停车场管理	1、队列的定义和存储 2、队列的基本操作 3、链队列与顺序队列 4、循环队列
3		串	字符串替换	1、串的存储 2、串的遍历 3、串的匹配 4、串的删除
4		树和二叉树	图片压缩编码	1、树、二叉树、三种遍历 2、树的带权路径长度 3、二叉链表、线索链表 4、哈夫曼树、哈夫曼编码、前缀编码
5		图	景区信息管理系统	1、带权无向图 2、图的定义与实现 3、图的存储结构
6		深度优先搜索	旅游景点导航	1、图的遍历

				2、深度优先搜索算法
7		Dijkstra 算法	搜索最短路径	1、最短路径的定义 2、Dijkstra 算法
8		最小生成树	铺设电路规划	1、生成树 2、最小生成树 3、Prime 算法

2、综合实践部分

针对线性结构和非线性结构，分别设计了 2 套“连连看游戏”案例。应用迭代开发思想，进行功能迭代开发，迭代根据功能和训练知识进行划分，每一个迭代内容包括“功能需求、设计思路、编码实现”。

编号	阶段	功能/迭代	实践内容	训练知识点
1	线性结构	创建工程	创建连连看游戏工程，绘制欢迎语。	1、VS2010 开发工具 2、对话框的属性 3、GDI 绘图
2		主界面	创建主界面，添加按钮控件，绘制背景图片。	1、按钮控件 2、图形绘制 3、位图资源的添加 4、BitBlt() 函数
3		开始游戏	进入游戏界面，绘制游戏地图。	1、数组 2、开局算法 3、坐标计算 4、光栅码、掩码
4		消子判断	在选中的两张相同，绘制提示框，同色图片间绘制连接线，并消除图片清除提示框。	1、鼠标点击事件 2、消子算法 3、界面重绘
5		判断胜负	判断棋盘子是否消完。并提示用户获胜。更新数据。	1、游戏状态控制 2、胜负判断算法
6	非线性结构	创建工程	创建对话框的工程，修改对话框标题，图标，添加最小化按钮。并在主界面上绘制显示欢迎语。	1、VS2010 开发工具 2、对话框的属性 3、GDI 绘图 4、程序调试
7		主界面	在主界面上绘制游戏背景，绘制模式选择提示语、模式选择按钮、显示排行榜按钮、帮助按钮、设置按钮、关于按钮。	1、按钮控件 2、图形绘制 3、位图资源的添加 4、BitBlt() 函数
8		开始游戏	在基本模式下，选择“开始新游戏”按钮，根据界面显示的图片，生成相应的图结构。	1、图的定义 2、邻接矩阵 3、图的初始化
9		相邻同色消子	将相邻并且相同的图片消掉，并更新图结构	1、顶点和边的更新 2、鼠标点击事件 3、坐标计算

10		消子判断	判断选择的两张图片是否能够连通,在选中可以消除的图片之间绘制连接线。	1、DFS 算法 2、连通判断算法 3、拐角计算算法 4、栈结构
11		判断胜负	判断棋盘子是否消完。并提示用户获胜。更新数据。	1、游戏状态控制 2、胜负判断算法
12		扩展功能	提供功能和算法的扩展思路,供读者自行扩展开发。	1、随机重排算法 2、提示

1.3 开发环境

1、Visual C++集成开发环境

Mircosoft Visual Studio 2010（简称 VS2010）是微软公司推出的开发环境。是目前最流行的 Windows 平台应用程序开发环境。VS2010 版本于 2010 年 4 月 12 日上市，其集成开发环境（IDE）的界面被重新设计和组织，变得更加简单明了。Visual Studio 2010 同时带来了 NET Framework 4.0、Microsoft Visual Studio 2010 CTP(Community Technology Preview--CTP)，并且支持开发面向 Windows 7 的应用程序。除了 Microsoft SQL Server，它还支持 IBM DB2 和 Oracle 数据库。

使用 VS2010 工具可以开发的程序非常多，有 Visual C++、Visual C#等等。第一次启动时，需要设置默认的开发环境，设置开发环境为“Visual C++ Development Settings”。



图 1-1 VS2010 C++开发设置

如果已经被设置为别的开发环境(如：默认的开发环境为 Visual C#)，可以进行修改，修改方法如下：

(1) 选择菜单“工具(Tools) --> 导入导出设置(Import and Export Settings..)”。打开设置

向导。

(2) 在设置向导中，选择“Reset all settings”。点击“Next”按钮。

(3) 显示“是否保存当前设置”对话框，需要保存当前设置选择 Yes，如果不需要选择 No，点击“Next”按钮。

(4) 在向导对话框中，选择默认开发环境为“Visual C++ Development Settings”，点击“Finish”按钮，完成。

2、使用 Visual C++开发应用程序

使用 Microsoft Visual Studio 2010 开发工具创建一个应用程序，会产生哪些文件？这些文件是如何组织的？VS2010 开发程序目录结构由解决方案文件夹和工程文件夹组成。

(1) 解决方案文件夹

解决方案文件夹中，存放解决方案文件(.sln 文件)，和解决方案中的工程文件夹。一个解决方案可以包含多个工程。

解决方案文件(.sln 文件)，包含当前解决方案中的工程信息，存储解决方案的设置。VS2010 中一个解决方案可以包含多个工程。



图 1-2 解决方案结构

(2) 工程文件夹

工程文件夹中存放工程文件(.vcxproj 文件)和工程中的代码文件。

工程文件(.vcxproj 文件)包含当前工程的设置和工程所包含的文件等信息。



图 1-3 应用程序结构

第一部分 技术专题

2 栈与表达式求值实践

2.1 实验目标

- 1、掌握栈、数组、链表等知识和应用。
- 2、使用 C++ 语言和栈实现“表达式求值”专题编程。

2.2 实验任务

本程序是一个控制台程序，要求用户从控制台输入语法正确的、不含变量的表达式，并利用给定的优先关系实现对算术四则混合表达式的求值。表达式可以使用加减乘除四则运算符，支持括号、浮点数。

1、表达式组成

表达式符号表如下表所示：

单词	符号组成	数据类型
操作数(operand)	123456789	浮点数(double)
运算符(operator)	+ - * /	符号(char)
界限符(delimiter)	()	符号(char)

2、四则运算规则

- (1) 先乘除，后加减；
- (2) 同级运算从左到右；
- (3) 先括号内、后括号外。

具体功能如下：

- (1) 输出程序界面，提示用户输入表达式。

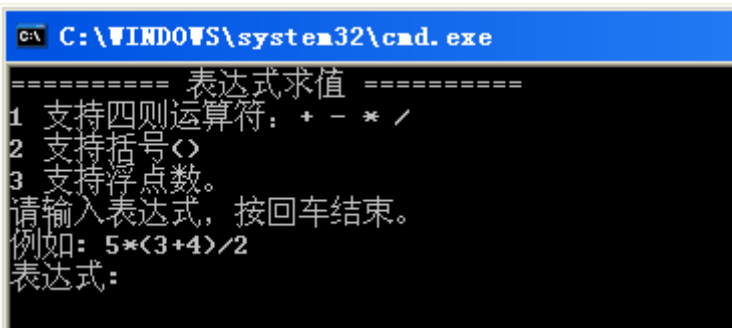


图 2-1 输入表达式

- (2) 用户在控制台下输入表达式，如：5*(3+4)/2。
- (3) 点击回车，程序解析用户输入的表达式，输出计算结果。

图 2-2 输出计算结果

2.3 分析和设计

表达式求值是栈应用的一个典型例子, 定义 2 个栈来存储表达式中的操作数和操作符, 通过比较符号的优先级, 实现四则运算规则。栈的实现有顺序栈(数组)和链式栈(链表)两种方式, 本程序将采用链式栈来实现表达式求值。

程序中将创建两个文件 Expr.h 与 Expr.cpp, 其中 Expr.h 用于定义数据结构、声明函数, Expr.cpp 用于实现具体的函数功能。

在 Expr.cpp 文件中, 定义 `int _tmain(int argc, _TCHAR* argv[])` 函数, 作为应用程序入口, 程序从这里开始运行。

2.3.1 表达式组成

任何一个表达式都是由操作数(operand)、运算符(operator)和界限符(delimiter)组成, 我们称它们为单词(token)。一般的, 操作数既可以是常数, 也可以是被说明为变量或常量的标识符; 运算符可以分为算术运算符、关系运算符和逻辑运算符 3 类; 基本界限符有左右括号和表达式结束符等。

为了叙述的简洁, 我们仅仅讨论简单算术表达式的求值问题。这种表达式只包括加、减、乘、除 4 种运算符。

一般来说, 括号都是成对出现的, 它界定了运算的优先级。设定括号也是一种特殊的运算符, 这样就可以把表达式的组成分成 2 类: 操作数与操作符。以表达式“5*(3+4)/2”为例, 它包含: 4 个操作数: “5、于用户输入的表达式, 可以通过如下方式进行存储:

定义字符数组 `char expr[]` 保存用户输入的表达式。定义两个栈, 分别用于存储操作数和操作符。

定义 `double parse(char expr[])` 函数对其进行解析, 将表达式分离为操作数(operand)和操作符(operator), 分别保存到操作数栈(OPND)和操作符栈(OPTR)中。

2.3.2 核心算法

从左到右扫描表达式, 将表达式解析成一个个的数字和符号, 分别存入操作数栈

(OPND) 和操作符栈 (OPTR) 中。

解析到操作数时，不做处理直接入栈。

解析到操作符时，比较栈顶操作符和当前操作符的优先级，根据下列规则，进行操作和运算。

1、四则运算规则

比较 2 个操作符的优先级，找出优先级较高的那个，然后进行运算。参与比较的操作符有两种状态：

(1) 即将入栈的操作符。

该操作符刚刚从表达式中解析得到，准备入栈，称其为“当前操作符”。

(2) 已经入栈的操作符。

该操作符位于栈顶，称其为“栈顶操作符”。

用“栈顶操作符”与“当前操作符”相比，根据四则运算规则，应该有以下几种情况：

(1) 先乘除，后加减；

当前乘除号比栈顶加减号优先级高。

(2) 同级运算从左到右；

相同优先级的操作符，栈顶操作符比当前操作符优先级高。

(3) 先括号内、后括号外。

当前左括号比栈顶任何操作符优先级都高。

当前右括号与栈顶左括号优先级相同，比其他栈顶操作符优先级都低。

综上所述，可得到符号优先级表如下：

优先级 \	+	-	*	/	(	)
+	>	>	<	<	<	>
-	>	>	<	<	<	>
*	>	>	>	>		>
/	>	>	>	>		>
(	<	<	<	<	<	=
)	>	>	>	>	>	>

定义 **char precede(char a, char b)** 函数，在解析到操作符时，先用 OPTR 栈顶运算符和当前运算符比较优先级，再做出相应操作。

(1) OPTR 栈顶运算符优先级 < 当前运算符优先级

直接把当前运算符 push 进栈；

(2) OPTR 栈顶运算符优先级 > 当前运算符优先级

从 OPTR 栈 pop 出 1 个操作符，从 OPND 栈 pop 出 2 个操作数，定义 **double calc(double, char, double)** 函数进行计算。先将计算结果 push 进 OPND 栈，然后将当前操作符 push 进 OPTR 栈。

(3) OPTR 栈顶运算符优先级 = 当前运算符优先级

左右括号相遇，弹出左括号。

2、运算过程

以算术表达式“5*(3+4)/2”为例，从左到右扫描整个表达式，分别对 OPTR 栈和 OPND

栈进行操作。表达式扫描完毕后，再对 OPTR 栈剩余的运算符进行计算，最后 OPND 栈中剩下的一个操作数 17.5，即为最终运算的结果。

表达式求值过程如下图所示：

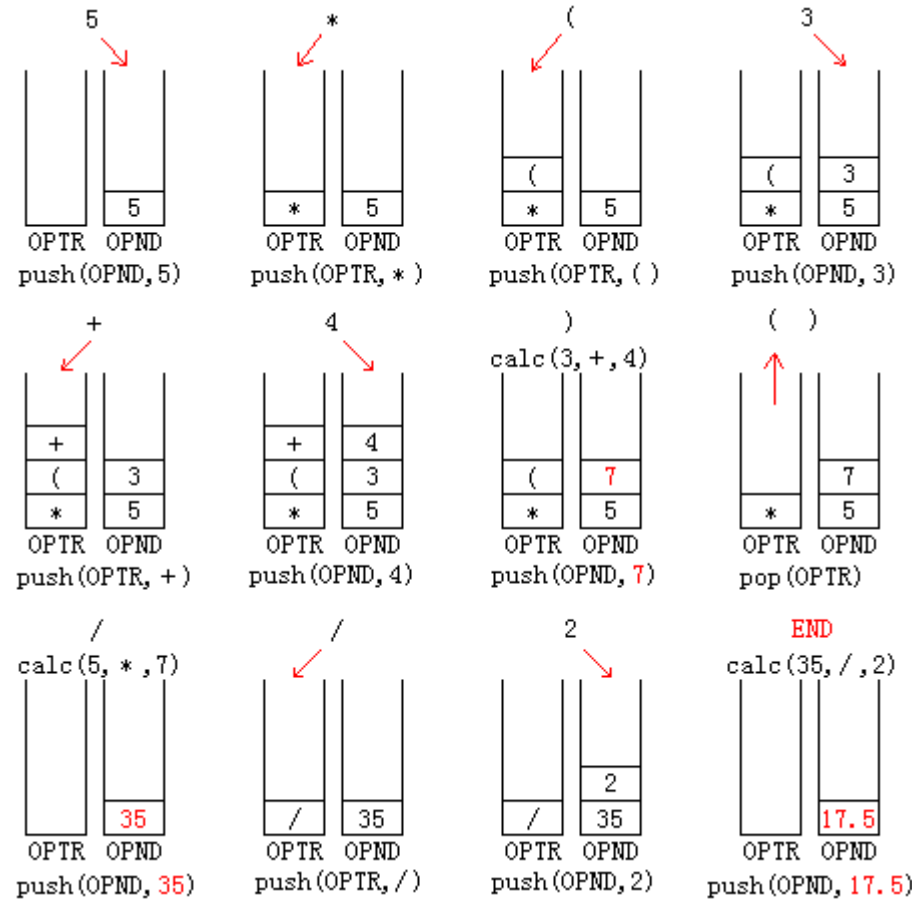


图 2-3 算数表达式求值过程示意图

2.3.3 栈(stack)

栈是限定仅在表尾进行插入或删除操作的线性表。对栈来说，表尾有特殊的含义，称为栈顶 **top**，表头端称为栈底 **bottom**。不含元素的空表称为空栈。栈的特点是先进后出。栈的存储方式有两种，一种是顺序栈另外一种为链式栈。

栈的基本操作如下：

- (1) 初始化栈 `init(&s)`。
- (2) 元素入栈 `push(&s, e)`。
- (3) 元素出栈 `pop(&s, &e)`。
- (4) 获得栈顶元素 `getTop(&s, &e)`。
- (5) 获得栈元素个数 `getLength(&s)`。
- (6) 判断栈是否为空 `isEmpty(&s)`。

1、链式栈

链式栈比顺序栈更容易实现增删改查等操作，尤其链表中的节点还可以有复杂的数据结构，能够携带更多的数据形式。

定义如下结构，作为操作数栈的节点：


```
// 操作数栈
typedef struct StackNodeOPND
{
    double num;
    StackNodeOPND *next;
} *OPND;
```

使用链表实现的操作数栈（OPND），基本函数如下：

(1) 初始化

```
int init(OPND &opnd)
{
    // 销毁链表元素
    while (opnd != NULL)
    {
        // 删除旧节点
        OPND temp = opnd;
        opnd = temp->next;
        free(temp);
    }
    opnd = NULL;
    return 0;
}
```

(2) 操作数入栈

```
int push(OPND &opnd, const double value)
{
    // 新增节点，栈顶上移
    OPND temp = (OPND)malloc(sizeof(StackNodeOPND));
    if (!temp)
    {
        return -1;
    }
    temp->value = value;
    temp->next = opnd;
    opnd = temp;

    return 0;
}
```

(3) 操作数出栈

```

int pop(OPND &opnd, double &value)
{
    if (opnd == NULL)// 空栈
    {
        return -1;
    }
    // 弹出操作数
    double value = opnd->value;

    // 删除旧节点
    OPND temp = opnd;
    opnd = temp->next;
    free(temp);

    return 0;
}

```

(4) 获得栈顶操作数

```

int getTop(OPND &opnd, double &value)
{
    if (opnd == NULL)
    {
        return -1;
    }
    value = opnd->value;
    return 0;
}

```

(5) 判断栈是否为空

```

int isEmpty (OPND &opnd)
{
    if (opnd == NULL)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

```

2、顺序栈

顺序栈，即栈的存储结构是利用一组地址连续的存储单元依次存放自栈底到栈顶的元素，同时附设指针 **top** 指示栈顶元素的位置。

```

#define SIZE 10

// 操作数栈
typedef struct OPND
{
    double stack[SIZE];
    int top;
}OPND;

```

通常的习惯做法是以 `top=0` 表示空栈，鉴于 C/C++ 语言中数组的下标约定从 0 开始，则当以 C/C++ 语言作为描述语言时，如此设定会带来很大不便；另一方面，由于栈在使用过程中所需最大空间的大小很难估计，因此，一般来说，在初始化设空栈时不应限定栈的最大容量。

一个较合理的做法是：先为栈分配一个基本容量，然后在应用过程中，当栈的空间不够使用时再逐段扩大。为此，可设定两个常量：`STACK_INIT_SIZE` 和 `STACKINCREMENT`，并以下述类型说明作为顺序栈的定义。

```

#define STACK_INIT_SIZE 10
#define STACKINCREMENT 10

// 操作数栈
typedef struct OPND
{
    double* base;
    double* top;
    int stacksize;
}OPND;

```

使用动态数组实现的操作数栈（OPND），基本函数如下：

(1) 初始化

```

int init(OPND &opnd)
{
    // 构造一个空栈
    opnd.base = (double *)malloc(STACK_INIT_SIZE * sizeof(double));
    if (!opnd.base)
    {
        exit(-1); // 内存分配失败
        return ERROR;
    }
    opnd.top = opnd.base;
    opnd.size = STACK_INIT_SIZE;
    return OK;
}

```

(2) 操作数入栈

```

int push(OPND &opnd, const double value)
{
    if(opnd.top - opnd.base >= opnd.size)
    {
        // 栈满，追加空间
        opnd.base = (double*) realloc(opnd.base, (opnd.size +
STACKINCREMENT) * sizeof(double));
        if (!opnd.base)
        {
            exit(-1); // 内存分配失败
            return ERROR;
        }
        opnd.top = opnd.base + opnd.size;
        opnd.size += STACKINCREMENT;
    }
    *(opnd.top++) = value;
    return OK;
}

```

(3) 操作数出栈

```

int pop(OPND &opnd, double &value)
{
    // 若栈不空，则删除栈顶元素，并返回操作数
    if (opnd.top == opnd.base)
    {
        return ERROR;
    }
    value = *(--opnd.top);
    return OK;
}

```

(4) 获得栈顶元素

```

int getTop(OPND &opnd, double &value)
{
    // 若栈不空，则返回操作数
    if (opnd.top == opnd.base)
    {
        return ERROR;
    }
    value = *(opnd.top - 1);
    return OK;
}

```

2.4 编码实现

使用链式栈来实现表达式求值，基本思路如下：

- 1、定义两个栈，分别用于存储操作数和操作符。
- 2、将表达式解析为操作数和操作符，分别入栈。
- 3、弹出操作符和操作数进行比较、运算。

2.4.1 创建工程

1、创建工程和解决方案

启动 Microsoft Visual Studio 2010，选择“File -> New -> Project”，新建“Win32 Console Application”，工程名“ExprCPro”。

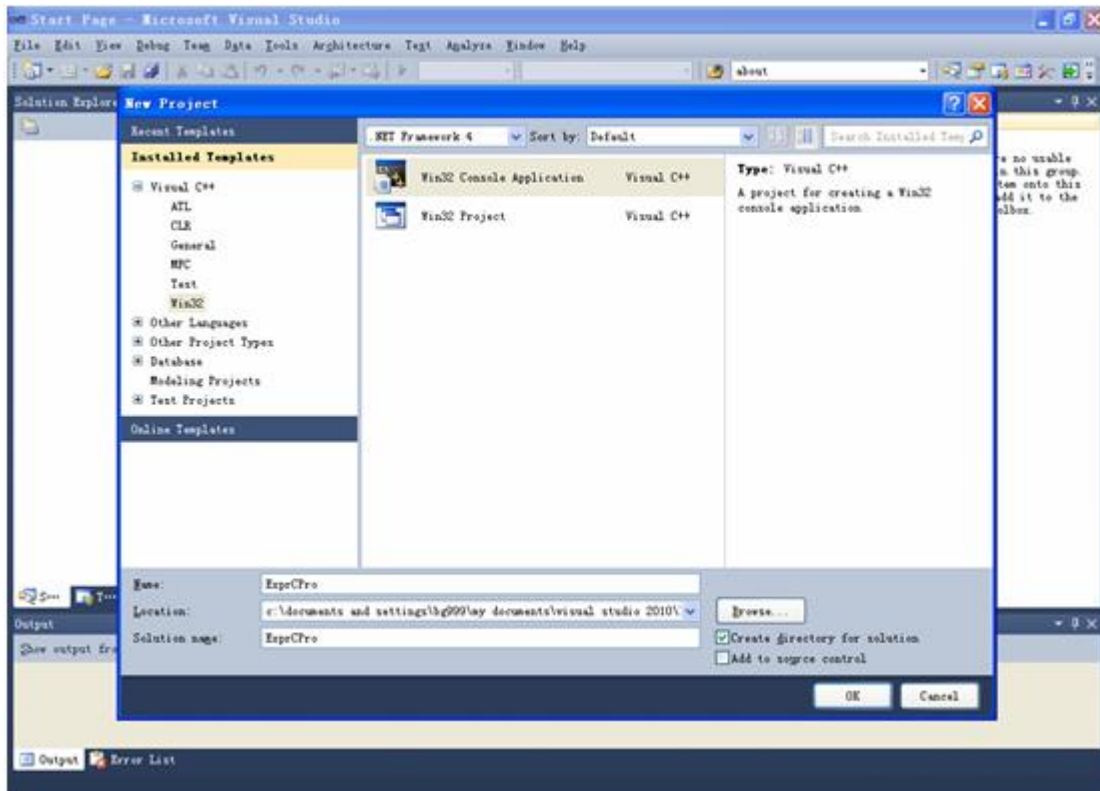


图 2-4 创建工程界面图

新建工程如下。

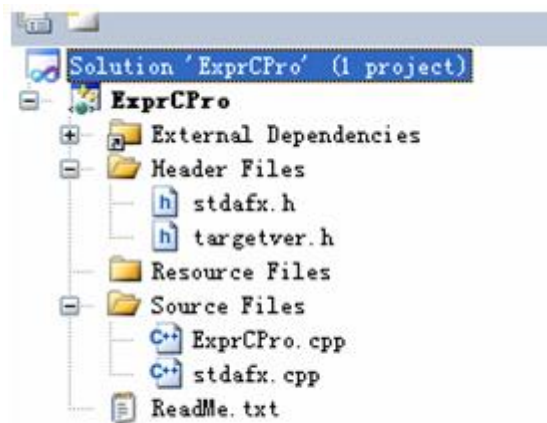


图 2-5 工程结构

2、输出程序界面

ExprCPro.cpp 文件中包含 `int _tmain(int argc, _TCHAR* argv[])` 函数，为应用程序入口，程序将从这里开始运行。

在“ExprCPro.cpp”文件头部导入输入输出流对象 `cout` 与 `cin` 的头文件，并使用 `std` 命名空间。在 `_tmain` 函数中使用 `cout` 输出程序界面。

```
#include <iostream>
using namespace std;
/*****
[函数名称]_tmain
[函数功能]程序主函数，即应用程序入口
[函数参数]int:      参数数目
[函数参数]_TCHAR:  参数列表
[返回值]int:说明程序的退出状态
*****/
int _tmain(int argc, _TCHAR* argv[])
{
    // 输出界面
    cout<<"===== 表达式求值 ====="<<endl;
    cout<<"1 支持四则运算符: + - * /"<<endl;
    cout<<"2 支持括号()"<<endl;
    cout<<"3 支持浮点数。"<<endl;
    cout<<"请输入表达式，按回车结束。"<<endl;
    cout<<"例如: 5*(3+4)/2"<<endl;

    // 输入表达式

    // 输出结果

    return 0;
}
```

3、编译运行

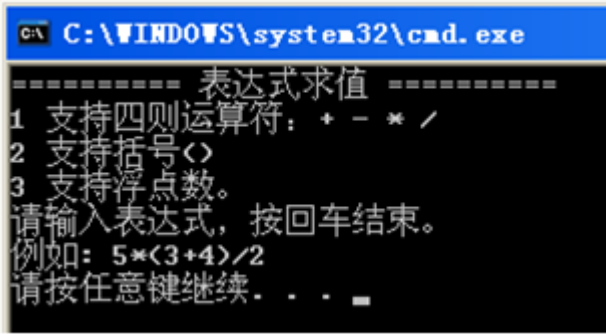


图 2-6 运行结果

2.4.2 定义栈 (链表)

新建头文件 Expr.h，分别定义操作数栈（OPND）和操作符栈（OPTR）的结构体。操作数栈定义为链表节点 StackNodeOPND，操作符栈定义为 StackNodeOPTR。操作栈的具体功能函数，将在 Expr.cpp 文件中实现。

1、添加“Expr.h”头文件

右键单击“Header Files”，选择“Add -> New Item”。

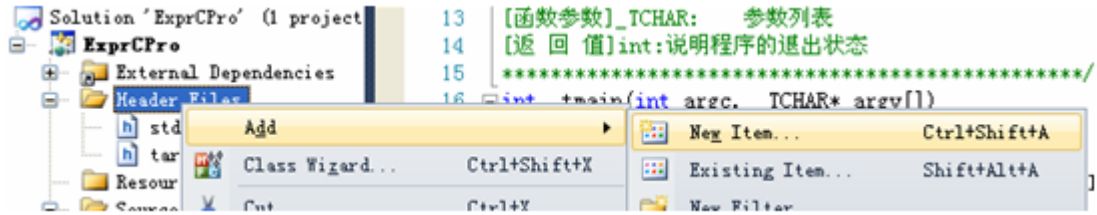


图 2-7 添加头文件操作

新建文件“Header File (.h)”，输入文件名为“Expr”，选择文件保存路径为工程文件夹。

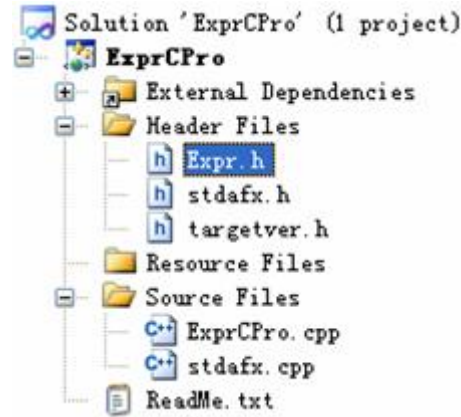


图 2-8 保存路径

2、定义操作数栈和操作符栈

在 Expr.h 文件中，定义操作数栈的结构体和栈的基本操作函数。

```
// 操作数栈
typedef struct StackNodeOPND
{
    double value;
    StackNodeOPND *next;
} *OPND;

// 操作符栈
typedef struct StackNodeOPTR
{
    char op;
    StackNodeOPTR *next;
} *OPTR;
```

2.4.3 表达式解析

把用户输入的表达式保存在字符数组 char expr[]中，定义 double parse(char expr[])函数对其进行解析，将表达式分离为操作数(operand)和操作符(operator)，分别添加到操作数栈(OPND)和操作符栈(OPTR)中。

1、创建 Expr.cpp 文件

在工程中添加 (C++ Files)，文件名为“Expr.cpp”。在 Expr.cpp 中引入 Expr.h 头文件，所有的函数先在 Expr.h 中声明，然后在 Expr.cpp 中实现。

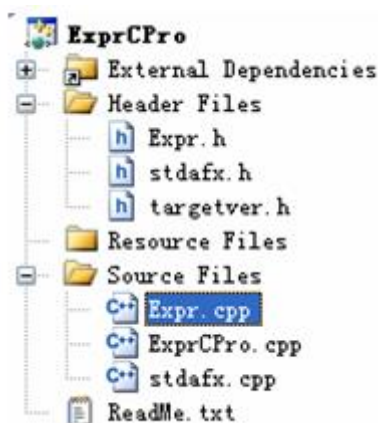


图 2-9 创建 Expr.cpp 文件

2、初始化栈:init

在 Expr.h 中声明 int init(OPND &opnd)函数和 int init(OPTR &optr)函数，分别用于初始化操作数栈和操作符栈。

在 Expr.cpp 中实现栈的初始化。初始化的方法，可参考“2.3.3 栈”中的叙述。

3、入栈 push

在 Expr.h 中声明 int push(OPND &opnd, double value) 函数和 int push(OPTR &optr, char c) 函数，在 Expr.cpp 中实现入栈功能函数。

4、解析表达式

定义 double parse(const char expr[]) 函数，用于解析表达式。将操作数和操作符分别压入操作数栈（OPND）和操作符栈（OPTR）。

在“Expr.h”文件中定义这个函数，在 Expr.cpp 中编程实现。

表达式解析的基本算法如下。

```
double parse(char expr[])
{
    OPND opnd = NULL; // 操作数栈
    OPTR optr = NULL; // 操作符栈

    // 初始化栈
    init(opnd);
    init(optr);

    for(int i=0; expr[i] != '\0'; i++)
    {
        // 读一个字符
        char c = expr[i];
        if (c >= '0' && c <= '9' || c == '.')
        {
            // 处理操作数
            double value = ...
            .....
            push(opnd, value);
        }
        else
        {
            // 处理操作符
            push(optr, c);
        }
    }
    return 0;
}
```

5、处理操作数

处理操作数，可定义一个字符串数组 char token[20]，用于保存扫描到的数字字符串。使用 atof 函数，将 token 转成浮点数。

```
double parse(char expr[])
{
    //.....
}
```

```

        if (c >= '0' && c <= '9' || c == '.')
        {
            // 处理操作数
            char token[20] = {0};
            int j = 0;
            while(expr[i+j] >= '0' && expr[i+j] <= '9' || expr[i+j] == '.')
            {
                token[j] = expr[i+j];
                j++;
            }
            token[j] = '\0';
            i = i + j - 1;

            // 将 token 转成数字，压到操作数栈
            double value = atof(token);
            push(opnd, value);
        }

    return 0;
}

```

6、输出栈元素

在 parse 函数最后，加入 2 段测试代码，输出栈中的元素。

```

double parse(char expr[])
{
    OPND opnd = NULL;// 操作数栈
    OPTR optr = NULL;// 操作符栈

    // 前略

    //////////// 测试代码 ////////////
    // 遍历链表，输出操作符
    cout<<"操作符:";
    while (optr != NULL)
    {
        // 删除旧节点
        OPTR temp = optr;
        optr = temp->next;
        cout<<temp->op<<" ";// 输出符号
        free(temp);
    }
    cout<<endl;
}

```

```

// 遍历链表，输出操作数
cout<<"操作数:";
while (opnd != NULL)
{
    // 删除旧节点
    OPND temp = opnd;
    opnd = temp->next;
    cout<<temp->value<<" ";// 输出数字
    free(temp);
}
cout<<endl;

return 0;
}

```

7、编译运行

在 ExprCPro.cpp 文件中引入 Expr.h 头文件。

在 _tmain 函数中定义字符串数组 char expr[256]，保存用户输入的表达式。并以 expr 为参数，调用 parse 函数。

```

int _tmain(int argc, _TCHAR* argv[])
{
    // 输出界面提示信息(略)

    // 输入表达式
    char expr[256] = {0};
    cout<<"表达式:";
    cin>>expr;

    // 输出结果
    parse(expr);

    return 0;
}

```

编译、运行结果如下。注意栈内元素出栈的顺序，与入栈的顺序相反，后入先出。

```
C:\WINDOWS\system32\cmd.exe

===== 表达式求值 =====
1 支持四则运算符: + - * /
2 支持括号<>
3 支持浮点数。
请输入表达式, 按回车结束。
例如: 5*(3+4)/2
表达式:5*(3+4)/2
操作符:/ > + < *
操作数:2 4 3 5
请按任意键继续. . .
```

图 2-10 运行结果

2.4.4 比较操作符

parse 函数解析表达式, 在处理操作符时, 首先要比较栈顶操作符和当前操作符的优先级, 再分别进行操作。

定义 getTop 函数, 获得操作符栈的栈顶操作符。

定义 precede(char, char)函数, 用于比较栈顶操作符和当前操作符的优先级。

在 Expr.h 文件中定义这 2 个函数, 在 Expr.cpp 中编程实现。

1、获得栈顶元素:getTop

参考“2.3.3 栈”, 实现获得栈顶元素函数。

2、比较操作符优先级

定义 preorder(char, char)函数, 使用二维数组来存储符号优先级关系表, 比较 2 个操作符之间的优先级。

```
/******
[函数名称]precede
[函数功能]判断符号优先级
[函数参数]char a: 栈顶操作符
[函数参数]char b: 当前操作符
[返回值]'<': 栈顶优先级较低; '=': 左右括号相遇; '>': 栈顶优先级较高
*****/

char precede(char a, char b)
{
    char aPriorTable[6][6] = { // 运算符之间的关系存在二维数组中
        //+   -   *   /   (   )
        {'>', '>', '<', '<', '<', '>'}, // + 加号
        {'>', '>', '<', '<', '<', '>'}, // - 减号
        {'>', '>', '>', '>', '<', '>'}, // * 乘号
        {'>', '>', '>', '>', '<', '>'}, // / 除号
        {'<', '<', '<', '<', '<', '='}, // ( 左括号
```



```

        case '=':// 弹出括号
            pop(optr, op);
            break;
        case '>':// 退栈并将运算结果入栈
            // 表达式计算，结果入栈
            i--;
            break;
    }// end switch
}
}
return 0;
}

```

2.4.5 表达式计算

当 OPTR 栈顶运算符优先级 $>$ 当前运算符优先级时，弹出 2 个操作数和 1 个操作符，调用 calc 函数进行计算，并将计算的结果重新压入操作数栈。

在 Expr.h 中定义出栈函数 pop()、表达式计算函数 calc()，并在 Expr.cpp 中实现。

1、出栈 POP

参考“2.3.3 栈”，实现出栈功能函数。

2、表达式计算

定义 calc(double, char, double)函数，计算表达式的值。

```

/*****
[函数名称]calc
[函数功能]表达式解析
[函数参数]double a: 被操作数
[函数参数]char theta: 操作符
[函数参数]double b: 操作数
[返回值]double: 表达式计算结果
*****/

double calc(double a, char theta, double b)
{
    // 运算
    double value = 0;
    switch (theta)
    {
        case '+': value = a + b; break;
        case '-': value = a - b; break;
        case '*': value = a * b; break;
        case '/': value = a / b; break;
        default :
    }
}

```

```

        {
            cout<<"表达式错误."<<endl;
            exit(0);
        }
    }

    return value;
}

```

3、表达式求值

在 parse 函数处理操作符时，若栈顶操作符优先级 > 当前操作符优先级，弹出 2 个操作数和 1 个操作符，调用 calc 函数进行计算，并将计算结果重新压入操作数栈。

由于进行了一次计算，栈顶操作符发生了改变。当前操作符需要与栈顶操作符再次进行比较，使用 “i--;” 语句让 parse 扫描表达式循环一次。

```

double parse(char expr[])
{
    //.....

    // 获得栈顶操作符，比较优先级
    char op = 0;
    getTop(optr, op);
    switch (precede(op, c))
    {
        case '<:// 栈顶元素优先权低，直接入栈
            push(optr, c);
            break;
        case '=':// 弹出括号
            pop(optr, op);
            break;
        case '>:// 退栈并将运算结果入栈
            double a = 0;
            double b = 0;
            // 弹出 2 个操作数和一个操作符
            pop(opnd, b);
            pop(opnd, a);
            pop(optr, op);

            // 计算结果，入栈
            push(opnd, calc(a, op, b));
            i--;
            break;
    } // end switch

    return 0;
}

```

```
}
```

2.4.6 最终结果

表达式扫描完毕后，操作符栈中可能会剩下一个操作符，因此还要调用 `calc` 函数进行一次计算。最后操作数栈顶剩下的元素，即为最终计算的结果。

在 `Expr.h` 中定义 `isEmpty` 函数，判断操作符栈是否为空，在 `Expr.cpp` 中编程实现。

在 `parse` 函数末尾调用 `isEmpty` 函数进行判断，若不为空，则调用 `calc` 函数进行最后一次计算。

1、判断栈是否为空 `is Empty`

参考“2.3.3 栈”，实现判断栈顶为空函数。

2、获得最终结果

```
double parse(char expr[])
{
    // 前略..

    // 最后计算一次
    if(!isEmpty(optr))
    {
        double a = 0;
        double b = 0;
        char op = 0;
        // 操作数、操作符出栈
        pop(opnd, b);
        pop(opnd, a);
        pop(optr, op);
        // 计算结果，入栈
        push(opnd, calc(a, op, b));
    }

    // 最终结果
    double value = 0;
    pop(opnd, value);

    return value;
}
```

3、表达式求值

在 `t_main` 函数中，调用 `parse` 函数，获得计算结果。


```

int _tmain(int argc, _TCHAR* argv[])
{
    // 输出界面

    // 输入表达式
    char expr[256] = {0};
    cout<<"表达式:";
    cin>>expr;

    // 输出结果
    cout<<"结果:"<<parse(expr)<<endl;

    return 0;
}

```

4、编译运行。

2.5 调试和运行

- (1) 按 Ctrl+F5，编译和运行程序。
输出程序界面，提示用户输入表达式。
- (2) 在控制台输入表达式，如：5*(3+4)/2。
- (3) 点击回车，程序解析用户输入的表达式，输出计算结果。

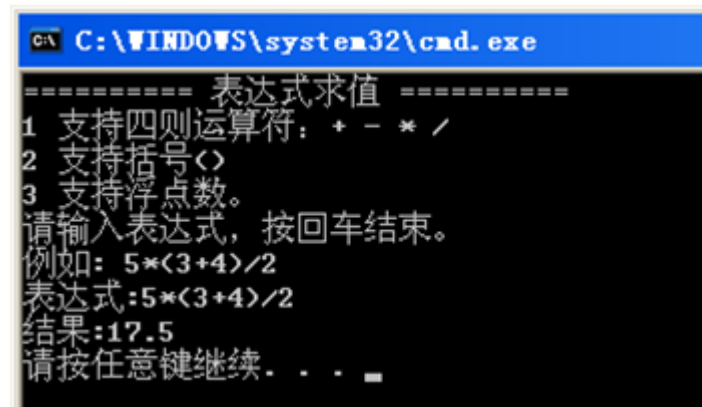


图 2-11 运行结果

习题

1. 表达式括号匹配配对判断问题

【问题描述】假设一个算术表达式中可包含三种括号：圆括号，方括号和花括号且这三种括号可按任意次序嵌套使用。试利用栈的运算，编写判别给定表达式中所含括号是否正确配对出现的算法。

2. 后缀表达式求值问题

【问题描述】从文件读取表达式，判断表达式是否合理，将表达式转换成后缀形式，按后缀表达式求值；题目涉及加减乘除，带括弧的混合运算；随时可以退出。

3. 回文判断问题

【问题描述】编写一个函数判断一个字符序列是否为回文序列，所谓回文序列就是正读与反读都相同的字符序列，例如：abba 和 abdba 均是回文序列。

4. 数制转换问题

【问题描述】任意给定一个 M 进制的数 x ，请实现如下要求 1) 求出此数 x 的 10 进制值（用 MD 表示） 2) 实现对 x 向任意的一个非 M 进制的数的转换。 3) 至少用两种或两种以上的方法实现上述要求（用栈解决，用数组解决，其它方法解决）。

5. 迷宫问题

【问题描述】以一个 $m \times n$ 的长方阵表示迷宫，0 和 1 分别表示迷宫中的通路和障碍。设计一个程序，对任意设定的迷宫，求出一条从入口到出口的通路，或得出没有通路的结论。

6. Hanoi 塔问题

【问题描述】现在有三根相邻的柱子，标号为 A,B,C。A 柱子上从下到上按金字塔状叠放着 n 个不同大小的圆盘，现在把所有盘子一个一个移动到柱子 B 上，并且每次移动同一根柱子上都不能出现大盘子在小盘子上方，请问至少需要多少次移动。

3 队列与停车场管理程序实践

3.1 实验目标

- 1、掌握队列的操作和应用。
- 2、使用 C++ 语言和队列实现“停车场管理程序”专题编程。

3.2 实验任务

1、停车场配置

有这样一个停车场，拥有过 100 个车位。车辆停放在一条单行道上，而且只能从停车场的一端进入，从另一端离开。停车场另外提供一个条过道，可供 10 辆汽车临时停靠。

停车场的平面图如下所示：

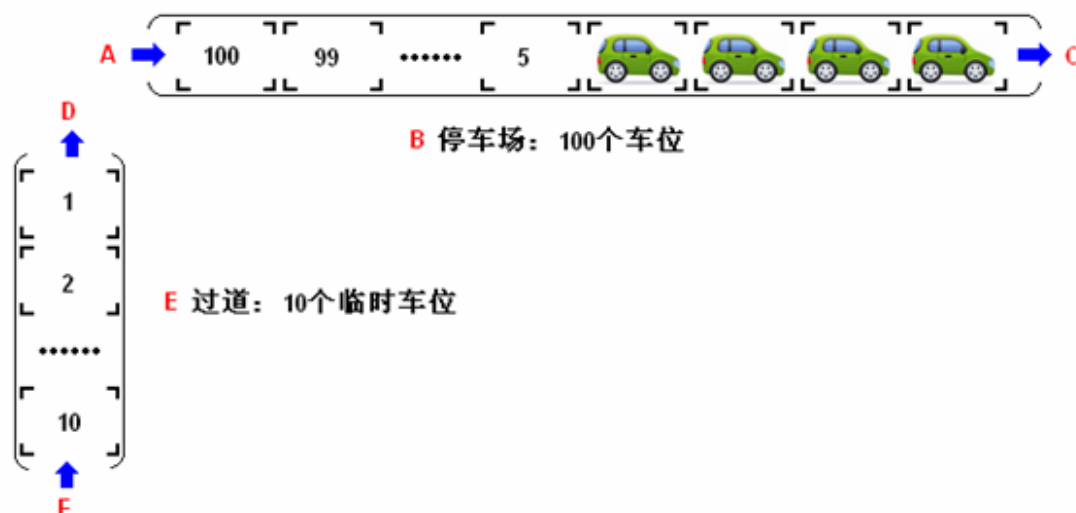


图 3-1 停车场示意图

B 为停车场，提供 100 个车位。其中 A 是入口，C 是出口。

E 为过道，提供 10 个临时车位。其中 F 是入口，D 是出口。

车辆进入停车场 B 时，只能从 A 端进入，从 C 端离开。若停车场 B 内已停满 100 辆车，那么后来的车只能在门外的过道 E 上等候。一旦停车场 B 中有空位，过道 E 上的第一辆车即可从 D 开出，从 A 进入停车场 B 中。

2、收费规则

每辆停放在车场的车，在离开停车场时，必须按停留的时间长短交纳费用。停车按 2 元/小时计费，不足 1 小时的不收费。

车辆从 A 端进入时开始计费，从 C 端离开时收费。只有进入停车场 B 的车辆收费，停在过道 E 上的车辆不收费。

3、停车场管理程序

停车场管理程序是一个控制台程序，提供“车辆入场”、“车辆离开”、“查看停车场” 3

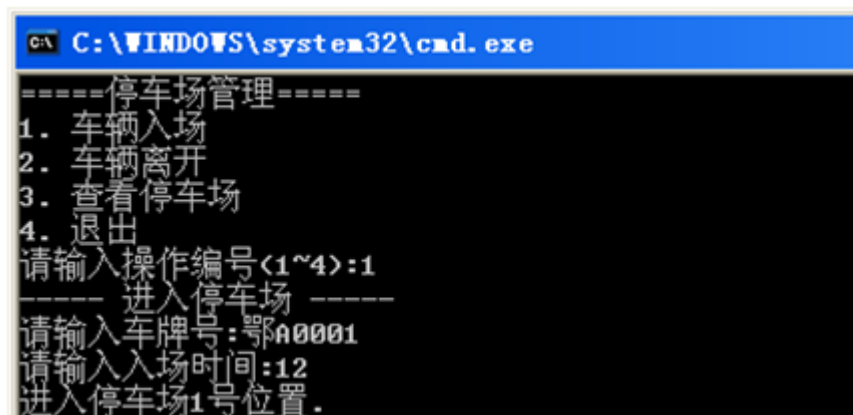
个功能。

(1) 车辆入场

输入：汽车的车牌号码和入场时间

处理：判断停车场是否有空位，若有空位则入场；若没有空位，则停到便道上；若便道也满了，则直接离开。

输出：显示车辆到达后的停车位置（停车场或者便道上）。



```
C:\WINDOWS\system32\cmd.exe
====停车场管理====
1. 车辆入场
2. 车辆离开
3. 查看停车场
4. 退出
请输入操作编号<1~4>:1
----- 进入停车场 -----
请输入车牌号:鄂A00001
请输入入场时间:12
进入停车场1号位置.
```

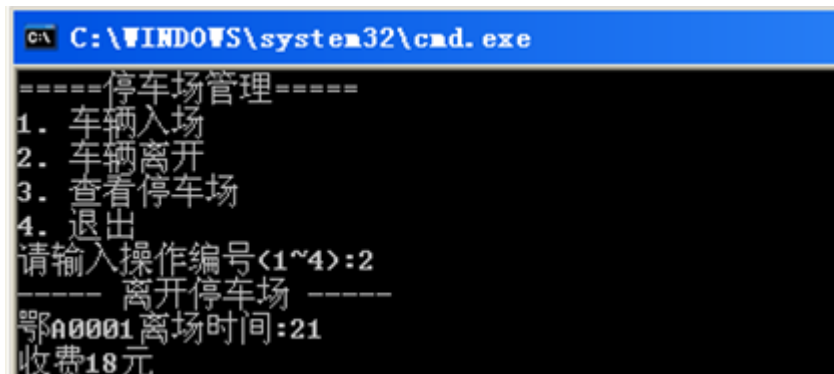
图 3-2 车辆进入停车场

(2) 车辆离开

输入：汽车离开的时间。

处理：计算车辆应缴纳的费用。若有车在过道上等候，则让排在过道上的第一辆车进入停车场，开始计时收费。

输出：缴费数额。



```
C:\WINDOWS\system32\cmd.exe
====停车场管理====
1. 车辆入场
2. 车辆离开
3. 查看停车场
4. 退出
请输入操作编号<1~4>:2
----- 离开停车场 -----
请输入离场时间:21
鄂A00001 离场时间:21
收费18元
```

图 3-3 车辆离开停车场

(3) 查看停车场

输入：已有停车场

处理：检车停车站中是否有车，若有则遍历打印所有的车辆信息。

输出：显示停车场中所有车牌号码及入场时间。

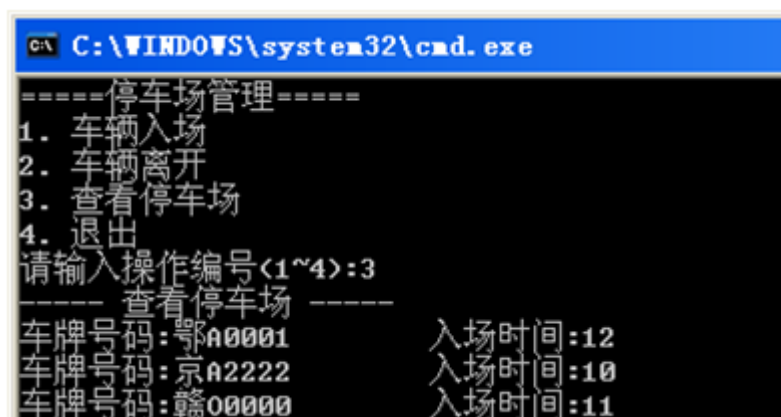


图 3-4 查看停车场停车情况

3.3 分析和设计

停车场具有先入先出的特点，而且停车位数量是固定的。因此本程序将选择数组作为车辆信息的存储方式，使用循环队列来操作车辆的入场和离开。

程序中将创建 App.cpp 文件，定义 int main(void) 函数，作为程序运行的入口。App.cpp 文件中还会定义 InPark、OutPark、ViewPark 等函数，分别实现“车辆入场”、“车辆离开”、“查看停车场”这三个功能。

创建 2 个文件 Park.h 与 Park.cpp，其中 Park.h 用于定义数据结构、声明函数，Park.cpp 用于实现具体的函数功能。

3.3.1 数据结构

根据需求可知，停车场和过道都是狭长通道，且具备先入先出 (FIFO) 的特点，适合采用队列来保存停车场和过道上的车辆信息。当有车辆要入场时，将车辆信息插入到队列的末尾。若有车辆要离开，删除队头元素。



图 3-5 停车场队列

1、队列

队列的表示和实现，可以分为链式表示与顺序表示。链式表示采用链表来实现，称为链队列；顺序表示采用数组来实现，称为顺序队列。一般情况下，若能估算出数据的规模，可以考虑顺序队列；如果无法确定，需要使用链式队列。

由于停车场和过道上的车位数量都是已知的，因此采用顺序队列作为本程序的数据结构，使用数组来存储车辆信息。

一般的，可以如下定义顺序队列的结构体。

```
#define MAXQSIZE 100 // 最大队列长度
typedef struct
{
    DataType data[MAXQSIZE]; // 存储空间
    int front; // 头指针，若队列不空，指向队列头元素
    int rear; // 尾指针，若队列不空，指向队列尾元素的下一个元素
} SqQueue;
```

顺序队列有如下几个基本操作：

- (1) 初始化：front = rear = 0
- (2) 插入元素：rear++
- (3) 删除元素：front++
- (4) 队列为空：front == rear

2、循环队列

从顺序存储的队列可以看出，有可能出现这样情况，尾指针指向一维数组最后，但前面有很多元素已经出队，即空出很多位置，这时要插入元素，就会发生溢出。例如，在下图中，若队列的最大容量 MAXQSIZE=12，若 rear=11，再进队时将发生溢出。我们将这种溢出称为“假溢出”。



图 3-6 队列元素进出示意图

要克服“假溢出”，可以将整个队列中元素向前移动，直到头指针 front 为 0，或者每次出队时，都将队列中元素前移一个位置。因此，顺序队列的队满判定条件为 rear==MAXQSIZE-1。

但是，在顺序队列中，这些克服假溢出的方法都会引起大量元素的移动，花费大量的

时间，所以在实际应用中很少采用，一般采用循环队列的形式。将一维数组 `data[0]` 到 `data[MAXQSIZE-1]` 看成是一个首尾相接的圆环，即 `data[0]` 与 `data[MAXQSIZE-1]` 相接在一起。将这种形式的顺序队列称为循环队列。再次插入元素时，操作如下图所示。

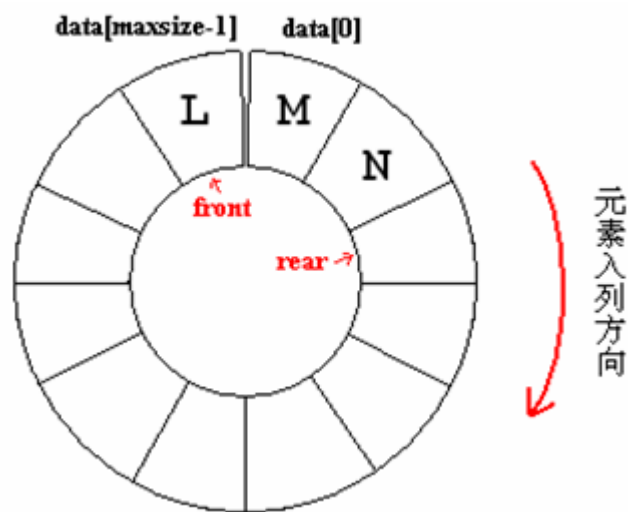


图 3-7 元素入列方向示意图

对于在使用顺序队列时可能会产生的“假溢出”问题，本程序中将使用循环队列的方式来解决。循环队列有如下几个基本操作：

- (1) 初始化： `front = rear = 0`
- (2) 插入元素： `rear = (rear + 1) % maxqsize`
- (3) 删除元素： `front = (front + 1) % maxqsize`
- (4) 队列为空： `front == rear`
- (5) 队列满： `(rear + 1) % maxqsize == front`

3.3.2 业务流程

1、车辆入场

- (1) 当有车辆要入场时，优先进入停车场的车位。
- (2) 若停车场没有车位了，则停到过道上。
- (3) 若过道也满了，则通知车辆离开。

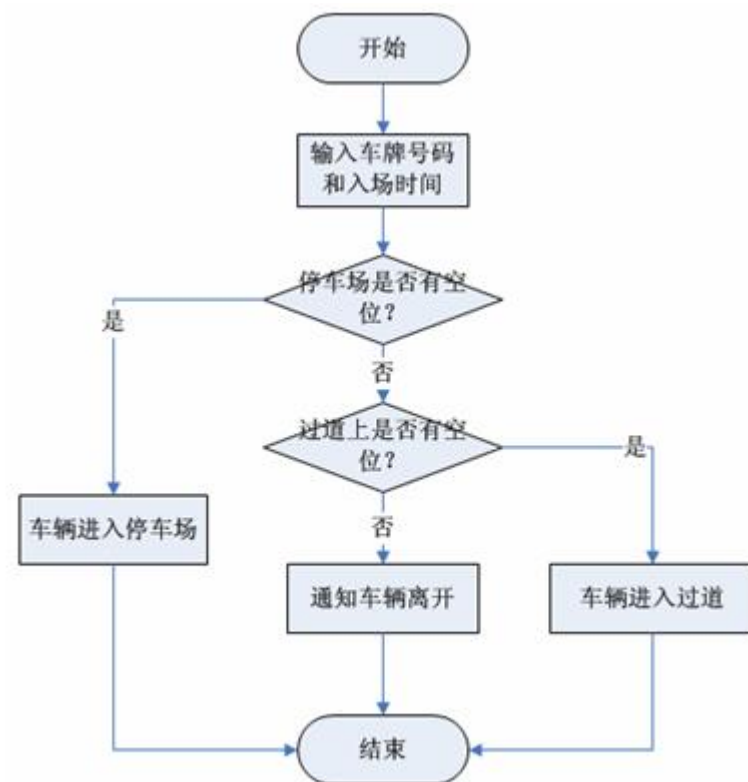


图 3-8 车辆入场流程图

2、车辆离开

- (1) 输入车辆离场时间，计算停车费用，缴费离场。
- (2) 判断过道上是否有车在等待，若有则进入停车场。

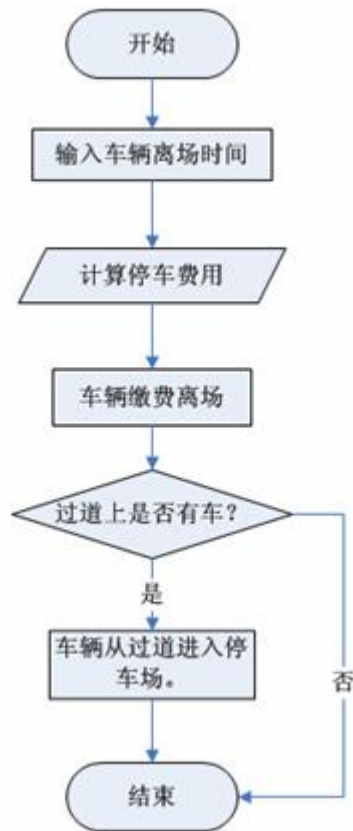


图 3-9 车辆离开流程图

3.3.3 技术分析

1、队列(Queue)

队列(queue)是一种先进先出(first in first out, 缩写为 FIFO)的线性表。它只允许在表的一端进行插入, 而在另一端删除元素。最早进入队列的元素最早离开。在队列中, 允许插入的一端叫做队尾(rear), 允许删除的一端叫做队头(front)。

队列的基本操作如下:

- (1) 初始化队列 InitQueue(&q)
- (2) 销毁队列 DestoryQueue(&q)
- (3) 清空队列 ClearQueue(&q)
- (4) 判断队列是否为空 QueueEmpty(q)
- (5) 获得队列长度 QueueLength(q)
- (6) 获得队首元素 GetHead(&q, &e)
- (7) 插入元素到队尾 EnQueue(&q, &e)
- (8) 删除队首元素 DeQueue(&q, &e)

2、循环队列

在队列顺序存储结构中, 使用一组地址连续的存储单元, 依次存放从队列头到队列尾的元素, 定义两个 int 型变量 front 和 rear 指向队列头元素和尾元素的位置, 定义一个数组

保存队列中的数据信息。将一维数组 data[0]到 data[MAXQSIZE-1]看成是一个首尾相接的圆环，即 data[0]与 data[MAXQSIZE-1]相接在一起。将这种形式的顺序队列称为循环队列。

定义数据结构如下：

```
#define ERROR -1
#define OK 0

// 汽车节点
struct Car
{
    char lisence_plate[10]; // 车牌
    int time;               // 进入停车场的时间
};

// 循环队列
typedef struct SqQueue
{
    Car *base;              // 数组
    int front;              // 队头指针
    int rear;               // 队尾指针
    int size;               // 最大队列长度
}Queue;
```

基本操作函数如下：

(1) 初始化队列 InitQueue(&q)

```
int InitQueue(Queue &q, int nSize)
{
    q.base = (Car *) malloc(nSize * sizeof(Car));
    if (!q.base)
    { // 内存分配失败
        return ERROR;
    }
    q.front = q.rear = 0;
    q.size = nSize;
    return OK;
}
```

(2) 销毁队列 DestoryQueue(&q)

```
int DestoryQueue(Queue &q)
{
    // 释放空间
    if (q.base)
    {
        free(q.base);
        q.base = NULL;
    }
}
```

```
    }  
    q.front = q.rear = 0;  
    q.size = 0;  
    return OK;  
}
```

(3) 清空队列 **ClearQueue(&q)**

```
int ClearQueue(Queue &q)  
{  
    q.front = q.rear = 0;  
    return OK;  
}
```

(4) 判断队列是否为空 **QueueEmpty(q)**

```
int QueueEmpty(Queue q)  
{  
    return (q.front == q.rear);  
}
```

(5) 获得队列长度 **QueueLength(q)**

```
int QueueLength(Queue q)  
{  
    return (q.rear - q.front + q.size) % q.size;  
}
```

(6) 获得队首元素 **GetHead(&q, &e)**

```
int GetHead (Queue &q, Car &car)  
{  
    car = q.base[q.front];  
    return OK;  
}
```

(7) 插入元素到队尾 **EnQueue(&q, &e)**

```
int EnQueue(Queue &q, Car car)  
{  
    // 队列满  
    if ((q.rear + 1) % q.size == q.front)  
    {  
        return ERROR;  
    }  
    q.base[q.rear] = car;  
    q.rear = (q.rear + 1) % q.size;  
    return OK;  
}
```

(8) 删除队首元素 DeQueue(&q, &e)

```
int DeQueue(Queue &q, Car &car)
{
    // 队列空
    if (q.front == q.rear)
    {
        return ERROR;
    }
    car = q.base[q.front];
    q.front = (q.front + 1) % q.size;
    return OK;
}
```

3、链队列

用链表表示的队列简称为链队列。一个链队列需要 2 个分别指向队头和队尾的指针才能唯一确定。为了方便操作，可以给链表添加一个头结点，并令头指针指向头结点。这样，空的链队列的判断条件为头指针与尾指针均指向头结点。

定义数据结构如下：

```
#define ERROR -1
#define OK 0

// 汽车节点
struct Car
{
    char lisence_plate[10]; // 车牌
    int time;               // 进入停车场的时间
    Car *next;
};

// 链队列
typedef struct LinkQueue
{
    Car *front;             // 队头指针
    Car *rear;             // 队尾指针
}Queue;
```

基本操作函数如下：

(1) 初始化队列 InitQueue(&q)

```
int InitQueue(Queue &q)
{
    // 构造一个空队列
    q.front = q.rear = (Car *)malloc(sizeof(Car));
}
```

```

        if(!q.front)
        {
            // 内存分配失败
            exit(-1);
            return ERROR;
        }
        q.front->next = NULL;
        return OK;
    }

```

(2) 销毁队列 DestoryQueue(&q)

```

int DestoryQueue(Queue &q)
{
    // 销毁队列
    while(q.front)
    {
        q.rear = q.front->next;
        free(q.front);
        q.front = q.rear;
    }
    return OK;
}

```

(3) 判断队列是否为空 QueueEmpty(q)

```

int QueueEmpty(Queue q)
{
    return (q.front == q.rear);
}

```

(4) 获得队列长度 QueueLength(q)

```

int QueueLength(Queue q)
{
    int length = 0;
    for (Car *p = q.front; p->next != q.rear; p = p->next)
    {
        length++;
    }

    return length;
}

```

(5) 获得队首元素 GetHead(&q, &e)

```

int GetHead(Queue &q, Car &car)
{
    car = *q.front->next;
    return OK;
}

```

(6) 插入元素到队尾 EnQueue(&q, &e)

```

int EnQueue(Queue &q, Car car)
{
    // 插入元素
    Car *p = (Car*)malloc(sizeof(Car));
    if(!p)
    {
        // 内存分配失败
        exit(-1);
        return ERROR;
    }
    strcpy(p->lisence_plate, car.lisence_plate);
    p->time = car.time;
    q.rear->next = p;
    q.rear = p;
    return OK;
}

```

(7) 删除队首元素 DeQueue(&q, &e)

```

int DeQueue(Queue &q, Car &car)
{
    // 若队列不空，删除队头元素，用&car 返回其值
    if(q.front == q.rear)
    {
        return ERROR;
    }
    // 返回元素
    Car *p = q.front->next;
    car = *p;

    q.front->next = p->next; // 指向下一个元素
    if(q.rear == p)
    {
        q.rear = q.front;
    }
    free(p); // 释放空间
    return OK;
}

```

3.4 编码实现

3.4.1 创建工程

VS2010 开发工具通过解决方案来组织和管理代码，以工程为单位进行项目开发。创建停车场管理程序的解决方案及工程，并创建程序运行入口，输出程序的主界面。

1、创建解决方案

启动 Microsoft Visual Studio 2010，选择“File -> New -> Project”，空白解决方案，方案名为“ParkCPro”。

2、创建工程

新建“Win32 Console Application”空工程，工程名“ParkCPro”。

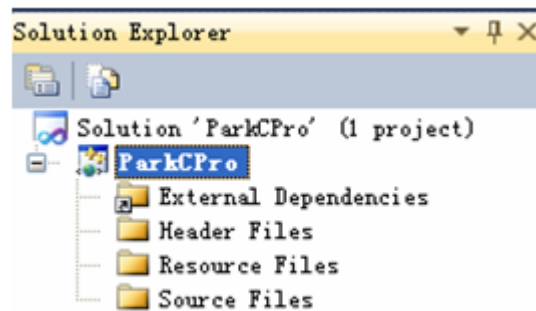


图 3-10 创建工程

3、程序入口

创建 App.cpp 文件，定义 int main(void)函数，作为程序的入口。

导入<iostream>头文件，使用 cout 在控制台下输出“====停车场管理====”。

```

#include <iostream>
using namespace std;

/*****
[函数名称]main
[函数功能]程序主函数，即应用程序入口
[函数参数]无
[返回值]int: 说明程序退出的状态
*****/

int main(void)
{
    // 输出界面
    cout<<"====停车场管理===="<<endl;

    return 0;
}

```

编译运行。



图 3-11 运行结果

4、主菜单

在控制台下输出菜单，供用户选择停车、离开、查看队列等操作。使用 `cin` 来获得用户的输入，使用 `switch-case` 语句来处理不同的选项，使用 `while` 语句和 `bool` 变量来控制菜单的循环和退出。


```

int main(void)
{
    bool running = true;// 程序运行标识
    char key[10] = {0};// 用户输入的选项
    while(running)
    {
        // 输出界面
        cout<<"====停车场管理===="<<endl;
        cout<<"1. 车辆入场"<<endl;
        cout<<"2. 车辆离开"<<endl;
        cout<<"3. 查看停车场"<<endl;
        cout<<"4. 退出"<<endl;

        // 输入选项
        cout<<"请输入操作编号(1~4):";
        cin>>key;

        // 处理用户输入
        switch(key[0]) {
            case '1':// 车辆入场
                break;
            case '2':// 车辆离开
                break;
            case '3':// 查看停车场
                break;
            case '4':// 退出
                running = false;
                cout<<"再见!"<<endl;
                break;
        }
    }
    return 0;
}

```

编译运行。

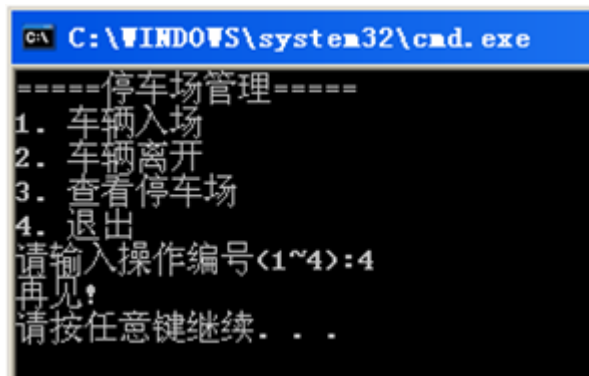


图 3-12 运行结果

3.4.2 定义数据结构

新建 Park.h 头文件，用于定义常量和数据结构。队列的操作函数，会先定义在 Park.h 文件中，然后在 Park.cpp 文件中实现。在 Park.h 中定义循环队列的数据结构，声明初始化队列、销毁队列这两个基本操作函数，并在 Park.cpp 中实现。

将 Park.h 引入到 App.cpp 文件中，就可以在 main 函数中使用队列了。在 main 中创建 2 个队列，分别代表停车场与过道，并对其进行初始化、销毁等基本操作。

1、添加文件

右键单击工程，选择“Add -> New Item...”。新建 Park.h 文件和 Park.cpp 文件。

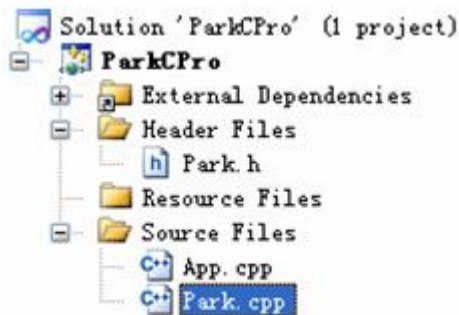


图 3-13 添加文件

2、定义循环队列

在 Park.h 文件中，定义循环队列数据结构。具体方法可参考“5.2 循环队列”。

```
// 汽车节点
struct Car
{
    char lisence_plate[10]; // 车牌
    int time;                // 进入停车场的时间
};

// 循环队列
typedef struct SqQueue
```

```
{
    Car *base;           // 数组
    int front;           // 队头指针
    int rear;            // 队尾指针
    int size;            // 队列长度
}Queue;
```

3、初始化队列

在 Park.h 中声明 InitQueue 函数，用于初始化队列。在 Park.cpp 中实现队列的初始化函数。可参考“3.3.3 技术分析”中的叙述。

将 Park.h 引入到 App.cpp 文件中，在 main 函数顶部声明两个队列，分别代表停车场和临时车道。对这两个队列进行初始化。

```
#include "Park.h"

int main(void)
{
    // 声明队列
    Queue sqPark;// 停车场
    Queue sqAisle;// 临时车道

    // 初始化队列
    InitQueue(sqPark, 100);// 初始化停车场
    InitQueue(sqAisle, 10);// 初始化临时车道

    // .....

    return 0;
}
```

4、销毁队列

在 Park.h 中声明 DestoryQueue 函数，用于销毁队列，释放内存。在 Park.cpp 中实现此函数。可参考“3.3.3 技术分析”中的叙述。

在 main 函数程序退出之前，调用 DestoryQueue 函数销毁队列。

```

#include "Park.h"

int main(void)
{
    // .....

    // Destory queue
    DestoryQueue(sqPark);
    DestoryQueue(sqPassage);
    return 0;
}

```

3.4.3 车辆入场

在 App.cpp 文件中，声明 InPark 函数，用于从控制台获得用户输入的车辆信息。在 main 函数的主菜单循环中调用 InPark 函数，实现“车辆入场”功能。

“车辆入场”功能将向队列尾部插入车辆信息，并需要获得队列长度。在 Park.h 中声明 EnQueue 函数和 QueueLength 函数的定义，并在 Park.cpp 中实现函数功能。

最后，创建一个 QueueTraverse 函数，遍历队列，将队列中的车辆信息输出到控制台中。

1、插入队尾元素

在 Park.h 中声明 EnQueue 函数，用于向队尾插入元素。在 Park.cpp 中实现此函数。可参考“3.3.3 技术分析”中的叙述。

2、获得队列长度

在 Park.h 中声明 QueueLength 函数，用于向队尾插入元素。在 Park.cpp 中实现此函数。可参考“3.3.3 技术分析”中的叙述。

3、进入停车场

在 App.cpp 文件中，声明 InPark 函数。调用 EnQueue 函数，将车辆信息插入到队列尾部。车辆入场时，遵循“3.3.2 业务流程”中描述的业务流程。

```

int InPark(Queue &sqPark, Queue &sqAisle)
{
    cout<<"----- 进入停车场 -----"<<endl;

    Car car;
    cout<<"请输入车牌号:";
    cin>>car.lisence_plate;
    cout<<"请输入入场时间:";
    cin>>car.time;

    if (OK == EnQueue(sqPark, car))
    {

```

```

        cout<<"进入停车场"<<QueueLength(sqPark)<<"号位置."<<endl;
    }
    else if (OK == EnQueue(sqAisle, car))
    {
        cout<<" 停车场 已满,进入临时车道 "<<QueueLength(sqAisle)<<" 号位
置."<<endl;
    }
    else
    {
        cout<<"没有空余车位，离开停车场."<<endl;
    }

    return 0;
}

```

4、车辆入场

在 main 函数的主菜单循环中调用 InPark 函数，实现“车辆入场”功能。

```

int main(void)
{
    // .....

    switch(key[0]) {
    case '1':// 车辆入场
        {
            InPark(sqPark, sqAisle);
            break;
        }
    case '2':// 车辆离开
        break;
    case '3':// 查看停车场
        break;
    case '4':// 退出
        running = false;
        cout<<"再见!"<<endl;
        break;
    }

    return 0;
}

```

编译运行。

```
C:\WINDOWS\system32\cmd.exe
=====停车场管理=====
1. 车辆入场
2. 车辆离开
3. 查看停车场
4. 退出
请输入操作编号<1~4>:1
----- 进入停车场 -----
请输入车牌号:鄂A00001
请输入入场时间:12
进入停车场1号位置.
```

图 3-14 运行结果

3.4.4 车辆离开

在 Park.h 中声明一个 DeQueue 函数，在 Park.cpp 中实现此函数。在 App.cpp 中创建一个 OutPark 函数，调用 DeQueue 函数实现“车辆离开”功能，并在 main 函数的主菜单循环中调用。

1、删除队首元素

在 Park.h 中声明 DeQueue 函数，若队列不空，则删除队首元素，并将其返回。在 Park.cpp 中实现此函数。可参考“3.3.3 技术分析”中的叙述。

2、离开停车场

在 App.cpp 文件中，声明 OutPark 函数。调用 DeQueue 函数，从队列头部获得车辆信息，并删除队首元素。车辆离开时，遵循“4.2 业务规则”中描述的业务流程。

```
int OutPark(Queue &sqPark, Queue &sqAisle)
{
    cout<<"----- 离开停车场 -----"<<endl;
    Car car;
    if (OK == DeQueue(sqPark, car))
    {
        int time;
        cout<<car.lisence_plate<<"离场时间:";
        cin>>time;

        int fee = (time - car.time) * 2;// 每小时 2 元
        cout<<"收费"<<fee<<"元"<<endl;

        if (OK == DeQueue(sqAisle, car))
        {
            cout<<car.lisence_plate<<"从过道进入停车场"<<endl;
            cout<<"请输入入场时间:";
```

```

        cin>>car.time;
        EnQueue(sqPark, car);
    }
}
else
{
    cout<<"停车场没有车要离开."<<endl;
}

return 0;
}

```

3、车辆离开

在 main 函数的主菜单循环中调用 OutPark 函数，实现“车辆离开”功能。

```

int main(void)
{
    // .....

    switch(key[0]) {
    case '1':// 车辆入场
        {
            InPark(sqPark, sqAisle);
            break;
        }
    case '2':// 车辆离开
        {
            OutPark(sqPark, sqAisle);
            break;
        }
    case '3':// 查看停车场
        break;
    case '4':// 退出
        running = false;
        cout<<"再见!"<<endl;
        break;
    }

    return 0;
}

```

编译运行。



```
C:\WINDOWS\system32\cmd.exe
=====停车场管理=====
1. 车辆入场
2. 车辆离开
3. 查看停车场
4. 退出
请输入操作编号<1~4>:2
----- 离开停车场 -----
鄂A00001 离场时间:21
收费18元
```

图 3-15 运行结果

3.4.5 查看停车场

在 Park.h 中声明一个 QueueTraverse 函数，用于遍历队列，将队列中的车辆信息输出到控制台中。在 Park.cpp 中实现此函数。

在 App.cpp 中创建一个 ViewPark 函数，调用 QueueTraverse 函数，实现“查看停车场”功能，并在 main 函数的主菜单循环中调用。

1、遍历队列元素

在 Park.h 中声明 QueueTraverse 函数，在 Park.cpp 中实现此函数。遍历队列，将队列中的车辆信息输出到控制台中。

```
int QueueTraverse(Queue q)
{
    if (q.front == q.rear)
    {
        return ERROR;
    }
    int i = q.front;
    while (i != q.rear)
    {
        Car car = q.base[i];
        cout<<"车牌号码:"<<car.lisence_plate<<"\t 入场时间:"<<car.time<<endl;
        i = (i + 1) % q.size;
    }

    return OK;
}
```

2、查看停车场

在 App.cpp 文件中，声明 ViewPark 函数。调用 QueueTraverse 函数，遍历输出 2 个队列中的车辆信息。


```

int ViewPark(Queue &sqPark, Queue &sqAisle)
{
    cout<<"----- 查看停车场 -----"<<endl;

    QueueTraverse(sqPark);
    QueueTraverse(sqAisle);
    return 0;
}

```

在 main 函数的主菜单循环中调用 ViewPark 函数，实现“查看停车场”功能。

```

int main(void)
{
    // .....

    switch(key[0]) {
    case '1':// 车辆入场
        {
            InPark(sqPark, sqAisle);
            break;
        }
    case '2':// 车辆离开
        {
            OutPark(sqPark, sqAisle);
            break;
        }
    case '3':// 查看停车场
        {
            ViewPark(sqPark, sqAisle);
            break;
        }
    case '4':// 退出
        running = false;
        cout<<"再见!"<<endl;
        break;
    }

    return 0;
}

```

编译运行。

```
C:\WINDOWS\system32\cmd.exe

=====停车场管理=====
1. 车辆入场
2. 车辆离开
3. 查看停车场
4. 退出
请输入操作编号<1~4>:3
----- 查看停车场 -----
车牌号码:鄂A00001      入场时间:12
车牌号码:京A2222      入场时间:10
车牌号码:赣00000      入场时间:11
```

图 3-16 运行结果

3.5 调试和运行

- (1) 按 Ctrl+F5，编译和运行程序。
- (2) 输出程序界面，提示用户输入操作编号，选择一个功能。

```
C:\WINDOWS\system32\cmd.exe

=====停车场管理=====
1. 车辆入场
2. 车辆离开
3. 查看停车场
4. 退出
请输入操作编号<1~4>:
```

图 3-17 运行结果

1、车辆入场

- (1) 输入 1，选择进入“车辆入场”功能
- (2) 输入汽车的车牌号码和入场时间
- (3) 程序显示车辆到达后的停车位置。

```
C:\WINDOWS\system32\cmd.exe

=====停车场管理=====
1. 车辆入场
2. 车辆离开
3. 查看停车场
4. 退出
请输入操作编号<1~4>:1
----- 进入停车场 -----
请输入车牌号:鄂A00001
请输入入场时间:12
进入停车场1号位置.
```

图 3-18 入场运行结果

2、车辆离开

- (1) 输入 2，选择进入“车辆离开”功能
- (2) 输入汽车离开的时间
- (3) 输出停车缴费数额

```
C:\WINDOWS\system32\cmd.exe
=====停车场管理=====
1. 车辆入场
2. 车辆离开
3. 查看停车场
4. 退出
请输入操作编号<1~4>:2
----- 离开停车场 -----
鄂A00001离场时间:21
收费18元
```

图 3-19 离开运行结果

3、查看停车场

- (1) 输入 3，选择进入“查看停车场”功能
- (2) 显示停车场中所有车牌号码及入场时间

```
C:\WINDOWS\system32\cmd.exe
=====停车场管理=====
1. 车辆入场
2. 车辆离开
3. 查看停车场
4. 退出
请输入操作编号<1~4>:3
----- 查看停车场 -----
车牌号码:鄂A00001    入场时间:12
车牌号码:京A2222    入场时间:10
车牌号码:赣00000    入场时间:11
```

图 3-20 查看停车情况

4、退出

- (1) 输入 4，选择进入“退出”功能
- (2) 退出程序

习题

1. 银行业务模拟与离散事件模拟

【问题描述】假设某银行有四个窗口对外接待客户，从早晨银行开门起不断有客户进入银行。由于每个窗口在某个时刻只能接待一个客户，因此在客户人数众多时需在每个窗口前顺次排队，对于刚进入银行的客户，如果某个窗口的业务员正空闲，则可上前办理业务，反之，若四个窗口均有客户所占，他便会排在人数最少的队伍后面。这个程序以模拟银行的这种业务活动并计算一天中客户在银行逗留的平均时间。

2. 舞伴配对程序

【问题描述】假设在某场舞会上，男士和女士进入舞厅时分别排成两队。跳舞开始时，依次从男队和女队的开始位置各出一个配成舞伴。若两初始人数不相同，则较长的那一队中未配对者等待下一轮舞曲。编写程序模拟上述舞伴配对程序。

3. 电梯运行模拟

【问题描述】综合办公楼有 14 层，设有 4 部自动电梯。每梯最多载客 12 人。从上班到下班不断有乘客使用电梯。电梯运行每上一层需 5 秒，空闲时，在某一层最多停留 10 秒，否则返回 1 层待命。乘客等待电梯时间不超过 T。

4. 打印杨辉三角

【问题描述】利用队列打印杨辉三角。如下图所示。

```
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
1 6 15 20 15 6 1
```

5. 模拟医院患者排队候诊

【问题描述】患者医院看病过程：先在诊室外等候，再看病治疗。排队过程中做两件事：患者到达诊室时将病历交给护士，排入等候的队列的尾部候诊（队尾入队）；二是护士从等候的队列中取出下一个患者的病历（队头元素出队），让患者进入诊室就诊。

【基本要求及提示】在排队时，按照“先到先服务”的原则。“患者到达”用 a 表示；“护士让下一位患者就诊”用 n 表示；“不再接收患者排队”用 q 表示。

(1) 当有“患者到达”命令时，则执行入队操作；

(2) 当有“护士让下一位患者就诊”命令时，则对头元素出队，进行诊疗；

(3) 当有“不再接收患者排队”命令时，则队列中所有元素出队，程序结束。(4) 要求程序通过主菜单进行控制，通过选择菜单项的序号来调用各功能函数。

6. 键盘输入缓冲区的问题

【问题描述】有两个进程同时存在于一个程序中。其中第一个进程在屏幕上连续显示‘A’字符，与此同时，程序不断检测是否有键盘输入，如果有，就读如用户键入的字符并保存到输入缓冲区中。在用户输入时键入的字符不立即显示在屏幕上。当用户渐入’；’或’，’时，第一个进程结束，第二个进程从缓冲区中依次读取用户输入的字符，并显示在屏幕上。第二个进程结束后，程序又进入第一个进程，重新显示‘A’，同时用户又可以继续键入字符，直到用户键入‘.’时，第二个进程结束。另外，当输入缓冲区满后，强行中止第一个进程，进入第二个进程。

4 串与字符串替换程序实践

4.1 实验目标

- 1、掌握串的操作和应用。
- 2、使用 C++ 和串实现“字符串的替换程序”专题编程。

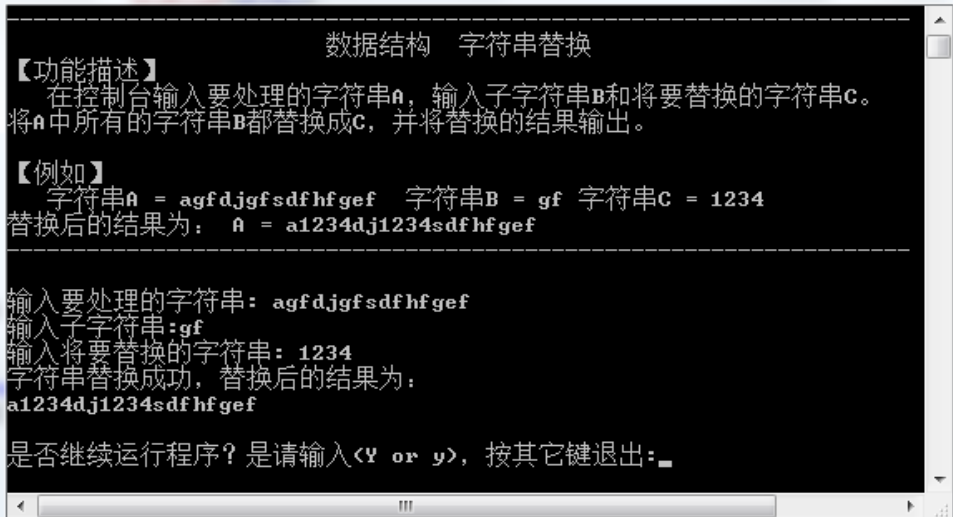
4.2 实验任务

1、功能描述

在控制台输入要处理字符串 A，输入子字符串 B 和将要替换的字符串 C。将 A 中所有的字符串 B 都替换成 C，并将替换的结果输出。

2、示例

例如：主串为 A = agfdjgfsdfhfgef，子串为 B = gf，要替换的串为 C = 1234。
成功替换后的输出结果为：a1234dj1234sdfhfgef



```
数据结构 字符串替换
【功能描述】
在控制台输入要处理的字符串A，输入子字符串B和将要替换的字符串C。
将A中所有的字符串B都替换成C，并将替换的结果输出。

【例如】
字符串A = agfdjgfsdfhfgef 字符串B = gf 字符串C = 1234
替换后的结果为： A = a1234dj1234sdfhfgef

-----
输入要处理的字符串： agfdjgfsdfhfgef
输入子字符串：gf
输入将要替换的字符串： 1234
字符串替换成功，替换后的结果为：
a1234dj1234sdfhfgef

是否继续运行程序？是请输入<y or y>，按其它键退出：_
```

图 4-1 字符替换

若子串不在主串当中，则输出：在主串中不包含子串，无需替换。

```
【例如】
字符串A = agfdjgfsdfhfgef 字符串B = gf 字符串C = 1234
替换后的结果为: A = a1234dj1234sdfhfgef

-----

输入要处理的字符串: agfdjgfsdfhfgef
输入子字符串: gf
输入将要替换的字符串: 1234
字符串替换成功, 替换后的结果为:
a1234dj1234sdfhfgef

是否继续运行程序? 是请输入(Y or y), 按其它键退出: y

输入要处理的字符串: sdjksadf kjs
输入子字符串: asf ds1
输入将要替换的字符串: asdf
在主串中不包含子串, 无需替换

是否继续运行程序? 是请输入(Y or y), 按其它键退出:
```

图 4-2 无需替换

4.3 分析和设计

串的实现方式可以使用顺序存储或链式存储, 这里使用顺序存储来实现。

程序中将创建 strReplaceMain.cpp 文件, 定义 int main(void)函数, 作为程序运行的入口。strReplaceMain.cpp 文件中还会定义 prompt()、input()函数, 分别实现“输出提示信息”、“初始化串”这两个功能。

创建 2 个文件 SString.h 与 SString.cpp, 其中 SString.h 用于定义数据结构、声明函数, SString.cpp 用于实现具体的函数功能。

4.3.1 数据结构

1、串类型的定义

字符串(简称为串), 可以将它看作是一种特殊的线性表, 其特殊性在于线性表的数据元素的类型总是字符型, 字符串的数据对象约束为字符集。串是由零个或多个字符组成的有限序列。一般记作

$S = \langle S_1 S_2 \dots S_n \rangle$

其中, S 是串名, 用双引号括起来的字符序列“ $S_1 S_2 \dots S_n$ ”是串的值, $S_i (1 \leq i \leq n)$ 称为串的元素, 可以是字母、数字或其他字符, 是构成串的基本单位, 字符个数 $n (n \geq 0)$ 称为串的长度。串中任意个连续的字符组成的子序列称为该串的子串。零个字符的串称为空串。通常称字符在序列中的序号为该字符在串中的位置。子串在主串中的位置则以子串的第一个字符在主串中的位置来表示。

2、串的表达

这里使用顺序存储的方式来实现串。类似于线性表的顺序存储结构, 用一组地址连续的存储单元存储串值的字符序列。在串的顺序存储结构中, 按照预定义的大小, 为每个定义的串变量分配一个固定长度的存储区, 可使用数组来描述, 串的数据结构定义如下:

```
#define MAXSTRLEN 255
```

```
// 定义串的存储结构: 顺序存储
typedef struct SSTRING
{
    char str[MAXSTRLEN]; // 串
    int nLen;             // 串的长度
}SString;
```

3、串的基本操作

```
/****** 定义串的基本操作 begin *****/
bool StrAssign(SString& T, char* ch); //串的赋值, 生成一个其值等于ch的串T
int StrLength(const SString& S); // 求串的长度
void StrPrint(const SString& T); // 输出串
int StrCompare(const SString& S, const SString& T); // 字符串比较
bool SubString(SString& sub, const SString& S, const int& nPos, const int& nLen); // 求子串
bool Concat(SString& T, const SString& S1, const SString& S2); // 字符串联接
// 从串S中删除第nPos个字符起长度为nLen的子串
bool StrDelete(SString& S, const int& nPos, const int& nLen);
// 在串S的第nPos个字符前插入串T
bool StrInsert(SString& S, const int& nPos, const SString& T);
// 求子串在主串中第一次出现的位置
int Index(const SString& S, const SString& T, const int& nPos);
// 字符串替换
bool StrReplace(SString& dest, const SString& src, const SString& rep);
/****** 定义串的基本操作 end *****/
```

4.3.2 业务流程

在字符串替换的程序中, 先将数据结构中串的常用操作封装到一个单独的.h 和.cpp 文件当中 (SString.h 和 SString.cpp), 然后在主函数中做以下四步主要操作:

- (1) 输出提示信息 prompt(): 包括本程序功能的简介、一个简单的测试用例;
- (2) 输入三个串 input(): 主串、子串、要替换的串;
- (3) 调用字符串替换处理函数 StrReplace();
- (4) 输出结果。

流程图如下:

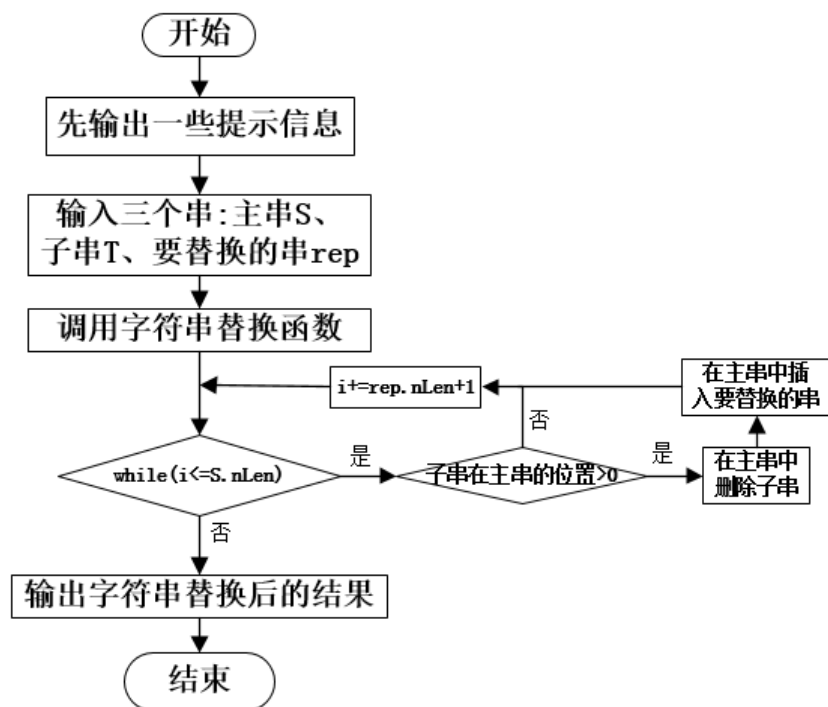


图 4-3 业务流程图

4.3.3 定位子串在主串的位置

可以通过求串长、求子串和串的比较等操作来实现定位子串在主串中的位置 $\text{Index}(S, T, \text{nPos})$ 。

算法思想：在主串 S 中取从第 i (i 的初值为 nPos) 个字符起、长度和串 T 相等的子串和串 T 比较，若相等，则求得函数值为 i ；否则 i 自增 1 直至串 S 中不存在和串 T 相等的子串为止。

4.3.4 字符串的替换

可以通过求串长、定位子串在主串中的位置、串的删除和串的插入等操作来实现字符串替换函数 $\text{StrReplace}(S, T, \text{rep})$ 。

算法思想：从主串 S 中第 i (i 初值为 1, $i \leq \text{StrLength}(S)$) 个字符开始遍历，每次自增(要替换的串 rep 的长度 + 1)，先获取子串 T 在主串 S 中的位置 j ，若 $j > 0$ (表示找到了)，则先主串 S 中从第 j 个位置开始，删除 $\text{StrLength}(T)$ 个字符，接着在主串的第 j 个位置开始插入要替换的串 rep ；若 $j \leq 0$ ，则表示主串 S 中不包含子串 T 。这里可以通过一个 bool 变量来标记字符串是否替换成功。

4.4 编码实现

4.4.1 创建工程

启动 Microsoft Visual Studio 2010，选择“文件 -> 新建 -> 项目”，新建“Win32 控制台应用程序”，工程名“StrReplaceCPro”。添加串的操作文件 SString.h 和 SString.cpp 以及主程序 strReplaceMain.cpp，添加完后的解决方案结构如下：

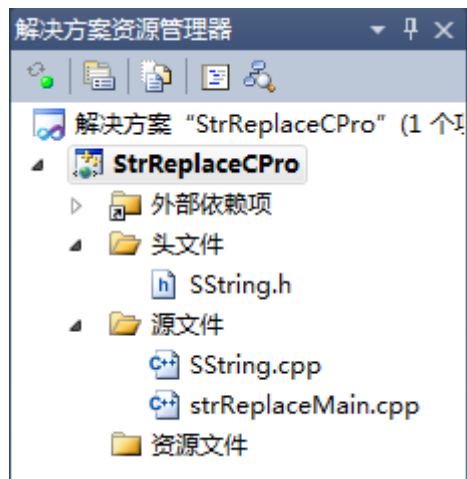


图 4-4 创建工程

4.4.2 定义串的存储结构

在 SString.h 文件中定义串的存储结构，这里使用顺序存储，借助数组来实现。串的顺序存储的数据结构的定义如下：

```
#define MAXSTRLEN 255
// 定义串的存储结构: 顺序存储
typedef struct SSTRING
{
    char str[MAXSTRLEN]; // 串
    int  nLen;           // 串的长度
}SString;
```

4.4.3 串的基本操作

在头文件 SString.h 中声明串的基本操作和在 SString.cpp 源文件中来实现。串的基本操作有：(1) 串的赋值、(2) 求串的长度、(3) 串的输出、(4) 串的比较、(5) 求子串、(6) 串的删除、(7) 串的插入。

1、串的赋值

```

bool StrAssign(SString& T, char* ch)
{
    if(strlen(ch) > MAXSTRLEN)
    {
        return false;
    }
    memset(&T, 0, sizeof(T)); // 清零
    T.nLen = strlen(ch); // 串的长度
    for(int i = 0; i < StrLength(T); i++)
    {
        T.str[i] = ch[i];
    }
    return true;
}

```

2、求串的长度

```

int StrLength(const SString& S)
{
    return S.nLen;
}

```

3、串的输出

```

void StrPrint(const SString& T)
{
    for(int i = 0; i < StrLength(T); i++)
    {
        cout<<T.str[i];
    }
    cout<<endl;
}

```

4、串的比较

```

int StrCompare(const SString& S, const SString& T)
{
    int nSlen = StrLength(S);
    int nTlen = StrLength(T);
    if(nSlen <= 0 || nTlen <= 0)
    {
        return -1;
    }
    for(int i = 0; i < nSlen && i < nTlen; i++)
    {
        if(S.str[i] != T.str[i])
        {

```

```

        return S.str[i] - T.str[i];
    }
}
return nSlen - nTlen;
}

```

5、求子串

```

bool SubString(SString& sub, const SString& S, const int& nPos, const int& nLen)
{
    if(nPos < 1 || nPos > StrLength(S) || nLen < 0 || nLen > StrLength(S) - nPos + 1)
    {
        return false;
    }
    for(int i = 0; i < nLen; i++)
    {
        sub.str[i] = S.str[nPos + i - 1];
    }
    sub.nLen = nLen;
    return true;
}

```

6、串的删除

```

bool StrDelete(SString& S, const int& nPos, const int& nLen)
{
    if(nPos >= 1 && nPos <= StrLength(S) - nLen + 1)
    {
        for(int i = 0; i < nLen; i++)
        {
            for(int j = nPos; j <= S.nLen; j++)
            {
                S.str[j - 1] = S.str[j];
            }
        }
        S.nLen -= nLen;
        return true;
    }
    return false;
}

```

7、串的插入

```

bool StrInsert(SString& S, const int& nPos, const SString& T)
{
    int nSlen = StrLength(S);
    int nTlen = StrLength(T);
    // 这里要确保插入后新的串的长度不超过最大值
    if( (nPos >= 1 && nPos <= nSlen + 1) && (nSlen + nTlen <= MAXSTRLEN) )
    {
        for(int i = nSlen; i >= nPos; i--)
        { // 从nPos位置开始，串S后的字符串依次后移nTlen个位置
            S.str[i + nTlen - 1] = S.str[i - 1];
        }
        // 插入串T
        for(int j = nPos; j < nPos + nTlen; j++)
        {
            S.str[j - 1] = T.str[j - nPos];
        }
        S.nLen += nTlen; // 更新插入后新串S的长度
        return true;
    }

    return false;
}

```

8、串的定位

```

int Index(const SString& S, const SString& T, const int& nPos)
{
    if(nPos >= 1 && nPos <= StrLength(S))
    {
        SString sub;
        memset(&sub, 0, sizeof(sub));
        int i = nPos;
        while(i <= S.nLen - T.nLen + 1)
        {
            SubString(sub, S, i, T.nLen);
            if(0 != StrCompare(sub, T))
            { ++i; }
            else
            { return i; }
        }
    }
    return 0;
}

```

9、串的替换

```

bool StrReplace(SString& S, const SString& T, const SString& rep)
{
    bool bFlag = false; // 标记字符串是否替换成功
    int i = 1;
    while(i <= S.nLen)
    {
        int j = Index(S, T, i);
        if(j > 0) // 找到子串
        {
            StrDelete(S, j, T.nLen);
            StrInsert(S, j, rep);
            bFlag = true;
        }
        i += rep.nLen + 1;
    }
    return bFlag;
}

```

4.4.4 主函数

在 strReplaceMain.cpp 文件中，封装了两个函数：(1) 打印输出一些提示信息、(2) 串的输入。然后在 mian 函数中主要分四步来处理：(1) 打印输出一些提示信息、(2) 串的输入(主串、子串和要替换的串)、(3) 字符串替换函数、(4) 输出处理后的结果。

1、提示输出函数

```

void prompt() // 输出提示信息
{
    cout<<"-----\n";
    cout<<"\t\t\t数据结构 字符串替换\n";
    cout<<"【功能描述】\n";
    cout<<" 在控制台输入要处理的字符串A，输入子字符串B和将要替换的字符串C。 \n";
    cout<<"将A中所有的字符串B都替换成C，并将替换的结果输出。 \n";
    cout<<"\n【例如】\n 字符串A = agfdjgfsdfhfgef 字符串B = gf 字符串C = 1234\n";
    cout<<"替换后的结果为： A = a1234dj1234sdfhfgef"<<endl;
    cout<<"-----\n";
}

```

2、串的输入

```

void input(SString& S) // 串的输入
{
    char szStrSrc[1024] = {0}; // 输入字符串缓冲区
    bool bFlag = false;
    while(! bFlag)
    {
        fflush(stdin);
        gets(szStrSrc); // 获取输入的字符串
        if(strlen(szStrSrc) > MAXSTRLEN)
        { ..... // 输出出错信息并设置bFlag = false; }
        else
        {
            if((bFlag = StrAssign(S, szStrSrc)) == false)
            { cout<<"input error! try again: "; }
        }
        memset(&szStrSrc, 0, sizeof(szStrSrc)); // 清零
    }
}

```

3、main 函数

```

int main()
{
    char szFlag; // 是否继续运行程序，Y或y 表示继续运行，否则结束程序
    prompt();
    do
    {
        SString strSrc, strSub, strReplace;
        ..... // 调用input()函数来初始化串

        if(StrReplace(strSrc, strSub, strReplace))
        {
            cout<<"字符串替换成功，替换后的结果为: "<<endl;
            StrPrint(strSrc);
        }
        else
        { cout<<"在主串中不包含子串，无需替换"<<endl; }
        cout<<"\n是否继续运行程序？是请输入(Y or y)，按其它键退出:";
        szFlag = getchar();
    }while(szFlag == 'Y' || szFlag == 'y');
    return 0;
}

```

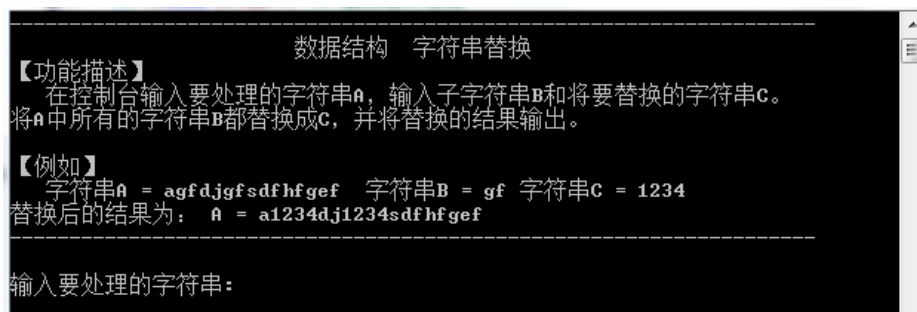
4.4.5 扩展优化

可以对本程序中使用的串的定位函数 Index()进行优化,优化的常见算法有 KMP 算法。

4.5 调试和运行

- (1) 按 F7, 生成解决方案。
- (2) 按 Ctrl+F5, 编译和运行程序。

先会输出一些提示信息, 然后提示用户输入要处理的字符串:



```
数据结构 字符串替换

【功能描述】
在控制台输入要处理的字符串A, 输入子字符串B和将要替换的字符串C。
将A中所有的字符串B都替换成C, 并将替换的结果输出。

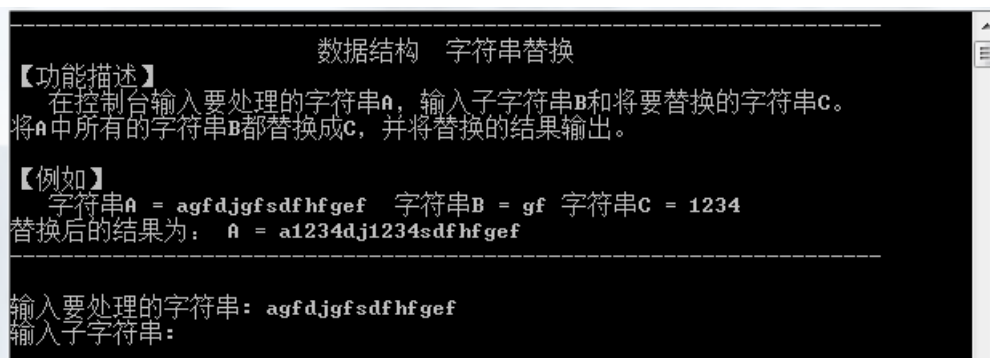
【例如】
字符串A = agfdjgfsdfhfgef 字符串B = gf 字符串C = 1234
替换后的结果为: A = a1234dj1234sdfhfgef

-----

输入要处理的字符串:
```

图 4-5 输入提示信息

- (3) 输入要处理的字符串 agfdjgfsdfhfgef, 然后按<Enter>, 提示输入子串:



```
数据结构 字符串替换

【功能描述】
在控制台输入要处理的字符串A, 输入子字符串B和将要替换的字符串C。
将A中所有的字符串B都替换成C, 并将替换的结果输出。

【例如】
字符串A = agfdjgfsdfhfgef 字符串B = gf 字符串C = 1234
替换后的结果为: A = a1234dj1234sdfhfgef

-----

输入要处理的字符串: agfdjgfsdfhfgef
输入子字符串:
```

图 4-6 提示输入字符串

- (4) 输入子字符串 gf, 然后按<Enter>, 提示输入要替换的字符串:



```
数据结构 字符串替换

【功能描述】
在控制台输入要处理的字符串A, 输入子字符串B和将要替换的字符串C。
将A中所有的字符串B都替换成C, 并将替换的结果输出。

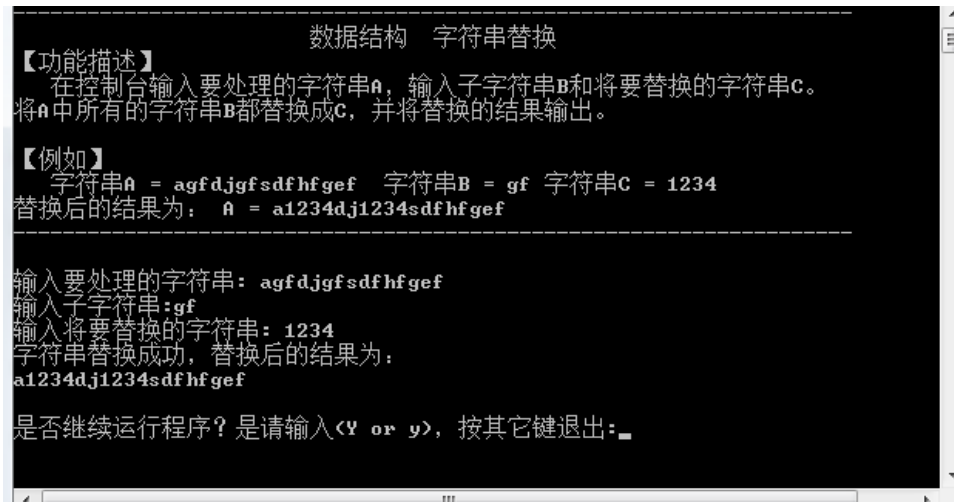
【例如】
字符串A = agfdjgfsdfhfgef 字符串B = gf 字符串C = 1234
替换后的结果为: A = a1234dj1234sdfhfgef

-----

输入要处理的字符串: agfdjgfsdfhfgef
输入子字符串: gf
输入将要替换的字符串: _
```

图 4-7 提示输入替换掉字符串

(5) 输入要替换成的字符串 1234，然后按<Enter>，输出字符串替换处理后的结果：



```
数据结构 字符串替换

【功能描述】
在控制台输入要处理的字符串A，输入子字符串B和将要替换的字符串C。
将A中所有的字符串B都替换成C，并将替换的结果输出。

【例如】
字符串A = agfdjgfsdfhfgef 字符串B = gf 字符串C = 1234
替换后的结果为： A = a1234dj1234sdfhfgef

-----

输入要处理的字符串： agfdjgfsdfhfgef
输入子字符串：gf
输入将要替换的字符串： 1234
字符串替换成功，替换后的结果为：
a1234dj1234sdfhfgef

是否继续运行程序？是请输入(Y or y)，按其它键退出：_
```

图 4-8 输出字符串替换后的结果

如果想继续运行程序，可输入 Y 或 y；如果想退出，可输入其它的任意键。

习题

1. 行编辑程序

【问题描述】设计一个简单的行编辑程序。程序包括 5 个基本命令：E（行编辑），W（写入文件），#（退格），@（清除整行）和 Q（存盘退出）。程序运行后提示用户输入命令：当用户输入“E”命令，开始编辑一行文本，程序为用户申请一个临时存放空间（栈），存储用户当前输入字符；当用户输入一个退格命令“#”，以表示当前栈顶字符无效；如果错误较多，可以键入退行符“@”，清除栈中整行；当行输入完毕后，输入“W”命令，则将该行写入文件中。文件编辑结束后，输入“Q”命令，存盘退出。

2. 文学研究助手与模式匹配算法 KMP

【问题描述】文学研究人员需要统计某篇英文小说中某些形容词的出现次数和位置。试写一个实现这一目标的文字统计系统。

3. 文本文件单词的检索与计数

【问题描述】要求编程建立一个文本文件，每个单词不包含空格且不跨行，单词由字符序列构成且区分大小写；统计给定单词在文本文件中出现的总次数；检索输出某个单词出现在文本中的行号、在该行中出现的次数以及位置。

4. 串基本操作演示系统

【问题描述】如果计算机语言没有把串作为一个预先定义好的基本类型对待，又需要用该语言写一个涉及串操作的软件系统时，用户必须自己实现串类型。试着实现串类型，并写一个串的基本操作演示系统。

5. 关键词提取程序

【问题描述】设计一个简单的英文关键词提取程序，可实现对一段英文短文中出现的频率最高的三个到五个词或短语进行提取。要求：（1）从文件中读取一篇英文短文（300 词以内），并显示在屏幕上。（2）按照出现频率顺序显示三个到五个词语，并注明出现的次数。

6. 小型文字编辑软件

【问题描述】设计一小型文字编辑软件，系统主要功能如下：1. 从键盘输入文字，以文本文件形式存储；2. 统计该文本文件中英文字母数、空格数及整篇文章总字数；3. 统计某一字符串在文章中出现的次数，并输出该次数；4. 删除某一子串，并将后面的字符前移。

5 二叉树与赫夫曼图片压缩实践

5.1 实验目标

- 1、掌握树的存储结构。
- 2、掌握二叉树的三种遍历方法。
- 3、掌握 Huffman 树、Huffman 编码等知识和应用。
- 4、使用 C++、文件操作和 Huffman 算法实现“图片压缩程序”专题编程。

5.2 实验任务

压缩软件是利用特定算法来压缩数据的工具，压缩后生成的文件称为压缩包(archive)。如果想使用其中的数据，就得用压缩软件对数据进行解压。利用压缩软件对文件中重复的数据进行压缩，可以减小文件中的字节总数，使文件能够通过互联网连接实现更快传输，此外还可以减少文件的磁盘占用空间。常用的压缩软件有 rar、zip 等。

压缩可以分为无损压缩与有损压缩两种。无损压缩后的文件，经过解压能够完全恢复原始数据；有损压缩的文件则无法完全恢复。rar、zip 等格式都是无损压缩格式。音乐文件格式 mp3、图片文件格式 jpg 都是有损压缩格式。

计算机文件是由一个个字节组成的，1 个字节有 0~255 共 256 种可能的取值，每个字节的编码长度都是 8 位。由于文件中的字节总是会重复出现，可以对不同的字节设计长度不等的编码，让出现次数较多的字节，采用尽可能短的编码，那么文件编码的总长便可减少。

统计文件中 256 种不同的字节重复的次数，以每种字节重复的次数作为权值(weight)，构造一棵有 256 个叶子节点的二叉树。若带权路径长度达到最小，称这样的二叉树为最优二叉树，即 Huffman 树(Huffman tree)。

Huffman 树从根到每个叶子都有一条路径。对路径上的各分支，约定指向左子树根的分支编码为“0”，指向右子树根的分支编码为“1”。从根到每个叶子相应路径上的“0”和“1”组成的序列，就是这个叶子节点的编码，称为 Huffman 编码。

功能要求：使用 Huffman 压缩算法，对一幅 BMP 格式的图片文件进行压缩。图片文件名为“Pic.bmp”，内容如下图所示。压缩后保存为“Pic.bmp.huf”文件。

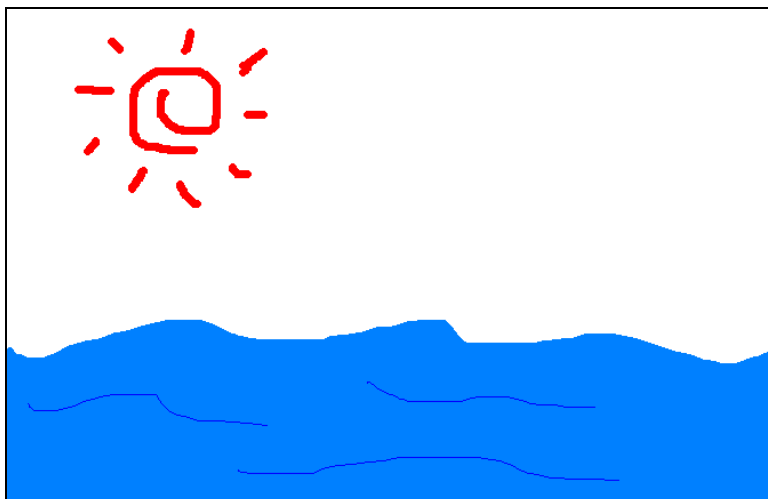


图 5-1 压缩示例图片

使用 VS2010 作为开发工具，开发一个控制台程序，使用 Huffman 压缩算法对图片文件 “Pic.bmp” 进行压缩。具体要求如下：

1、读取原文件，统计权值

运行程序，输入文件名。

以 “Pic.bmp” 文件为例。若文件存放在 F 盘根目录下，输入文件完整路径 “F:\Pic.bmp”。按回车结束。



图 5-2 输入文件名

以字节流的方式，只读打开 “Pic.bmp” 文件。

逐字节读取文件，统计文件中 256 种字节重复的次数，保存到一个数组中 `int weight[256]` 中。

```

C:\ F:\HfmCompressCPro.exe
===== Huffman文件压缩 =====
请输入文件名: F:\Pic.bmp
原文件每个字符的权值为:
Byte Weight
0x00 108393
0x01 2
0x02 1
0x03 0
0x04 0
0x05 0
.....
0xFB 0
0xFC 0
0xFD 0
0xFE 0
0xFF 677672

```

图 5-3 读取文件

2、生成 Huffman 树

根据（1）中统计的结果，构建 Huffman 树。定义一个结构体来记录每个节点的权值、父节点、左孩子和右孩子。使用结构体数组来存储这个 Huffman 树。

```

C:\ F:\HfmCompressCPro.exe
===== Huffman文件压缩 =====
请输入文件名: F:\Pic.bmp
哈夫曼树的每个节点信息为:
Byte Weight Parent Lchild Rchild
pHT[1] 108393 510 0 0
pHT[2] 2 502 0 0
pHT[3] 1 495 0 0
pHT[4] 0 257 0 0
pHT[5] 0 257 0 0
pHT[6] 0 258 0 0
.....
pHT[508] 16 509 506 507
pHT[509] 98197 510 508 129
pHT[510] 206590 511 509 1
pHT[511] 884262 0 510 256

```

图 5-4 生成 Huffman 树

3、生成 Huffman 编码

遍历（2）中生成的 Huffman 树，记录 256 个叶子节点的 Huffman 编码，保存在字符串数组中。

```
C:\ F:\HfmCompressCPro.exe
===== Huffman文件压缩 =====
请输入文件名: F:\Pic.bmp
先序遍历哈夫曼树输出编码信息:
Byte HufCode
0x01 000000
0x0D 000001
0x18 0000100
0x26 0000101
0x28 0000110
.....
0xEC 0001111011111111
0x02 00011111
0x80 001
0x00 01
0xFF 1
```

图 5-5 生成 Huffman 码

4、压缩原文件

使用 Huffman 编码对原文件中的字节重新编码，获得压缩后的文件数据。

```
C:\ F:\HfmCompressCPro.exe
===== Huffman文件压缩 =====
请输入文件名: F:\Pic.bmp
压缩后的文件编码:
0001000000100100001010001011000001010101010000111010100001100101000011000001000000
11111010100011010000000101000000010000100010101010100011100001010000001010101010
1010101010101010101010101001011001011001011001011001011001011001011001011001011001011
00101100101100101100101100101100101100101100101100101100101100101100101100101100101100
101100101100101100101100101100101100101100101100101100101100101100101100101100101100101
11001011001011001011001011001011001011001011001011001011001011001011001011001011001011
00101100101100101100101100101100101100101100101100101100101100101100101100101100101100
10110010110010110010110010110010110010110010110010110010110010110010110010110010110010
1100101100101100101100101100101100101100101100101100101100101100101100101100101100101100
00101100101100101100101100101100101100101100101100101100101100101100101100101100101100
10110010110010110010110010110010110010110010110010110010110010110010110010110010110010
11001011001011001011001011001011001011001011001011001011001011001011001011001011001011
00101100101100101100101100101100101100101100101100101100101100101100101100101100101100
10110010110010110010110010110010110010110010110010110010110010110010110010110010110010
11001011001011001011001011001011001011001011001011001011001011001011001011001011001011
```

图 5-6 压缩文件

5、保存压缩文件

将编码过的数据，保存到文件“Pic.bmp.huf”中。

```
C:\ F:\HfmCompressCPro.exe
===== Huffman文件压缩 =====
请输入文件名: F:\Pic.bmp
884262 字节
生成压缩文件: F:\Pic.bmp.huf
149682 字节
压缩比率: 16.9273%
```

图 5-7 保存压缩文件

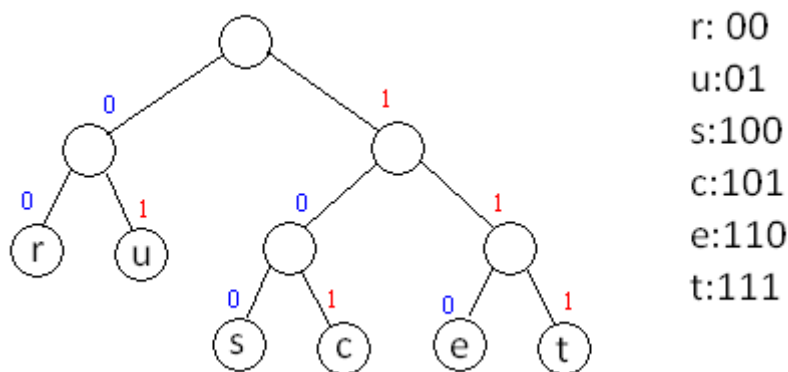


图 5-8 单词 “structure” 生成的 Huffman 树

使用 Huffman 编码对原文件中的字节重新编码。重复次数较多的字节，Huffman 编码的长度较短。原文件中每个字节的编码长度都是 8 位，而重复次数较多的字节的 Huffman 编码，长度比 8 位短得多，因此可实现压缩。

5.3 分析和设计

通过需求可知，本程序首先需要通过读取文件，生成一棵带权二叉树。树在程序中可使用链式结构、顺序结构两种方式来表示。已知这棵树的叶子节点数为 256，其存储空间是固定的，因此，本程序将使用顺序结构来表示这棵二叉树。

1、编码构思

(1) 定义存储结构

定义一个结构体 HTNode 来表示二叉树的叶子节点，记录每个节点的权值、父节点、左孩子和右孩子。创建结构体数组 HTNode aHuffmanTree[512]来存储这棵带权二叉树。

(2) 遍历算法

其次，生成 Huffman 树后，要遍历这棵二叉树，根据叶子节点的路径来生成 Huffman 编码表。遍历二叉树有先序遍历、中序遍历、后序遍历三种方式。本程序将使用先序遍历的方式，查找二叉树上所有的叶子节点，生成对应的 Huffman 编码。定义二维字符数组 char[256][]来存储所有叶子节点的 Huffman 编码，供编码时使用。

(3) 压缩编码

由于 Huffman 编码表是以字符数组的形式保存的，重新编码后的数据将是一个很长的字符串。定义 Str2byte 函数，将形如 “01010101” 字符串转换成字节，才能得到最终的编码。将其保存到 “*.huf” 中，即实现了文件压缩。

(4) 解压缩

最后，为了保证压缩后的数据能够被正确解压，除了保存压缩的数据之外，还应保存原文件的长度及 256 种字节的重复次数。定义一个文件头，用于保存这些信息。在保存压缩文件时，同时向文件中写入文件头和压缩数据，以保证文件能够被还原。

2、程序设计

(1) 创建 Main.cpp 文件，定义 int main(void)函数，作为程序的入口函数。

(2) 创建 Huffman.h 文件与 Huffman.cpp 文件，实现 Huffman 算法相关的函数。

- 1) 定义 HTNode 结构体。
- 2) 定义 HuffmanTree 函数，用于生成 Huffman 树。
- 3) 定义 HuffmanCode 函数，用于生成 Huffman 编码。

(3) 创建 Compress.h 文件与 Compress.cpp 文件，实现文件的压缩算法。Compress.cpp 文件中将包含 Huffman.h 文件，使用其中的 Huffman 算法相关的函数。

- 1) 定义 Compress 函数，用于实现文件压缩。
- 2) 定义 InitHead 函数，读取原文件，初始化文件头数据信息。
- 3) 定义 Encode 函数，利用 Huffman 编码实现文件压缩。
- 4) 定义 Str2byte 函数，用于将“01010101”形式的字符串字节。
- 5) 定义 WriteFile 函数，用于将压缩后的数据写入新文件。

3、业务流程

使用 Huffman 算法压缩图片，业务流程如下：

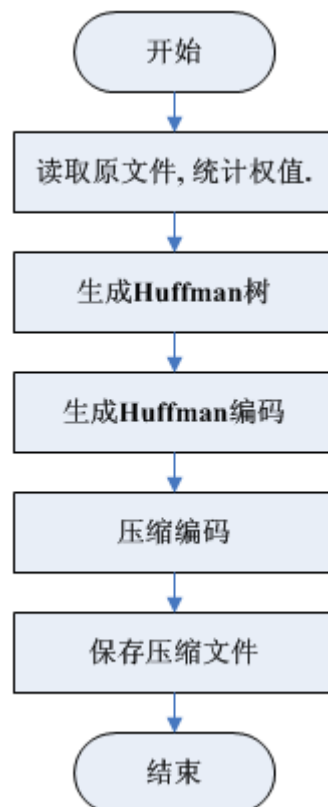


图 5-9 业务流程图

5.3.1 Huffman 树

Huffman 算法是一种经典的无损压缩算法。这是一种通过字符出现的频率和二叉树来进行的一种压缩算法。统计 256 种字节出现的次数，构造一棵有 256 个叶子节点的二叉树，每个叶子节点的权值即为字符出现的次数。其中带权路径长度最小的二叉树称为最优二叉树，即 Huffman 树。

读取 Pic.bmp 文件，生成 Huffman 树如下。

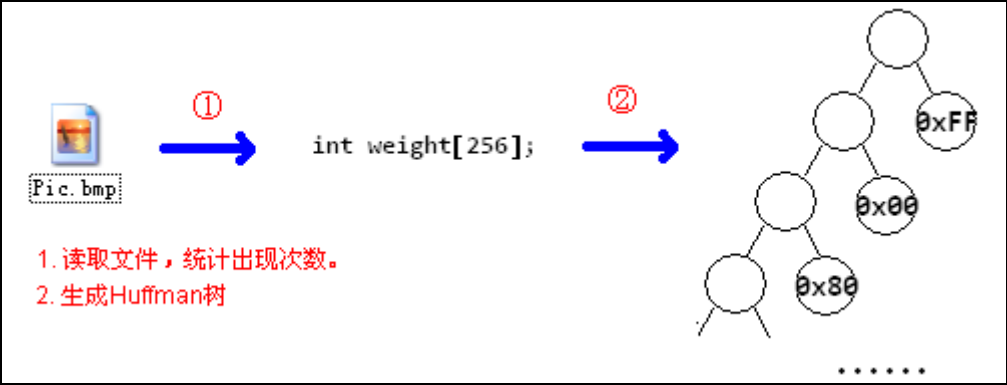


图 5-10 生成 Huffman 树的过程

二叉树的存储结构可分为顺序结构和链式结构。已知 Huffman 树的叶子节点数为 256，其存储空间是固定的，适合使用顺序结构来表示。创建结构体数组来存储 Huffman 树。

```
// Huffman 树节点
struct HTNode
{
    int weight;// 权值
    int parent;// 父节点
    int lchild;// 左孩子
    int rchild;// 右孩子
};

// Huffman 树
typedef HTNode *HuffmanTree;// 动态分配数组存储 Huffman 树
```

文件数据在内存中实际的存储结构，如下图所示：

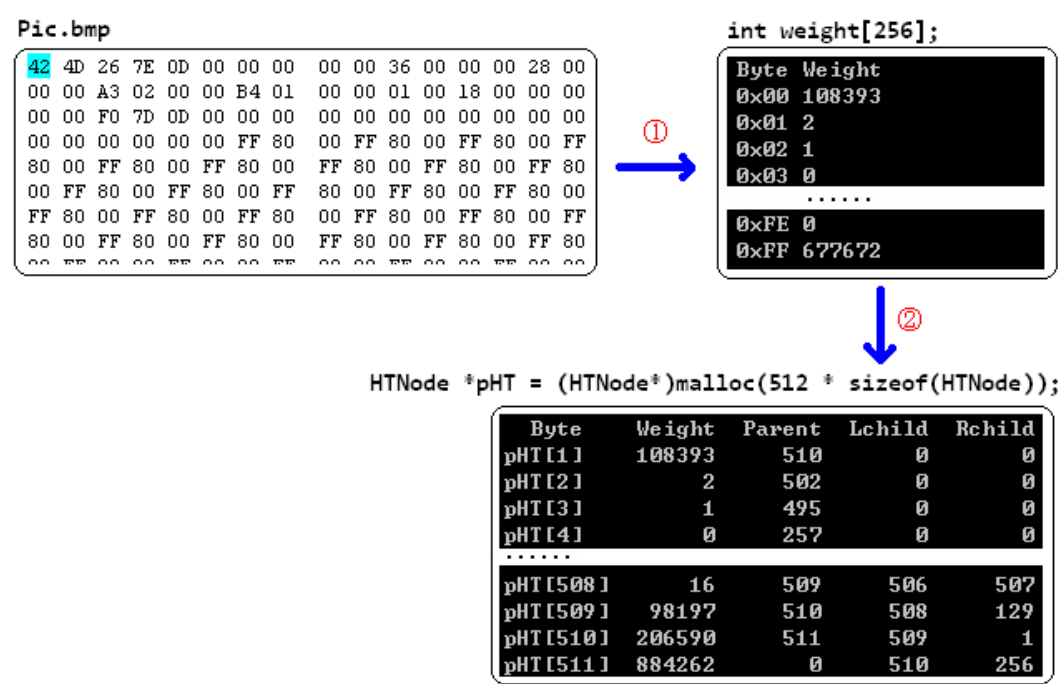


图 5-11 数据在内存里的存储结构

以字符串“structure”为例，其中一共出现过 6 个不同的字符。生成二叉树的节点树为 12 个。使用结构体数组保存这个带权二叉树，初始如下：

	1	2	2	2	1	1
	s	t	r	u	c	e
HT	weight	parent	lchild	rchild		
HT[0]	0	0	0	0		
HT[1]	1	0	0	0		
HT[2]	2	0	0	0		
HT[3]	2	0	0	0		
HT[4]	2	0	0	0		
HT[5]	1	0	0	0		
HT[6]	1	0	0	0		
HT[7]	0	0	0	0		
HT[8]	0	0	0	0		
HT[9]	0	0	0	0		
HT[10]	0	0	0	0		
HT[11]	0	0	0	0		

图 5-12 二叉树的初始化

生成最优二叉树后，存储结果如下：

HT	weight	parent	lchild	rchild	
HT[0]	0	0	0	0	
HT[1]	1	7	0	0	
HT[2]	2	8	0	0	
HT[3]	2	9	0	0	
HT[4]	2	9	0	0	
HT[5]	1	7	0	0	
HT[6]	1	8	0	0	
HT[7]	2	10	1	5	
HT[8]	3	10	6	2	
HT[9]	4	11	3	4	
HT[10]	5	11	7	8	
HT[11]	9	0	9	10	

图 5-13 最优二叉树的存储结果

5.3.2 Huffman 编码

Huffman 树从根到每个叶子都有一条路径，对路径上的各分支约定指向左子树根的分支编码为“0”，指向右子树根的分支编码为“1”，从根到每个叶子相应路径上的“0”和“1”组成的序列就是这个叶子节点的编码。Huffman 编码的特点是：每个字符都有唯一的一个编码，并且任意一个字符的编码都不是另一个字符编码的前缀。

遍历 Huffman 树，记录访问叶子节点的路径，即可生成 Huffman 编码表。遍历由 Pic.bmp 文件生成的 Huffman 树，将生成如下 Huffman 编码。

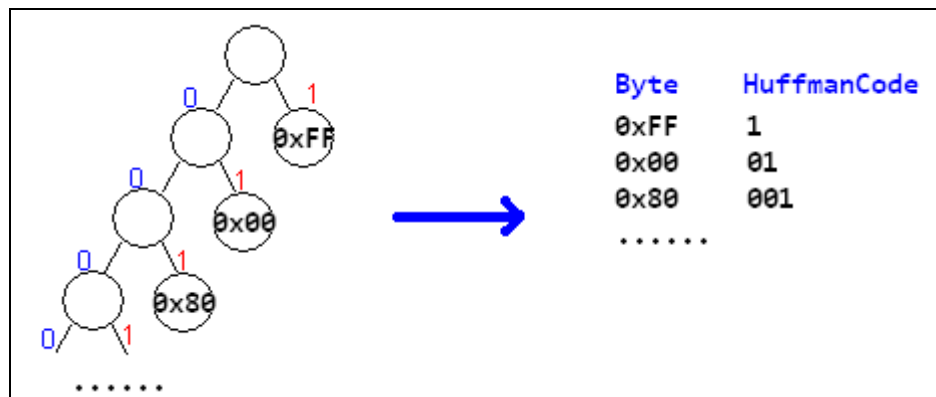


图 5-14 生成 Huffman 的过程

遍历二叉树有先序遍历、中序遍历、后序遍历三种方式。先序遍历 Huffman 树，方法如下：

```
//先序遍历输出权值
void PreorderTraverse(int root, HuffmanTree pHT)
{
    cout<<pHT[root].weight<<" ";// 访问节点
    if(pHT[root].lchild)// 左孩子
    {
        PreorderTraverse(pHT[root].lchild, pHT);
    }
    if(pHT[root].rchild)// 右孩子
    {
        PreorderTraverse(pHT[root].rchild, pHT);
    }
}
```

遍历 Huffman 树时，要有记录访问每个叶子节点的路径，因此需要对先序遍历算法进行一些改进。

- 1、定义一个二维字符串数组 char[256][256]来保存所有叶子节点的编码。
- 2、记录访问路径，左孩子为'0'，右孩子为'1'；
- 3、只记录叶子节点，访问到叶子节点时，保存路径。

改进算法如下：

```

int HuffmanCoding(HuffmanCode &pHC, HuffmanTree &pHT)
{
    // 无栈非递归遍历 Huffman 树，求 Huffman 编码
    char cd[256] = {'\0'}; // 记录访问路径
    int cdlen = 0; // 记录当前路径长度
    for(int i = 1; i < 512; i++)
    {
        pHT[i].weight = 0; // 遍历 Huffman 树时用作节点的状态标志
    }

    int p = 511; // 根节点
    while(p != 0)
    {
        if(pHT[p].weight == 0) // 向左
        {
            pHT[p].weight = 1;
            if(pHT[p].lchild != 0)
            {
                p = pHT[p].lchild;
                cd[cdlen++] = '0';
            }
            else if(pHT[p].rchild == 0) // 登记叶子节点的字符的编码
            {
                pHC[p] = (char*)malloc((cdlen+1) * sizeof(char));
                cd[cdlen] = '\0';
                strcpy(pHC[p], cd); // 复制编码
            }
        }
        else if(pHT[p].weight == 1) // 向右
        {
            pHT[p].weight = 2;
            if(pHT[p].rchild != 0) // 右孩子为叶子节点
            {
                p = pHT[p].rchild;
                cd[cdlen++] = '1';
            }
        }
        else
        {
            // 退回父节点，编码长度减 1
            pHT[p].weight = 0;
            p = pHT[p].parent;
            --cdlen;
        }
    }
}

```

```
return OK;  
}
```

5.3.3 压缩过程

1、统计字符出现的频度

扫描文件的全部数据，统计 256 个字符各自出现的次数，即为带权二叉树所有叶子节点的权值。

以字符串“structure”为例，统计结果如下：

"structure"

字符	s	t	r	u	c	e
次数	1	2	2	2	1	1

图 5-15 统计字符串出现的频率

2、建立 Huffman 树

依据各个字符出现的次数，建立 Huffman 树。

- (1) 每次选择权值最小的 2 棵树作为左右子树构造一个新的二叉树，且置新的二叉树的根节点权值为其左右子树上根节点权值之和。
- (2) 删除原有 2 棵树，将新构造的二叉树加入队列中。
- (3) 重复(1)、(2)，直到只剩一棵二叉树，即为 Huffman 树。

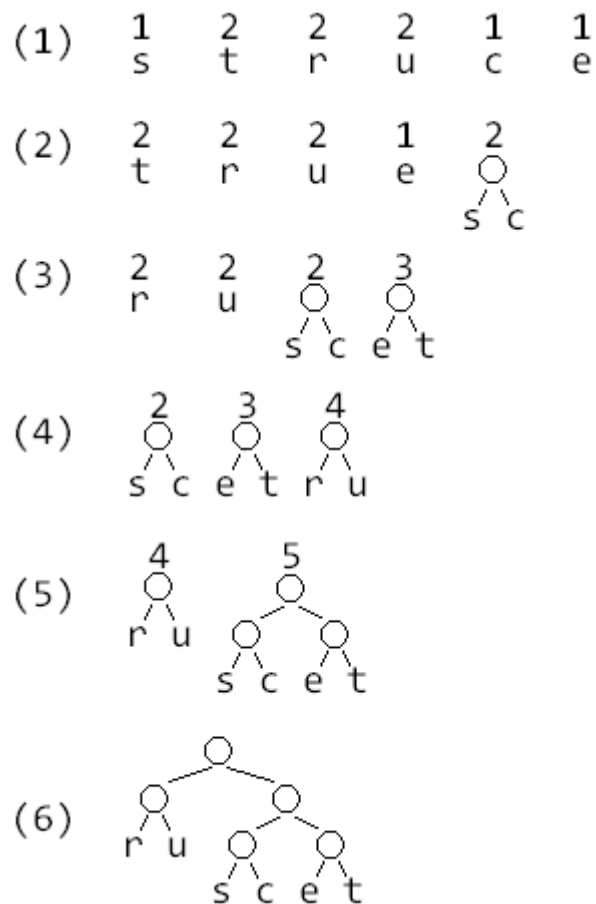


图 5-16 建立 Huffman 树

3、生成 Huffman 编码

Huffman 树从根到每个叶子都有一条路径，对路径上的各分支约定指向左子树根的分支编码为“0”，指向右子树根的分支编码为“1”，从根到每个叶子相应路径上的“0”和“1”组成的序列就是这个叶子节点的编码。

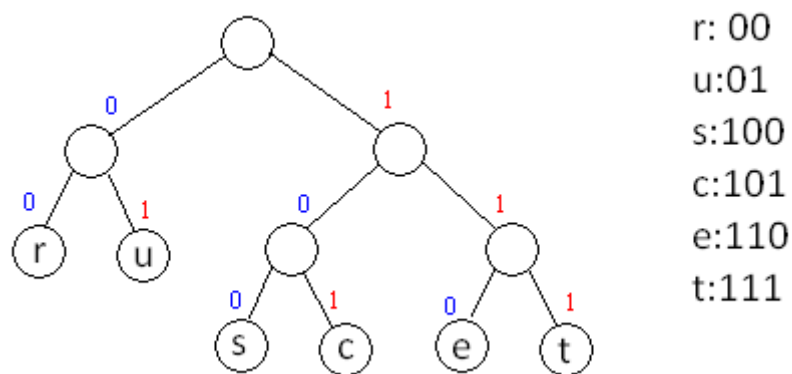


图 5-17 生成 huffman 树

4、压缩编码

根据 Huffman 编码，对原文件逐字节编码，编码结果按 bit 记录。

字符串“structure”的原始 ASCII 编码为：

二进制：01110011 01110100 01110010 01110101 01100011 01110100 01110101
01110010 01100101

进行 Huffman 编码，结果为：

二进制：10011100011011110100110

Huffman 编码的特点是每个字符的编码都不会是另一个字符编码的前缀。由于编码时是没有分隔符的，这种前缀编码就保证了能够被正常解码。

5、保存压缩文件

将编码结果中，每八个 Huffman 编码为一组，将 8 个 bit 转化成 1 个 byte，然后保存到输出文件中。末尾不足 8 个 bit 的补 0。

10011100 01101111 01001100

十六进制：0x9C 0x6F 0x4C

字符串“structure”原本为 9 个字符，压缩后为 3 个字符。可以看到，Huffman 编码的结果相对于原始内容，得到了较大的压缩。

5.3.4 压缩与解压

为了保证压缩后的数据能够被正确还原，编码与解码时必须使用同样的 Huffman 编码表。一般有两种做法：

1、固定编码

编码与解码时，始终使用一套编码表。在编码和解码之前，已经有一份确定的编码表。对于任意数据，都使用相同的方式来编码和解码。

固定编码常用于发送电报。把常用字符的编码缩短，即可实现压缩的目的，编码速度也比较快。这种方式也常用于数据加密。编码前的数据为明文，编码后的数据为密文，只有拥有编码表的人能够解读密文。

2、动态编码

在保存压缩数据的同时，向文件中写入原始文件的信息，让解压程序能够根据这些信息重新生成 Huffman 编码，并以此为依据进行解码。这种做法会增加数据的总长度，但是能提高压缩的效率。

可将原文件各字节出现次数保存在一个数组中，并将其与压缩数据一同保存。解压缩时，先根据这个数组重构 Huffman 编码，就可以正确解压了。

可定义一个文件头，保存原文件基本信息。结构体如下：

```
// 文件头
struct HEAD
{
    char type[4];      // 文件类型
    int length;        // 原文件长度
    int weight[256];   // 权值数值
};
```

5.3.5 文件操作

本程序将使用字节流的方式来读、写 Pic.bmp 文件。在程序中包含 `stdlib.h` 头文件，即可使用 `fopen`、`fclose` 等函数。

1、读文件步骤

- (1) 打开文件: `File *in = fopen(filename, "rb");`
- (2) 读取文件: `char b = fgetc(in)`
- (3) 关闭文件: `fclose(in);`

2、写文件步骤

- (1) 打开文件: `File *out = fopen(filename, "wb");`
- (2) 写文件: `fwrite(data, size, count, out);`
- (3) 关闭文件: `fclose(out);`

5.4 编码实现

使用 Huffman 算法实现图片压缩程序，可分为 6 个步骤。

1、创建工程

创建 HuffmanCompressCPro 工程，定义入口函数 `int main();`

2、读取原文件

读取文件，统计 256 种字节重复的次数；

3、生成 Huffman 树

根据上一步统计的结果，构建 Huffman 树；

4、生成 Huffman 编码

遍历 Huffman 树，记录 256 个叶子节点的路径，生成 Huffman 编码；

5、压缩编码

使用 Huffman 编码，对原文件中的字节重新编码，获得压缩后的文件数据；

6、保存文件

将编码过的数据，保存到文件“Pic.bmp.huf”中。

5.4.1 创建工程

启动 Microsoft Visual Studio 2010，创建一个空的解决方案，解决方案名为“Huffman SLN”。在“HuffmanSLN”解决方案下面新建一个空的 Win32 控制台工程，工程名为“HfmCompressCPro”。

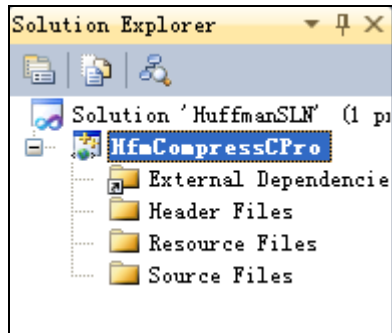


图 5-18 创建工程

在工程中新建“Main.cpp”文件，创建 `int main()` 函数作为程序运行的入口函数。

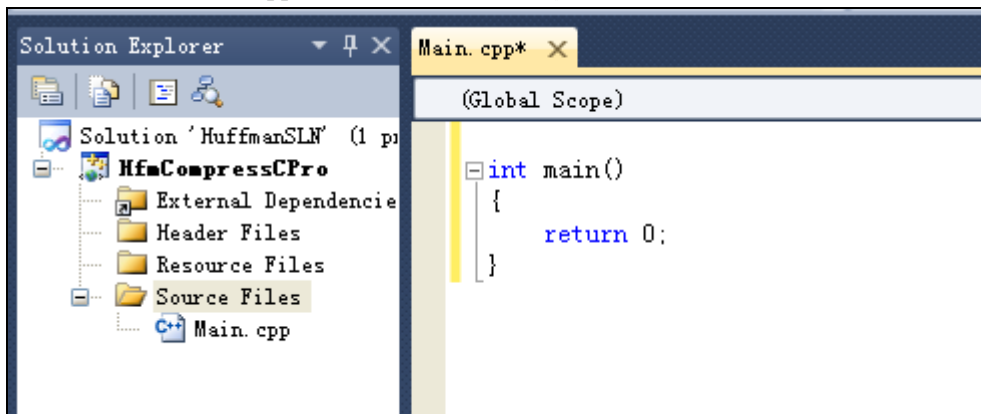


图 5-19 创建入口函数

在“Main.cpp”文件中导入 `<iostream>` 头文件，声明 `using namespace std`。在 `main()` 函数中使用 `cout` 输出界面提示信息，同时用 `cin` 接受用户从键盘输入的信息。

```
#include <iostream>
using namespace std;
int main()
{
    cout<<"==== Huffman 文件压缩 =====<<endl;
    cout<<"请输入文件名: ";
    char filename[256];
    cin>>filename;
    return 0;
}
```

运行结果:

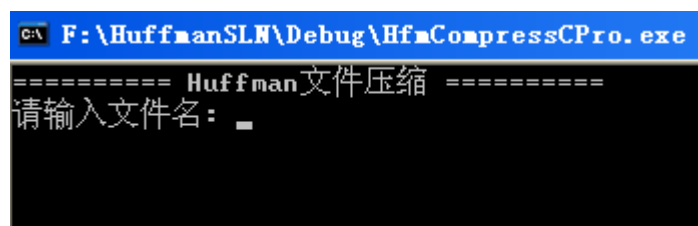


图 5-20 运行结果

5.4.2 读取原文件

以二进制流的方式，只读打开文件。逐字节读取文件数据，统计文件中 256 种字节重复的次数，保存到一个数组中 `int weight[256]` 中。

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main()
{
    // ...

    int weight[256] = {0};
    // 以二进制流形式打开文件
    FILE *in = fopen(filename, "rb");
    ...
    // 扫描文件，获得权重
    while((ch = getc(in)) != EOF)
    {
        weight[ch]++;
    }
    // 关闭文件
    fclose(in);
    return 0;
}
```

将扫描的结果在控制台打印出来。

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main()
{
    // ...

    // 显示 256 种字节的出现的次数
    cout<<"Byte "<<"Weight"<<endl;
    for(int i = 0; i < 256; i++)
    {
        printf("0x%02X %d\n",i,weight[i]);
    }

    return 0;
}
```

编译运行。

```
C:\F:\Hf\CompressCPro.exe
===== Huffman 文件压缩 =====
请输入文件名: F:\Pic.bmp
原文件每个字符的权值为:
Byte Weight
0x00 108393
0x01 2
0x02 1
0x03 0
0x04 0
.....
0xFC 0
0xFD 0
0xFE 0
0xFF 677672
```

图 5-21 运行结果

5.4.3 生成 Huffman 树

根据上一步中统计的结果，构建一棵 Huffman 树。定义一个结构体来记录每个节点的权值、父节点、左孩子和右孩子。使用结构体数组来存储这个 Huffman 树。

1、添加文件

右键单击工程，选择“Add -> New Item...”，新建 Huffman.h 和 Huffman.cpp 文件。我们将在 Huffman.h 文件中定义操作 Huffman 树的相关函数，然后在 Huffman.cpp 文件中实现这些函数。

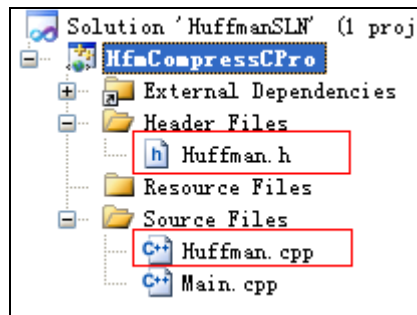


图 5-22 添加文件

2、创建 Huffman 树

在 Huffman.h 文件中，定义一个用于存储 Huffman 树节点信息的结构体，该结构体中包含四个信息，分别是节点的权值、父节点、左孩子和右孩子。

这些信息为 Huffman 树的构造和遍历，以及 Huffman 编码提供了依据。

```
// Huffman 树节点
struct HTNode
{
    int weight;// 权值
    int parent;// 父节点
```

```
int lchild;// 左孩子
int rchild;// 右孩子
};
```

在 Huffman.cpp 文件中，定义 HuffmanTree 函数，用于创建 Huffman 树。
具体的代码参见技术分析。

3、查找最小权值节点

在构建 Huffman 树的过程中，涉及到一个选择最小权值节点的问题，可以在 Huffman.h 文件中，定义一个 Select()函数，用来查找 Huffman 树结点数组中权值最小的节点，并在 Huffman.cpp 文件中去实现这个函数。

```
int Select(HuffmanTree pHT, int nSize)
{
    int minValue = 0x7FFFFFFF;// 最小值
    int min = 0;// 元素序号

    // 找到最小权值的元素序号
    for (int i = 1; i <= nSize; i++)
    {
        if (pHT[i].parent == 0 && pHT[i].weight < minValue)
        {
            minValue = pHT[i].weight;
            min = i;
        }
    }
    return min;
}
```

4、编译运行

编写测试函数 TestHufTree()，在控制台输出 Huffman 树每个节点的信息。

```
void TestHufTree(HuffmanTree pHT)
{
    // ..
    for(int i = 1; i < 512; i++)
    {
        printf("pHT[%d]\t%d\t%d\t%d\t%d\n",i,    pHT[i].weight,    pHT[i].parent,
pHT[i].lchild, pHT[i].rchild,);
    }
}
```

编译运行。

```

C:\ F:\HfmCompressCPro.exe
===== Huffman文件压缩 =====
请输入文件名: F:\Pic.bmp
哈夫曼树的每个节点信息为:
  Byte      Weight  Parent  Lchild  Rchild
pHT[1]      108393   510     0       0
pHT[2]         2    502     0       0
pHT[3]         1    495     0       0
pHT[4]         0    257     0       0
pHT[5]         0    257     0       0
pHT[6]         0    258     0       0
.....
pHT[508]      16    509    506    507
pHT[509]     98197   510    508    129
pHT[510]    206590   511    509     1
pHT[511]    884262     0    510   256

```

图 5-23 运行结果

5.4.4 生成 Huffman 编码

遍历生成的 Huffman 树，记录每个叶子节点的路径，生成 Huffman 编码。在 Huffman.h 文件中定义 HuffmanCoding()函数，用来生成 Huffman 编码。定义一个字符串数组，用来保存 Huffman 编码表。

1、先序遍历

编写测试函数 TestHufTreeN()，采用先序遍历的方法，在控制台输出 Huffman 树每个节点的信息。先序遍历 Huffman 树，方法如下：

```

void TestHufTreeN(int root, HuffmanTree pHT)
{
    cout<<pHT[root].weight<<" ";
    if(pHT[root].lchild != 0)
    {
        TestHufTreeN(pHT[root].lchild, pHT);}
    }
    if(pHT[root].rchild != 0)
    {
        TestHufTreeN(pHT[root].rchild, pHT);
    }
}

```

编译运行。


```
C:\ F:\HfmCompressCPro.exe
===== Huffman文件压缩 =====
请输入文件名: F:\Pic.bmp
先序遍历哈夫曼树输出编码信息:
Byte HufCode
0x01 000000
0x0D 000001
0x18 0000100
0x26 0000101
0x28 0000110
.....
0xEC 0001111011111111
0x02 00011111
0x80 001
0x00 01
0xFF 1
```

图 5-25 生成 Huffman 码运行结果

5.4.5 压缩原文件

创建 Compress.h 和 Compress.cpp 文件，定义 Compress()函数用于压缩原文件。

创建 Encode 函数，以二进制流方式只读打开原文件，逐字节读取文件中的数据，使用 Huffman 编码对原文件中的字节重新编码。获得压缩后的文件数据，保存在一个 char 数组中。

1、获得 Huffman 编码

右键单击工程，选择“Add -> New Item...”，新建 Compress.h 和 Compress.cpp 文件。我们将在 Compress.h 文件中定义压缩文件的相关函数，然后在 Compress.cpp 文件中实现这些函数。

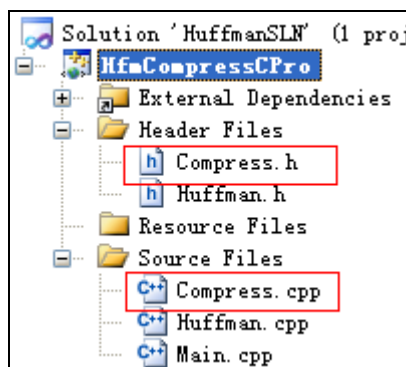


图 5-26 创建 Compress.h 和 Compress.cpp 文件

在 Compress.h 文件中定义 Compress()函数，用来得到编码文件，然后在 Compress.cpp 文件中实现这个函数。

在 Compress 函数中将会调用 HuffmanTree()和 HuffmanCoding 两个函数来生成 Huffman 编码。

```
int Compress(const char *pFilename)
{
    //...
    HuffmanTree(pHT, weight);
    //...
    HuffmanCoding(pHC, pHT);
}
```

2、字符串转码

由于 Huffman 编码表是以字符数组的形式保存的，重新编码后的数据将是一个很长的字符串。定义 Str2byte 函数，将形如“01010101”字符串转换成字节，才能得到最终的编码。

在 Compress.h 文件中定义 Str2byte() 函数用来将八个字符转换成一个字符，在 Compress.cpp 文件中实现这个函数。

```
char Str2byte(const char *pBinStr)
{
    char b = 0x00;
    for(int i = 0; i < 8; i++)
    {
        b = b<<1; // 左移 1 位
        if (pBinStr[i] == '1') {
            b = b | 0x01;
        }
    }
    return b;
}
```

3、编码原文件

创建 Encode 函数，以二进制流方式只读打开原文件，逐字节读取文件中的数据，使用 Huffman 编码对原文件中的字节重新编码。获得压缩后的文件数据，保存在一个 char 数组中。

(1) 计算压缩数据字节数

得到 Huffman 编码之后，需要计算编码完整文件需要的空间大小，计算的方法为：先计算每个字符的权值乘以该字符编码的长度，这样即为同一个字符在编码文件中所占的大小，将 256 个这样的值相加即为编码整个文件所需的空间大小。除以 8 即为总字节数。

```
int Compress(const char *pFilename)
{
    // ...
    // 计算编码缓冲区大小
    int nSize = 0;
    for (int i = 0; i < 256; i++)
    {
        nSize += weight[i] * strlen(pHC[i]);
    }
}
```

```

        nSize = (nSize % 8) ? nSize / 8 + 1 : nSize / 8;
        // ...

    }

```

Encode 函数将根据字节数 nSize，创建一个 char 数组作为缓冲区，将压缩后的数据保存到缓冲区中。

(2) 压缩文件

一边读取一边查找 Huffman 编码表，将每个字符用相应的编码替换掉，这样就得到了重新编码后的文件。此时，我们会发现得到的新的文件比原来的文件还要大，所以我们需要对得到的编码进行压缩。

在 Compress.h 文件中定义 Encode() 函数来实现压缩编码，在 Compress.cpp 文件中实现这个函数。

```

int Encode(const char *pFilename, const HuffmanCode pHC, char *pBuffer, const int
nSize)
{
    // ...
    // 开辟缓冲区
    pBuffer = (char *)malloc(nSize * sizeof(char));
    if (!pBuffer)
    {
        cerr<<"开辟缓冲区失败"<<endl;
        return ERROR;
    }

    char cd[SIZE] = {0}; // 工作区
    int pos = 0; // 缓冲区指针
    int ch;
    // 扫描文件，根据 Huffman 编码表对其进行压缩，压缩结果暂存到缓冲区中。
    while((ch = fgetc(in)) != EOF)
    {
        strcat(cd, pHC[ch]); // 从 HC 复制编码串到 cd

        // 压缩编码
        while(strlen(cd) >= 8)
        {
            // 截取字符串左边的 8 个字符，编码成字节
            pBuffer[pos++] = str2byte(cd);
            // 字符串整体左移 8 字节
            for (int i = 0; i < SIZE - 8; i++)
            {
                cd[i] = cd[i+8];
            }
        }
    }
}

```



```

    }
}
if (strlen(cd) > 0)
{
    pBuffer[pos++] = str2byte(cd);
}
....
}

```

(3) 编码文件

在 Compress()函数中，调用 Encode()函数，实现文件编码。编码后的数据保存在内存缓冲区中。

```

int Compress(const char *pFilename)
{
    // ...
    // 对原文件进行压缩编码
    char *pBuffer = NULL;
    Encode(pFilename, pHc, pBuffer, nSize);
    if (!pBuffer)
    {
        return ERROR;
    }
    // ...
}

```

4、编译运行

在控制台下显示压缩结果。

```

F:\Hf\CompressCPro.exe
===== Huffman文件压缩 =====
请输入文件名: F:\Pic.bmp
压缩后的文件编码:
00010000001001000010100010110000010101010100001101010100001100101010001100000
1111101010001101000000010100000001000100010101010100011100001010000001010101010
1010101010101010101010101001011001011001011001011001011001011001011001011001011
00101100101100101100101100101100101100101100101100101100101100101100101100101100
10110010110010110010110010110010110010110010110010110010110010110010110010110010
11001011001011001011001011001011001011001011001011001011001011001011001011001011
00101100101100101100101100101100101100101100101100101100101100101100101100101100
10110010110010110010110010110010110010110010110010110010110010110010110010110010
11001011001011001011001011001011001011001011001011001011001011001011001011001011
00101100101100101100101100101100101100101100101100101100101100101100101100101100
10110010110010110010110010110010110010110010110010110010110010110010110010110010
11001011001011001011001011001011001011001011001011001011001011001011001011001011
00101100101100101100101100101100101100101100101100101100101100101100101100101100
10110010110010110010110010110010110010110010110010110010110010110010110010110010
11001011001011001011001011001011001011001011001011001011001011001011001011001011

```

图 5-27 压缩文件运行结果

5.4.6 保存压缩文件

建立一个新的文件，文件名为“原文件名字+.huf”，将压缩后的数据写入文件。

为了保证压缩后的数据，能够被重新正确解压，我们必须把相关的解压缩规则写入到文件中去。这里的解码规则实际上就是权值数组 **weight**，解压缩时只要根据权值信息表构建 **Huffman** 树，然后得到 **Huffman** 编码，就可以根据编码进行解压缩了。

为了验证解压缩的文件是否正确，还需要将文件的大小保存到文件中去，同时也可以将文件的类型存入文件中，用于判断是否需要解压的文件，这里默认的是 **HUF** 类型。

因此，压缩文件中的数据由 2 部分组成：

(1) 文件头

包含文件类型、文件长度和权值数组

(2) 压缩数据

即 6.5.3 中，压缩原文件后得到的数据

1、初始化文件头

在 **Compress.h** 文件中定义一个文件头结构体，包含文件类型、原文件长度和权值数组三个信息。

```
// 文件头
struct HEAD
{
    char type[4];        // 文件类型
    int length;           // 原文件长度
    int weight[256];     // 权值数组
};
```

在 **Compress.h** 文件中定义 **InitHead()** 函数，并在 **Compress.cpp** 文件中实现它，用来扫描文件和初始化文件头的信息。

在扫描文件的时候，可以使用累加的方法，计算出原文件的大小。

```
int InitHead(const char *pFilename, HEAD &sHead)
{
    // 初始化文件头
    strcpy(sHead.type, "HUF"); // 文件类型
    sHead.length = 0; // 原文件长度
    for (int i = 0; i < SIZE; i++)
    {
        sHead.weight[i] = 0; // 权值
    }
    // 以二进制流形式打开文件
    FILE *in = fopen(pFilename, "rb");
    ...
    // 扫描文件，获得权重
    while((ch = fgetc(in)) != EOF)
    {
        sHead.weight[ch]++;
        sHead.length++;
    }
}
```

```

    }
    // 关闭文件
    fclose(in);
    in = NULL;

    return OK;
}

```

2、写文件

在得到了文件头信息，压缩后的编码之后，将这些信息写入到文件中去，即可得到压缩后的新的文件。在 `Compress.h` 文件中定义 `WriteFile()`函数，用来生成压缩文件，在 `Compress.cpp` 文件中实现函数功能。

```

int WriteFile(const char *pFilename, const HEAD sHead, const BUFFER
pBuffer, const int nSize)
{
    // 生成文件名
    char filename[256] = {0};
    strcpy(filename, pFilename);
    strcat(filename, ".huf");

    // 以二进制流形式打开文件
    FILE *out = fopen(filename, "wb");

    ...
    // 写文件头
    fwrite(&sHead, sizeof(HEAD), 1, out);
    // 写压缩后的编码
    fwrite(pBuffer, sizeof(char), nSize, out);
    // 关闭文件，释放文件指针
    fclose(out);
    out = NULL;

    cout<<"生成压缩文件: "<<filename<<endl;
    int len = sizeof(HEAD) + strlen(pFilename) + 1 + nSize;
    return len;
}

```

3、编译运行

当压缩文件成功时，会在控制台显示压缩信息，包括原文件名字和大小，压缩后的文件名和大小，以及压缩比率。

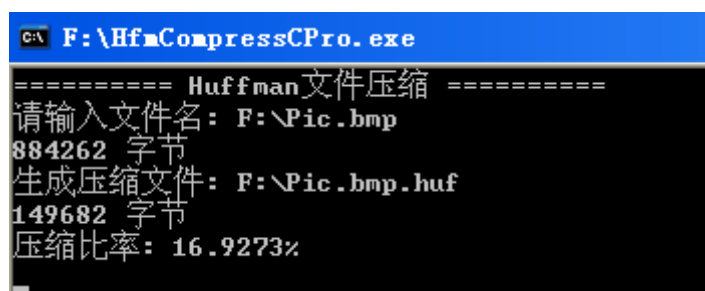


图 5-28 压缩文件显示信息

5.4.7 扩展功能

1、Huffman 编码

在“6.4 生成 Huffman 编码”中，利用二叉树的特点，遍历二叉树可以得到所有叶子节点的编码。除此之外，还可利用结构体数组的特点，反过来从叶子节点寻找父节点，逆向记录访问路径，生成 Huffman 编码。

试用这种方法，按顺序生成 0x00~0xFF 每个字节所对应的 Huffman 编码。

2、文件解压

经过前面的开发，Pic.bmp 文件已经能够被压缩成 Pic.bmp.huf 文件了。那么这个压缩后的文件，要如何解压呢？

主要思路如下：

- (1) 读取 Pic.bmp.huf 文件，扫描文件头。
- (2) 获得文件头中的权值数组，生成 Huffman 树和 Huffman 编码表。
- (3) 读取压缩包的数据体，查找 Huffman 编码表，将数据还原。

根据上述思路，请大家自行开发解压软件。

5.5 调试和运行

- (1) 按 Ctrl+F5，编译运行程序。

- (2) 输入文件名

若文件存放在 F 盘根目录下，输入文件完整路径“F:\Pic.bmp”。按回车结束。



图 5-29 输入文件名

- (3) 压缩文件

对文件进行压缩，控制台输出文件长度及压缩比。

```
C:\ F:\HfmCompressCPro.exe
===== Huffman文件压缩 =====
请输入文件名: F:\Pic.bmp
884262 字节
生成压缩文件: F:\Pic.bmp.huf
149682 字节
压缩比率: 16.9273%
```

图 5-30 压缩文件

压缩后生成“Pic.bmp.huf”文件，保存在与 Pic.bmp 相同的目录下。



图 5-31 压缩后生成 Pic.bmp.huf 文件

习题

1. 0-1 背包问题

【问题描述】有 n 件物品和一个容量为 c 的背包。第 i 件物品的容量是 $w[i]$ ，价值是 $p[i]$ 。求解将哪些物品装入背包可使价值总和最大。

2. 文件目录管理与显示

【问题描述】给出目录和文件信息，要求编程实现将其排列成一棵有一定缩进的树。要求：
(1) 设计文件和目录信息树的存储结构。(2) 从文件或键盘输入目录和文件信息，输入格式采用绝对路径法，即：\A \A\AA1 \A\AA1\aa1.doc ... 创建时要检查同一路径下不能有同名的目录或文件名。(3) 设计文件和目录信息树的输出格式(以凹入表的形式显示)。

3. 并查集

【问题描述】若某个家族人员过于庞大，要判断两个是否是亲戚，确实还很不容易，给出某个亲戚关系图，求任意给出的两个人是否具有亲戚关系。规定： x 和 y 是亲戚， y 和 z 是亲戚，那么 x 和 z 也是亲戚。如果 x,y 是亲戚，那么 x 的亲戚都是 y 的亲戚， y 的亲戚也都是 x 的亲戚。

4. 八皇后问题

【问题描述】八皇后问题 19 世纪著名的数学家高斯于 1850 年提出的。他的问题是：在 $8*8$ 的棋盘上放置 8 个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列、同一斜线上。请设计算法打印所有可能的摆放方法。

5. 树与二叉树转换

【问题描述】利用双亲表示法创建一棵树，将该树转换为二叉链表表示，并给出转换后的二叉树的先序、中序和后序遍历结果以及对该二叉树进行中序遍历线索化。要求：给出树的双亲表示和二叉链表表示的存储结构；要求二叉树三种遍历要采用非递归算法。

6. 二叉树遍历程序

【问题描述】设计一个程序：输入一个二叉树,对该树分别以先序、中序和后序三种方式进行遍历每个节点，并输出遍历结果，要求采用非递归算法。

求二叉树中两结点最近共同祖先

【问题描述】假设二叉树采用二叉链的结构存储， $p^{\wedge}$ 和 $q^{\wedge}$ 为二叉树中的两个结点，编写程序计算它们最近共同祖先并输出。

7. 2-3 树的建立

【问题描述】2-3 树的性质为：一个节点包含一个或者两个关键码；每个内部节点有 2 个子女（如果它包含一个关键码），或者 3 个子女（包含 2 个关键码）；所有叶子节点在树的同一层，因此树总是高度平衡的。2-3 树每一个节点的左子树中所有后继节点的值都小于其父节点第一个关键码的值；而中间子树所有后继节点的值都大于或等于其父节点第一个关键码的值而小于第二个关键码的值；如果有右子树，则右子树所有后继节点都大于或等于其父节点第二个关键码的值。从键盘输入以整数序列，建立一棵 2-3 树。

8. m 叉树的后序遍历

【问题描述】 M 叉树的定义为： m 叉树是指任意一个节点最多由 m 棵子树构成； m 叉树的后序遍历是指：先后序遍历第一棵子树，第二棵，,,,,，第 m 棵最后遍历根节点；用非递归的算法写出 m 叉树的后序遍历过程。

6 图与景区信息管理系统实践

6.1 实验目标

- 1、掌握图的定义和图的存储结构。
- 2、掌握图的创建方法和图的应用。
- 3、使用 C++语言，定义图的数据结构，结合迭代开发思路实现“景区信息管理系统”。

6.2 实验任务

1、任务背景

现有一个景区，景区里面有若干个景点，景点之间满足以下条件：

- (1) 某些景点之间铺设了道路(相邻)
- (2) 这些道路都是可以双向行驶的(无向图)
- (3) 从任意一个景点出发都可以游览整个景区(连通图)

开发景区信息管理系统，对景区的信息进行管理。使用图的数据结构来保存景区景点信息，为用户提供创建图、查询景点信息、旅游景点导航、搜索最短路径、铺设电路规划等功能，本次实验和后面章节实验一起来完成这些功能。

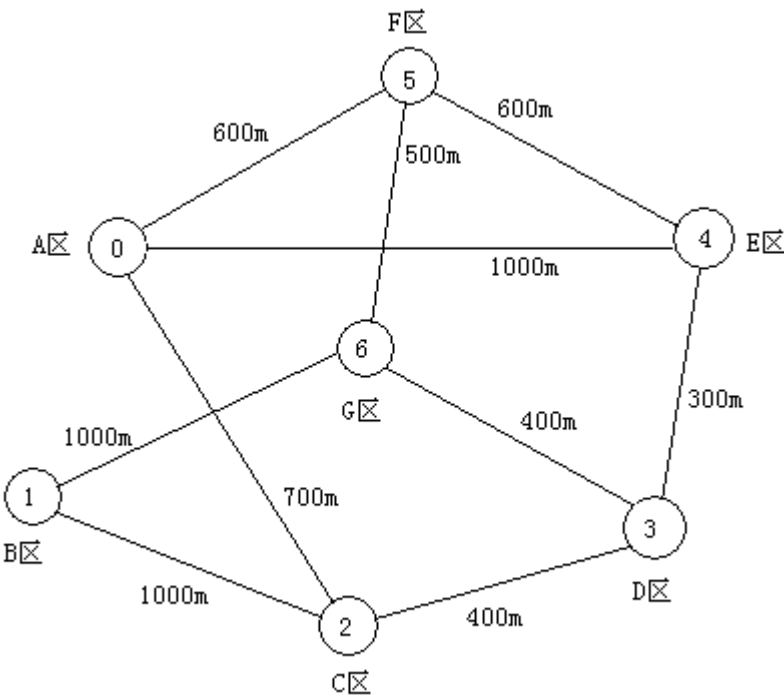


图 6-1 景区连通图

2、景点数据

景区的数据包含景点信息和景点之间的道路信息。分别由两个文本文件存储。Vex.txt 文件用来存储景点信息；Edge.txt 文件用来存储道路信息。

- (1) 景点信息：景点编号、名字和介绍

表 6-1 景点信息表

编号	名字	介绍
0	A 区	
1	B 区	
2	C 区	
3	D 区	
4	E 区	
5	F 区	
6	G 区	

Vex.txt 文件第 1 行，记录景区的景点个数。从第 2 行开始，每 3 行记录一个景点信息。
每个景点信息的记录格式为：

- 1) 景点编号(编号从 0 开始，逐个加 1)
- 2) 景点名字
- 3) 景点介绍

文件格式与示例如下：

景点个数

景点编号

景点名字

景点介绍

景点编号

景点名字

景点介绍

.....

→

Vex.txt - 记事本

文件(F) 编辑(E) 格式(O)

7
0
A区
风景优美，气候宜人。
1
B区
风景优美，气候宜人。
2
C区
风景优美，气候宜人。

图 6-2 Vex.txt

(2) 道路信息：景点 1，景点 2、两个景点之间的距离。

表 6-2 道路信息表

景 点 1	景 点 2	距 离(m)
A	C	70 0
A	E	10 00
A	F	60 0
B	C	10 00
B	G	10 00
C	D	40 0

D	E	30 0
D	G	40 0
E	F	60 0
F	G	50 0

Edge.txt 文件中，每 1 行记录 1 条道路信息。格式为：“景点 1 的编号 景点 2 的编号 道路的长度”。（每个字段使用空格符分割）
 若文件中没有某两个景点的信息，就表示这两个景点之间没有直接的路径。
 文件格式及示例如下：

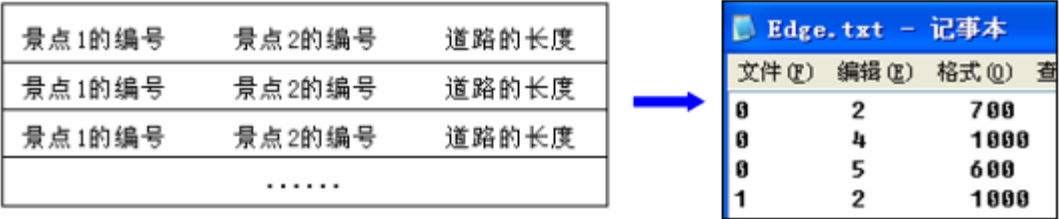


图 6-3 Edge.txt

3、任务要求

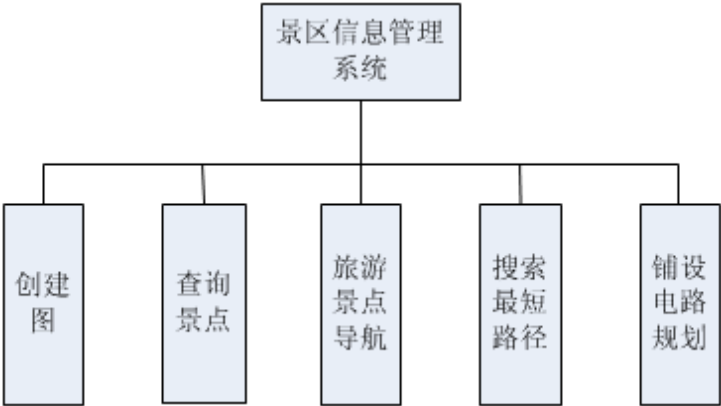


图 6-4 景区信息管理系统功能图

本次实验主要完成“创建图”和“查询景点”功能开发。

(1) 创建图

输入：从 Vex.txt 文件中读取景点信息，从 Edge.txt 文件中读取道路信息。
 处理：根据读取的景区信息创建景区景点图。
 输出：
 创建成功，依次输出：

- 1) 顶点数目
- 2) 顶点编号
- 3) 顶点名字
- 4) 边两端的顶点编号

5) 边的权值

创建失败，输出失败的提示信息。

输出格式如下：

```
C:\WINDOWS\system32\cmd.exe
===== 创建景区景点图 =====
顶点数目: 7
----- 顶点 -----
0-A区
1-B区
2-C区
3-D区
4-E区
5-F区
6-G区
----- 边 -----
(v0,v2) 700
(v0,v4) 1000
(v0,v5) 600
(v1,v2) 1000
(v1,v6) 1000
(v2,v3) 400
(v3,v4) 300
(v3,v6) 400
(v4,v5) 600
(v5,v6) 500
```

图 6-5 创建图

(2) 查询景点

输入：想要查询的景点的编号。

处理：根据输入的景点编号，查询该景点及相邻景点的信息。

输出：

- 1) 景点名字
- 2) 景点介绍
- 3) 相邻景区的名字
- 4) 到达相邻景区的路径长度

输出格式如下：

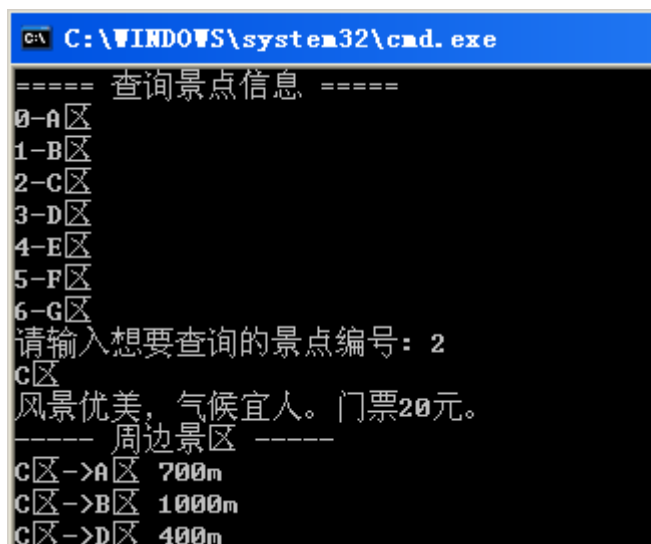


图 6-6 查询景点

6.3 分析和设计

程序输入的数据,可从文件中直接读取,也可以从界面中输入。本程序将直接从 Vex.txt 和 Edge.txt 这两个文件中读取数据,采用图的存储结构,应用深度优先搜索算法(DFS)、迪杰斯特拉(Dijkstra)算法、普里姆(Prim)算法等,开发景区信息管理系统。

6.3.1 程序设计

使用 Microsoft Visual Studio 2010 开发工具,创建一个空的控制台工程(Win32 Console Application)。利用图的存储结构来保存景区景点图,使用 C++语言开发景区信息管理系统,工程名为 GraphCPro。

- (1) 添加 Graph.h 和 Graph.cpp 文件,用来定义图的数据结构,实现图的相关操作。
- (2) 添加 Tourism.h 和 Tourism.cpp 文件,用来实现景区信息管理系统的相关功能。
- (3) 添加 Main.cpp 文件,在文件中创建程序入口函数 int main(void)。

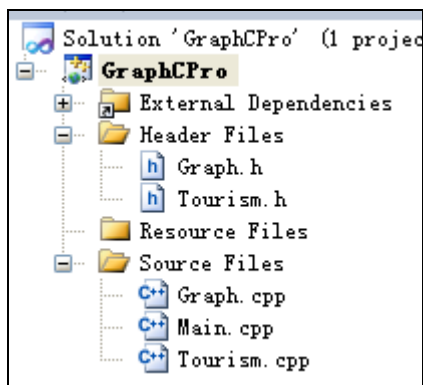


图 6-7 创建工程

6.3.2 界面设计

在 int main(void)函数中输出菜单，将系统功能列出来，供用户选择。

表 6-3 功能菜单

输入数字	功能
1	创建景区景点图
2	查询景点信息
3	旅游景点导航
4	搜索最短路径
5	铺设电路规划
0	退出

使用 while 循环输出主菜单。使用 cin 获得用户的输入，使用 switch-case 语句判断具体是哪个功能。界面效果图如下：

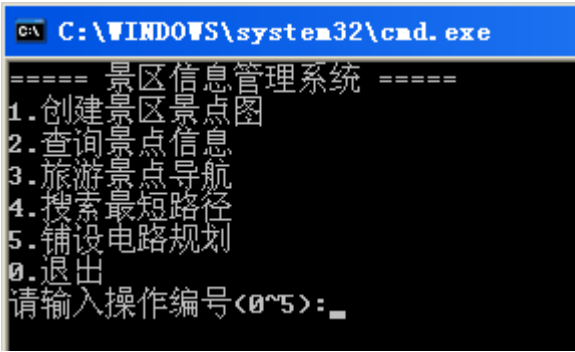


图 6-8 界面效果图

6.3.3 算法设计

本程序将使用两个主要文件：Graph.cpp 与 Tourism.cpp。Graph 用于实现图的数据结构和相关操作。Tourism 用于实现系统的主要功能。

1、Graph 文件

Graph 为图的数据结构，用于存储景区景点图，实现图的相关操作。

(1) 数据结构

```
struct Graph{
    int m_aAdjMatrix[20][20]; // 邻接矩阵
    Vex m_aVexs[20]; // 顶点数组
    int m_nVexNum; // 顶点个数
} m_Graph; // Graph 对象，用于存储景区景点图
```

(2) 函数

```
void Init(void), 初始化图结构。
int InsertVex(Vex sVex), 将顶点添加到顶点数组中。
int InsertEdge(Edge sEdge), 将边保存到邻接矩阵中。
```

Vex GetVex(int nVex), 查询指定顶点信息。
int FindEdge(int nVex, Edge aEdge[]), 查询与指定顶点相连的边。
int GetVexnum(void), 获取当前顶点数。

2、Tourism 文件

Tourism 为景点旅游信息管理功能，用于实现系统的主要功能。
void CreateGraph(void), 读取文件，创建景区景点图。
void GetSpotInfo(void), 查询指定景点信息，显示到相邻景点的距离。

6.3.4 数据结构设计

1、图的存储

当保存图结构时，即要保存顶点信息，也要保存边。图可用数组或链表来存储。
① 数组表示，常用一维数组来保存顶点的集合，使用二维数组来保存边的集合；
② 链表表示，常用邻接表、十字链表等方式存储图的顶点和边的信息。
本程序中使用数组表示法存储图：
定义一维数组 Vex m_aVexs[20]保存顶点信息，最多允许有 20 个顶点。
定义二维数组(邻接矩阵)int m_AdjMatrix[20][20]保存边的集合，数组中每个元素的值即为边的权值。

2、景区景点图

景区的地图可以看做是一个带权无向图，使用邻接矩阵来保存。
(1) 景区中的所有景点即为图的顶点。
(2) 当两个景点之间铺设的有道路时，表示两个顶点相连，为一条边。
(3) 两个景点之间的距离，即为边的权值。权值为 0 表示两个顶点不相连。

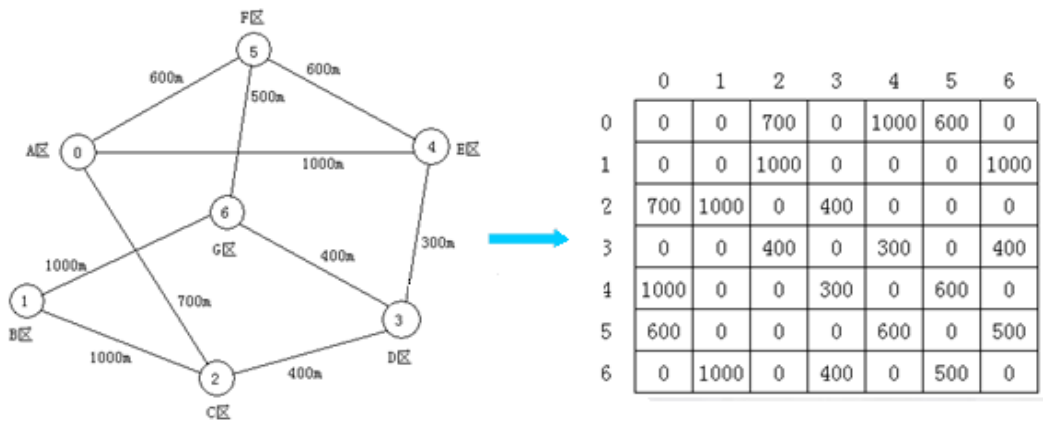


图 6-9 景区景点图

3、顶点和边的信息

定义 Vex 结构体，存储图的顶点信息。

```
struct Vex
{
    int num;           // 景点编号
    char name[20];     // 景点名字
}
```

```
char desc[1024]; // 景点介绍
};
```

定义 Edge 结构体，存储图的边的信息。

```
struct Edge
{
    int vex1;    //边的第一个顶点
    int vex2;    //边的第二个顶点
    int weight;  //权值
};
```

4、图

(1) 图的定义

无向图：顶点、边、邻接点、连通性

(2) 图的存储结构

顶点的存储：一维数组

边的存储：关系矩阵

(3) 图的创建

5、文件操作

(1) 打开文件：fopen()

(2) 格式化读文件：fscanf()

(3) 关闭文件：fclose()

6.4 编码实现

使用 Microsoft Visual Studio2010 开发工具创建控制台工程，用 C++语言开发景区旅游信息管理系统。

从文件中读取景区信息，然后创建景区景点图。

用户可以通过输入景点编号来查询景点的相关信息。

6.4.1 创建工程

1、创建工程

- (1) 使用 Microsoft Visual Studio2010 开发工具创建空的控制台工程 GraphCPro。
- (2) 创建 Graph.h 文件和 Graph.cpp 文件，用来存储图的信息和实现图的相关操作。
- (3) 创建 Tourism.h 和 Tourism.cpp 文件，用来实现景区管理系统的相关业务操作。
- (4) 添加 Main.cpp 文件，用来定义程序入口函数。

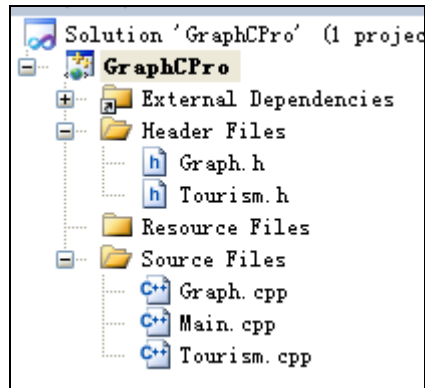


图 6-10 创建工程

2、定义程序入口函数

- (1) 在 Main.cpp 文件中，添加 int main(void) 函数作为程序入口函数。
- (2) 在 main 函数里添加以下代码，显示菜单。

```
int main(void)
{
    //.....
    while(bRunning)
    {
        //输出界面
        cout<<"===景区信息管理系统===<<endl;
        cout<<"1.创建景区景点图"<<endl;
        cout<<"2.查询景点信息"<<endl;
        cout<<"3.旅游景点导航"<<endl;
        cout<<"4.搜索最短路径"<<endl;
        cout<<"5.铺设电路规划"<<endl;
        cout<<"0.退出"<<endl;
    }
}
```

编译运行：

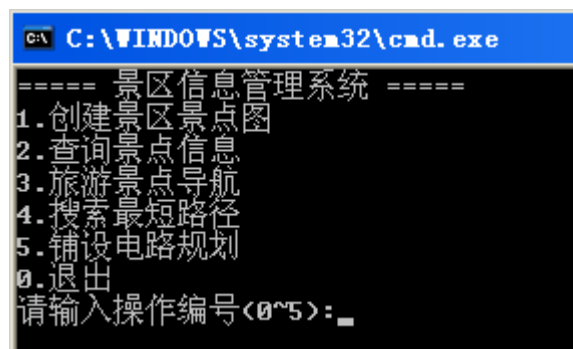


图 6-11 运行结果

6.4.2 定义图

在 Graph.h 文件中定义 Vex 和 Edge 结构体，来分别存储图的顶点和边的信息。

定义顶点数组和邻接矩阵，用来保存图的信息。

(1) 定义顶点和边

1) Vex 结构体

```
struct Vex
{
    int num;          // 景点编号
    Char name[20];    // 景点名字
    Char desc[1024];  // 景点介绍
};
```

2) Edge 结构体

```
struct Edge
{
    int vex1;         // 边的第一个顶点
    int vex2;         // 边的第二个顶点
    int weight;       // 权值
};
```

(2) 初始化图

1) 在 Graph 添加私有成员变量，定义邻接矩阵 m_aAdjMatrix，顶点信息数组 m_aVexs，当前顶点数目 m_nVexNum。

```
Class CGraph
{
private:
    int m_aAdjMatrix[20][20]; // 邻接矩阵
    Vex m_aVexs[20];          // 顶点信息数组
    int m_nVexNum;             // 当前图的顶点个数
}
```

2) 在 Graph.cpp 文件中定义 void Init(void)函数，用来实现图的初始化。

其中权值信息都初始化为 0，表示所有的顶点之间都不存在边。

(3) 创建景区景点图

1) 读取文件

工程目录下有两个文件 Vex.txt 和 Edge.txt，分别存入了景点信息和道路信息。读取这两个文件，创建景区景点图。

在 Tourism.cpp 文件中定义 void CreateGraph(void)函数，读取文件，获得景点信息和道路信息。

景点信息保存为结构体 Vex；道路信息保存为结构体 Edge。

编译运行，在控制台下输出读取到的信息。

2) 插入顶点信息

在 Graph.cpp 文件中添加 bool InsertVex(Vex sVex) 函数，通过传入 Vex 结构体，将顶点的相关信息插入到顶点信息数组中。

```
bool Cgraph::InsertVex(Vex sVex)
{
    if(m_nVexNum == MAX_VERTEX_NUM)
    {
        //顶点已满
        return false;
    }
    M_aVexs[m_nVexNum++] = sVex; //插入顶点信息
    return true
}
```

3) 插入边信息

在 Graph.cpp 文件中添加 bool InsertEdge(Edge sEdge) 函数，通过传入 Edge 结构体，将边的相关信息插入到权值矩阵中。

```
bool CGraph::InsertEdge(Edge sEdge)
{
    if (sEdge.vex1<0 || sEdge.vex1 >=m_nVexNum || sEdge.vex2<0 || sEdge.vex2 >=m_nVexNum
    {
        // 下标越界
        return false
    }
    // 插入边的信息
    m_aAdjMatrix[sEdge.vex1][ sEdge.vex2] = sEdge.weight
    m_aAdjMatrix[sEdge.vex2][ sEdge.vex1] = sEdge.weight
    return true
}
```

4) 创建景区景点图

在 Tourism.cpp 文件中添加一个变量 Graph m_Graph，将景点信息保存成图。

第一步，在 CreateGraph() 函数中，调用 Init() 函数，初始化图。

第二步，调用 InsertVex() 函数，将顶点信息插入到图中。

第三步，调用 InsertEdge() 函数，将边的信息插入到图中。

```
void CreateGraph(void)
{
    //////////////// 1 初始化图
    Init();
    //////////////// 2 设置图的顶点
    //...
    InsertVex(vex);
    //////////////// 3 设置图的边
    //...
```

```
InsertEdge(edge);  
}
```

(4) 编译运行

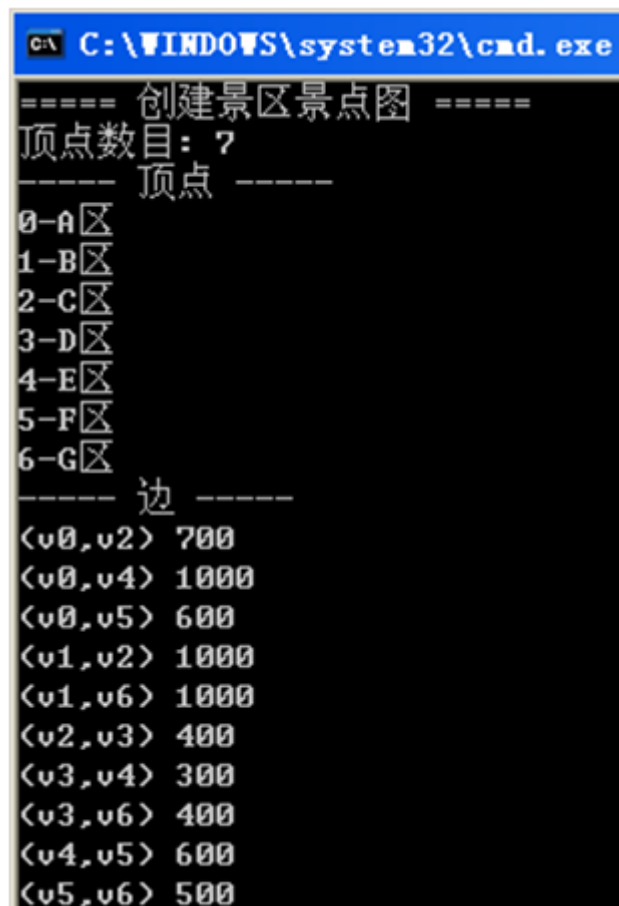


图 6-12 运行结果

6.4.3 查询景点信息

显示所有景点的编号和名字供用户选择。

输入景点编号，显示景点详细信息。查询所有与该景点相邻的景点，显示到达这些景点的距离。

(1) 显示景点列表

1) 在 Graph.cpp 文件中创建 Vex GetVex(int v)函数，通过传入的顶点编号，得到对应的顶点信息，并返回顶点结构体。

```
Vex GetVex(int v)  
{  
    return m_aVexs[v];  
}
```

2) 在 Tourism.cpp 文件中创建 void GetSpotInfo(void)函数，调用 GexVex()函数，得到所有

顶点信息，并输出到界面上。

3) 编译运行。

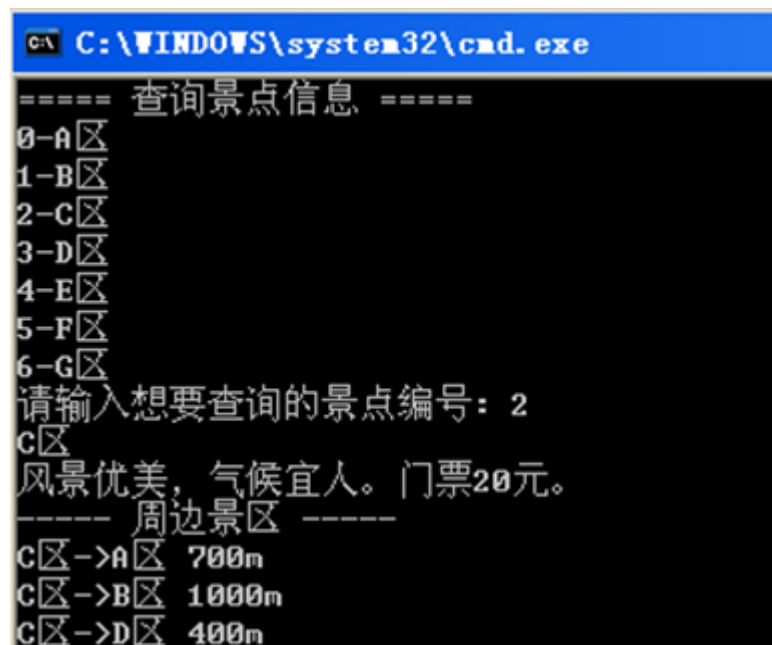
(2) 查询相邻景点

1) 在 Graph.cpp 文件中添加 int FindEdge(int v, Edge aEdge[])函数，查询所有与指定顶点相连的边。

```
int FindEdge(int v, Edge aEdge[])
{
    int k = 0;
    for(int i = 0; i < m_nVexNum; i++)
    {
        // 得到边的消息
        .....
        K++
    }
    return k; // 返回边的条数
}
```

2) 在 Tourism.cpp 文件的 GetSpotInfo() 函数中调用 FindEdge() 函数，得到所有相邻的边。根据边找到所有相邻顶点，在控制台输出到这些相邻景点的距离。

3) 编译运行

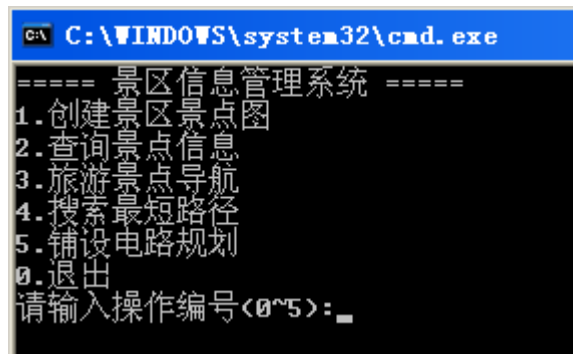


```
C:\WINDOWS\system32\cmd.exe
==== 查询景点信息 ====
0-A☐
1-B☐
2-C☐
3-D☐
4-E☐
5-F☐
6-G☐
请输入想要查询的景点编号: 2
C☐
风景优美, 气候宜人。门票20元。
----- 周边景区 -----
C☐->A☐ 700m
C☐->B☐ 1000m
C☐->D☐ 400m
```

图 6-13 运行结果

6.5 调试和运行

(1) 按 Ctrl+F5，编译运行程序。



```
C:\WINDOWS\system32\cmd.exe

===== 景区信息管理系统 =====
1. 创建景区景点图
2. 查询景点信息
3. 旅游景点导航
4. 搜索最短路径
5. 铺设电路规划
0. 退出
请输入操作编号(0~5):
```

图 6-14 主菜单

(2) 在主菜单中输入 1，则进行对应“创建图”操作，输入 2，则进行对应“查询景点信息”操作。

习题

1. 欧拉图问题

【问题描述】欧拉图起源于哥尼斯堡七桥问题，通过图中所有边一次且仅一次行遍图中所有顶点的通路称为欧拉通路，通过图中所有边一次并且仅一次行遍所有顶点的回路称为欧拉回路。具有欧拉回路的图称为欧拉图。编程随机生成 n 个结点的无向图并能进行（半）欧拉图的判定，若是则给出欧拉（回）路。要求：对给定 n 个结点，随机生成邻接矩阵以确定某无向简单图并进行欧拉图和半欧拉图的判定，若符合则给出至少一条欧拉回路或欧拉路。

2. 中国邮递员问题

【问题描述】中国邮递员问题是我国数学家管梅谷先生在 20 世纪 60 年代提出来的。该问题描述如下：一个邮递员从邮局出发，到所管辖的街道投递邮件，最后返回邮局，若必须走遍所辖各街道中每一条至少一次，则怎样选择投递路线使所走的路程最短？

3. 哈密尔顿回路问题

【问题描述】所谓哈密尔顿回路就是指经过图（有向图或无向图）中所有顶点一次且仅一次的通路。求解一个无向图中是否存在哈密尔顿回路。

4. 图的建立及输出

【问题描述】建立图的存储结构（图的类型可以是有向图、无向图、有向网、无向网，学生可以任选两种类型），能够输入图的顶点和边的信息，并存储到相应存储结构中，而后输出图的邻接矩阵。

5. 检测数据库死锁的等待图法

【问题描述】事务等待图是一个有向图 $G=(T,U)$ 。 T 为结点的集合，每个结点表示正运行的事务； U 为边的集合，每条边表示事务等待的情况。若 T_1 等待 T_2 ，则 T_1 、 T_2 之间划一条有向边，从 T_1 指向 T_2 。事务等待图动态地反映了所有事务的等待情况，如果发现图中存在回路，则表示系统中出现了死锁。

7 深度优先搜索与旅游景点导航实践

7.1 实验目标

- 1、掌握图的两种遍历方法和应用。
- 2、基于已经实现“景区信息管理系统”功能，采用迭代开发，使用 C++语言和深度优先搜索算法实现“旅游景点导航”功能开发。

7.2 实验任务

在“创建图和查询景点”工程的基础上，为景区信息管理系统增加“旅游景点导航”的功能。

- (1) 输入
起始景点的编号。
- (2) 处理
从起始景点开始，遍历景区所有的景点，记录所有无重复的路径。
- (3) 输出
将查询到的旅游路线显示到控制台中。

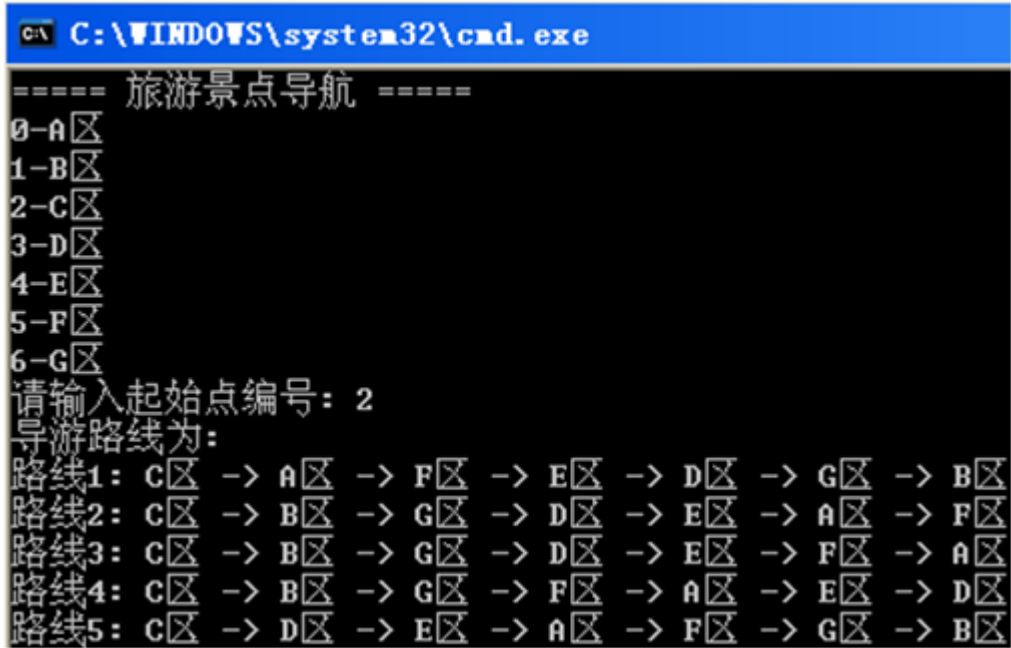


图 7-1 图的遍历

7.3 分析和设计

在“创建图和查询景点”工程的基础上进行迭代开发。

1、算法设计

旅游景点导航实际上就是从某一顶点出发，搜索出一条能够游览完所有景点的路径，其中搜索的过程就是图的遍历。

图的遍历方式有两种为深度优先搜索和广度优先搜索，这里采用深度优先搜索的方式遍历图。

深度优先搜索 (Depth First Search)

从顶点 v_0 出发：

- (1) 访问 v_0 ；
- (2) 依次访问 v_0 的邻接点 v_1 ， v_1 的邻接点 v_2 ， v_2 的邻接点 v_3 ……直到所有顶点都被访问过。

2、旅游景点导航

从一个景点出发游览整个景区时，路线可能不止一条，因此需要在深度优先搜索 (DFS) 的算法上进行改进，用来得到多条路线。

改进思路：

- (1) 定义数组 `bool aVisited [20]` 保存图中顶点的访问状态。
- (2) 定义整数 `int nIndex` 记录图的访问深度。
- (3) 若所有顶点都被访问过，就保存一条路径。
- (4) 访问结束后，将顶点的访问状态改为未访问，访问深度减 1，以便于生成其他访问路线。

定义链表 `PathList` 来保存所有路径。

```
typedef struct Path
{
    int vexs[20]; //保存一条路径
    Path *next; //下一条路径
}*PathList;
```

根据改进后的深度优先搜索算法，从图中的顶点 2 开始遍历，得到 5 种遍历结果

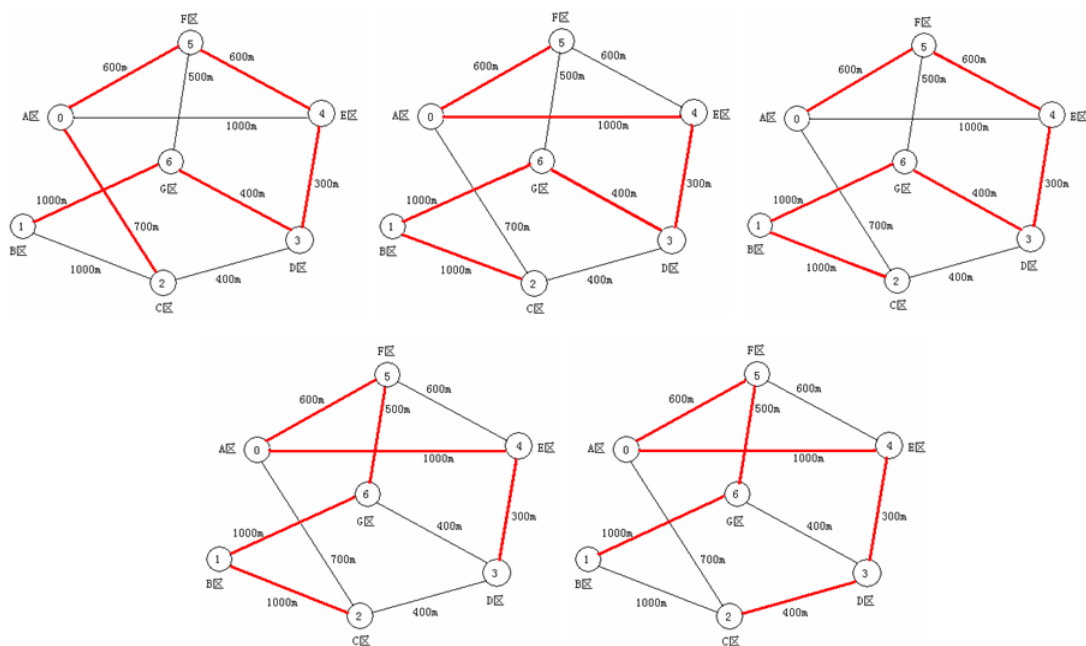


图 7-2 5 种遍历结果

3、算法的设计

(1) Graph.cpp 文件

增加函数：

1) void DFS(int nVex, bool bVisted[], int &nIndex, PathList &pList)

输入参数：int nVex，顶点编号。

输入参数：bVisted[]，bool 类型的数组，用来记录某个顶点是否被遍历过。

输入参数：int &nIndex，记录遍历的深度。

输出参数：PathList &pList，遍历得到的结果。

功能：使用深度优先搜索算法遍历图。

2) void DFSTraverse(int nVex, PathList &pList)。

输入参数：int nVex，顶点编号。

输出参数：PathList &pList，遍历得到的结果。

功能：通过调用 DFS() 函数，得到深度优先搜索遍历结果。

(2) Tourism.cpp 文件

增加函数：

void TravelPath()

输入：void

输出：void

功能：通过调用 DFSTraverse() 函数，实现旅游景点导航功能，将查询到的景点导航路线显示在界面上。

7.4 编码实现

为景区信息管理系统增加旅游景点导航功能。

- (1) 使用深度优先搜索算法实现图的遍历，得到一条导航路线。
- (2) 改进深度优先搜索算法，用来得到多条导航路线。

在实现过程中采用迭代思路，具体实现步骤如下：

步骤一：导入工程。

步骤二：遍历景区景点图(一条路线)。

步骤三：优化遍历算法(多条路线)。

7.4.1 导入工程

在实现了创建景区景点图和查询景点信息的功能之后，在原有工程的基础上新增旅游景点导航功能。

- (1) 打开 Microsoft Visual Studio 2010 开发工具。
- (2) 导入“GraphCPro”工程。

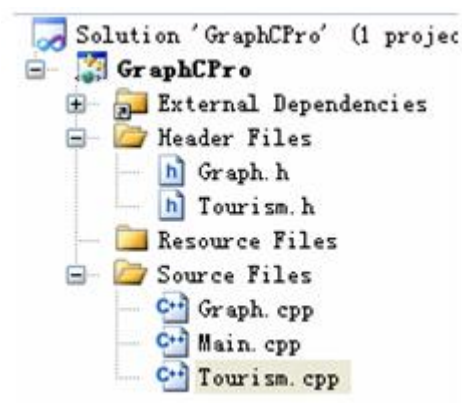


图 7-3 Tourism.cpp

7.4.2 遍历景区景点图(一条路线)

采用深度优先搜索方式遍历图。对于同一顶点上的多个邻接点，按照它们的存储顺序来访问。

- (1) 在 Graph.cpp 文件中添加 **DFS()**方法，用来实现图的深度优先搜索遍历。

```
void DFS(int nVex, bool bVisited[], int &nIndex, PathList &pList)
{
    aVisited[nVex] = true; // 改为已访问
    pList->vexs[nIndex++] = nVex; //访问顶点 nVex
    for(...) //搜索 v 的所有邻接点
    {
        if(i 是 nVex 的邻接点 && ! bVisited[i])
        {
            DFS(i, aVisited, nIndex, pList); //递归调用 DFS
        }
    }
}
```

```

    }
}

```

(2) 在 Graph.cpp 文件中添加 DFSTraverse()方法, 通过调用 DFS()函数, 进行图的深度优先遍历。

```

void Cgraph::DFSTraverse(int nVex, PathList *pList)
{
    int nIndex = 0;
    bool aVisited[MAX_VERTEX_NUM] = {false};
    DFS(nVex, aVisited, nIndex, pList);
}

```

(3) 在 Tourism.cpp 文件中添加 TravelPath()方法, 通过调用 DFSTraverse()函数, 得到景点导航路线, 并显示在界面上。

```

void CTourism::TravelPath(void)
{
    //输入景点编号
    //遍历景区景点图
    //输出遍历结果
}

```

(4) 编译运行

在 main()函数 case 3 语句中调用 TravelPath()函数, 进入旅游景点导航功能。编译运行, 得到结果如下。

```

C:\WINDOWS\system32\cmd.exe
===== 旅游景点导航 =====
0-A区
1-B区
2-C区
3-D区
4-E区
5-F区
6-G区
请输入起始点编号: 2
导游路线为:
路线1: C区 -> A区 -> F区 -> E区 -> D区 -> G区 -> B区

```

图 7-4 导航一条路线

7.4.3 优化遍历算法(多条路线)

改进 DFS 算法:

(1) 若所有顶点都被访问过, 就保存一条路径。

(2) 访问结束后，将顶点的访问状态改为未访问，访问深度减 1。

```
void DFS(int v,bool bVisited[],int aPth[],int index)
{
    bVisited[v] = true; // 改为已访问
    aPath[index++] = v; // 访问顶点 v
    if(所有的顶点都被访问过)
    {
        // 1、保存一条路径
    }
    else
    {
        for(...;...;...) // 搜索 v 的所有邻接点
        {
            if(w 是 v 的邻接点 && ! bVisited[w])
            {
                DFS(w,bVisited,aPath,index); // 递归调用 DFS
                bVisited[w] = false; // 2、改为未访问
                index--; // 索引值减 1
            }
        }
    }
}
```

7.5 调试和运行

(1) 按 Ctrl+F5，编译运行程序。

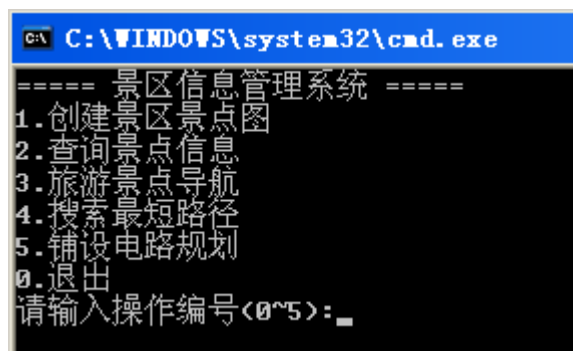


图 7-5 主菜单

(2) 在主菜单中输入 3，则进行对应“旅游景点导航”操作。

在 main() 函数 case 3 语句中调用 TravelPath() 函数，进入旅游景点导航功能。编译运行，得到结果：

```
C:\WINDOWS\system32\cmd.exe

===== 旅游景点导航 =====
0-A区
1-B区
2-C区
3-D区
4-E区
5-F区
6-G区
请输入起始点编号: 2
导游路线为:
路线1: C区 -> A区 -> F区 -> E区 -> D区 -> G区 -> B区
路线2: C区 -> B区 -> G区 -> D区 -> E区 -> A区 -> F区
路线3: C区 -> B区 -> G区 -> D区 -> E区 -> F区 -> A区
路线4: C区 -> B区 -> G区 -> F区 -> A区 -> E区 -> D区
路线5: C区 -> D区 -> E区 -> A区 -> F区 -> G区 -> B区
```

图 7-6 旅游景点导航

习题

1. 国际象棋的马踏棋盘问题

【问题描述】国际象棋共有 8 行 8 列，共 64 个单元格，无论将马放于棋盘的哪个单元格，都可以让马踏遍棋盘的每个单元格。马只能走日字，当马位于棋盘中间位置时，马可以向 8 个方向跳动，当马位于棋盘的边或角时，马可以跳动的方向将少于 8 个。另外，当马所跳向的 8 个方向中的某一个或几个方向若已经被马走过，也将跳至马下一步要走的位置。

2. 八数码问题

【问题描述】将分别标有数字 1, 2, 3, ..., 8 的八块正方形数码牌任意地放在一块 3×3 的数码盘上。放牌时要求不能重叠。于是，在 3×3 的数码盘上出现了一个空格。现在要求按照每次只能将与空格相邻的数码牌与空格交换的原则，将任意摆放的数码盘逐步摆成某种特殊的排列。如下图表示了一个具体的八数码问题求解。

2		7
5	8	4
1	6	3

1	2	3
8		4
7	6	5

3. 全国交通咨询模拟

【问题描述】不同目的的旅客对交通工具具有不同的要求。例如，因公出差的旅客希望在旅途中的时间尽可能短，出门旅游的游客则希望旅费尽可能省，而老年旅客则要求中转次数最少。编制一个全国城市间的交通咨询程序，为旅客提供两种或三种最优决策的交通咨询。

4. 迷宫的生成与路由

【问题描述】设计算法生成一个 $N \times M$ (N 行 M 列) 的迷宫，并完成迷宫的组织 and 存储。实现两种不同的迷宫路由算法：广度优先，深度优先算法。要求：1. N 和 M 是用户可配置的，缺省值为 50 和 50。 2. 迷宫的入口和出口分别在第 0 行和第 $N-1$ 行上，随机选择。3. 生成的迷宫要求是连通的。

8 Dijkstra 算法与搜索最短路径实践

8.1 实验目标

- 1、掌握迪杰斯特拉(Dijkstra)算法和应用。
- 2、基于已经实现“景区信息管理系统”功能，采用迭代开发，使用 C++语言和 Dijkstra 算法实现“搜索最短路径”功能开发。

8.2 实验任务

在“旅游景点导航”工程的基础上，为景区信息管理系统增加“搜索最短路径”功能。

(1) 输入

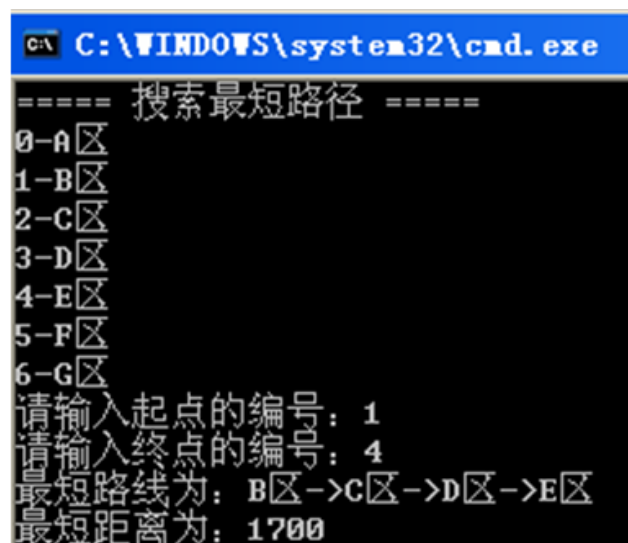
- 1) 起点编号 int nVexStart;
- 2) 终点编号 int nVexEnd。

(2) 处理

搜索两个景点之间的所有路径，找到其中距离最短的路径。

(3) 输出

在控制台输出最短路径及路径的总长度。



```
C:\WINDOWS\system32\cmd.exe
==== 搜索最短路径 =====
0-A区
1-B区
2-C区
3-D区
4-E区
5-F区
6-G区
请输入起点的编号: 1
请输入终点的编号: 4
最短路线为: B区->C区->D区->E区
最短距离为: 1700
```

图 8-1 搜索最短路径

8.3 分析和设计

在“创建图和查询景点”或“旅游景点导航”工程的基础上进行迭代开发。

1、算法设计

常用的求最短路径的算法有两个:

- (1) 迪杰斯特拉(Dijkstra)算法

特点：求得从某个源点到其余各顶点的最短路径

(2) 弗洛伊德(Floyd)算法

特点：求得每一对顶点之间的最短路径

本程序首先使用 Dijkstra 算法来求得从某个起点到其余各顶点之间的最短路径,然后从中选出我们需要的那条。

对于同一顶点上的不同邻接点,按照其存储顺序进行访问。

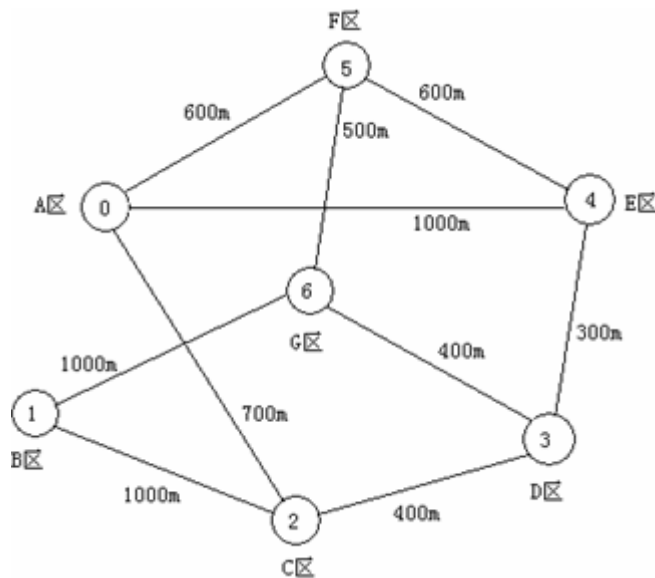


图 8-2 景区图

2、算法设计

(1) Graph.cpp 文件

增加函数：`int FindShortPath(int nVexStart, int nVexEnd, Edge aPath[])`

输入：起始景点的编号 v1 和目的景点的编号 v2。

输出：最短路径。

功能：通过 Dijkstra 算法求得 v1 到 v2 的最短路径。

(2) Tourism.cpp 文件

增加成员函数：`void FindShortPath(void)`

功能：通过调用 `m_Graph()` 函数查询两个景点间的最短路径和距离。

8.4 编码实现

为景区信息管理系统增加搜索最短路径功能。使用 Dijkstra 算法求得两个顶点间的最短路径和最短距离,并显示在界面上。

在实现过程中采用迭代思路,具体实现步骤如下:

步骤一：导入工程。

步骤二：搜索最短路径。

步骤三：查询最短路径。

步骤四：编译和运行。

8.4.1 导入工程

在实现了旅游景点导航的功能之后，要在原有工程的基础上新增搜索最短路径功能。

- (1) 打开 Microsoft Visual Studio 2010 开发工具。
- (2) 导入“GraphCPro”工程。

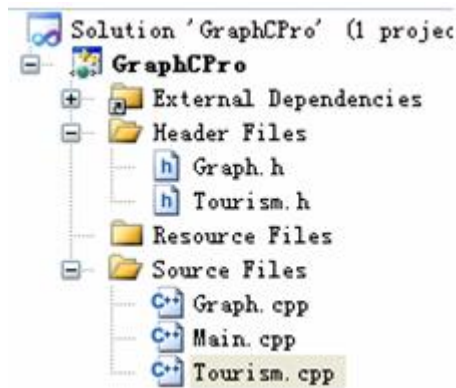


图 8-3 程序结构

8.4.2 搜索最短路径

使用 Dijkstra 算法搜索最短路径，对于同一顶点上的多个邻接点，按照它们的存储顺序来访问。

在 Graph.cpp 文件中添加 **FindShortPath()**方法，通过 Dijkstra 算法搜索源点到终点之间的最短路径。

```
int FindShortPath(int nVexStart,int nVexEnd,Edge aPath[])
{
    //初始化最短路径
    for(...;...;...){
        if(w 是距离 nVexStart 最近的点&&!aVisited[w]){
            aVisited[w] = true
        }
        // 将 w 作为中间点计算 nVexStart 到所有顶点的最短距离
        if(有新的最短距离){
            //更新最短路径
        }
    }
}
```

8.4.3 查询最短路径

在 Tourism.cpp 文件中添加 **FindShortPath()**方法，通过调用 FindShortPath 函数，查询

起始景点到目的景点之间的的最短路径和最短距离。

```
void FindShortPath(void)
{
    //输入两个景点的编号
    //搜索最短路径
    //得到最短路径和最短距离
}
```

8.5 调试和运行

(1) 按 Ctrl+F5，编译运行程序。

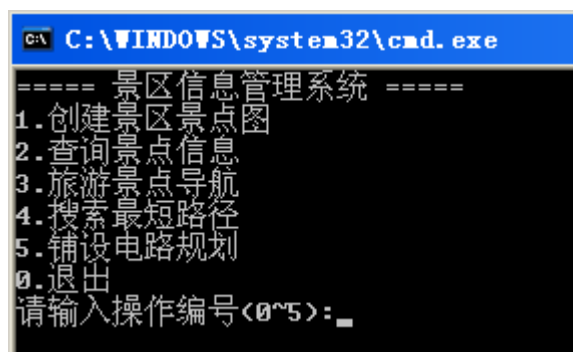


图 8-4 主菜单

(2) 在主菜单中输入 4，则进行对应“旅游景点导航”操作。

在 main()函数 case 4 语句中调用 FindShortPath()函数，进入旅游景点导航功能。编译和运行，得到结果：



图 8-5 运行结果

习题

1. 医院选址问题

【问题描述】 n 个村庄之间的交通图可以用有向网图来表示，图中边 $\langle v_i, v_j \rangle$ 上的权值表示从村庄 i 到村庄 j 的道路长度。现在要从这 n 个村庄中选择一个村庄新建一所医院，问这所医院应建在哪个村庄，才能使所有的村庄离医院都比较近？

2. 教学计划编制问题

【问题描述】针对计算机系本科课程，根据课程之间的依赖关系（如离散数学应在数据结构之前开设）制定课程安排计划，并满足各学期课程数目大致相同。

3. 旅行商问题(TSP 问题)

【问题描述】已知 n 个城市之间的相互距离，现有一个推销员必须遍访这 n 个城市，并且每个城市只能访问一次，最后又必须返回出发城市。如何安排他对这些城市的访问次序，可使其旅行路线的总长度最短？

4. 利用弗洛伊德(Floyd)算法求解最短路径

【问题描述】给出一张无向图，图上的每个顶点表示一个城市，顶点之间的边表示城市间存在路径，边上的权值表示城市间的路径长度。利用弗洛伊德(Floyd)算法求解最短路径求解任意两个城市之间的最短路径问题。

9 最小生成树与铺设电路规划实践

9.1 实验目标

- 1、理解最小生成树的概念。
- 2、掌握普里姆(Prim)算法和应用。
- 3、基于已经实现“景区信息管理系统”功能，采用迭代开发，使用 C++语言和最小生成树算法实现“铺设电路规划”功能开发。

9.2 实验任务

在“搜索最短路径”工程的基础上，为景区信息管理系统增加“铺设电路规划”的功能。

(1) 输入

景区景点信息和道路信息。

(2) 处理

根据景区景点图，构造一棵最小生成树，设计出一套铺设线路最短，但能满足每个景点都能通电的方案。

(3) 输出

- 1) 需要铺设电路的道路。
- 2) 每条道路铺设电路的长度。
- 3) 铺设电路的总长度。

9.3 分析和设计

在“创建图和查询景点”或“搜索最短路径”工程的基础上进行迭代开发。

1、算法设计

铺设电路的规划方案要求铺设线路最短，但能满足每个景点都能通电，从图的角度来说，就是在图的所有路径中，选择其中的几条，使得这几条路径可以连通所有的顶点，并且权值和最小。

得到这几条路径的过程，实际上就是构造最小生成树的过程。

构建最小生成树常用的方法有两种，普里姆(Prim)算法和克鲁斯卡尔(Kruskal)算法。本程序使用 Prim 算法来构建最小生成树。

对于下边的图，以顶点 0 为源点，按照顶点的存储顺序查找顶点，构建最小生成树。

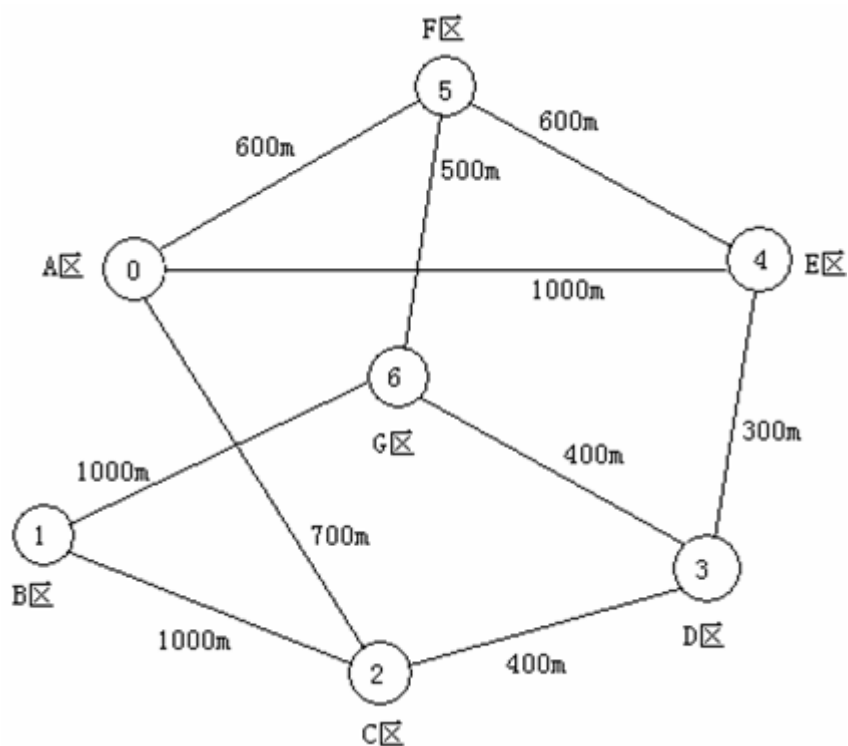


图 9-1 原图

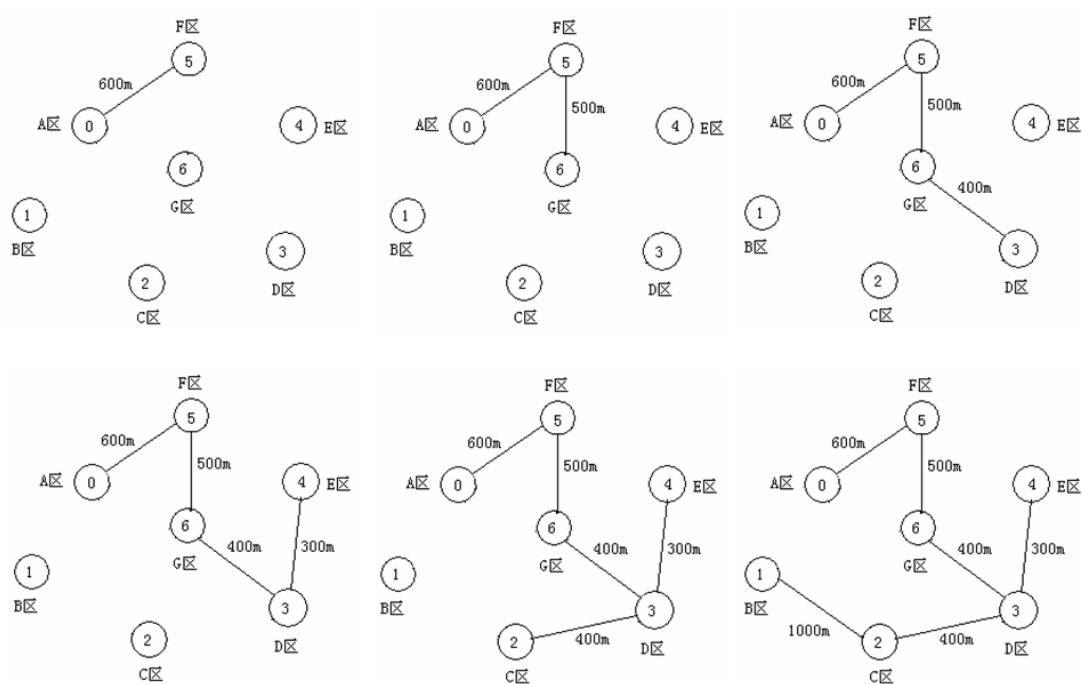


图 9-2 最小生成树

查找 v1 和 v2 的顺序不同，得到的最小生成树可能会不同。

下面是按照顶点的存储顺序查找和逆序查找得到的不同的最小生成树：

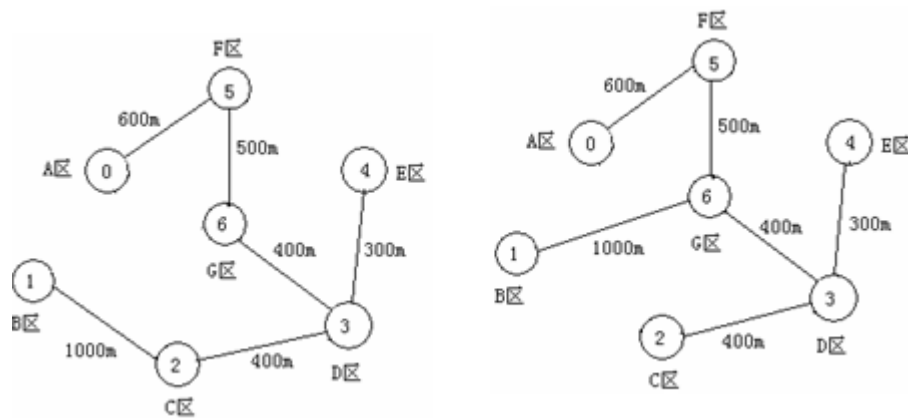


图 9-3 不同的最小生成树

图的结构决定了最小生成树的个数，相同权值的边越多，最小生成树的个数就可能越多。

对于同一个图，得到多个最小生成树的方式有两种：

- (1) 改变顶点的查找顺序
- (2) 当有多条权值最小的边时，使用栈将每条边都保存下来

2、类的设计

(1) Graph.cpp 文件

增加函数：int FindMinTree(Edge aPath[])

输入：Edge aPath[]

输出：最小生成树。

功能：通过 Prim 算法构建最小生成树。

(2) Tourism.cpp 文件

增加函数：void DesignPath(void)

输入：viod

输出：铺设的线路。

功能：通过调用 Graph.cpp 文件中的 FindMinTree()方法查询铺设电路规划图。

3、最小生成树

在一给定的无向图 $G = (V, \{E\})$ 中， (u, v) 代表连接顶点 u 与顶点 v 的边，而 $w(u, v)$ 代表此边的权重，若存在 T 为 E 的子集且为无循环图，使得 $w(T)$ 最小，则此 T 为 G 的最小生成树。

4、Prim 算法

假设 $N = \{V, \{E\}$ 是连通图 $\}$ ， TE 和 TV 分别是最小生成树的边和顶点的集合，以 v_0 作为源点，构造最小生成树：

- (1) 将 v_0 从 V 移动到 TV 中；
- (2) 在 V 中找到顶点 v_1 ， TV 中找到顶点 v_2 ，使得边 $E(v_1, v_2)$ 的权值最小；
- (3) 将 $E(v_1, v_2)$ 加入到 TE 中， v_1 移动到 TV 中，重复步骤 2，直到 V 为空，此时 TE 即为 N 的最小生成树。

9.4 编码实现

为景区信息管理系统增加铺设电路规划功能。当用户选择铺设电路规划功能时，系统会设计出一套铺设线路最短，但能满足每个景点都能通电的方案。

在实现过程中采用迭代思路，具体实现步骤如下：

步骤一：导入工程

步骤二：构建最小生成树

步骤三：查询铺设电路规划图

步骤四：编译和运行

9.4.1 导入工程

在实现了搜索最短路径的功能之后，在原有工程的基础上新增铺设电路规划功能。

(1) 打开 Microsoft Visual Studio 2010 开发工具。

(2) 导入“GraphCPro”工程。

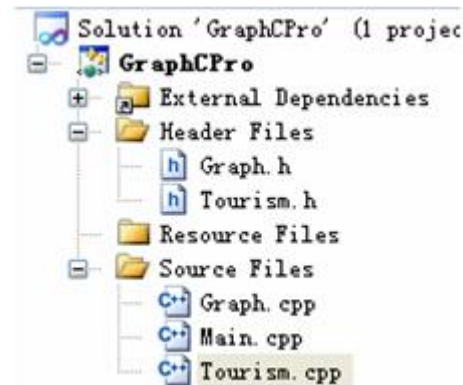


图 9-4 导入工程

9.4.2 构建最小生成树

使用 Prim 算法构建最小生成树，按照顶点的存储顺序来查找顶点。

在 Graph.cpp 文件中增加 FindMinTree() 方法，通过普里姆算法构建最小生成树。

```
int FindMinTree(Edge aPrth[])
{
    //使用 Prim 算法构建最小生成树
}
```

9.4.3 查询铺设电路规划图

在 Tourism.cpp 文件中增加 DesignPath() 方法，通过调用 FindMinTree() 方法查询铺设电

路规划图。

```
int DesignPath()
{
    M_Graph.Find MinTree() //构建最小树

    //输出铺设线路图
}
```

9.5 调试和运行

(1) 按 Ctrl+F5，编译和运行程序。

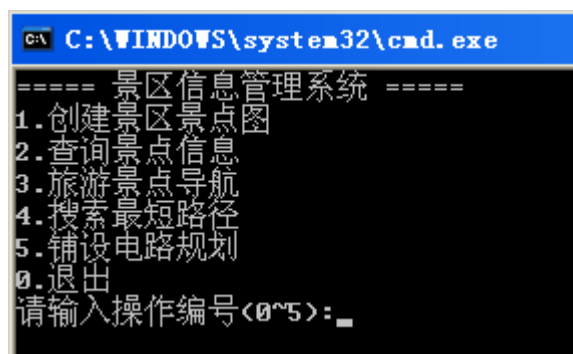


图 9-5 主菜单

(2) 在主菜单中输入 5，则进行对应“铺设电路规划”操作。

在 main() 函数 case 5 语句中调用 DesignPath() 方法，进入铺设电路规划功能。编译和运行，得到结果：



图 9-6 运行结果

习题

1. KRUSKAL 算法求最小生成树

【问题描述】以合适方便的方式输入一个边带权值的无向图，采用邻接表存储结构存储该无向图，然后根据 KRUSKAL 算法求该无向图的最小生成树并输出。

2. 有向图的拓扑排序

【问题描述】以合适方便的方式输入一个有向图，采用邻接表存储结构存储该有向图，然后对该有向图进行拓扑排序，如果该有向图有环存在，程序需要给出图中有环的输出结果，否则输出对图进行拓扑排序的结果。

3. 求有向图中所有简单回路

【问题描述】以合适方便的方式输入一有向图，并建立有向图的邻接表存储结构，然后求该有向图中所有的简单回路。输入有向图的方式要尽量简单方便，要能够形象方便地观察有向图的图形结构。

4. 实现求有向图强连通分量的算法

【问题描述】以合适方便的方式输入一有向图,并建立有向图的合适的存储结构. 判断该有向图是否强连通,如果是强连通图,则求出该图的所有的强连通分量并输出字符。

5. 连通图着色求解

【问题描述】从键盘输入一个无向图，给图上的每一个节点标记一种颜色，在保证任何相邻节点颜色不同的同时，求解出该图所需要的最少颜色数，并给出每个节点的具体颜色。

第二部分 综合实践

10 线性结构与连连看游戏综合实践

10.1 实践目标

- 1、调研连连看游戏，了解连连看游戏的功能和规则等。
- 2、掌握集成开发工具(Visual C++ 6.0 或 Microsoft Visual Studio 2010)。
- 3、掌握 C++ 的基础编程。
- 4、理解 MFC 框架，包括 MFC Dialog 应用程序和 GDI 编程。
- 5、理解线性结构，重点掌握数组和栈操作，掌握数组的遍历、消子和胜负判断等算法。
- 6、理解企业软件开发过程，理解系统需求分析和设计，应用迭代开发思路进行项目开发。
- 7、养成良好的编码习惯和培养软件工程化思维，综合应用“C++编程、MFC Dialog、算法、线性结构”等知识，开发“连连看游戏”桌面应用程序，达到掌握和应用线性结构核心知识。

10.2 需求分析

10.2.1 项目简介

“连连看游戏”是给一堆图案中的相同的图案进行配对的简单游戏，在一定的规则之内对相同的图案进行消除处理，在规定时间内消除所有图案后玩家就获胜。

“连连看游戏”只要将相同的两张元素用三根以内的直线连在一起就可以消除，规则简单容易上手，游戏速度节奏快，画面清晰可爱。类似游戏界面如下图所示：

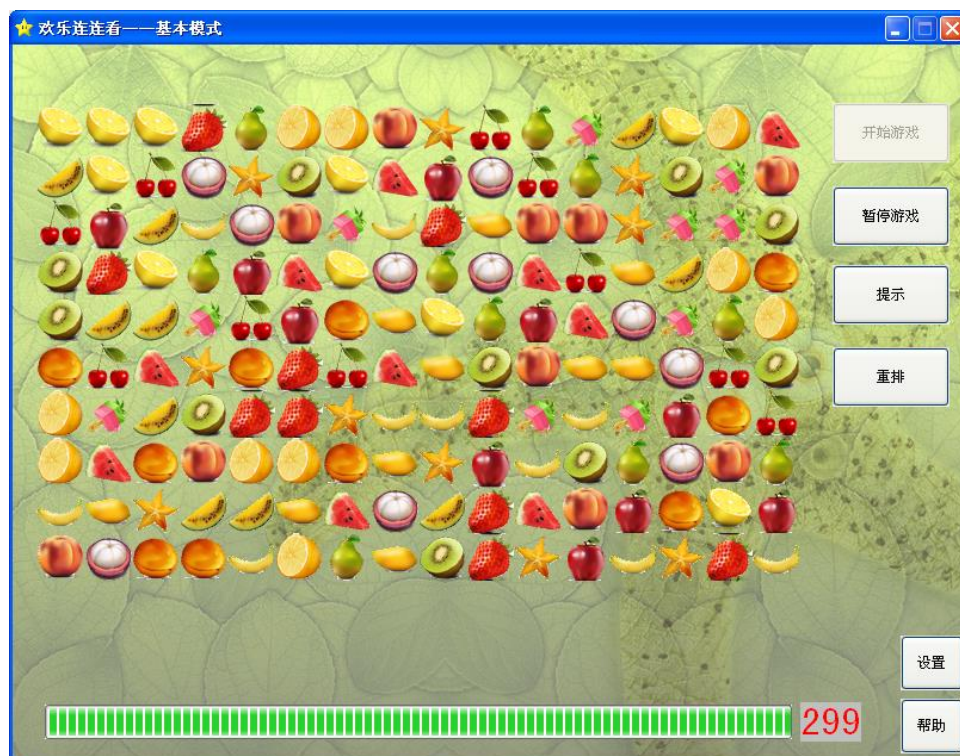


图 10-1 连连看游戏参考界面

10.2.2 游戏规则

(1) 一条直线消子

选择的两张图片花色相同，并且处于同一条水平线或者同一条垂直线上，并且两张图片之间没有其余的图片，则可以进行一条直线消子。

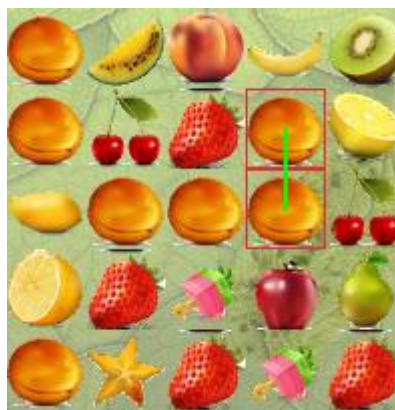


图 10-2 一条直线消子

(2) 两条直线消子

选择的两条图片花色相同，既不在同一水平线上，也不再同一垂直线上，两个图片的连通路径至少要有两条直线组成，两条直线经过的路径必须是空白，中间只要有一个非同种类的图片，该路径无效。

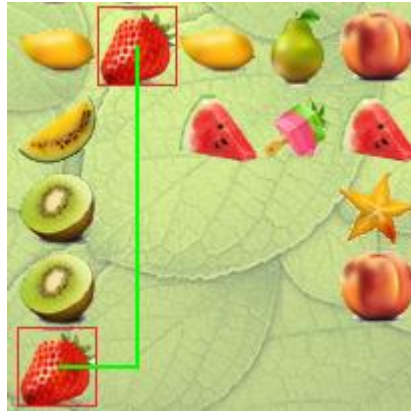


图 10-3 两条直线消子

(3) 三条直线消子

使用一个折点的路径无法连通的两个图片，只能如图中连线所示连通，即连通路径有三条直线，在该直线的路径上没有图案出现，只能是空白区域。

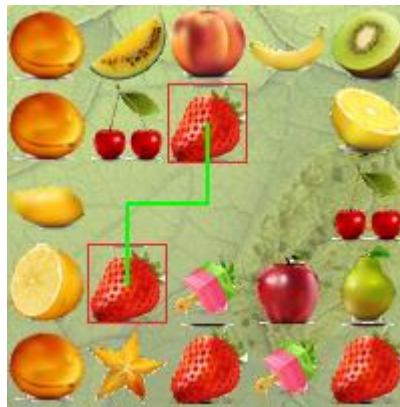


图 10-4 三条直线消子

系统主要业务流程图如下：

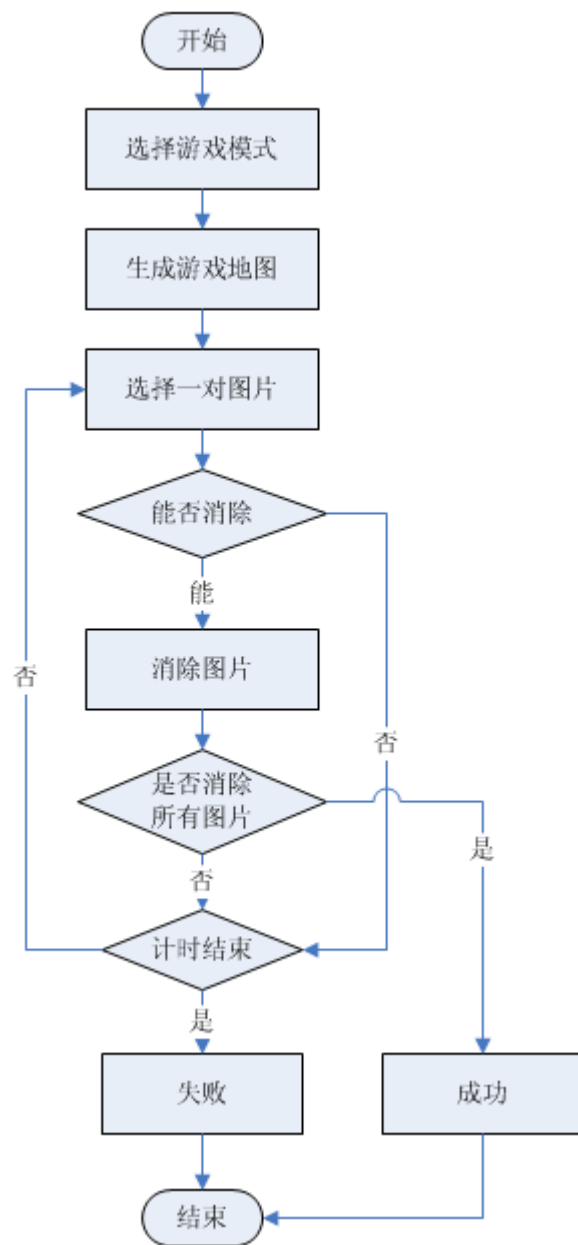


图 10-5 业务流程图

10.2.3 功能简介

待开发连连看游戏，称为“欢乐连连看”，使用二维数组来保存游戏地图中的数据，实现了连连看的核心功能。

欢乐连连看游戏功能结构图如下：

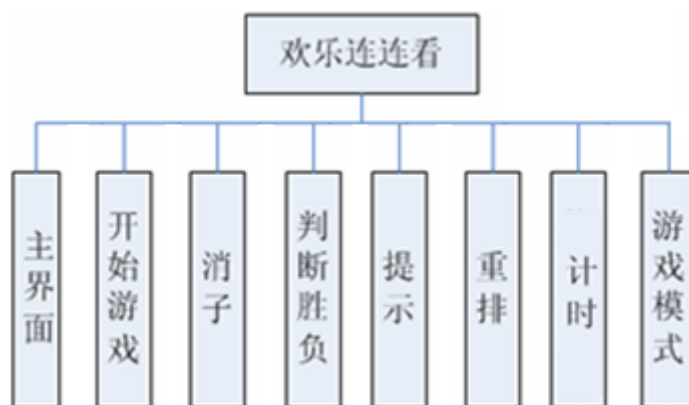


图 10-6 功能结构图

1、主界面

为游戏主界面，进行各项操作入口。

2、开始游戏

玩家选择游戏模式，进入游戏后，选择开始游戏，系统根据设置的主题风格生成一个图片布局(游戏地图)，以供玩家点击消除。

游戏地图大小为 640*400，是一个 16 行 * 10 列矩形，分成 160 个小正方形，存放 160 张图片，每张图片大小为 40*40。

3、消子

对玩家选中的两个图案进行判断，是否符合消除的规则。只有符合以下条件的图案对才会消失：

- (1) 一条直线连通
- (2) 两条直线连通
- (3) 三条直线连通

如果可以消除，从游戏地图中提示连接路径，然后消除这两种图片，并计算相应的积分。如果不能消除，则保持原来的游戏地图。

4、判断胜负

当游戏完成后，需要判断游戏的胜负。不同模式下，判断胜负的规则不同。

- (1) 基本模式时，如果在 5 分钟内，将游戏地图中所有的图片都消除，则提示玩家获胜。
- (2) 休闲模式时，如果游戏地图中所有的图片都被消除，则提示玩家获胜。

5、提示

可以提示界面上能够消除的一对图片。

6、重排

根据随机数，重新排列游戏地图上图片。

7、定时

设定一定时间来辅助游戏是否结束。

8、游戏模式

基本模式、休闲模式和关卡模式三种，可以根据是否定时等规则进行设置，增强趣味性。

10.3 系统设计

10.3.1 界面设计

1、主界面

主界面为启动游戏时出现的界面，在该界面上进行游戏模式的选择、游戏的设置、查看帮助信息、关于“欢乐连连看”。主界面大小为 800*600。

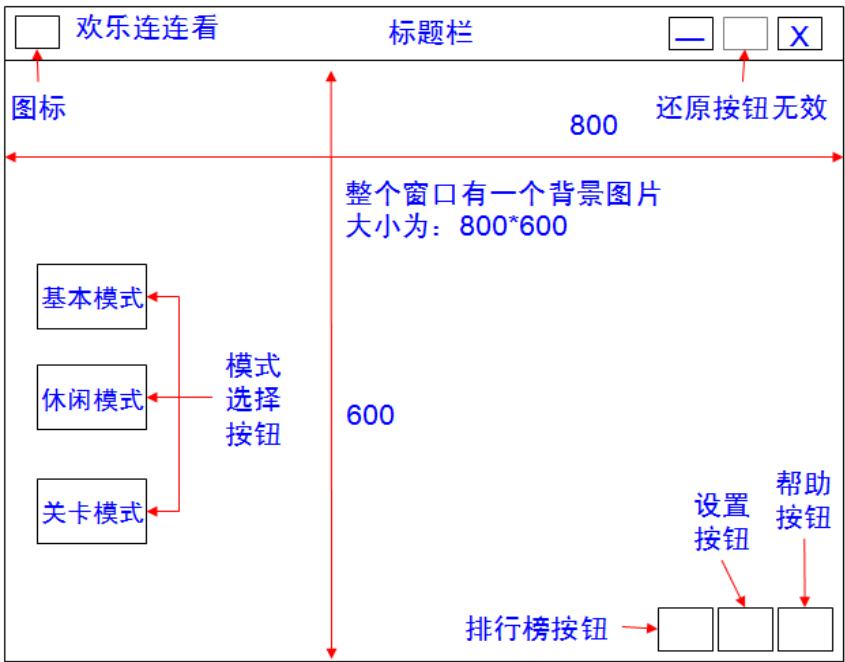


图 10-7 主界面设计

使用画图软件制作一张 800*600 大小的 BMP 图片，背景图片设计时需要考虑主界面上按钮位置的摆放。



图 10-8 主界面效果

2、游戏界面

根据设置的主题生成的游戏地图、开始新游戏按钮、暂停按钮、提示按钮、重排按钮、计时、设置按钮、帮助按钮。游戏地图像素大小为 640*400。每张图片像素大小为 40*40。游戏地图是一个 16*10 的矩形。游戏地图有 160 张图片。图片出现的位置为随机的。



图 10-9 游戏界面

10.3.2 程序结构设计

1、工程结构设计

(1) 解决方案名称: Lianliankan

(2) 工程名称: LLK

(3) 工程目录结构

工程目录结构如下图所示:

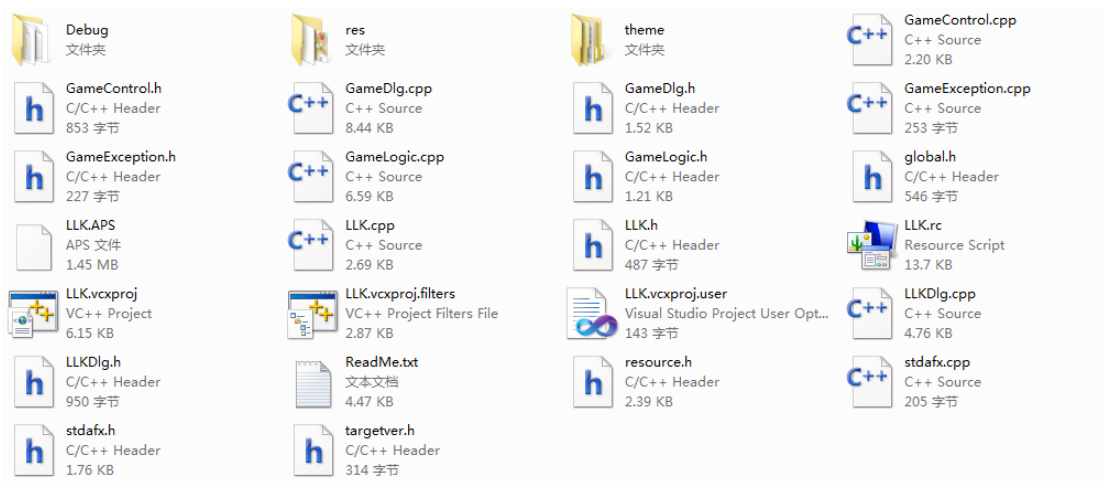


图 10-10 工程目录结构

- 1) res: 程序资源文件夹, 存放图标。
- 2) theme: 主题文件夹, 存放游戏的主题资源文件。

2、程序类关系设计

游戏程序按分层的思路来设计, 主要分为: 界面层(主窗口类 CCLKDlg、游戏窗口类 CGameDlg), 游戏控制和业务逻辑层(游戏控制类 CGameControl、游戏逻辑操作类 CGameLogic)。

各层之间使用结构体(顶点信息 Vertex)来传递数据。

应用程序中公共的常量, 定义在 global.h 头文件中。

10.3.3 数据结构设计

1、顶点存储结构

添加 global.h 文件, 定义结构体 Vertex, 用于保存游戏地图中一个点的行号、列号、值信息。

代码如下:

```
typedef struct tagVertex{
    int row;           // 行
    int col;           // 列
    int info;          // 信息类
}Vertex;
```

2、游戏地图存储结构

使用二维数组来保存连连看游戏地图, 给每种图片一个编号, 并将这些编号保存在二维数组中。

用户在屏幕上选择 2 张图片, 对应为数组中的两组坐标。分别实现三个消子判断算法: “一条直线消子”、“两条直线消子”、“三条直线消子”, 并使用这三个算法进行消子判断。若符合消子规则, 就在屏幕上消除一对图片, 并把数组对应元素清空。

(1) 游戏地图中的图片种类和重复次数与游戏的级别和难度有关。图片种类越多, 重复次数越小, 游戏的难度越大, 反之则越容易。

(2) 因为 2 张同类的图片才能消。为保证游戏能完全消完, 每种图片重复的次数一定要是偶数, 即 2 的倍数。

(3) 地图的大小与图片元素种类之间的关系

地图的行数 * 地图的列数 = 图片的种类数 * 每种图片重复的次数

(4) 地图数据的存储

1) 用 int 类型动态二维数组(int** m_pGameMap)存储地图中元素图片的编号。

2) 获得某行某列对应的元素数值

10.3.4 算法设计

1、随机开局算法

(1) 计算游戏中元素个数: 行数 * 列数。

- (2) 计算每一种花色重复数：行数 * 列数 / 花色数。
- 1) 判断（行数 * 列数 % 花色数）是否为 0。如果不为 0，则进行异常处理。
 - 2) 判断每一种花色重复数是否能被 2 整除，如果不能被 2 整除，则进行异常处理。
- (3) 按从左到右，从上到下的顺序，将花色数填入游戏地图。

0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1
1	1	1	1	2	2	2	2	2	2	2	2	2	2	3	3
3	3	3	3	3	3	3	3	4	4	4	4	4	4	4	4
4	4	5	5	5	5	5	5	5	5	5	5	6	6	6	6
6	6	6	6	6	6	7	7	7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8	8	8	9	9	9	9	9	9
9	9	9	9	10	10	10	10	10	10	10	10	10	10	11	11
11	11	11	11	11	11	11	11	12	12	12	12	12	12	12	12
12	12	13	13	13	13	13	13	13	13	13	13	14	14	14	14
14	14	14	14	14	14	15	15	15	15	15	15	15	15	15	15

实现伪代码如下：

```
int nRepeatNum = 得到花色重复数;
int nCount = 0; // 记录当前是第几个元素
for(int i = 0; i < nPicNum; i++) // nPicNum 为花色数
{
    // 重复数
    for(int j = 0; j < nRepeatNum; j++)
    {
        pGameMap[nCount / nCols][nCount % nCols] = i; // nCols 是游戏地图的列数
        nCount++;
    }
}
```

- (4) 由于生成的地图是规则的，因此，需要将地图中的花色打乱。实现思路是，随机选择两个元素，将其值对调。重复若干次（当前游戏重复了元素总数次）。

2、消子判断的流程

- (1) 获得选中的两张图片的行号与列号。
- (2) 判断选中的图片是否同色，不同色，则不能相消。判断选中的图片是否为同一个图片，如果为同一个图片，不能相消。
- (3) 判断连通性，如以下三种情况均不满足，则结束。
 - 1) 首先判断能否一条直线连通。
 - 2) 如果不能一条直线连通，则判断能否两条直线连通。
 - 3) 如果不能两条直线连通，则判断能否三条直线连通。
- (4) 获得连通路径，绘制连通线。
- (5) 消除图片。
- (6) 更新游戏地图。

3、一条直线消子算法

(1) 判断两个顶点，行是否相同，若相同，则判断两个顶点在 X 方向是否连通。

在 CGameLogic 类中定义 LinkInRow()函数实现 X 方向连通判断。依次判断在 X 方向两个顶点间每一个顶点，是否都为空，全为空，表示可以连通，否则不能连通。

实现伪代码如下：

```
bool CGameLogic::LinkInRow(int** pGameMap, Vertex v1, Vertex v2)
{
    1: 将顶点的列进行调整，使 nCol1 的值小于 nCol2
    2: 使用 for 循环，判断两个顶点间是否不为空的图片
    for(int i = nCol1 + 1; i <= nCol2; i++)
    {
        if(i == nCol2)      return true;
        if(pGameMap[nRow][i] != BLANK) break; // BLANK 表示空
    }
    return false;
}
```

(2) 判断两个顶点，列是否相同，若相同，则判断两个顶点在 Y 方向是否连接。

在 CGameLogic 类中定义 LinkInCol()函数实现 Y 方向连通判断。依次判断在 Y 方向两个顶点间每一个顶点，是否都为空，全为空，表示可以连通，否则不能连通。

4、两条直线消子算法

若一条直线无法连通，则判断二条直线的情况。在 CGameLogic 类中定义 OneCornerLink ()函数判断两点是否能两条直线连通。

先判断两个顶点的 X 和 Y 方向的直线相交的两个顶点，是否为空。若能构成两条直线连通，那么这个相交的顶点必须为空才行。

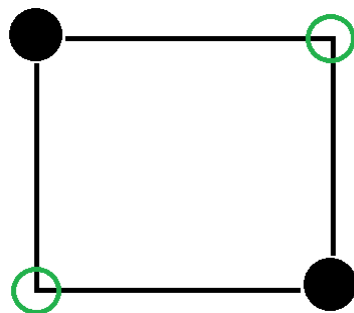


图 10-11 两条直线消子算法示意图

若顶点有一个可以相消，则判断该顶点与两个顶点，横向或纵向一条直线是否连通。若都连通，则表示两条直线消子成功。否则不能相消。

实现伪代码如下：

```
bool CGameLogic::OneCornerLink(int** pGameMap, Vertex v1, Vertex v2)
{
    1: 判断相交的顶点是否为空
```

```

if (pGameMap[v1.row][v2.col] == BLANK)
{
    2: 判断两个同行的顶点是否一条直线连通
    3: 判断两个同列的顶点是否一条直线连通
    if(纵向连通判断(pGameMap, v1.row, v2.row, v2.col)
    && 横向连通判断(pGameMap, v1.row, v1.col, v2.col))
    {
        return true;
    }
}

if(pGameMap[v2.row][v1.col] == BLANK)
{
    if(纵向连通判断(pGameMap, v1.row, v2.row, v1.col)
    && 横向连通判断(pGameMap, v2.row, v1.col, v2.col))
    {
        return true;
    }
}

return false;
}

```

5、三条直线消子算法

若二条直线无法连通，则判断三条直线的情况。在 CGameLogic 类中定义 TwoCornerLink ()函数判断两点是否能三条直线连通。

三条直线消子时，假设选择的两个图片的位置为(nRow1, nCol1)和(nRow2, nCol2)，则先寻找与 Y 轴平行的连通线段。

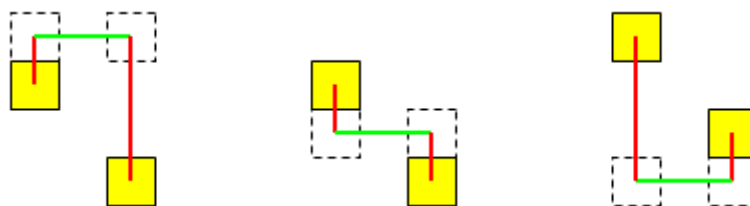


图 10-12 寻找与 Y 轴平行的连通线段

如果 Y 轴没有找到可以连通的三条直线，则寻找与 X 轴平行的连通线段。

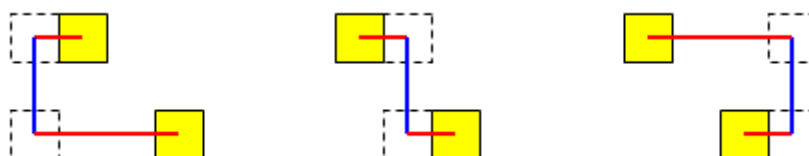


图 10-13 寻找与 X 轴平行的连通线段

(1) 搜索关键路径

如何找到这样的一条关键路径呢？以搜索水平方向上的关键路径为例。
假设玩家选择的两个顶点为 $v0(row0, col0)$, $v3(row3, col3)$ 。

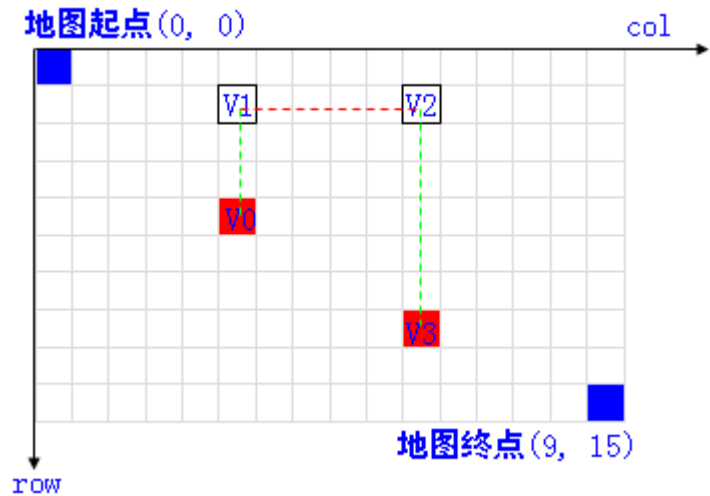


图 10-14 搜索关键路径

步骤如下：

第一步，从地图的第一行开始扫描，当前扫描到 $nRow$ 行；

第二步，设置拐点： $V1(nRow, col0)$, $V2(nRow, col3)$ ；

第三步，判断顶点 $V1$ 和 $V2$ 是否为空；

第四步，判断顶点 $V1$ 和 $V2$ 是否水平方向上连通，如果连通，则 $V1$ 到 $V2$ 之间的连线即为关键路径。如果不连通则接着扫描下一行，重复②③④的步骤。

(2) 判断三条直线连通

采用枚举法判断三条直线连通，假设玩家选择的两个顶点为 $V0$ 和 $V3$ ，判断三条直线连通具体实现步骤如下：

- 1) 找到其中一条关键路径 $V1, V2$ ；
- 2) 判断 $V1$ 和 $V0$ 是否连通；
- 3) 判断 $V2$ 和 $V3$ 是否连通；
- 4) 如果同时满足 $V1$ 和 $V0$ 连通， $V2$ 和 $V3$ 连通，则 $V0$ 和 $V3$ 满足三条直线连通；
否则，在此关键路径下 $V0$ 和 $V3$ 不连通，找到下一条关键路径，重复步骤②③④，直到判断出 $V0$ 和 $V3$ 是否连通。

(3) 保存连通路径

使用栈来保存连通路径中的关键点：起始点 $V0$ 、拐点 $V1$ ，拐点 $V2$ 和终点 $V3$ 。

保存连通路径的步骤如下：

- 1) 保存起始点 $V0$ ；
- 2) 判断是否存在能够满足三条直线消子的关键路径 $V1, V2$ ；
- 3) 如果存在，保存顶点 $V1, V2, V3$ ；如果不存在，删除起始点 $V0$ 。

6、胜负判断算法

当有元素被消掉后，进行胜负判断，遍历地图中所有元素的值，当所有的元素都为空时，表示获胜，游戏结束，否则继续游戏。

10.4 创建工程

10.4.1 功能需求

创建“欢乐连连看”游戏工程。

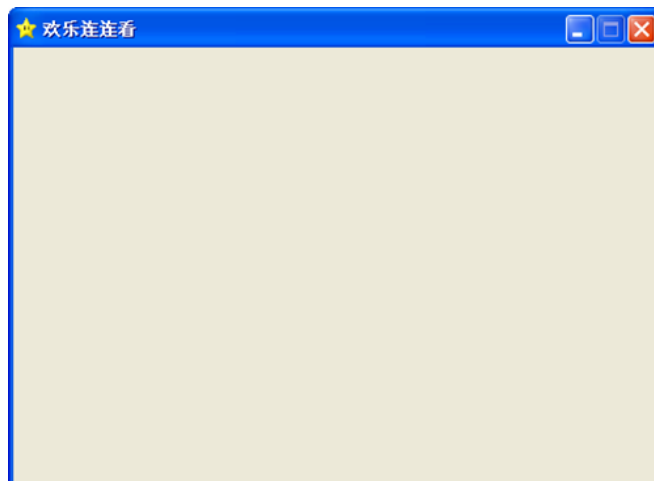


图 10-15 游戏主窗口

1、创建工程

- (1) 开发工具：Microsoft Visual Studio 2010
- (2) 解决方案：Lianliankan
- (3) 工程类型：MFC 对话框(Dialog)工程
- (4) 项目名称：欢乐连连看
- (5) 工程名称：LLK
- (6) 主对话框：CLLKDlg

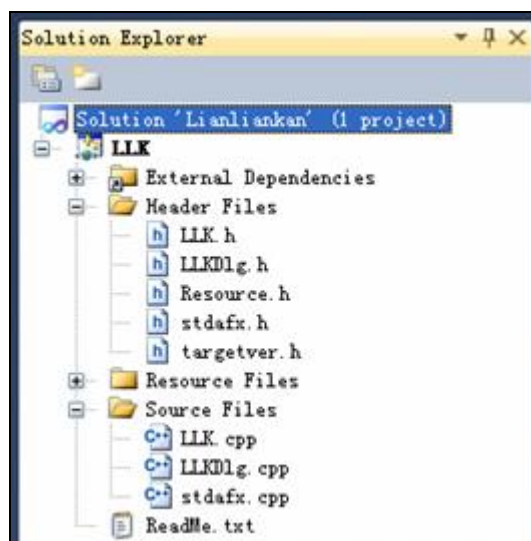


图 10-16 解决方案目录

2、游戏主窗口

- (1) 窗口标题：欢乐连连看。
- (2) 标题栏按钮：最小化按钮、关闭按钮。最大化与还原按钮不可使用。
- (3) 对话框图标：自定义图标，ico 文件。



图 10-17 程序图标

10.4.2 设计思路

选用 Microsoft Visual Studio 2010 作为开发工具，简称 VS2010。选择 MFC 对话框工程作为游戏工程。

10.4.3 编码实现

实现步骤如下：

- 步骤一：创建解决方案。
- 步骤二：创建工程。
- 步骤三：修改主界面对话框属性。

1、创建解决方案

- (1) 选择“开始 -> 程序 -> Microsoft Visual Studio 2010 -> Microsoft Visual Studio 2010”，打开 VS2010。
- (2) 在 VS2010 开发工具中选“File -> New -> Project”菜单,出现新建对话框。
- (3) 在新建对话框中，选择解决方案类型为“Other Project Type -> Visual Studio Solutions -> Blank Solution”,解决方案名为“Lianliankan”，保存路径。

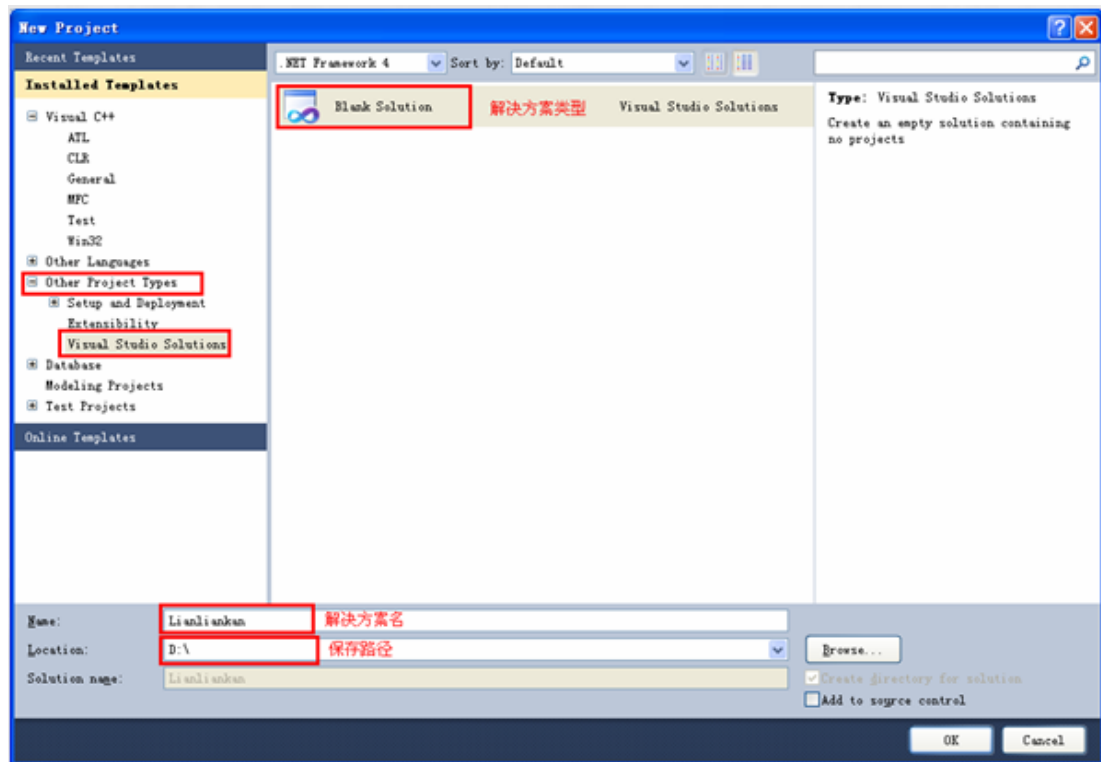


图 10-18 创建解决方案

(4) 创建完成后，解决方案保存路径中，生成解决方案文件夹，在解决方案文件夹中，生成解决方案文件(.sln 后缀)。



图 10-19 生成解决方案的文件

2、创建工程

(1) 创建解决方案之后，选择“File -> New -> Project”，显示新建对话框。

(2) 选择工程类型为“Visual C++ -> MFC -> MFC Application”，输入工程名称 LLK，选择“Solution”为 “Add to solution”，点击“OK”，进入应用程序向导。

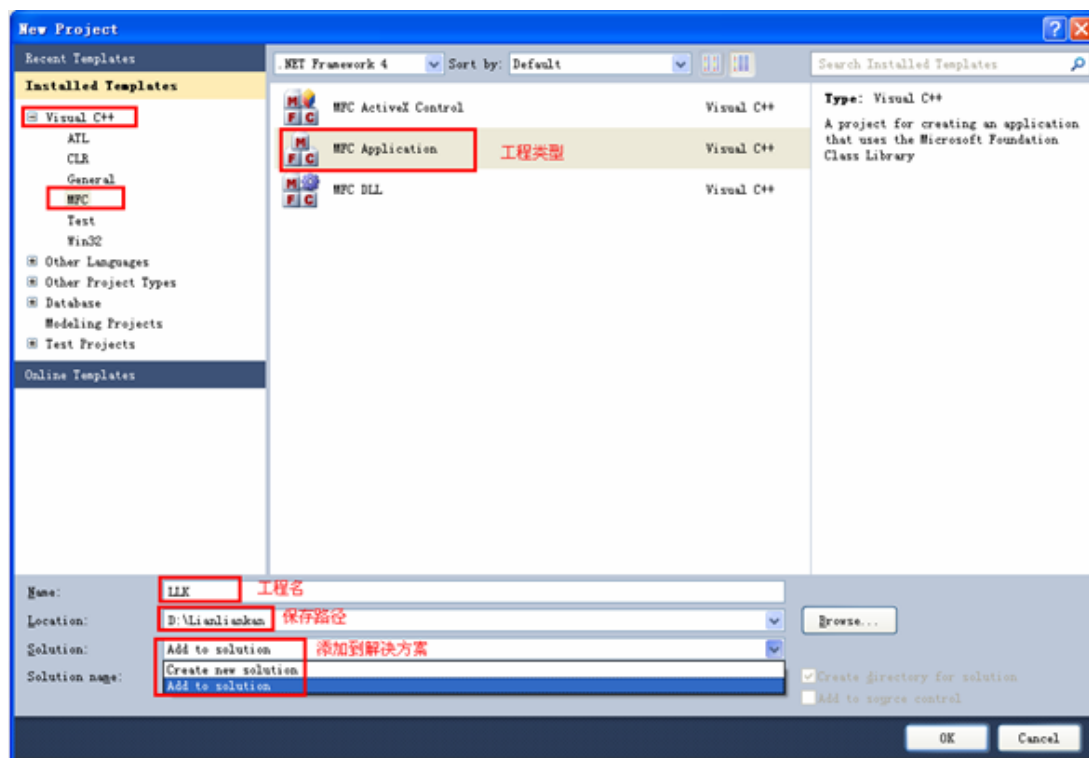


图 10-20 创建工程

(3) 选择应用程序类型

在应用程序向导的“Application Type”中，选择应用程序类型为“Dialog based”。然后点击“Next”进入下一步。

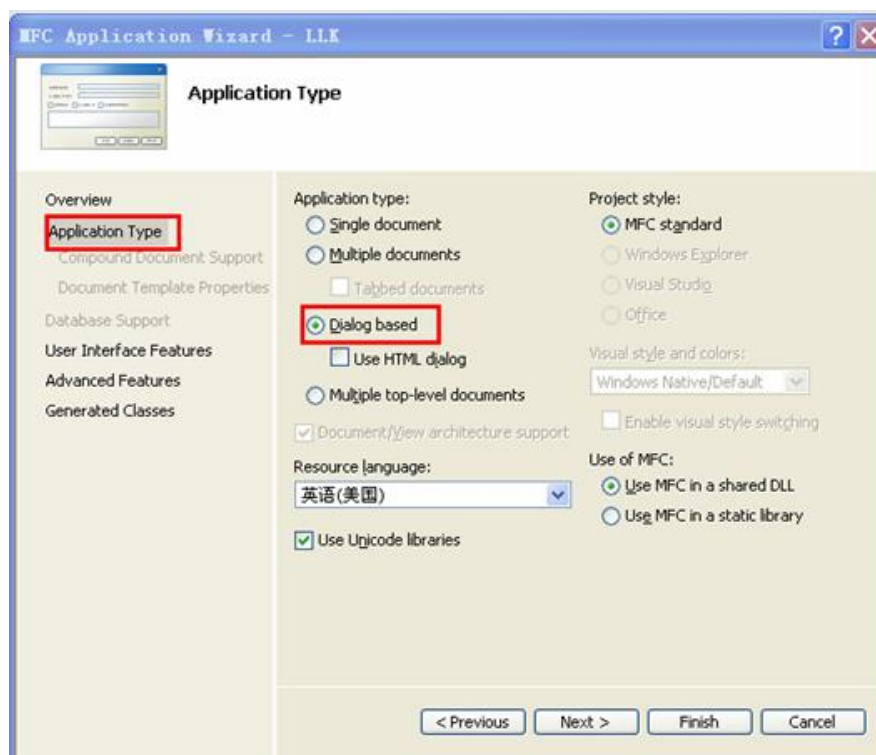


图 10-21 选择应用类型

(4) 在“User Interface Features”中，勾选“Minimize box”，给对话框窗口添加一个最小化按钮。然后点击“Finish”完成工程的创建。

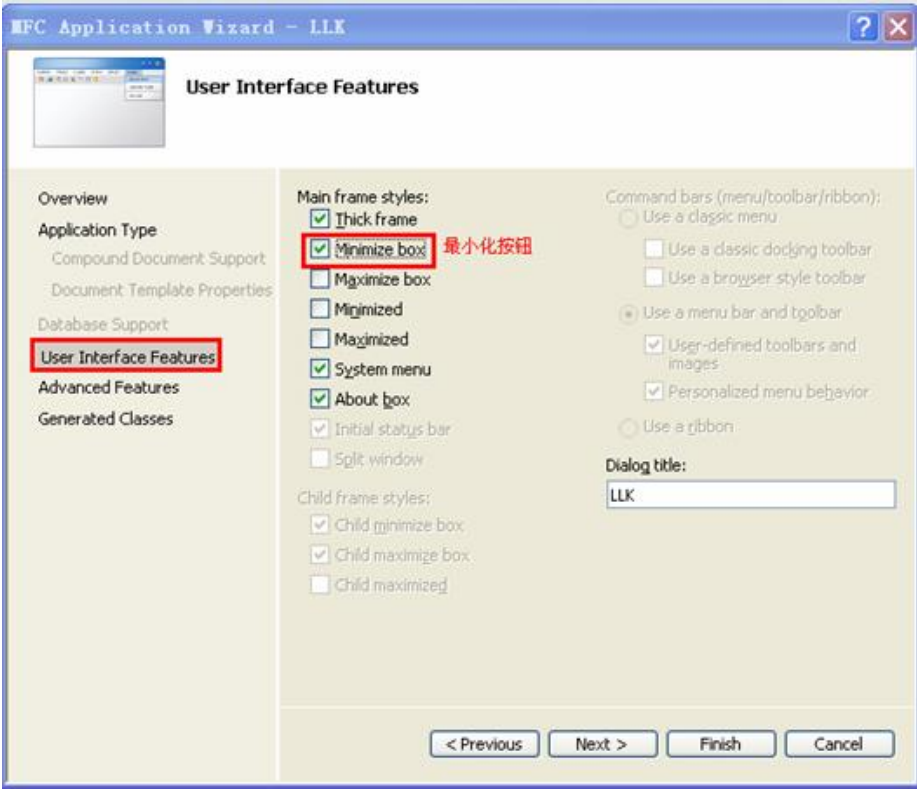


图 10-22 添加最小化按钮

(5) 创建完成后，在工程中可以看到如下内容。

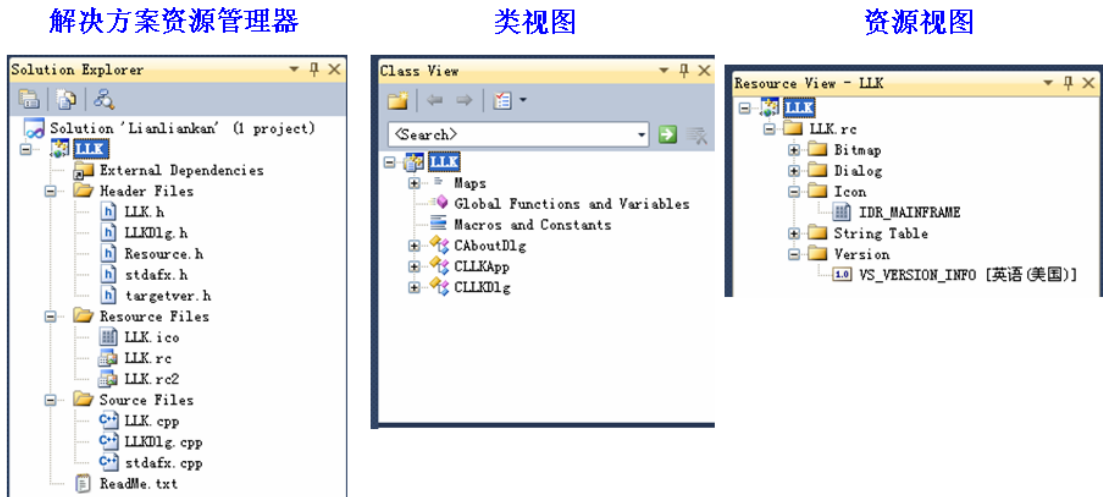


图 10-23 程序的三种视图

(6) 在工程保存路径下可以看到如下文件：

1) 解决方案文件夹



图 10-24 解决方案文件夹

2) 工程文件夹

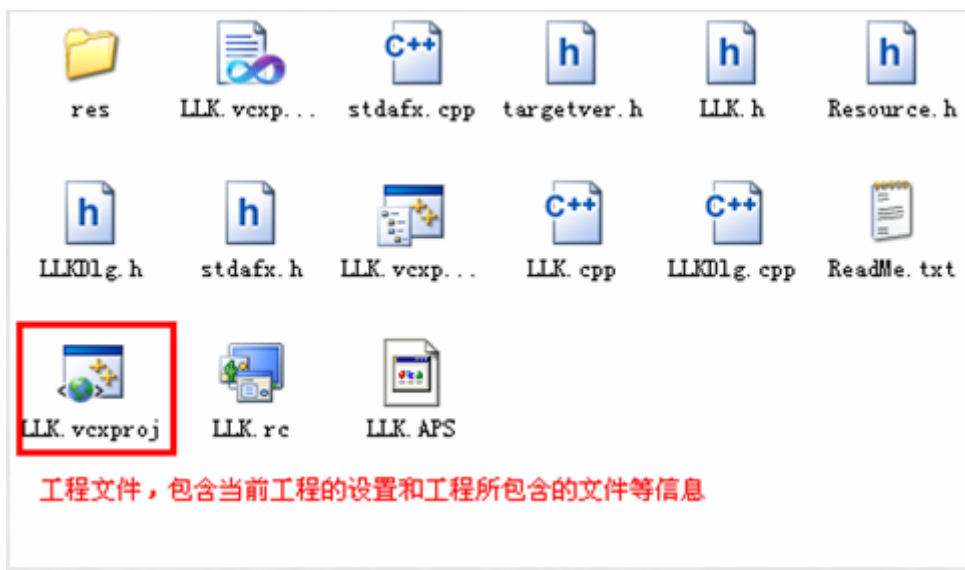


图 10-25 工程文件夹

(7) 编译并运行程序。

使用 VS2010 应用程序向导创建好工程之后，也就完成的代码的编辑工作。现在可以对代码进行编译、连接和运行。

1) 编译程序(F7)

在 VS2010 中，将编译和连接的功能放到一起，直接编译就可以产生可执行文件。

编译后，产生可执行文件。

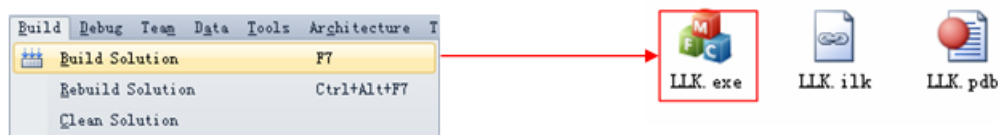


图 10-26 产生可执行文件

2) 运行程序(ctrl+F5)

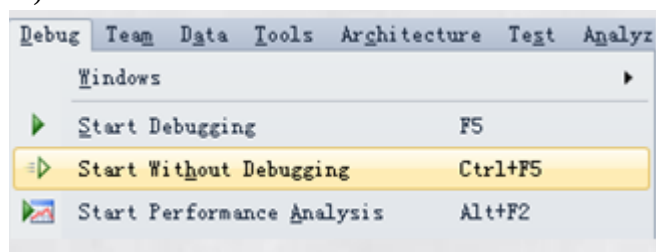


图 10-27 运行程序

3、修改主界面对话框属性

(1) 打开主界面对话框资源

方法一：选择主界面对话框类 CLLKDlg，右键选择“Go To Dialog”，打开主界面对话框

资源。

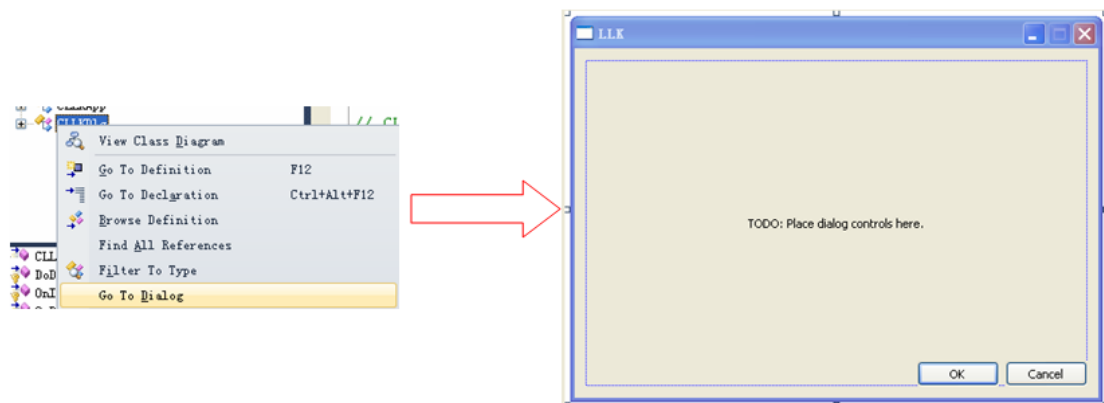


图 10-28 打开对话框方法一

方法二：在资源视图，双击主界面对话栏资源。打开对话框资源。

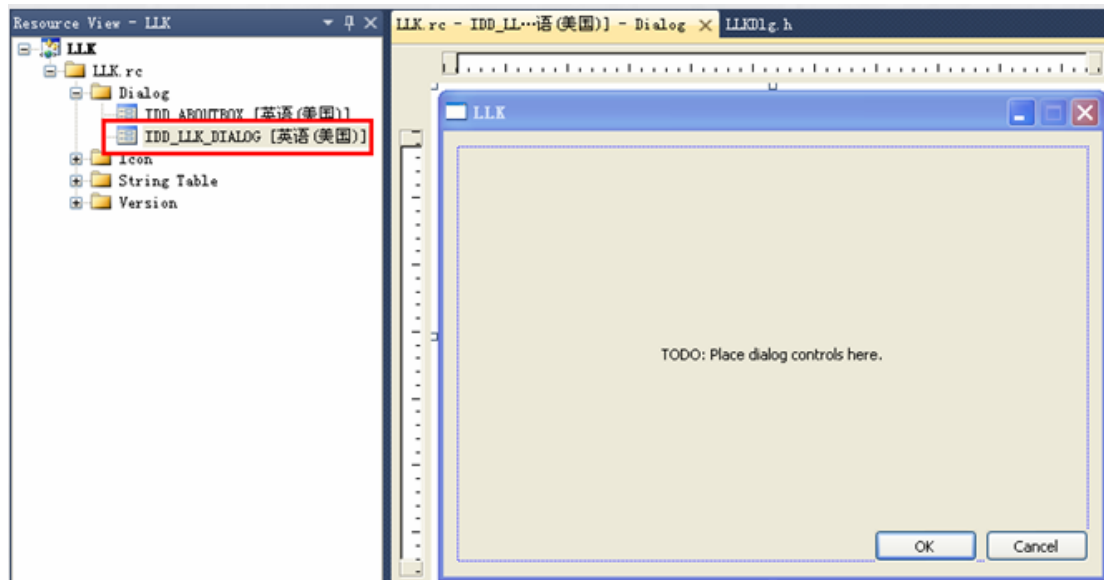


图 10-29 打开对话框方法二

(2) 修改对话框标题为“欢乐连连看”。

- 1) 在对话框编辑器中，删除对话框资源中默认产生的控件。
- 2) 在对话框资源上右键，选择“Properties”，打开对话框属性编辑器。
- 3) 在对话框属性编辑器中修改对话框标题栏为“欢乐连连看”。
- 4) 编译并运行程序。

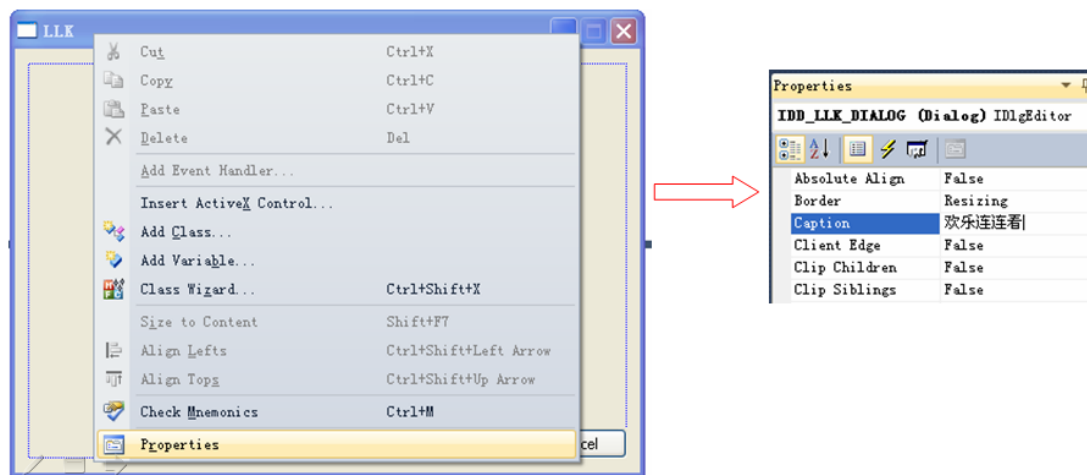


图 10-30 修改标题的运行结果

(3) 修改对话框图标。

1) 在工程目录 res 文件夹中，找到对话框图标“LLK.ico”。将需要设置为对话框图片的 ico 文件命名为“LLK.ico”，替换工程目录 res 中默认的 LLK.ico 文件。



图 10-31 修改图标

2) 编译并运行程序。

由于修改了资源文件，必须要先把原来编译的文件清除后，全部重新编译才行。否则 VS 中默认是增量编译的，已编译的内容不会重新编译。只替换了图标的文件，工具并不会重新编译图标。

10.5 主界面

10.5.1 功能需求

设计“欢乐连连看”项目的主界面，在主界面上添加一个背景图片，并在适当的地方添加“基本模式”、“休息模式”、“关卡模式”、“帮助”、“排行榜”与“设置”按钮。

1、主界面为启动游戏时出现的界面。



图 10-32 主界面效果图

2、主界面布局

- (1) 主界面客户区大小：800*600，单位像素。
- (2) 背景图片：背景图片大小为 800*600，背景图片看起来欢快、轻松。



图 10-33 位图格式的背景图片

- (3) 模式选择按钮：基本模式、休息模式、关卡模式。
- (4) 辅助功能按钮：排行榜、设置、帮助。
- (5) 界面要求：界面协调、美观、符合大部分玩家操作习惯。

10.5.2 设计思路

在“创建工程”的基础上进行迭代开发。

使用 VS2010 工具中对话框编辑器，布置主界面，窗口的背景图一个位图文件，使用 MFC 中的 GDI 技术来给窗口添加一个背景。

1、背景图片绘制步骤

(1) 背景图片的导入

在工程的资源视图中，导入背景图片，背景图片资源 ID 为 IDB_MAIN_BG。

(2) 背景图片的显示

1) 在 `CLLKDlg` 类中，添加一个初始化背景的方法 `void InitBackground()` 函数，将位图选入位图内存。

2) 在 `CLLKDlg::OnInitDialog()` 函数中，调用 `InitBackground()` 函数。在对话框启动时，将位图选进位图内存。

3) 在主界面窗口的 `WM_PAINT` 消息响应函数 `CLLKDlg::OnPaint()` 函数中，将选入位图内存中的图片选到视频内存中，然后调用 `CDC::BitBlt()` 函数，将位图显示在主界面上。

2、CLLKDlg 类设计

(1) 属性

`CDC m_dcMem`; 表示内存 DC，访问权限为 `protected`。

(2) 方法

1) `void InitBackground()`

将背景图片位图资源保存到位图内存中。

2) `void Paint()`

对话框 `WM_PAINT` 消息响应函数，将位图内存中的背景图片绘制到对话框中。

3) `BOOL OnInitDialog()`

对话框初始化函数，在对话框初始化时，调用 `InitBackground()` 函数，加载位图资源到位图资源中。

10.5.3 编码实现

导入“创建工程”的解决方案，在此基础上进行迭代开发，实现步骤如下：

步骤一：位图导入。

步骤二：绘制窗口背景。

步骤三：添加主界面的功能按钮。

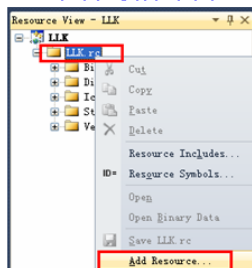
1、位图导入

(1) 将位图资源文件放到物理磁盘工程目录下的 `res` 文件夹。

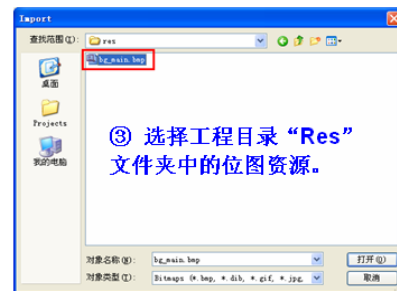
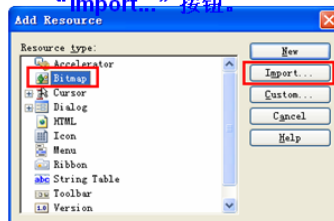
(2) 将位图资源导入到工程中。

(3) 修改位图资源为 `IDB_MAIN_BG`。

① 在资源视图中，选择工程名，右键菜单选择“Add Resource...”菜单项。显示添加资源对话框。



② 添加资源对话框中，选择资源类型为“Bitmap”。点击“Import...”按钮。



③ 选择工程目录“Res”文件夹中的位图资源。

图 10-34 位图导入

2、绘制窗口背景

(1) 创建一个内存 DC

给 CLLKDlg 类添加一个 protected 属性 CDC m_dcMem，表示位图内存。

(2) 在 CLLKDlg 类添加 void InitBackground()函数。

```
void CLLKDlg::InitBackground(void)
{
    //.....
}
```

(3) 加载位图

CBitmap 类封装了 Windows 的图形设备(GDI)中的位图，并且提供了操纵位图的成员函数。静态位图的加载：

- 1) 定义 CBitmap 位图对象。
- 2) 调用 CBitmap::LoadBitmap()函数加载位图资源。
- 3) 将位图选入到内存 DC 中。

加载背景图片位图代码如下：

```
CBitmap bmpMain;
bmpMain.LoadBitmapW(IDB_MAIN_BG);
```

(4) 创建兼容 DC

在对话框的非 OnPaint()函数中，可以使用 CClientDC 类来得到当前对话框的视频内存 DC。通过调用 CDC::CreateCompatibleDC()函数来创建一个与视频 DC 兼容的内存 DC (m_dcMem)，最后，将位图对象选入 (CDC::SelectObject) 到内存 DC 中。

```
void CLLKDlg::InitBackground(void)
{
    // 获得当前对话框的视频内存
    CClientDC dc(this);
    //.....
    // 创建与视频内存兼容的内存 DC
    m_dcMem.CreateCompatibleDC(&dc);
    // 将位图资源选入 DC
    m_dcMem.SelectObject(bmpMain);
}
```



```
//.....
}
```

(5) 在 CLLKDlg::OnInitDialog()函数中，调用 InitBackground()函数。

(6) 调用 CDC::BitBlt()函数。

在 CLLKDlg::OnPaint()函数中，调用 CDC::BitBlt()函数从位图内存中拷贝到视频内存中，进行显示。

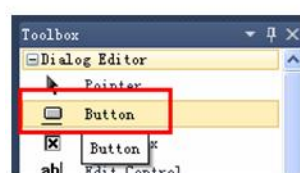
```
void CLLKDlg::OnPaint()
{
    if (IsIconic())
    {
        //.....
    }
    else
    {
        // 创建 CPaintDC 对象
        CPaintDC dc(this);
        // 绘制背景图片
        dc.BitBlt(0, 0, 800, 600, &m_dcMem, 0, 0, SRCCOPY);

        CDialogEx::OnPaint();
    }
}
```

3、添加主界面的功能按钮

(1) 给界面添加控件

在对话框编辑器中，将背景图片导入，调整对话框的大小。从 Toolbox 中将按钮控件添加到对话框中合适的位置。



VS工具的Toolbox中，有界面布局所需要用到的所有控件。用鼠标选中控件并拖到对话框中，即可将控件添加到对话框中。



图 10-35 添加功能按钮

(2) 修改按钮文本(Caption)和 ID。

鼠标右键选择控件，选择“Properties”，可以打开属性编辑器。在属性编辑器中可以查看和修改控件属性。

控件 ID 由大小字母和下划线组成，切忌用中文。

ID	Caption
IDC_BTN_BASIC	基本模式
IDC_BTN_RELAX	休息模式
IDC_BTN_LEVEL	关卡模式
IDC_BTN_RANK	排行榜
IDC_BTN_SETTING	设置
IDC_BTN_HELP	帮助

(3) 编译运行，调整对话框大小和控制位置。

(4) 设置主界面客户区的大小。

通过调用 MoveWindow()函数可以设置窗口的位置和大小。在 InitBackground 函数中添加如下代码，以调整窗口的大小。

```
void CLLKDlg::InitBackground(void)
{
    //.....
    // 调整窗口大小
    CRect rtWin;
    CRect rtClient;
    this->GetWindowRect(rtWin);           // 获得窗口大小
    this->GetClientRect(rtClient);        // 获得客户区大小
    // 标题栏和外边框的大小
    int nSpanWidth = rtWin.Width() - rtClient.Width();
    int nSpanHeight = rtWin.Height() - rtClient.Height();

    // 设置窗口大小
    MoveWindow(0, 0, 800 + nSpanWidth, 600 + nSpanHeight);
    //.....
}
```

(5) 调用 CenterWindow()函数，使窗口居中。

10.6 开始游戏

10.6.1 功能需求

当玩家在主界面选择【基本模式】时，出现基本游戏界面，并隐藏主界面。

玩家点击【开始游戏】按钮，生成游戏地图。基本模式的游戏地图为 10 行 16 列，图片的花色 16 个。

1、客户区

- (1) 背景图片。
- (2) 开始游戏按钮、暂停按钮、提示按钮、重排按钮、设置按钮、和帮助按钮。
- (3) 当选择最小化按钮时，将游戏界面最小化到窗口左下角。

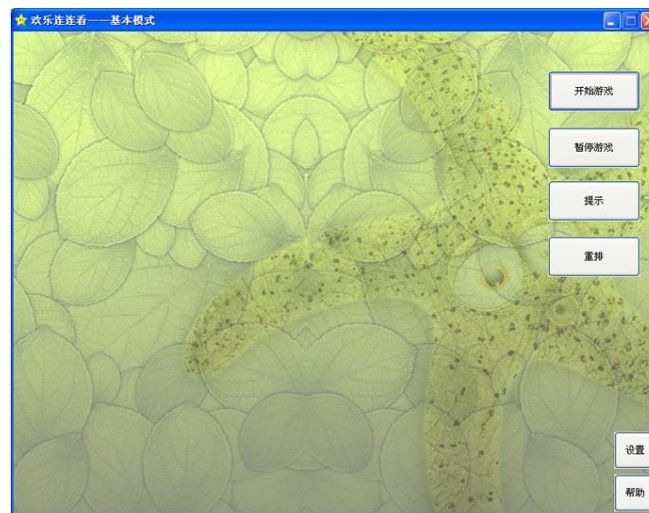


图 10-36 界面效果

2、游戏地图区域

- (1) 游戏地图起始点 (50,50)，单位像素。
- (2) 游戏地图：10 行，16 列。
- (3) 每张图片大小：40*40，单位像素。
- (4) 游戏地图中包含 16 种图片。

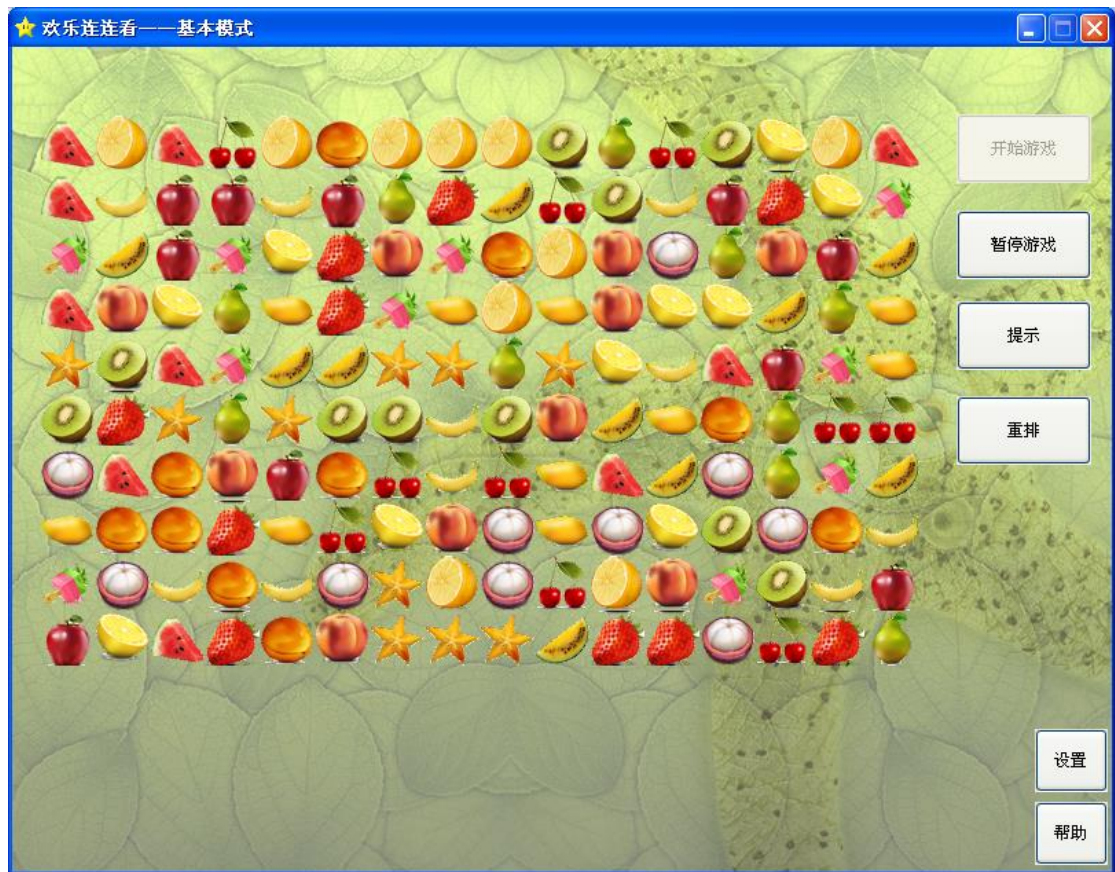


图 10-37 游戏地图

3、游戏元素

(1) 游戏元素如下图所示(图片根据确定的主题自定义)。

- 1) 图片背景为白色。
- 2) 单张图片为 40*40，单位像素。
- 3) 元素图片由单个小图片按顺序拼接得到。

(2) 例如前 6 个图片元素，分别用 0、1、2、3、4、5 来表示这 6 种图片。



图 10-38 游戏元素

(3) 物理磁盘保存路径

图片名：fruit_element.bmp，放在 LLK 工程目录下 theme\picture 文件夹中。图片为一张 40*800 的 BMP 图片，该图片中由 20 种大小为 40*40 的小图片组成。



图 10-39 游戏元素的保存

10.6.2 设计思路

在“主界面”的基础上进行迭代开发。

1、数据设计

添加 global.h 文件，定义结构体 Vertex，用于保存游戏地图中一个点的行号、列表、花色。

2、去除元素背景

(1) 图片在内存中以像素点存储，用 RGB 的方式表示。

RGB 的各项的值范围为：0~255，使用 8 位二进制的 bit 表示，即一个字节。因此，一个像素用 3 个字节来存储。

如，某一像素点颜色的RGB值为(224, 162, 67)
则，在内存中的值为：11000001010001001000011

色调 (H): 24
饱和度 (S): 173
颜色 | 纯色 (C): 亮度 (L): 137

红 (R): 224
绿 (G): 162
蓝 (B): 67

RGB	R								G								B								
十进制	224								162								67								
十六进制	E				0				A				2				4				3				
内存中的值	1	1	1	0	0	0	0	0	1	0	1	0	0	0	1	0	0	1	0	0	0	0	0	1	1

RGB换算

图 10-40 图片存储和表示

(2) 黑色的 RGB 值为 RGB(0, 0, 0)，白色的 RGB 值为 RGB(255, 255, 255)。

黑色	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
白色	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

可以看出，黑色为全0，白色为全1，其它的颜色为0或1的组合
所以图片的数据在内存中实际上就是0与1的组合

图 10-41 黑和白 RGB 值

- (3) BitBlt()函数对图片的操作，实际上是对像素进行位块的操作。
- 常用位操作包括：与(&)、或(|)。
- BitBlt()函数位操作类型由最后一个参数，光栅码来控制。参数意思为：
- 与(SRCAND): $D1 = D1 \& D2$ 。
- 或(SRCPAINT): $D1 = D1 | D2$ 。
- 拷贝(SRCCOPY): $D1 = D2$ 。

A	B	$P = A \& B$
1	0	0
0	0	0
1	1	1
0	1	0

A	B	$P = A B$
1	0	1
0	0	0
1	1	1
0	1	1

图 10-42 位操作

- (4) 利用掩码消除图片背景的原理

	显示DC m_dcMem D1	掩码DC m_dcMask D2	显示DC m_dcMem $D1 = D1 D2$	元素DC n_dcElement D3	显示DC m_dcMem $D1 = D1 \& D3$
图片的白边区域	X0	0	X0	1	X0
图片元素区域	X0	1	1	X	X

X0代表0或1，背景图
X代表0或1，显示元素图

图 10-43 消除图片背景

3、类设计

(1) CLLKDlg 类

CLLKDlg 类为主界面对话框类。在该类中添加一个“基本模式”按钮 BN_CLICKED 消息响应函数 CLLKDlg::OnBnClickedBtnBasic()。创建并显示游戏界面对话框，并控制主界面的隐藏和显示。

(2) CGameDlg 类

显示游戏界面，并与用户交换信息，不关心数据是以何种结构存储到内存中。

CGameDlg 类为游戏界面对话框类。

1) 数据成员

成员变量	描述
CDC m_dcMem	内存 DC
CDC m_dcBG	背景 DC
CDC m_dcElement	元素内存 DC
CDC m_dcMask	掩码内存 DC
CPoint m_ptGameTop	游戏区起始点（游戏第一张图片的顶点坐标, 坐标相对于窗口客户区）
CSize m_sizeElem	元素图片的大小
CRect m_rtGameRect	游戏区域大小(该区域为游戏更新区域大小, 考虑到后面画的线, 可能会超出图片区域一定范围)
CGameControl m_GameC	游戏控制类

2) 成员函数

成员函数	描述
virtual BOOL OnInitDialog()	窗口初始化函数
afx_msg void OnPaint()	WM_PAINT 消息函数
afx_msg void OnBnClickedBtnStart()	开始按钮
void InitBackground(void)	初始化窗口背景
void InitElement(void)	初始化元素图片与 DC
void UpdateWindow(void)	更新界面
void UpdateMap(void);	更新游戏地图

(3) CGameControl 类

保存游戏数据, 统一协调和调用各逻辑类完成相应的功能。

1) 静态成员变量

静态成员变量	描述
static int s_nRows	游戏行数
static int s_nCols	游戏列数
static int s_nPicNum	图片数

2) 数据成员

数据成员	描述
int** m_pGameMap	游戏地图数组指针
CGameLogic m_GameLogic	游戏逻辑操作对象

3) 成员函数

成员函数	描述
void StartGame(void)	开始游戏
int GetElement(int nRow, int nCol)	得到某一个元素

(4) CGameLogic 类

根据不同的数据结构，完成数据逻辑的判断功能。

成员函数	描述
int** InitMap()	初始化游戏地图
void ReleaseMap(int** &pGameMap)	释放地图

(5) CGameException 类

游戏异常类，处理程序中的一些异常。程序中类是层层调用的，当下一层出现异常，则可以抛出一个异常对象。在上一层，则使用 try{}catch{} 语句来捕获异常。

10.6.3 编码实现

导入“主界面”的解决方案，在此基础上进行迭代开发，实现步骤如下：

步骤一：添加游戏对话框资源。

步骤二：创建并显示游戏对话框。

步骤三：绘制游戏界面背景。

步骤四：游戏界面布局。

步骤五：加载游戏元素图片。

步骤六：绘制游戏地图。

1、添加游戏对话框资源

在资源视图中，右键选择“Dialog”，在弹出菜单中选择“Insert Dialog”菜单项。即可添加一个对话框资源。

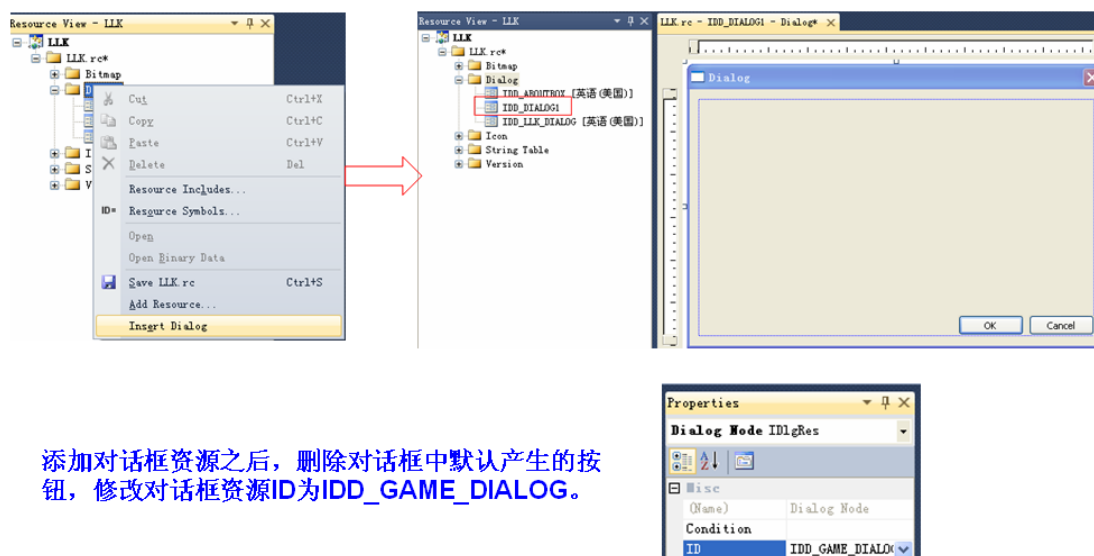


图 10-44 添加游戏对话框资源

2、创建并显示游戏对话框

(1) 添加游戏界面对话框类 CGameDlg。

1) 双击对话框资源，弹出“MFC Add Class Wizard”对话框。

- 2) 在对话框中输入对话框类的类名为“CGameDlg”。点击“Finish”，完成添加。
- 3) 添加完成后，在“Class View”可以看到添加 CGameDlg 类，在“Solution Explorer”可以看到 CGameDlg 类的源文件和头文件。

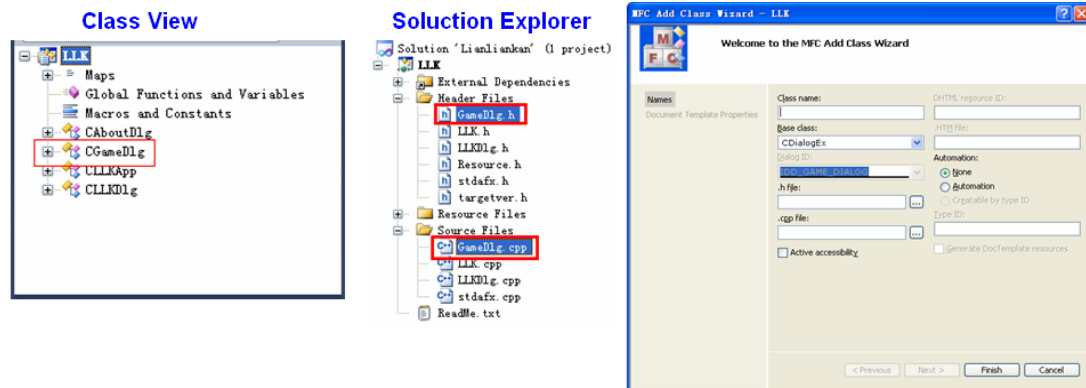


图 10-45 创建游戏对话框类

在 MFC 中，对话框资源和对话框类是通过对话框的资源 ID 进行关联的，每个对话框类都有一个对应的对话框资源。

(2) 创建并显示游戏对话框

- 1) 使用“Class Wizard”，给主界面对话框中的“基本模式”按钮(IDC_BTN_BASIC)，添加 BN_CLICKED 消息响应函数 OnBnClickedBtnBasic()。

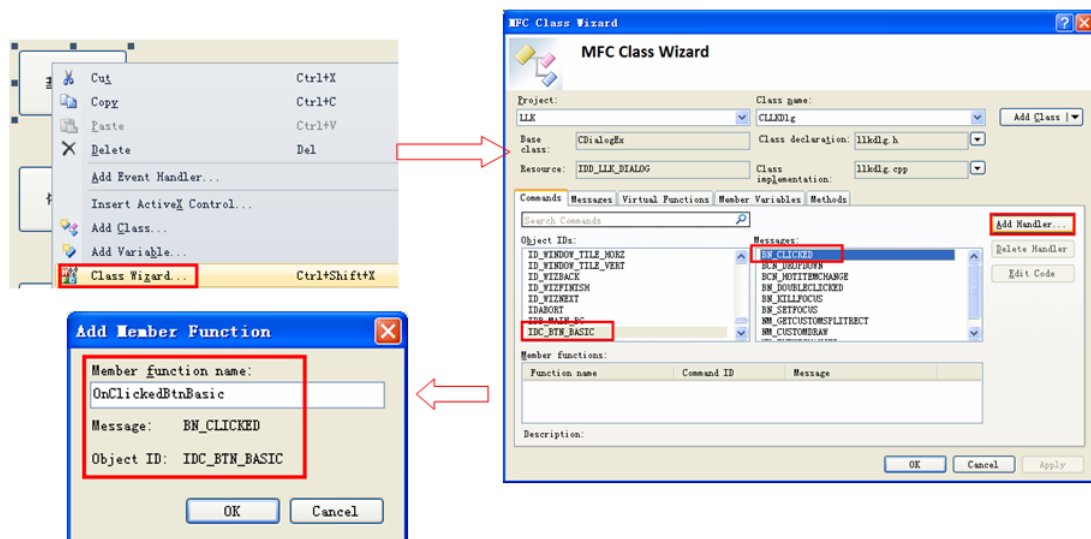


图 10-46 运行程序

- 2) 在 CLLKDlg::OnBnClickedBtnBasic()函数中，调用 CGameDlg::DoModal()函数创建并显示基本游戏对话框。

```
void CLLKDlg::OnBnClickedBtnBasic()
{
    CGameDlg dlg;
    dlg.DoModal();
}
```

```
}
```

3) 控制主界面的隐藏与显示

在 `CLLKDlg::OnBnClickedBtnBasic()` 函数中, 调用 `CWnd::ShowWindow()` 函数, 来控制主界面的隐藏和显示。

3、绘制游戏界面背景

(1) 加载游戏界面背景图片

1) 位图的加载方式

方式一：静态方式，位图直接导入到工程中。

如在“主界面设计”中，由于主界面的背景不会改变，这时，就可以使用这一种方式，直接将要显示的位图导入到工程中，在工程中，通过资源 ID 加载位图。

优点：可直接使用 `CBitmap` 类的 `LoadBitmap()` 函数完成。缺点：图片是不可修改的。

方式二：动态方式，通过传入的位图文件名加载位图。

由于“欢乐连连看”游戏，是可以更换主题的，即背景与元素图片都是可以在程序运行时修改的，这样，使用静态的方式就会很不灵活，也无法实现用户自定义。

优点：使用程序很灵活，可以通过修改位图文件名，改变加载的位图。缺点：需要用到系统 API 函数 `LoadImageW()` 函数来实现，程序会复杂一点。

2) 采用动态方式加载位图

在工程目录下新建一个“theme”文件夹，将游戏背景图片放到该文件夹中的“picture”文件夹中。

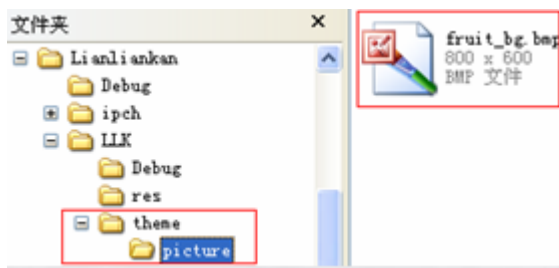


图 10-47 放置背景图片

给 `CGameDlg` 类添加成员函数 `void InitBackground()`，在该函数中调用 Windows API 函数 `LoadImage()` 加载位图。

(2) 将图片选入位图内存。

在对话框初始化时，将位图保存到位图内存中。给 `CGameDlg` 类添加 `OnInitDialog()` 函数，在 `OnInitDialog()` 函数中，调用 `InitBackground()` 函数。

1) 右键选择“`CGameDlg`”类，在弹出菜单中选择“Class Wizard”。

2) 在“Class Wizard”对话框中，选择“Virtual Functions”标签页，选择添加的函数为“`OnInitDialog`”。

3) 点击“Add Function”，完成 `OnInitDialog()` 函数的添加。

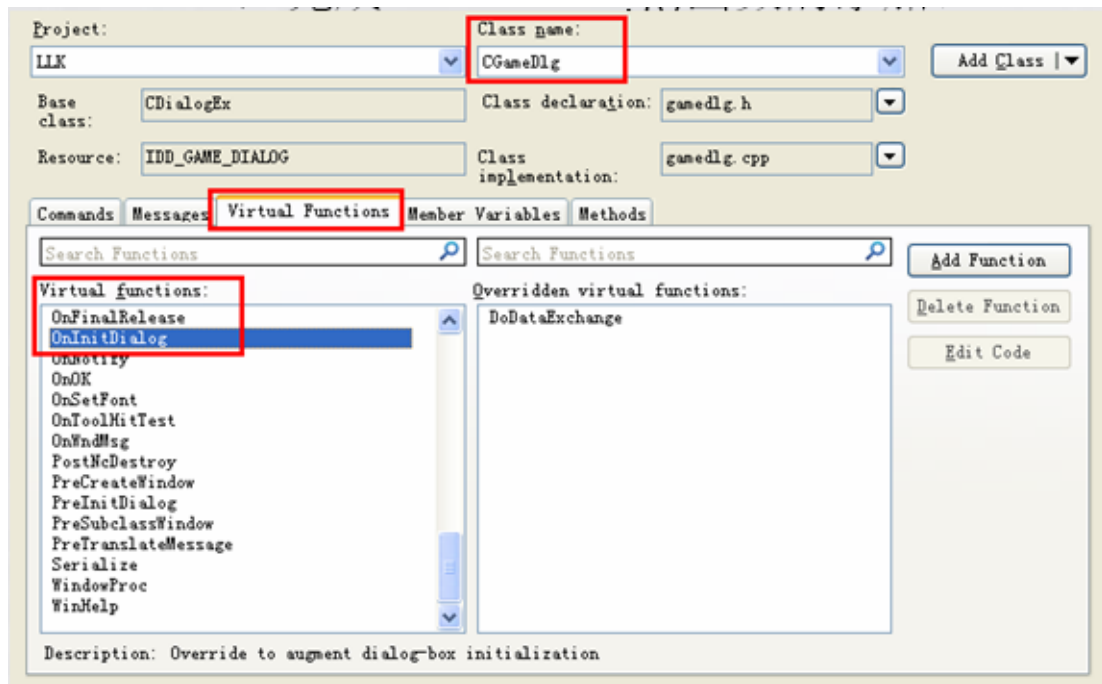


图 10-48 添加 OnInitDialog 函数

(3) 将图片从位图内存拷贝到视频内存。

给 CGameDlg 类添加 OnPaint()函数，在绘制界面时，从位图内存中拷贝位图到视频内存，进行显示。

1) 打开 CGameDlg 类的 Class Wizard。

2) 在 Class Wizard 中选择“Messages”标签页，选择“WM_PAINT”消息。点击“Add Handler”按钮，完成 WM_PAINT 消息响应函数 OnPaint()的添加。

3) 在 CGameDlg::OnPaint()函数中，调用 CPaintDC::BitBlt()函数，绘制游戏界面背景。

```
void CGameDlg::OnPaint()
{
    CPaintDC dc(this); // device context for painting

    // 绘制背景图片
    dc.BitBlt(0, 0, GAMEWND_WIDTH, GAMEWND_HEIGHT, &m_dcMem, 0, 0,
    SRCCOPY);
}
```

(4) 添加 CGameDlg::UpdateWindow()函数，调整游戏窗口大小。

设置游戏界面客户区大小为 800*600，在 OnInitDialog()中调用 UpdateWindow()函数。

```
void CGameDlg::UpdateWindow(void)
{
    // 调整窗口大小
    CRect rtWin;
    CRect rtClient;
```

```

this->GetWindowRect(rtWin);           // 获得窗口大小
this->GetClientRect(rtClient);        // 获得客户区大小

// 标题栏和外边框的大小
int nSpanWidth = rtWin.Width() - rtClient.Width();
int nSpanHeight = rtWin.Height() - rtClient.Height();

// 设置窗口大小
MoveWindow(0, 0, GAMEWND_WIDTH + nSpanWidth, GAMEWND_HEIGHT +
nSpanHeight);

// 设置对话框显示是，在 windows 窗口正中央。
CenterWindow();
}

```

4、游戏界面布局

(1) 设置游戏界面对话框标题

在 CGameDlg::OnInitDialog()函数中，设置对话框的标题为“欢乐连连看——基本模式”。

(2) 设置游戏界面对话框图标

(3) 添加控件

在对话框中添加控件，并调整按钮在对话框中的位置。

5、加载游戏元素图片

(1) 将游戏元素图片加载到程序中

1) 给 CGameDlg 类添加成员函数 CGameDlg::InitElement(void)，访问权限为 protected。

2) 调用::LoadImageW()函数，将游戏元素图片加载到程序中。

3) 创建元素图片的位图内存 DC。

CGameDlg 类添加 CDC 数据成员 m_dcElement，m_dcElement 与视频 DC 兼容。

4) 将位图资源选入创建的 DC。

5) 在对话框初始化时，调用 InitElement()函数。

```

void CGameDlg::InitElement(void)
{
    // 获得当前对话框的视频内存
    CClientDC dc(this);

    // 加载 BMP 图片资源
    HANDLE hBmp = ::LoadImageW(NULL, _T("theme\\picture\\fruit_element.bmp"),

```

```

IMAGE_BITMAP, 0, 0, LR_LOADFROMFILE);

// 创建与视频内存兼容的内存 DC
m_dcElement.CreateCompatibleDC(&dc);
// 将位图资源选入 DC
m_dcElement.SelectObject(hBmp);

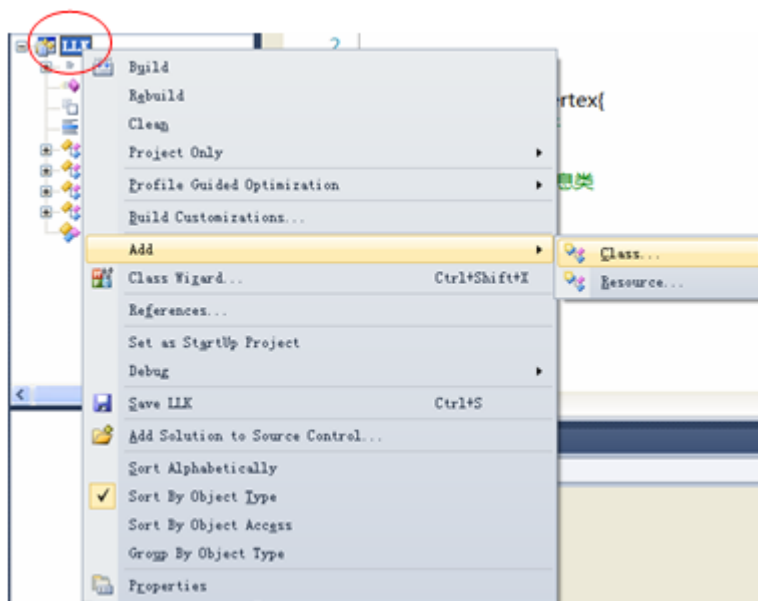
// 加载掩码 BMP 图片资源
HANDLE hMask = ::LoadImageW(NULL, _T("theme\\picture\\fruit_mask.bmp"),
IMAGE_BITMAP, 0, 0, LR_LOADFROMFILE);

// 创建与视频内存兼容的内存 DC
m_dcMask.CreateCompatibleDC(&dc);
// 将位图资源选入 DC
m_dcMask.SelectObject(hMask);
}

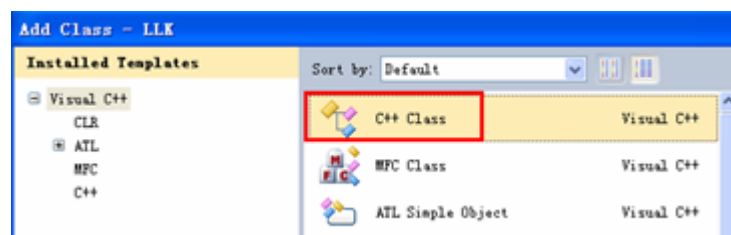
```

(2) 添加 CGameLogic 类

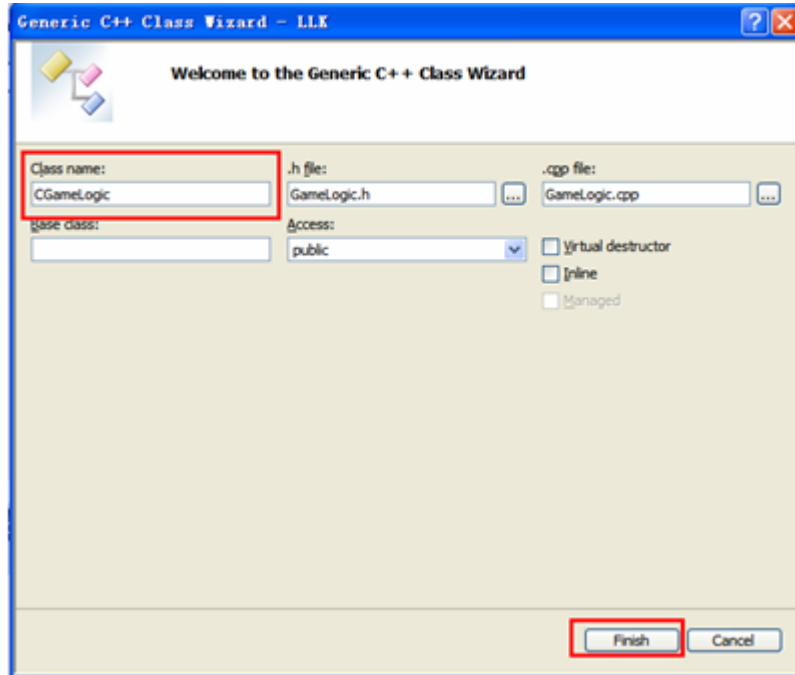
1) 切换到类视图，在工程上右击，在弹出菜单中选择“Add” -> “Class”。



2) 选择类的类型：C++ Class，点击“Add”。



3) 在 Class name 中输入类名：CGameLogic，点击“Finish”，完成类的创建。



(3) 在 CGameLogic 类中添加初始化游戏地图函数

1) 获取地图花色和大小

```
int** CGameLogic::InitMap()
{
    // 获取地图大小和花色
    int nRows = CGameControl::s_nRows;
    int nCols = CGameControl::s_nCols;
    int nPicNum = CGameControl::s_nPicNum;
}
```

2) 为游戏地图开辟内存空间

```
int** CGameLogic::InitMap()
{
    //.....
    // 开辟内存区域
    int** pGameMap = new int*[nRows];
    if(NULL == pGameMap)
    {
        throw new CGameException(_T("内存操作异常！ \n"));
    }
    else
    {
        for (int i = 0; i < nRows; i++)
        {
            pGameMap[i] = new int[nCols];
            if(NULL == pGameMap)

```

```

        {
            throw new CGameException(_T("内存操作异常！ \n"));
        }
        memset(pGameMap[i], NULL, sizeof(int) * nCols);
    }
}
}

```

3) 根据花色的种类计算出每种花色的图片的平均个数，依次给数组赋值。

```

int** CGameLogic::InitMap()
{
    //.....
    // 多少花色
    if ((nRows * nCols) % (nPicNum * 2) != 0)
    {
        ReleaseMap(pGameMap);
        throw new CGameException(_T("游戏花色与游戏地图大小不匹配！"));
    }
    int nRepeatNum = nRows * nCols / nPicNum;
    int nCount = 0;
    for(int i = 0; i < nPicNum; i++)
    {
        // 重复数
        for(int j = 0; j < nRepeatNum; j++)
        {
            pGameMap[nCount / nCols][nCount % nCols] = i;
            nCount++;
        }
    }
}
}

```

4) 随机找到两个位置的图片，进行交换。

```

int** CGameLogic::InitMap()
{
    //.....
    // 设置种子
    srand((int)time(NULL));

    // 随机任意交换两个数字
    int nVertexNum = nRows * nCols;
    for(int i = 0; i < nVertexNum; i++)
    {
        // 随机得到两个坐标
        int nIndex1 = rand() % nVertexNum;

```

```

        int nIndex2 = rand() % nVertexNum;

        // 交换两个数值
        int nTmp = pGameMap[nIndex1 / nCols][nIndex1 % nCols];
        pGameMap[nIndex1 / nCols][nIndex1 % nCols] = pGameMap[nIndex2 /
nCols][nIndex2 % nCols];
        pGameMap[nIndex2 / nCols][nIndex2 % nCols] = nTmp;
    }

    return pGameMap;
}

```

(4) 在 CGameLogic 类中创建释放游戏地图函数: void ReleaseMap(int** &pGameMap);

```

void CGameLogic::ReleaseMap(int** &pGameMap)
{
    for (int i = 0; i < CGameControl::s_nRows; i++)
    {
        delete []pGameMap[i];
    }
    delete []pGameMap;
}

```

(5) 调用初始化游戏地图函数，并进行异常处理

1) 添加 CGameControl 类。

2) 在 CGameControl 类中定义游戏地图数组指针: int** m_pGameMap。

3) 在CGameControl类中添加StartGame()函数，调用CGameLogic中的InitMap()函数，初始游戏数组。

(6) 生成地图数据

1) 给“开始游戏”按钮，添加 BN_CLICKED 消息响应函数 OnBnClickedBtnStart()。

2) 在 CGameDlg::OnBnClickedBtnStart()中，调用 CGameControl 类中的 StartGame()初始游戏地图的数据。

6、绘制游戏地图

(1) 调用 CGameControl 类中的 GetElement()获取相应行列位置图片的元素编号值，并将对应编号的图片区域的数据绘制到 m_dcMem 中的相应位置。

1) 制作一张与元素图片大小一致的掩码图片。掩码图片中，原图中白色的背景改为黑色，原图片区域填成白色。

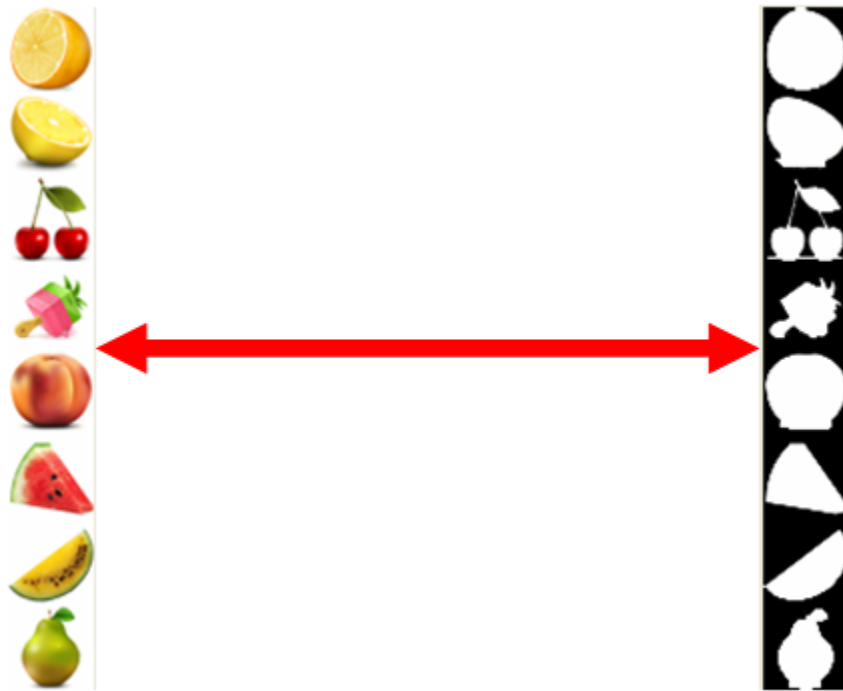


图 10-49 掩码图片

2) 创建元素图片和掩码图片的位图内存 DC。

3) 先将 `m_dcMem` 与 `m_dcMask` 相或(光栅码为: `SRCPAINT`), 再将此时的 `m_dcMem` 与元素 `m_dcElement` 相与(光栅码为: `SRCAND`)。

(2) 游戏地图的起始点为客户区中的(20,50)。游戏地图分为 10 行 16 列, 由 `CGameControl` 类的静态成员变量 `s_nRows` 和 `s_nCols` 得到。每格的大小和元素图片一致, 每个元素大小一致(40*40)。

(3) 在 `CGameDlg` 类中定义 `UpdateMap()`函数, 来绘制游戏界面。

```
void CGameDlg::UpdateMap(void)
{
    // 计算图片的顶点坐标与图片大小
    //.....

    //获取行数和列数
    //.....

    for(int i = 0; i < nRows; i++)
    {
        for(int j = 0; j < nCols; j++)
        {
            // 得到图片编号的值
            int nElemVal = m_GameC.GetElement(i, j);
```

```

// 将背景与掩码相或，边保留，图像区域为 1
m_dcMem.BitBlt(nLeft + j * nElemW, nTop + i * nElemH, nElemW, nElemH,
               &m_dcMask, 0, nElemVal * nElemH, SRCPAINT);

// 与元素图片相与，边保留，图像区域为元素图片
m_dcMem.BitBlt(nLeft + j * nElemW, nTop + i * nElemH, nElemW, nElemH,
               &m_dcElement, 0, nElemVal * nElemH, SRCAND);
    }
}
}

```

(4) 在绘制游戏地图之后，调用 `InvalidateRect()` 函数，更新游戏区域。

(5) 编译运行程序。

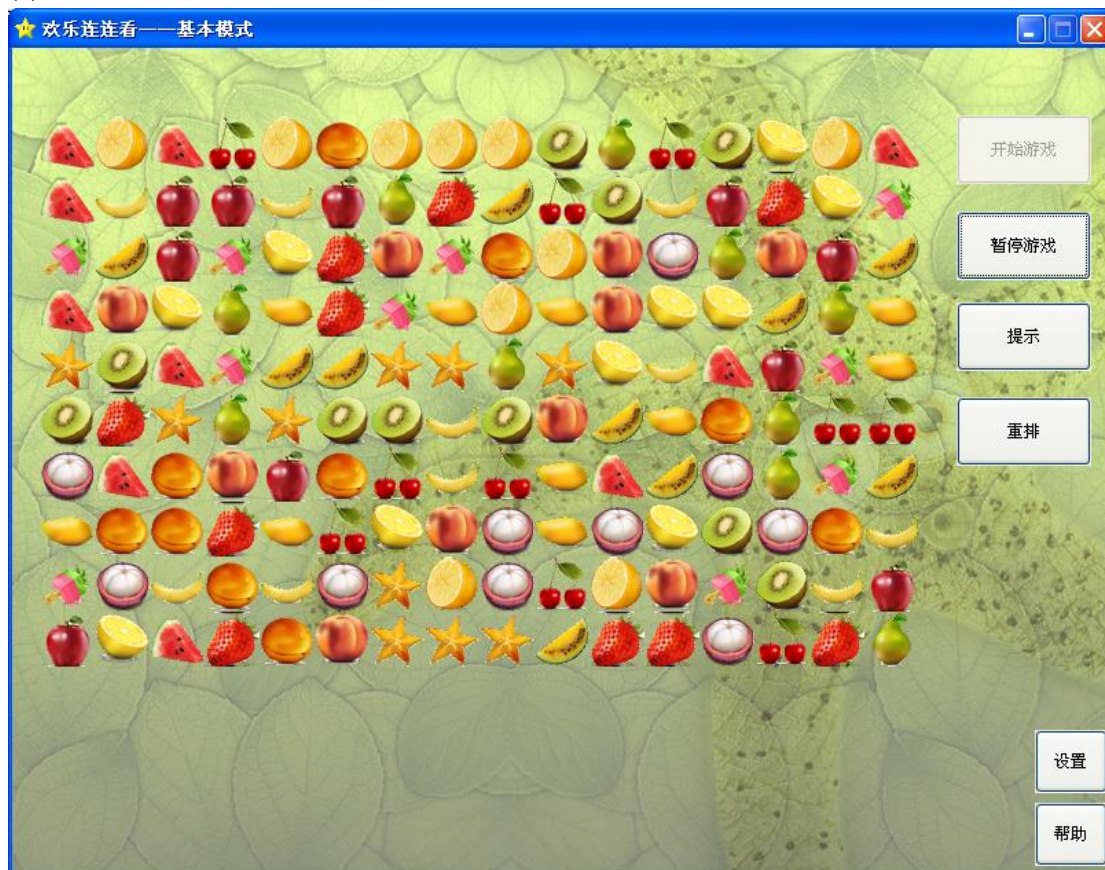


图 10-50 运行结果

10.7 消子判断

10.7.1 功能需求

1、介绍

实现游戏消子功能，玩家选中两个图片，判断是否符合消子规则，如果符合，则消掉

这两个图片。否则保持不变。



图 10-51 消子规则

2、输入

选择的两个图片在游戏地图中的位置坐标、图案编号。

3、处理

(1) 若选中的两张图片满足下面条件，则进行连通判断。

- 1) 选中 2 张图片。
- 2) 选中的 2 张图片元素完全相同。

(2) 除选中的两个顶点外，连通线只能经过游戏地图上没有图片的空白点。

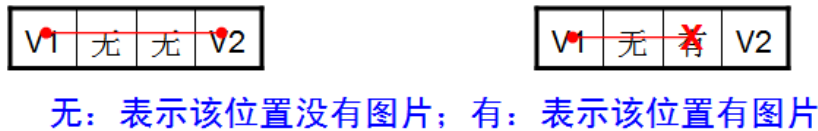


图 10-52 连通判断

(3) 如果有以下三类连通线中满足一种，则可进行消子。

1) 一条直线连通

- 情况一：两图片紧密相邻，中间既没有空白也没有其它种类的图案。
- 情况二：两图片并非紧密相邻，中间没有其它图案，但是有一个或者多个空白。

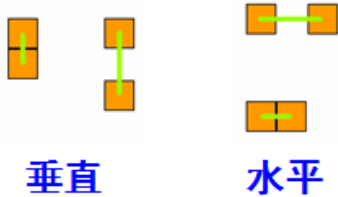
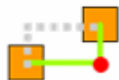


图 10-53 一条直线连通

2) 两条直线连通

两图片既不在同一水平线上，也不在同一垂直线上，两个图片的连通路径有两条直线组成，即有一个折点(图中红色圆点)，两条直线经过的路径必须是空白，中间只要有一个有其它图片，该路径无效，两图片无法连通。

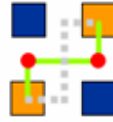


如图中的绿色实线和灰色虚线是两条直线连通的路径。

图 10-54 两条直线连通

3) 三条直线连通

连通路径有三条直线，两个折点组成(图中 2 个红色圆点)，在该直线的路径上没有图片出现，只能是空白区域。



如图中的绿色实线和灰色虚线是三条直线连通的路径。

图 10-55 三条直线连通

(4) 提示框

单击图片会使得图片转变成被选中状态，给图片加一个红色的边框提示玩家。

(5) 消子

图片满足消失条件，显示连通路径(绿色线条)，然后被选中的图片立即消失，位置为空白。

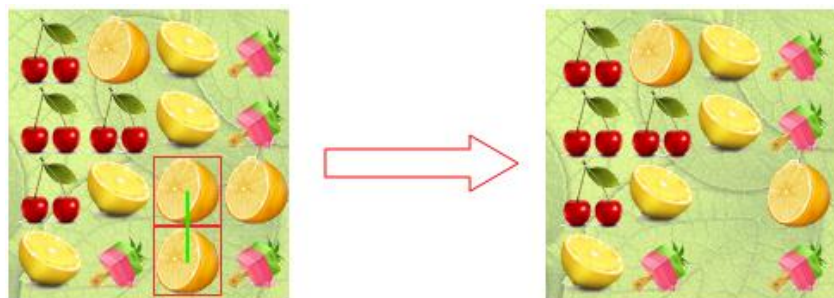


图 10-56 消子

4、输出

最新的游戏地图。

10.7.2 设计思路

在“开始游戏”的基础上进行迭代开发。

1、消子流程

- (1) 获得选中的两张图片的行号与列号。
- (2) 判断图片的有效性。
- (3) 判断连通性，如以下三种情况均不满足，则结束。
 - 1) 首先判断能否一条直线连通。
 - 2) 如果不能一条直线连通，则判断能否两条直线连通。
 - 3) 如果不能两条直线连通，则判断能否三条直线连通。

(4) 获得连通路径，绘制连通线。

(5) 消除图片。

(6) 更新游戏地图。

2、消子算法与连接路径

(1) 消子算法

利用枚举法，分别枚举出三种连通方式的算法：

- 1) 一条直线消子算法。
- 2) 两条直线消子算法。
- 3) 三条直线消子算法。

(2) 连接路径

连接路径为两张图片连接线上的各个点，用栈结构进行存储。

3、类设计

(1) CGameLogic 类 —— 逻辑判断类。

1) 数据成员

数据成员	描述
Vertex m_avPath[4]	保存在进行连接判断时所经过的顶点
int m_nVexNum	顶点数

2) 公有成员函数

成员函数	描述
bool IsLink(int** pGameMap, Vertex v1, Vertex v2)	判断是否连通
void Clear(int** pGameMap, Vertex v1, Vertex v2)	消子
int GetVexPath(Vertex avPath[4])	得到路径，返回的是顶点数

3) 保护类型成员函数

成员函数	描述
bool LinkInRow(int** pGameMap, Vertex v1, Vertex v2)	判断横向是否连通
bool LinkInCol(int** pGameMap, Vertex v1, Vertex v2)	判断纵向是否连通
bool OneCornerLink(int** pGameMap, Vertex v1, Vertex v2)	一个拐点连通判断
bool LineY(int** pGameMap, int nRow1, int nRow2, int nCol)	直线连通 Y 轴
bool LineX(int** pGameMap, int nRow, int nCol1, int nCol2)	直线连通 X 轴
void PushVertex(Vertex v)	添加一个路径顶点
void PopVertex()	取出一个顶点
void ClearStack()	清除栈
bool TwoCornerLink(int** pGameMap, Vertex v1, Vertex v2)	三条直线消子判断

(2) CGameControl 类 —— 游戏控制类。

1) 数据成员

数据成员	描述
------	----

Vertex m_svSelFst	选中的第一个点
Vertex m_svSelSec	选中的第二个点

2) 成员函数

成员函数	描述
void SetFirstPoint(int nRow, int nCol)	设置第一个点
void SetSecPoint(int nRow, int nCol)	设置第二个点
bool Link(Vertex avPath[4], int &nVexnum)	消子判断(路径暂定为 2 个顶点，后面再对该函数进行修订)

10.7.3 编码实现

导入“开始游戏”的解决方案，在此基础上进行迭代开发，实现步骤如下：

步骤一：添加鼠标事件。

步骤二：选择图片。

步骤三：消除相同元素图片。

步骤四：一条直线消子。

步骤五：两条直线消子。

步骤六：三条直线消子。

1、添加鼠标事件

(1) 鼠标点击事件

1) 用户使用鼠标点击窗口时，当点击的位置在当前应用程序窗口所在区域，操作系统会发
消息通知当前的应用程序，并且会把鼠标点击的坐标点和事件类型传入。

2) WM_LBUTTONDOWN 消息

当鼠标左键按钮被释放是响应。而 WM_LBUTTONDOWN 消息是在按下时响应。这个
两个消息都可以代表用户的点击事件。

3) WM_LBUTTONDOWN 消息响应函数

函数原型：void OnLButtonUp(UINT nFlags, CPoint point);

函数参数：

nFlags：标识在鼠标点击的同时是否有功能键，如：Ctrl、Shift 键。

point：鼠标点击的坐标点，以当前窗口客户区为坐标系。

(2) 添加鼠标点击事件

1) 选择 CGameDlg 类，打开类向导(ClassWizard)。

2) 给 CGameDlg 类添加 WM_LBUTTONDOWN 消息响应函数 OnLButtonUp()。

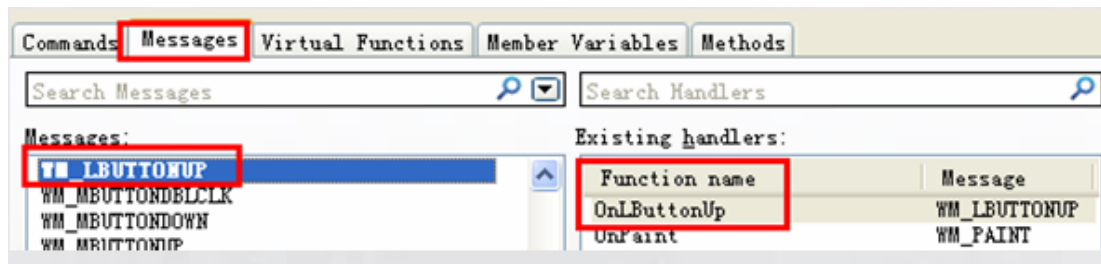


图 10-57 添加响应事件

鼠标输入消息如下：

消息	消息触发器
WM_LBUTTONDOWN	鼠标停留在窗口的客户区上，同时鼠标左键按钮被按下
WM_LBUTTONUP	鼠标左键按钮被释放
WM_RBUTTONDOWN	鼠标停留在窗口的客户区上，同时鼠标右键按钮被按下
WM_RBUTTONUP	鼠标右键按钮被释放
WM_MBUTTONDOWN	鼠标停留在窗口的客户区上，同时鼠标的中间按钮被按下
WM_MBUTTONUP	鼠标中间按钮被释放
WM_MOUSEMOVE	鼠标在窗口的客户区上移动

2、选择图片

(1) 判断点击位置是否在游戏地图中

游戏矩形区域为 `m_rtGameRect`，点击位置的坐标必须和游戏矩形之内。

鼠标消息响应函数 `OnLButtonDown()` 函数的参数 `CPoint point` 保存了鼠标点击位置的坐标。`CPoint.x` 表示横坐标，`CPoint.y` 表示纵坐标。

在 `CGameDlg::OnLButtonDown()` 函数中判断点击位置是否在游戏地图中。

```
void CGameDlg::OnLButtonDown(UINT nFlags, CPoint point)
{
    // 判断鼠标点击的区域
    if(point.y < m_rtGameRect.top || point.y > m_rtGameRect.bottom
        || point.x < m_rtGameRect.left || point.x > m_rtGameRect.right)
    {
        return CDialogEx::OnLButtonDown(nFlags, point);
    }
}
```

(2) 计算鼠标点击位置的行号和列号

游戏地图在游戏界面对话框中的起始坐标为(50, 50)。假设点击位置的坐标为(x, y)。

1) 首先排除坐标(x, y)不在游戏区域的情况。

2) 根据(x, y)计算点击位置的行号和列号。

将 `x>50` 的区域按照图片的长度，分为 `N` 列，将 `y>50` 的区域按照图片的高度，分为 `N` 行。

点击位置的行号 `row` 和列号 `col` 分别为：`row = (y - 50)/图片高度` 和 `col = (x - 50)/图片长度`。

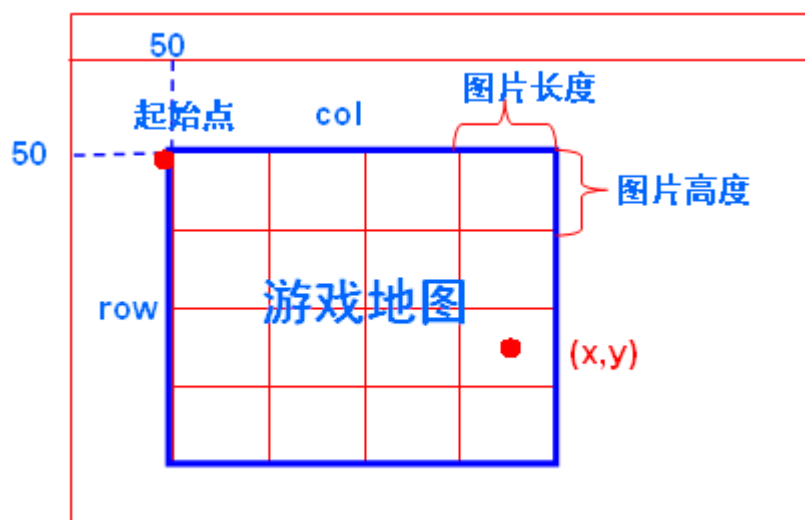


图 10-58 计算图片的行和列

根据点击位置的坐标、游戏地图中每个图片的大小，计算鼠标点击位置的行号和列号。

行号 = (点击位置 y 坐标 - 游戏地图的起始 y 坐标点)/元素的高度。

列号 = (点击位置 x 坐标 - 游戏地图的起始 x 坐标点)/元素的长度。

(3) 在鼠标选中的图中周围绘制矩形提示框。

在 CGameDlg 类中添加 DrawTipFrame() 函数，绘制提示框。

当鼠标选中游戏地图中的一张图片时，在该图片的四周绘制矩形提示框。矩形框的颜色为 RGB(233, 43, 43)，大小和选中的图片大小一致。

1) 在 CGameDlg 类中，添加成员函数 DrawTipFrame()，根据选择的图片的行号和列号，在选择的图片周围绘制矩形提示框。

函数定义格式为：void DrawTipFrame(int nRow, int nCol)

2) 绘制矩形时，可以根据矩形的左上角坐标和右下角坐标进行绘制。

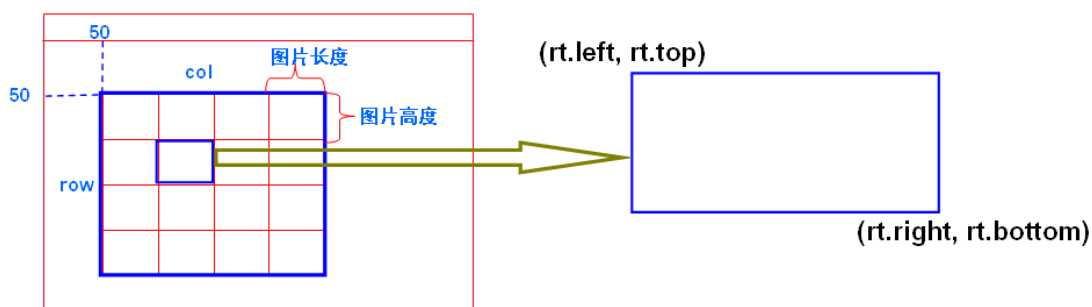


图 10-59 计算矩形框的坐标

3、一条直线消子

(1) 添加 IsLink 函数进行连通判断

```
bool CGameLogic::IsLink(int** pGameMap, Vertex v1, Vertex v2)
{
    PushVertex(v1);
```



```

// X 直连方式
if(v1.row == v2.row)
{
    //.....
}
//Y 直连方式
if(v1.col == v2.col)
{
    //.....
}
return false;
}

```

(2) 行号相同时，判断横向是否连通。

1) 添加 CGameLogic::LinkInRow()函数

```

bool CGameLogic::LinkInRow(int** pGameMap, Vertex v1, Vertex v2)
{
    int nCol1 = v1.col;
    int nCol2 = v2.col;
    int nRow = v1.row;
    //保证 nCol1 的值小于 nCol2
    if(nCol1 > nCol2)
    {
        //数据交换
        int nTemp = nCol1;
        nCol1 = nCol2;
        nCol2 = nTemp;
    }

    //直通
    for(int i = nCol1 + 1; i <= nCol2; i++)
    {
        if(i == nCol2)    return true;
        if(pGameMap[nRow][i] != BLANK)    break;
    }
    return false;
}

```

2) 算法：从 nCol1 + 1 遍历到 nCol2，判断这条直线上是否都为空白区域。如果全为空白区域，则横向连通。

(3) 列号相同时，判断能否纵向连通。

1) 添加 CGameLogic::LinkInCol()函数

```

bool CGameLogic::LinkInCol(int** pGameMap, Vertex v1, Vertex v2)

```

```

{
    int nRow1 = v1.row;
    int nRow2 = v2.row;
    int nCol = v1.col;
    if(nRow1 > nRow2)
    {
        int nTemp = nRow1;
        nRow1 = nRow2;
        nRow2 = nTemp;
    }
    //直通
    for(int i = nRow1+1; i <= nRow2; i++)
    {
        if(i == nRow2)            return true;
        if(pGameMap[i][nCol] != BLANK) break;
    }

    return false;
}

```

2) 算法：从 $nRow1 + 1$ 遍历到 $nRow2$ ，判断这条直线上是否全为空白区域。如果全为空白区域，则纵向连通。

4、两条直线消子

如果满足下列 2 个条件中的一个，则可以两条直线消子：

条件 1: $(nRow1, nCol1)$ 到 $(nRow1, nCol2)$ ， $(nRow1, nCol2)$ 到 $(nRow2, nCol2)$ 可以连通，并且 $(nRow1, nCol2)$ 位置的图片为空。

条件 2: $(nRow1, nCol1)$ 到 $(nRow2, nCol1)$ ， $(nRow2, nCol1)$ 到 $(nRow2, nCol2)$ 可以连通，并且 $(nRow2, nCol1)$ 位置的图片为空。

(1) 判断横向、纵向的线段是否能够连通

1) 判断 $(nRow1, nCol1)$ 到 $(nRow1, nCol2)$ 能否连通。

在 `CGameDlg::LineX()`函数中，判断 $(nRow1, nCol1)$ 到 $(nRow1, nCol2)$ 能否连通。

`LineX()`定义格式为： `bool LineX(int** pGameMap, int nRow, int nCol1, int nCol2)`

2) 判断 $(nRow1, nCol2)$ 到 $(nRow2, nCol2)$ 能否连通

在 `CGameDlg::LineY()`函数中，判断 $(nRow1, nCol2)$ 到 $(nRow2, nCol2)$ 能否连通。

`LineY()`定义格式为： `bool LineY(int** pGameMap, int nRow1, int nRow2, int nCol)`

(2) 判断 $(nRow1, nCol1)$ 到 $(nRow2, nCol2)$ 能否连通。

在 `CGameLogic` 类中定义数组 `Vertex m_avPath[4]`，使用栈来保存连通路径中的关键点：起始点 $V0$ 、拐点 $V1$ 和终点 $V2$ 。

在 `CGameLogic::OneCornerLink()`函数中，判断 $(nRow1, nCol1)$ 到 $(nRow2, nCol2)$ 能否连通。

OneCornerLink()定义格式为: bool OneCornerLink(int anMap[][4], Vertex v1, Vertex v2)

```
bool CGameLogic::OneCornerLink(int** pGameMap, Vertex v1, Vertex v2)
{
    // 直角能够消子, 那么顶点一定在与两个点的行和列相交的点, 只有这两个点为空, 才有可能实现二条直线消子

    if (pGameMap[v1.row][v2.col] == BLANK)
    {
        if (LineY(pGameMap, v1.row, v2.row, v2.col) && LineX(pGameMap, v1.row, v1.col, v2.col))
        {
            Vertex v = {v1.row, v2.col, BLANK};
            PushVertex(v);
            return true;
        }
    }

    if (pGameMap[v2.row][v1.col] == BLANK)
    {
        if (LineY(pGameMap, v1.row, v2.row, v1.col) && LineX(pGameMap, v2.row, v1.col, v2.col))
        {
            Vertex v = {v2.row, v1.col, BLANK};
            PushVertex(v);
            return true;
        }
    }

    return false;
}
```

(3) 在 CGameLogic::IsLink()中调用 CGameDlg::OneCornerLink(), 判断能否两条直线消子。

5、三条直线消子

在 CGameLogic::TwoCornerLink()函数中, 判断是否能进行三条直线消子。

```
bool CGameLogic::TwoCornerLink(int** pGameMap, Vertex v1, Vertex v2)
{
    //
    for(int nCol = 0; nCol < CGameControl::s_nCols; nCol++)
    {
        // 找到一条与 Y 轴平行的连通线段
        if (pGameMap[v1.row][nCol] == BLANK && pGameMap[v2.row][nCol] == BLANK)
        {
```

```

        if(LineY(pGameMap, v1.row, v2.row, nCol))
        {
            if(LineX(pGameMap, v1.row, v1.col, nCol) && LineX(pGameMap, v2.row,
v2.col, nCol))
            {
                // 保存节点
                Vertex vx1 = {v1.row, nCol, BLANK};
                Vertex vx2 = {v2.row, nCol, BLANK};
                PushVertex(vx1);
                PushVertex(vx2);
                return true;
            }
        }
    }
}

for(int nRow = 0; nRow < CGameControl::s_nRows; nRow++)
{
    // 找到一条与 X 轴平行的连通线段
    if(pGameMap[nRow][v1.col] == BLANK && pGameMap[nRow][v2.col] == BLANK)
    {
        if(LineX(pGameMap, nRow, v1.col, v2.col))
        {
            if(LineY(pGameMap, nRow, v1.row, v1.col) && LineY(pGameMap, nRow,
v2.row, v2.col))
            {
                // 保存节点
                Vertex vx1 = {nRow, v1.col, BLANK};
                Vertex vx2 = {nRow, v2.col, BLANK};
                PushVertex(vx1);
                PushVertex(vx2);
                return true;
            }
        }
    }
}

return false;
}

```

6、消除相同元素图片

(1) 判断选择的图片是否是同一种图片。

1) 如何判断连续两次选的图片，是否是要判断消除的一对图片？

要判断连续两次选择的图片是否是一组图片，需要设置一个标志位，CGameDlg 类的

属性 `bool m_bFirstPoint` 用于标识是否是第一个点，`true` 表示是第一次点击的，`false` 表示是第二次点击的。

2) 如果选择的一对图片是同一种的图片，那么它们在表示地图的二维数组中的数值相等。

定义 `CGameControl::Link()` 函数，在该函数中添加代码，先这一对图片是否是同一张，然后判断是否是同一种图片。

```
bool CGameControl::Link(Vertex avPath[4], int &nVexnum)
{
    // 判断是否同一张图片
    if(m_svSelFst.row == m_svSelSec.row && m_svSelFst.col == m_svSelSec.col)
    {
        return false;
    }
    // 判断图片是否相同
    if(m_pGameMap[m_svSelFst.row][m_svSelFst.col] != m_pGameMap[m_svSelSec.row][m_svSelSec.col])
    {
        return false;
    }
    //.....
}
```

(2) 对选中的两个图片进行连通判断。

1) 三种连通方式：

- ① 一条直线消子
- ② 两条直线消子
- ③ 三条直线消子

2) 添加 `CGameLogic::IsLink()` 函数来判断选中的两点是否能够消子，并记录关键点。

`IsLink()` 定义格式为：`bool IsLink(int** pGameMap, Vertex v1, Vertex v2)`

3) `CGameControl::Link()` 中调用 `CGameLogic::IsLink()` 函数进行连通判断。

(3) 获取连接路径

1) 在 `CGameLogic` 中，添加成员函数 `GetVexPath()`，获取连接路径。通过参数返回路径，通过返回值返回数组中实际元素个数。

```
int CGameLogic::GetVexPath(Vertex avPath[4])
{
    for(int i = 0; i < m_nVexNum; i++)
    {
        avPath[i] = m_avPath[i];
    }
    return m_nVexNum;
}
```

2) 在 CGameControl::Link() 函数中，判断一对图片是否能够连通，调用 CGameLogic::GetVexPath()函数获取连接路径。

```
bool CGameControl::Link(Vertex avPath[4], int &nVexnum)
{
    //.....
    // 判断是否连通
    if(m_GameLogic.IsLink(m_pGameMap, m_svSelFst, m_svSelSec))
    {
        // 消子
        m_GameLogic.Clear(m_pGameMap, m_svSelFst, m_svSelSec);

        // 返回路径顶点
        nVexnum = m_GameLogic.GetVexPath(avPath);

        return true;
    }
    return false;
}
```

3) 在 CGameDlg::OnLButtonUp()函数中调用 Link()函数，进行判断是否为相同的图片，判断是否能够连通，以及消子操作。

```
void CGameDlg::OnLButtonUp(UINT nFlags, CPoint point)
{
    //.....
    if(m_bFirstPoint)           // 第一个点
    {
        //.....
    }
    else                         // 第二个点
    {
        .....
        // 连子判断
        bool bSuc = m_GameC.Link(avPath, nVexnum);
        if(bSuc == true)
        {
            // 画提示线
            DrawTipLine(avPath, nVexnum);
            // 更新地图
            UpdateMap();
        }
        //.....
    }

    m_bFirstPoint = !m_bFirstPoint;
```

```
}
```

(4) 绘制连接线

当选中的两张图片是同种图片时，根据顶点绘制直线在选中的两张图片间绘制一条直线。起始点为第一次选中的图片的中心位置，终点为第二次选中图片的中心位置。

连接线颜色为 RGB(0, 255, 0)。

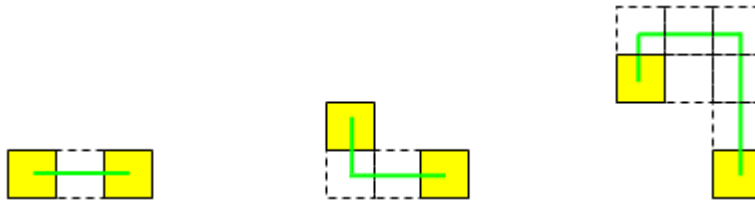


图 10-60 绘制连线

1) 在 CGameDlg 类中添加成员函数 DrawTipLine()。

函数定义格式为：void DrawTipLine(void)

2) 绘制连接线时，调用 CDC::MoveTo()函数和 CDC::LineTo()函数。

3) 连接线的起点为第一次点击的图片的中心位置，终点为第二次点击图片的中心位置。

```
void CGameDlg::DrawTipLine(Vertex asvPath[4], int nVexnum)
{
    CClientDC dc(this);
    CPen penLine(PS_SOLID, 2, RGB(0, 255, 0));
    CPen* pOldPen = dc.SelectObject(&penLine);

    dc.MoveTo(m_ptGameTop.x + asvPath[0].col * m_sizeElem.cx + m_sizeElem.cx / 2,
              m_ptGameTop.y + asvPath[0].row * m_sizeElem.cy + m_sizeElem.cy / 2);

    for(int i = 1; i < nVexnum; i++)
    {
        dc.LineTo(m_ptGameTop.x + asvPath[i].col * m_sizeElem.cx + m_sizeElem.cx / 2,
                  m_ptGameTop.y + asvPath[i].row * m_sizeElem.cy + m_sizeElem.cy / 2);
    }

    dc.SelectObject(pOldPen);
}
```

4) 为了方便清除提示线，将提示线直接绘制到视频内存 DC 中，而不要绘制到 m_dcMem 中。

10.8 判断胜负

10.8.1 功能需求

1、介绍

在基本模式下如果将游戏地图中所有的图片都消除，则提示玩家获胜，并且可以重新开始新游戏。

2、输入

游戏地图中剩余图片数量。

3、处理

判断游戏地图中所有的图片是否都被消除。

4、输出

(1) 如果游戏地图中所有图片都消除了，则提示玩家获胜，并且可以通过开始游戏按钮重新开始新游戏。

(2) 如果没有都消除，玩家不能开始新游戏。直到所有图片都消除完。

10.8.2 设计思路

在“消子判断”的基础上进行迭代开发。

1、判断胜负的条件

界面上游戏地图中所有的图片都被消除。

2、判断胜负的功能实现

每次成功消除一对图片后，在 CGameLogic 中，判断 CGameControl 类保存游戏地图图结构 m_graph 中，顶点数组所有元素的值是否为空(-1)。为空表示该顶点已经被消除。

3、类设计

(1) CGameLogic 类

成员函数	描述
bool IsBlank(int** pGameMap)	判断图中顶点是不是全为空

(2) CGameControl 类

成员函数	描述
bool IsWin(void)	判断是否获胜

(3) CGameDlg 类

数据成员	描述
bool m_bPlaying	游戏状态标识

10.8.3 编码实现

导入“消子判断”的解决方案，在此基础上进行迭代开发，实现步骤如下：

步骤一：判断胜负。

步骤二：控制开始游戏按钮状态。

1、判断胜负

判断胜负的规则为：游戏地图中所有的图片都被消除完，也就是图结构中的顶点数组中所有元素都为空(-1)。

(1) 逻辑层判断胜负

在 CGameLogic 类中，添加 IsBlank()函数。判断图中所有的顶点是否都为空。

IsBlank()函数定义为：bool IsBlank(int** pGameMap)。

(2) 控制层判断胜负

在 CGameControl 类中，添加 IsWin()函数，调用 CGameLogic::IsBlank()函数，判断是否已经获胜。IsWin()函数定义为：bool IsWin(void)。

(3) 界面层判断胜负

在 CGameDlg::OnLButtonUp()函数中，调用 CGameControl::IsWin()函数，判断是否已经获胜。如果获胜，则提示用户获胜。

```
void CGameDlg::OnLButtonUp(UINT nFlags, CPoint point)
{
    //.....
    // 判断胜负
    if(bSuc && m_GameC.IsWin())
    {
        //.....
    }
    //.....
}
```

2、控制开始游戏按钮状态

游戏开始后，到游戏获胜前，开始按钮的状态为禁用。直到游戏获胜之后，才可用。

设置按钮的禁用状态，可以调 CWnd::EnableWindow()函数，参数为 FALSE 表示该对象不可用。参数为 TRUE 表示该对象可用。

(1) 给 CGameDlg 类添加数据成员 bool m_bPlaying，表示游戏是否正在进行。

(2) 在构造函数中初始化 m_bPlaying = false。

(3) 在 CGameDlg::OnBnClickedBtnStart()函数中，开始游戏时，将 m_bPlaying 赋值为 true。并将开始游戏按钮设置为禁用状态。

(4) 在 CGameDlg::OnLButtonUp()函数中，首先判断游戏是否正在进行，如果没有正在进行，则不响应鼠标事件。然后在游戏获胜后将 m_bPlaying 赋值为 false，最后将设置开始游戏按

钮的状态为可用。

(5) 编译并运行程序。

11 非线性结构与连连看游戏综合实践

11.1 实践目标

- 1、调研连连看游戏，了解连连看游戏的功能和规则等。
- 2、掌握集成开发工具(Visual C++ 6.0 或 Microsoft Visual Studio 2010)。
- 3、掌握 C++的基础编程。
- 4、理解 MFC 框架，包括 MFC Dialog 应用程序和 GDI 编程。
- 5、掌握图的数据结构和应用，重点理解邻接矩阵的存储结构。
- 6、掌握图的常用算法，比如深度优先遍历、图的联通判断等，能够利用图的算法实现游戏联通判断和胜负判断。
- 7、理解企业软件开发过程，理解系统需求分析和设计，应用迭代开发思路进行项目开发。
- 8、养成良好的编码习惯和培养软件工程化思维，综合应用“C++编程、MFC Dialog、算法、图”等知识，开发“连连看游戏”桌面应用程序，达到掌握和应用非线性结构核心知识。

11.2 需求分析

本案例的功能与“线性结构”中“连连看游戏”的功能完全相同，不同点在于游戏数据存储结构和算法。

在线性结构“连连看游戏”中，使用二维数组来保存游戏地图中的数据，基本实现了连连看的核心功能。在非线性结构“连连看游戏”中，采用图结构来存储游戏地图。其中，图的顶点对应游戏地图中的图片元素编号，边对应游戏地图中相邻图片元素的连通状态，能连通则有边，不能连通则没有边。

连连看游戏功能结构图如下：

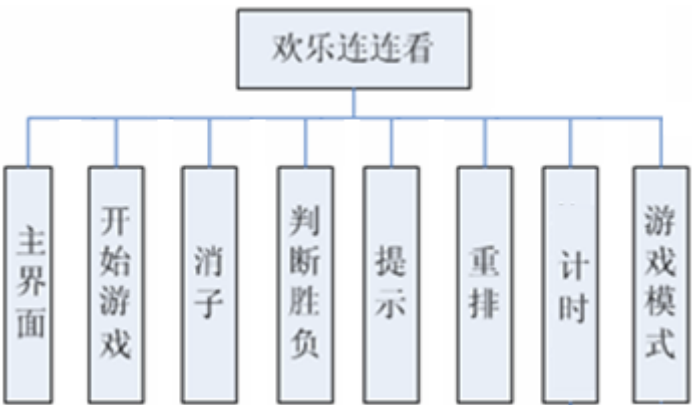


图 11-1 功能结构图

1、开始游戏

玩家选择游戏模式，进入游戏后，选择开始游戏，系统根据设置的主题风格生成一个图片布局(游戏地图)，以供玩家点击消除。

游戏地图大小为 640*400，是一个 16 行 * 10 列矩形，分成 160 个小正方形，存放 160 张图片，每张图片大小为 40*40。

2、消子

对玩家选中的两个图案进行判断，是否符合消除的规则。只有符合以下条件的图案对才会消失：

- (1) 一条直线连通
- (2) 两条直线连通
- (3) 三条直线连通

如果可以消除，从游戏地图中提示连接路径，然后消除这两种图片，并计算相应的积分。如果不能消除，则保持原来的游戏地图。

3、判断胜负

当游戏完成后，需要判断游戏的胜负。不同模式下，判断胜负的规则不同。

- (1) 基本模式时，如果在 5 分钟内，将游戏地图中所有的图片都消除，则提示玩家获胜。
- (2) 休闲模式时，如果游戏地图中所有的图片都被消除，则提示玩家获胜。

4、提示

当玩家选择提示功能时，提示玩家符合游戏规则可以消除的一对图片.如果游戏地图中没有能够消除的一组图片，则提示玩家没有能够消除的图片。

提示游戏地图从左上脚开始，查找满足消子规则，可以消除的一对图片，在该图片间绘制连接线。

查找的顺序从第一行第一列开始，按照从左到右的顺序，直到第一行最后一个，如果没有相同的，从第二行第一列开始查找，依次类推。

5、重排

对游戏地图中剩下的图片进行重新排列，重新排列只是将所有的图片的位置随机互换，不会增加图片的种类与个数。重排之前没有图片的位置，重排之后也不会有图片。

6、计时

在基本模式时，使用进度条对游戏进行计时。每局游戏时间为 300 秒，游戏开始从 300 秒开始倒计时。当时间为 0 秒时，游戏结束。计时效果图如下：



图 11-2 游戏计时效果图

11.3 系统设计

11.3.1 程序结构设计

1、工程结构设计

(1) 解决方案名称: Lianliankan

(2) 工程名称: LLK

(3) 工程目录结构

工程目录结构如下图所示:

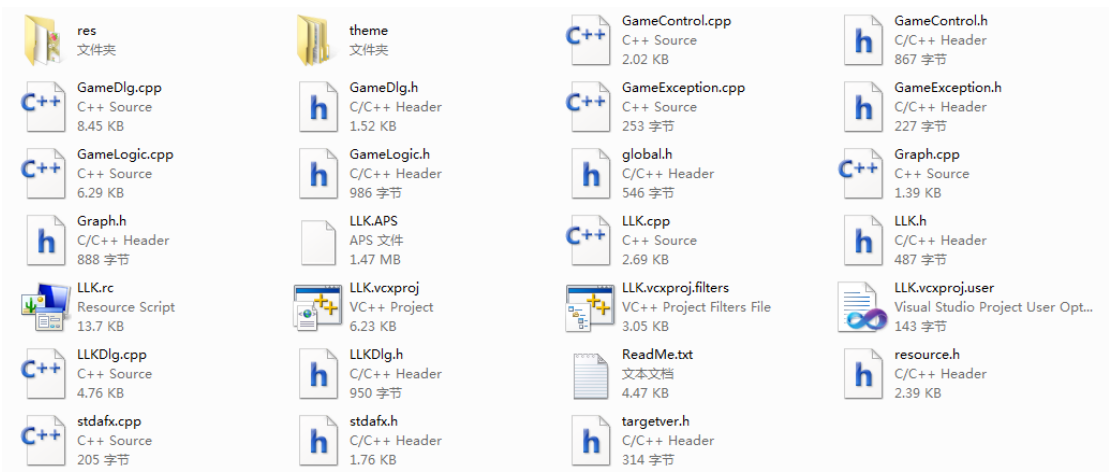


图 11-3 工程目录结构

- 1) res: 程序资源文件夹, 存放图标。
- 2) theme: 主题文件夹, 存放游戏的主题资源文件。

2、程序类关系设计

游戏程序按分层的思路来设计, 主要分为: 界面层 (主窗口类 CLLKDlg、游戏窗口类 CGameDlg), 游戏控制和业务逻辑层 (游戏控制类 CGameControl、CEasyGame、CBaseGame)。

各层之间使用结构体 (顶点信息 Vertex) 与实体类 (地图存储类 CGraph) 来传递数据。应用程序中公共的常量, 定义在 global.h 头文件中。

11.3.2 数据结构设计

1、顶点存储结构

添加 global.h 文件, 定义结构体 Vertex, 用于保存游戏地图中一个点的行号、列号、值信息。

代码如下:

```
typedef struct tagVertex{
    int row;           // 行
    int col;           // 列
    int info;          // 值信息
```

}Vertex;

2、游戏地图存储结构

连连看游戏中，生成的图是一个无向图。无向图是由顶点和边构成。当保存图结构时，即要保存顶点信息，也要保存边。在本案例中采用邻接矩阵来存储图。

(1) 顶点

1) 使用一个一维数组 V 来保存图的顶点。一维数组中元素的个数和游戏地图中图片的个数一致。如果游戏地图中图片数量为 N ，则顶点数组 V 中元素个数也为 N 。

2) 顶点中的数值就是游戏地图中图片元素对应的编号。

(2) 边

1) 使用一个邻接矩阵 E (也就是二维数组) 来保存图的边。二维数组的大小也是由游戏地图中图片的个数决定。如果顶点数为 N ，则二维数组大小为 $N*N$ 。

2) 如果两个顶点 $V1$ 和 $V2$ 有边，则在边数组 E 中， $E(v1, v2) = E(V2, V1) = 1$ 。

例如，以一个 4 行 4 列的游戏地图为例，顶点数组为 16 个元素，而邻接矩阵 E 则为 $16*16$ 的二维数组。如下图 (3-8) 所示，在游戏地图中用红色框框住的两张图片，在顶点数组中索引号为 0 和 4，在邻接矩阵中这两张图片有边存在，用 1 表示。

$E[0][4] = E[4][0] = 1$ 。

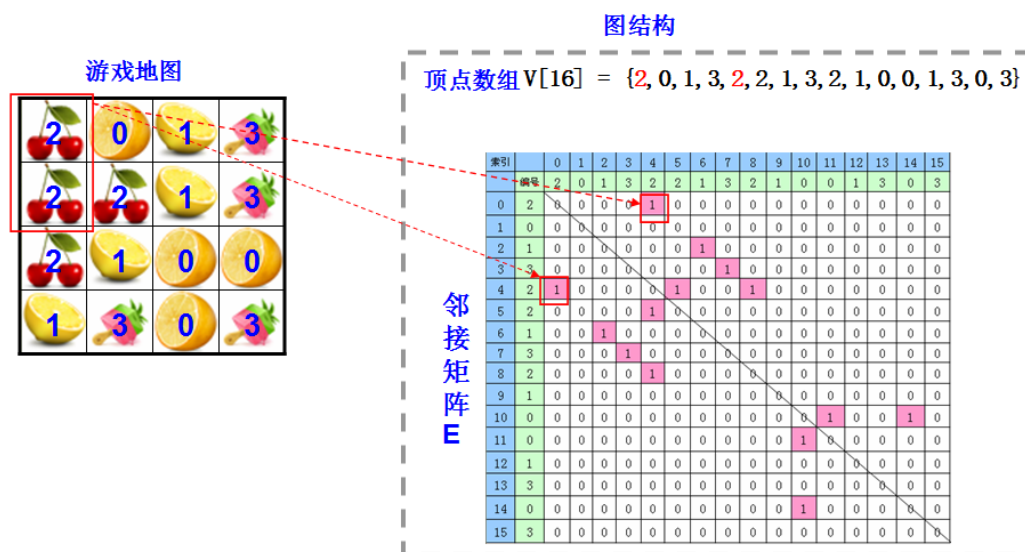


图 11-4 图结构的设计

3) 形成边的规则

每个顶点与相邻的四个方向的顶点可以形成边。如图 (3-9) 所示，顶点 V 与 V_i ($V1$ 、 $V2$ 、 $V3$ 、 $V4$) 有两种情况可以形成边：

情况 1：顶点都不为空，但为同色图片。

情况 2：两个顶点中至少有一个顶点为空。

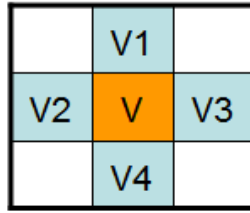


图 11-5 边的定义

11.3.3 开局与重排算法

(1) 计算游戏中元素个数：行数 * 列数。

(2) 计算每一种花色重复数：行数 * 列数 / 花色数。

① 判断（行数 * 列数 % 花色数）是否为 0。如果不为 0，则进行异常处理。

② 判断每一种花色重复数是否能被 2 整除，如果不能被 2 整除，则进行异常处理。

(3) 按从左到右，从上到下的顺序，将花色数填入游戏地图。

0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1
1	1	1	1	2	2	2	2	2	2	2	2	2	2	3	3
3	3	3	3	3	3	3	3	4	4	4	4	4	4	4	4
4	4	5	5	5	5	5	5	5	5	5	5	6	6	6	6
6	6	6	6	6	6	7	7	7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8	8	8	9	9	9	9	9	9
9	9	9	9	10	10	10	10	10	10	10	10	10	10	11	11
11	11	11	11	11	11	11	11	12	12	12	12	12	12	12	12
12	12	13	13	13	13	13	13	13	13	13	13	14	14	14	14
14	14	14	14	14	14	15	15	15	15	15	15	15	15	15	15

实现伪代码如下：

```
int nRepeatNum = 得到花色重复数;
int nCount = 0; // 记录当前是第几个元素
for(int i = 0; i < nPicNum; i++) // nPicNum 为花色数
{
    // 重复数
    for(int j = 0; j < nRepeatNum; j++)
    {
        pGameMap[nCount / nCols][nCount % nCols] = i; // nCols 是游戏地图的列数
        nCount++;
    }
}
```

(4) 由于生成的地图是规则的，因此，需要将地图中的花色打乱。实现思路是，随机选择

两个元素，将其值对调。重复若干次（当前游戏重复了元素总数次）。
重排的算法，其实就是对地图当前所有的元素，随机交换 100 次。

11.3.4 消子判断算法

1、判断连通

首先在顶点数组 V 中，判断是否是同一元素。满足 $V[v1] = V[v2]$ 条件，表示这两个顶点图片元素相同。

然后遍历图结构的邻接矩阵 E ，判断是否有边。满足 $E[vi][vj] = E[vj][vi] = 1$ 条件，表示这两个顶点间有边。(其中：无向图，邻接矩阵具有对称性。)

从一个顶点 $V1$ 出发，找到一个相邻的连通顶点 $Vi1$ ，如果 $Vi1$ 不等于 $V2$ ，则继续寻找 $Vi1$ 的一个相邻的连通顶点 $Vi2$依次类推，直到找到一个顶点 ViN 与 $V1$ 相同，则表示在 $V1$ 与 $V2$ 之间有一个连通路程($V1$ 、 $Vi1$ 、 $Vi2$、 ViN 、 $V2$)。若遍历了所有顶点都找不到该路径则表示 $V1$ 与 $V2$ 之间不连通。



图 11-6 判断连通

2、拐角判断实现的思路

- (1) 在连连看的规则中，最多只有二个拐角。
- (2) 拐角的判断
 - 1) 在查找连通路程的过程中，对路径中出现的拐角数进行限制，超过二个拐角的路径舍弃。
 - 2) 相邻三个顶点能形成一个拐角，在将顶点保存到路径数组时，判断相邻的三个顶点，是否形成拐角，如果形成则记录拐角数。
 - 3) 在同一直线的三个顶点的索引值，成等差数列；而如果有拐点的话，索引值不能形成等差数列。

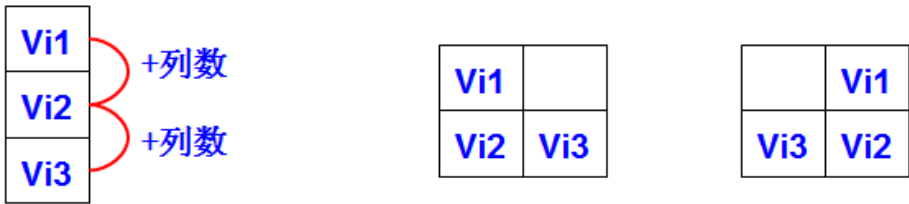


图 11-7 拐角判断

11.3.5 消子算法

顶点数组为 V，邻接矩阵为 E，两个顶点在顶点数组中的索引分别为：v1,v2。如果这两个点能够消除，则需要更新图结构，并更新游戏地图。

1、顶点的更新

将顶点数组中索引号为 v1 和 v2 的元素的值修改为-1。

2、边的更新

v1 和 v2 消除后，v1、v2 与它四个方向(上下左右)相邻的顶点都有边。

3、游戏地图的更新

消子之后，重新绘制游戏地图，根据顶点数组更新之后的图片元素编号，获取图片元素。



图 11-8 更新数据

11.3.6 胜负判断算法

每次判断连通后，进行获胜判断。
若成功消除一对图片，则判断地图中所有元素是否都为空，如果都为空，则表示获胜，游戏结束。否则，继续游戏。
在计时情况下，先判断时间是否已到，如果时间到了，表示未能成功，游戏结束。

11.4 创建工程

11.4.1 功能需求

创建“欢乐连连看”游戏工程。

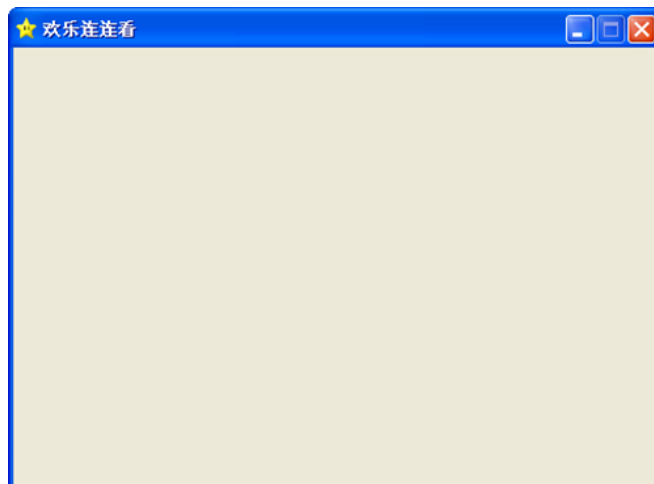


图 11-9 游戏主窗口

1、创建工程

- (1) 开发工具: Microsoft Visual Studio 2010
- (2) 解决方案: Lianliankan
- (3) 工程类型: MFC 对话框(Dialog)工程
- (4) 项目名称: 欢乐连连看
- (5) 工程名称: LLK
- (6) 主对话框: CLLKDlg

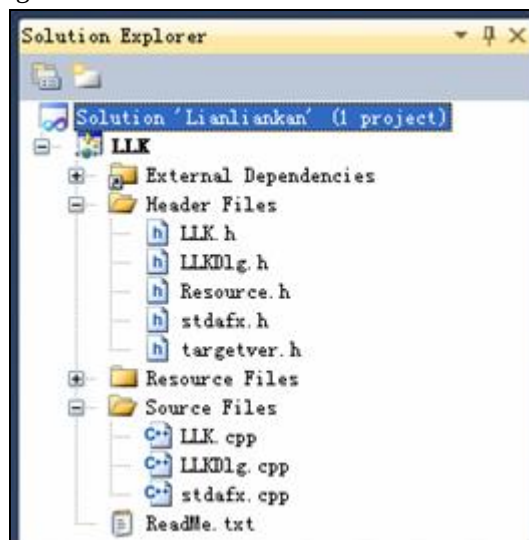


图 11-10 解决方案目录

2、游戏主窗口

- (1) 窗口标题: 欢乐连连看。
- (2) 标题栏按钮: 最小化按钮、关闭按钮。最大化与还原按钮不可使用。
- (3) 对话框图标: 自定义图标, ico 文件。



图 11-11 程序图标

11.4.2 设计思路

选用 Microsoft Visual Studio 2010 作为开发工具，简称 VS2010。选择 MFC 对话框工程作为游戏工程。

11.4.3 编码实现

实现步骤如下：

步骤一：创建解决方案。

步骤二：创建工程。

步骤三：修改主界面对话框属性。

1、创建解决方案

(1) 选择“开始 -> 程序 -> Microsoft Visual Studio 2010 -> Microsoft Visual Studio 2010”，打开 VS2010。

(2) 在 VS2010 开发工具中选“File -> New -> Project”菜单,出现新建对话框。

(3) 在新建对话框中，选择解决方案类型为“Other Project Type -> Visual Studio Solutions -> Blank Solution”,解决方案名为“Lianliankan”，保存路径。

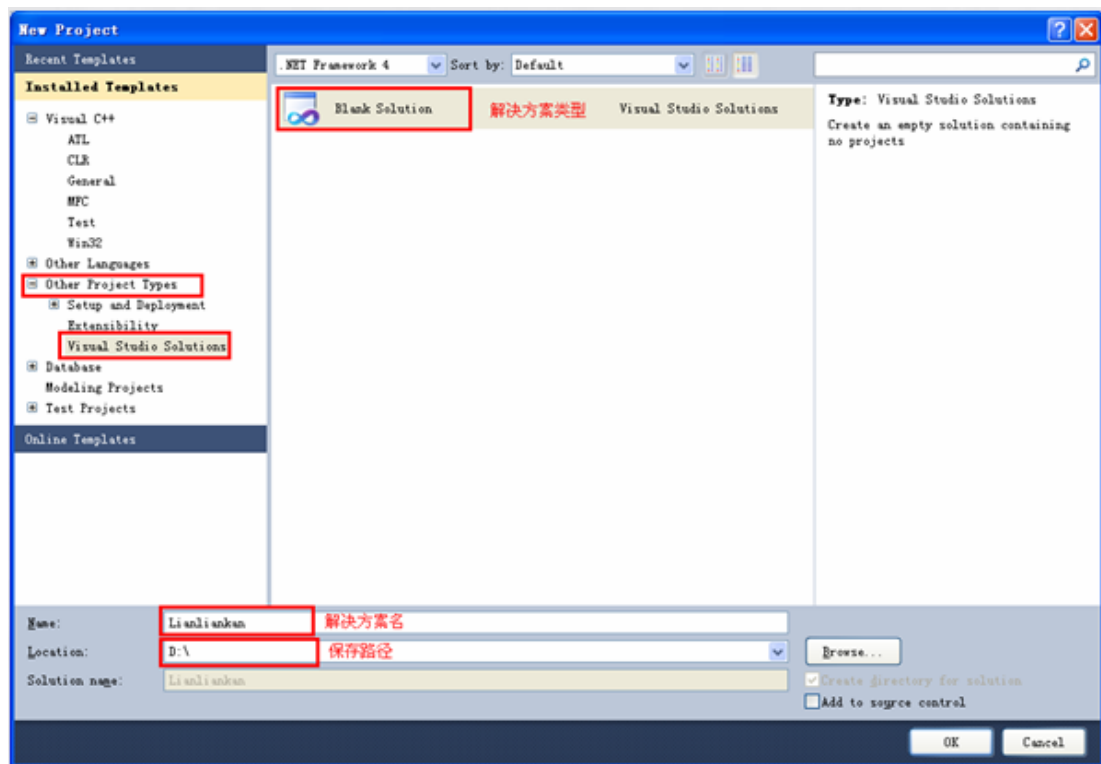


图 11-12 创建解决方案

(4) 创建完成后，解决方案保存路径中，生成解决方案文件夹，在解决方案文件夹中，生成解决方案文件(.sln 后缀)。



图 11-13 生成解决方案的文件

2、创建工程

(1) 创建解决方案之后，选择“File -> New -> Project”，显示新建对话框。

(2) 选择工程类型为“Visual C++ -> MFC -> MFC Application”，输入工程名称 LLK，选择“Solution”为 “Add to solution”，点击“OK”，进入应用程序向导

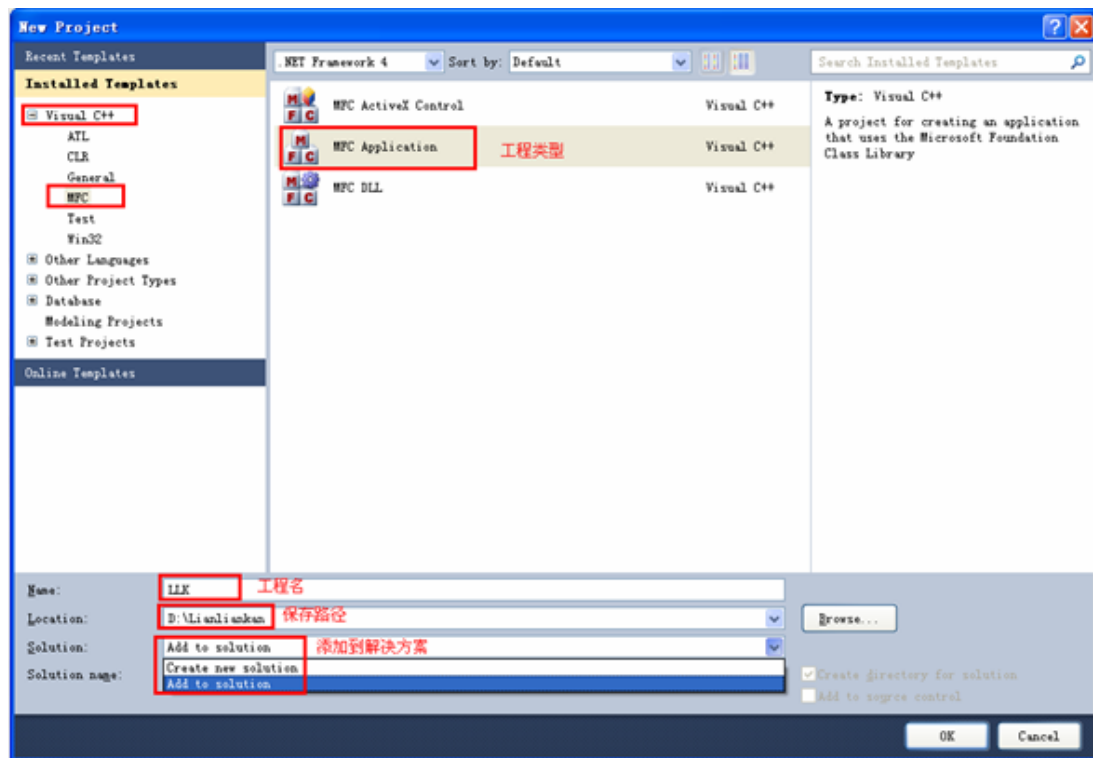


图 11-14 创建工程

(3) 选择应用程序类型

在应用程序向导的“Application Type”中，选择应用程序类型为“Dialog based”。然后点击“Next”进入下一步。

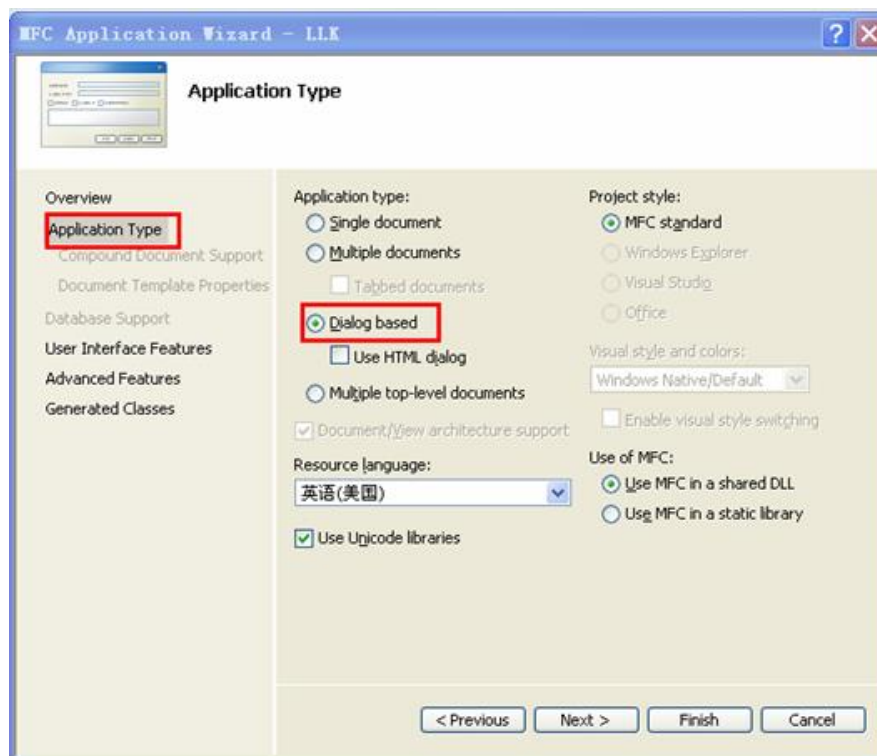


图 11-15 选择应用类型

(4) 在“User Interface Features”中，勾选“Minimize box”，给对话框窗口添加一个最小化按钮。然后点击“Finish”完成工程的创建。

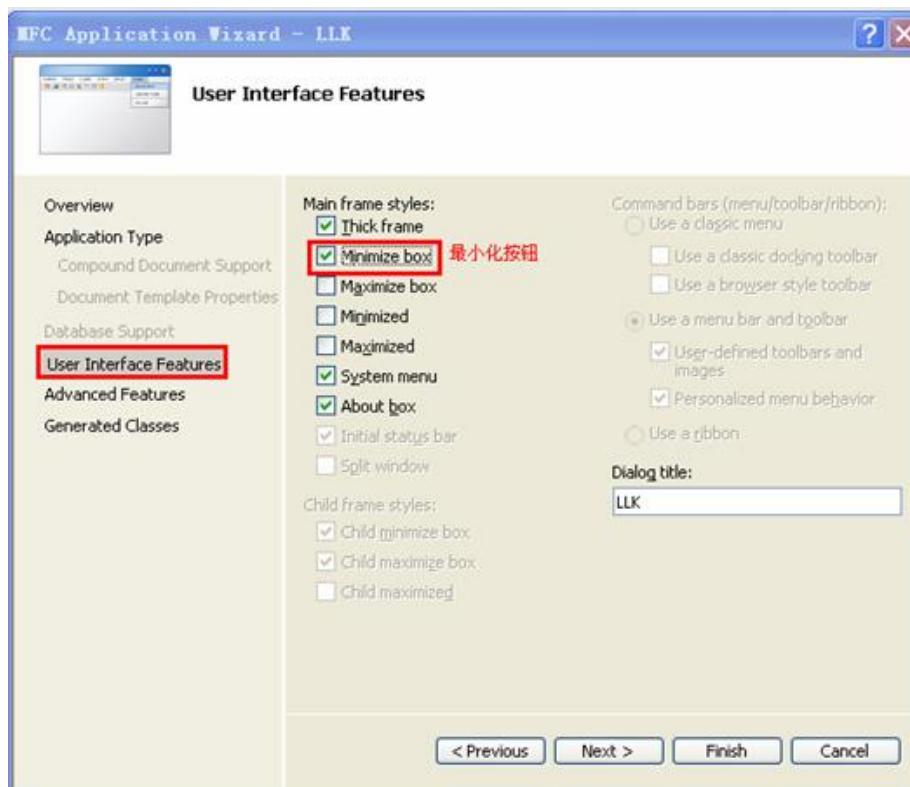


图 11-16 添加最小化按钮

(5) 创建完成后，在工程中可以看到如下内容。

解决方案资源管理器

类视图

资源视图

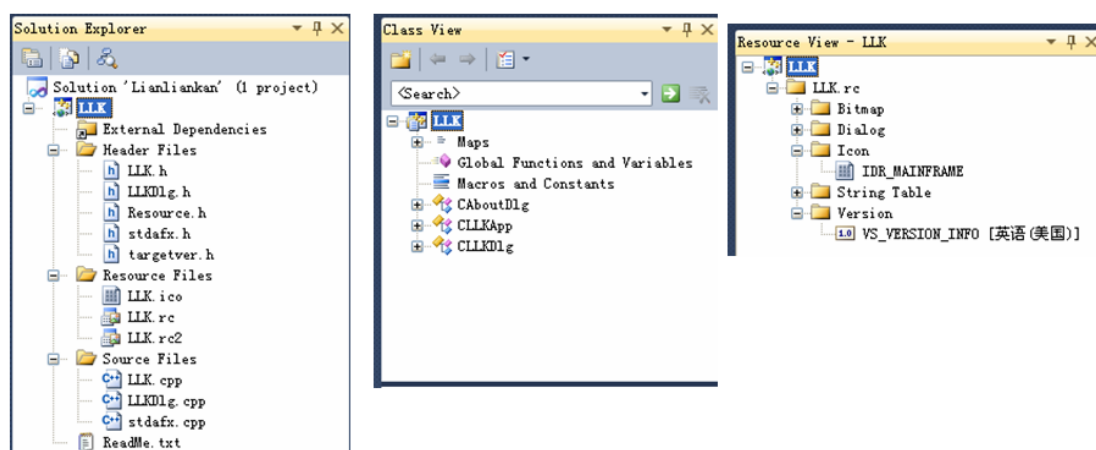


图 11-17 程序的三种视图

(6) 在工程保存路径下可以看到如下文件：

1) 解决方案文件夹



图 11-18 解决方案文件夹

2) 工程文件夹

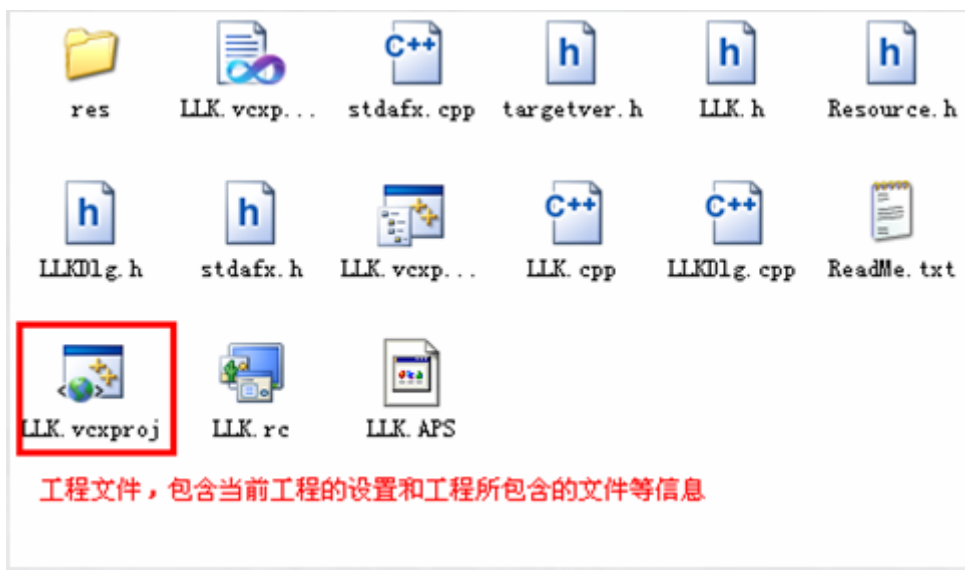


图 11-19 工程文件夹

(7) 编译并运行程序。

使用 VS2010 应用程序向导创建好工程之后，也就完成的代码的编辑工作。现在可以对代码进行编译、连接和运行。

1) 编译程序(F7)

在 VS2010 中，将编译和连接的功能放到一起，直接编译就可以产生可执行文件。

编译后，产生可执行文件。

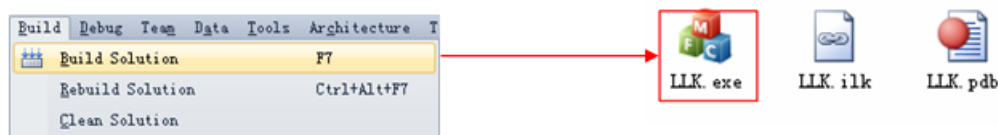


图 11-20 产生可执行文件

2) 运行程序(ctrl+F5)

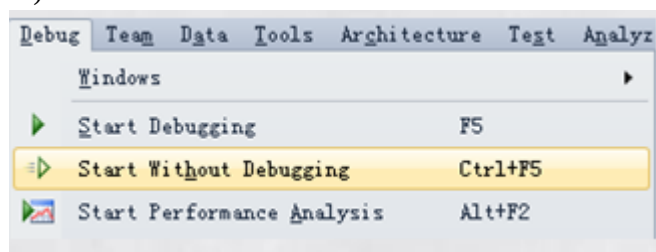


图 11-21 运行程序

3、修改主界面对话框属性

(1) 打开主界面对话框资源

方法一：选择主界面对话框类 CLLKDlg，右键选择“Go To Dialog”，打开主界面对话框

资源。



图 11-22 打开对话框方法一

方法二：在资源视图，双击主界面对话栏资源。打开对话框资源。

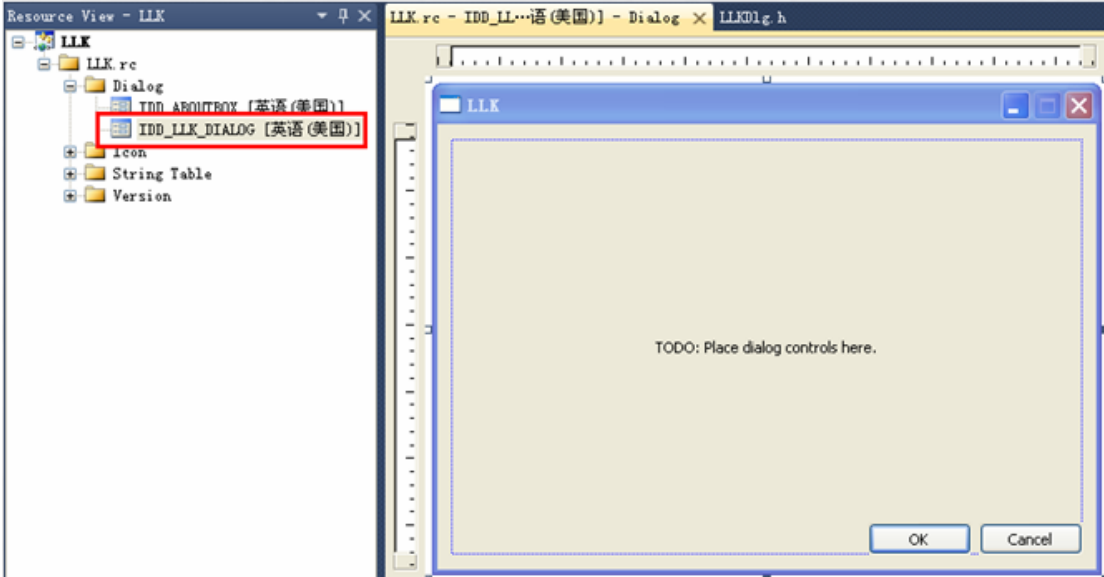


图 11-23 打开对话框方法二

- (2) 修改对话框标题为“欢乐连连看”。
- 1) 在对话框编辑器中，删除对话框资源中默认产生的控件。
 - 2) 在对话框资源上右键，选择“Properties”，打开对话框属性编辑器。
 - 3) 在对话框属性编辑器中修改对话框标题栏为“欢乐连连看”。
 - 4) 编译并运行程序。

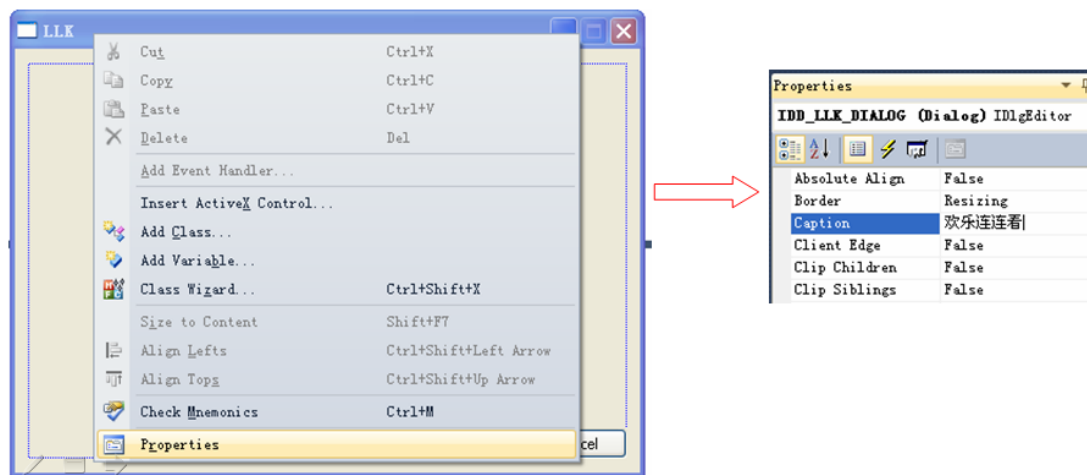


图 11-24 修改标题的运行结果

(3) 修改对话框图标。

1) 在工程目录 res 文件夹中，找到对话框图标“LLK.ico”。将需要设置为对话框图片的 ico 文件命名为“LLK.ico”，替换工程目录 res 中默认的 LLK.ico 文件。



图 11-25 修改图标

2) 编译并运行程序。

由于修改了资源文件，必须要把原来编译的文件清除后，全部重新编译才行。否则 VS 中默认是增量编译的，已编译的内容不会重新编译。只替换了图标的文件，工具并不会重新编译图标。

11.5 主界面

11.5.1 功能需求

设计“欢乐连连看”项目的主界面，在主界面上添加一个背景图片，并在适当的地方添加“基本模式”、“休息模式”、“关卡模式”、“帮助”、“排行榜”与“设置”按钮。

1、主界面为启动游戏时出现的界面。



图 11-26 主界面效果图

2、主界面设计

- (1) 主界面客户区大小：800*600，单位像素。
- (2) 背景图片：背景图片大小为 800*600，背景图片看起来欢快、轻松。

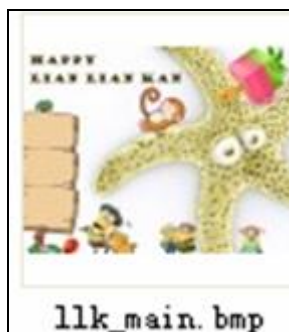


图 11-27 位图格式的背景图片

- (3) 模式选择按钮：基本模式、休息模式、关卡模式。
- (4) 辅助功能按钮：排行榜、设置、帮助。
- (5) 界面要求：界面协调、美观、符合大部分玩家操作习惯。

11.5.2 设计思路

在“创建工程”的基础上进行迭代开发。

使用 VS2010 工具中对话框编辑器，布置主界面，窗口的背景图一个位图文件，使用 MFC 中的 GDI 技术来给窗口添加一个背景。

1、背景图片绘制步骤

(1) 背景图片的导入

在工程的资源视图中，导入背景图片，背景图片资源 ID 为 IDB_MAIN_BG。

(2) 背景图片的显示

1) 在 `CLLKDlg` 类中，添加一个初始化背景的方法 `void InitBackground()` 函数，将位图选入位图内存。

2) 在 `CLLKDlg::OnInitDialog()` 函数中，调用 `InitBackground()` 函数。在对话框启动时，将位图选进位图内存。

3) 在主界面窗口的 `WM_PAINT` 消息响应函数 `CLLKDlg::OnPaint()` 函数中，将选入位图内存中的图片选到视频内存中，然后调用 `CDC::BitBlt()` 函数，将位图显示在主界面上。

2、CLLKDlg 类设计

(1) 属性

`CDC m_dcMem`: 表示内存 DC，访问权限为 `protected`。

(2) 方法

1) void InitBackground()

将背景图片位图资源保存到位图内存中。

2) void Paint()

对话框 `WM_PAINT` 消息响应函数，将位图内存中的背景图片绘制到对话框中。

3) BOOL OnInitDialog()

对话框初始化函数，在对话框初始化时，调用 `InitBackground()` 函数，加载位图资源到位图资源中。

11.5.3 编码实现

导入“创建工程”的解决方案，在此基础上进行迭代开发，实现步骤如下：

步骤一：位图导入。

步骤二：绘制窗口背景。

步骤三：添加主界面的功能按钮。

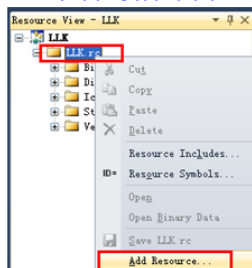
1、位图导入

(1) 将位图资源文件放到物理磁盘工程目录下的 `res` 文件夹。

(2) 将位图资源导入到工程中。

(3) 修改位图资源为 `IDB_MAIN_BG`。

① 在资源视图中，选择工程名，右键菜单选择“Add Resource...”菜单项。显示添加资源对话框。



② 添加资源对话框中，选择资源类型为“Bitmap”。点击“Import...”按钮。

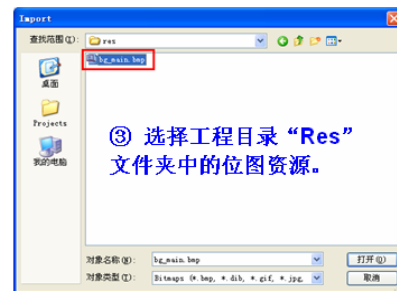
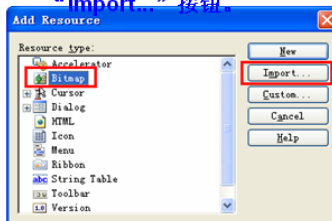


图 11-28 位图导入

2、绘制窗口背景

(1) 创建一个内存 DC

给 CLLKDlg 类添加一个 protected 属性 CDC m_dcMem，表示位图内存。

(2) 在 CLLKDlg 类添加 void InitBackground()函数。

```
void CLLKDlg::InitBackground(void)
{
    //.....
}
```

(3) 加载位图

CBitmap 类封装了 Windows 的图形设备(GDI)中的位图，并且提供了操纵位图的成员函数。静态位图的加载：

- 1) 定义 CBitmap 位图对象。
- 2) 调用 CBitmap::LoadBitmap()函数加载位图资源。
- 3) 将位图选入到内存 DC 中。

加载背景图片位图代码如下：

```
CBitmap bmpMain;
bmpMain.LoadBitmapW(IDB_MAIN_BG);
```

(4) 创建兼容 DC

在对话框的非 OnPaint()函数中，可以使用 CClientDC 类来得到当前对话框的视频内存 DC。通过调用 CDC::CreateCompatibleDC()函数来创建一个与视频 DC 兼容的内存 DC (m_dcMem)，最后，将位图对象选入 (CDC::SelectObject) 到内存 DC 中。

```
void CLLKDlg::InitBackground(void)
{
    // 获得当前对话框的视频内存
    CClientDC dc(this);
    //.....
    // 创建与视频内存兼容的内存 DC
    m_dcMem.CreateCompatibleDC(&dc);
    // 将位图资源选入 DC
    m_dcMem.SelectObject(bmpMain);
```

```
//.....
}
```

(5) 在 `CLLKDlg::OnInitDialog()` 函数中，调用 `InitBackground()` 函数。

(6) 调用 `CDC::BitBlt()` 函数。

在 `CLLKDlg::OnPaint()` 函数中，调用 `CDC::BitBlt()` 函数从位图内存中拷贝到视频内存中，进行显示。

```
void CLLKDlg::OnPaint()
{
    if (IsIconic())
    {
        .....
    }
    else
    {
        // 创建 CPaintDC 对象
        CPaintDC dc(this);
        // 绘制背景图片
        dc.BitBlt(0, 0, 800, 600, &m_dcMem, 0, 0, SRCCOPY);

        CDialogEx::OnPaint();
    }
}
```

2、添加主界面的功能按钮

(1) 给界面添加控件

在对话框编辑器中，将背景图片导入，调整对话框的大小。从 **Toolbox** 中将按钮控件添加到对话框中合适的位置。



VS工具的Toolbox中，有界面布局所需要用到的所有控件。用鼠标选中控件并拖到对话框中，即可将控件添加到对话框中。



图 11-29 添加功能按钮

(2) 修改按钮文本(Caption)和 ID。

鼠标右键选择控件，选择“Properties”，可以打开属性编辑器。在属性编辑器中可以查看和修改控件属性。

控件 ID 由大小字母和下划线组成。切忌用中文和数字！

ID	Caption
IDC_BTN_BASIC	基本模式
IDC_BTN_RELAX	休息模式
IDC_BTN_LEVEL	关卡模式
IDC_BTN_RANK	排行榜
IDC_BTN_SETTING	设置
IDC_BTN_HELP	帮助

(3) 编译运行，调整对话框大小和控制位置。

(4) 设置主界面客户区的大小。

通过调用 MoveWindow()函数可以设置窗口的位置和大小。在 InitBackground 函数中添加如下代码，以调整窗口的大小。

```
void CLLKDlg::InitBackground(void)
{
    //.....
    // 调整窗口大小
    CRect rtWin;
    CRect rtClient;
    this->GetWindowRect(rtWin);           // 获得窗口大小
    this->GetClientRect(rtClient);        // 获得客户区大小
    // 标题栏和外边框的大小
    int nSpanWidth = rtWin.Width() - rtClient.Width();
    int nSpanHeight = rtWin.Height() - rtClient.Height();

    // 设置窗口大小
    MoveWindow(0, 0, 800 + nSpanWidth, 600 + nSpanHeight);
    //.....
}
```

(5) 调用 CenterWindow()函数，使窗口居中。

11.6 开始游戏

11.6.1 功能需求

当玩家在主界面选择【基本模式】时，出现基本游戏界面，并隐藏主界面。玩家点击【开始游戏】按钮，绘制游戏地图。

11.6.2 设计思路

在“主界面”的基础上进行迭代开发。

1、CGraph 类

CGraph 类为程序的数据结构类，该类的对象为游戏地图的结构。

(1) 数据成员，访问权限 protected

数据成员	描述
Vertices m_Vertices	顶点数组
AdjMatrix m_AdjMatrix	边数组
int m_nVexnum	顶点个数
int m_nArcnum	边数量

(2) 保护型成员函数

保护型成员函数	描述
void InitGraph(void)	初始化图结构

(3) 公有成员函数

公有成员函数	描述
int AddVertex(int info)	将顶点 info 添加到顶点数组中
int GetVertex(int nIndex)	获取顶点数组中索引为 nIndex 的顶点
void AddArc(int nV1Index, int nV2Index)	添加关系矩阵中， nV1Index 与 nV2Index 的关系

2、CGameLogic 类

CGameLogic 类为游戏业务逻辑类。

成员函数	描述
void InitMap(CGraph &g)	根据固定的一组数据，生成游戏地图的图结构，并通过参数将图结构返回给调用该函数的函数
void UpdateArc(CGraph &g, int nRow, int nCol)	判断游戏地图中 nRow 行，nCol 列的顶点与它上下左右的顶点是否有边，如果有边，则更新图结构，并返回

3、CGameControl 类

CGameControl 类为游戏控制类。

(1) 静态成员变量

静态成员变量	描述
--------	----

static int s_nRows	游戏行数
static int s_nCols	游戏列数
static int s_nPicNum	图片数

(2) 数据成员

数据成员	描述
CGraph m_graph	保存生成的游戏地图的图结构

(3) 成员函数

成员函数	描述
void StartGame(void)	开始游戏，调用 CGameLogic 类的成员函数，生成游戏地图的图结构
int GetElement(int nRow, int nCol)	从游戏地图的图结构中，取出顶点，用于界面上第 nRow 行，nCol 列显示元素的根据

4、CGameDlg 类

CGameDlg 类为游戏界面类。

显示游戏界面，并与用户交换信息，不关心数据是以何种结构存储到内存中。

CGameDlg 类为游戏界面对话框类。

(1) 数据成员

数据成员	描述
CDC m_dcMem	位图内存 DC，游戏界面的背景图片保存到该 DC 中
CDC m_dcElement	元素内存 DC
CDC m_dcMask	掩码内存 DC
CGameControl m_GameC	游戏控制类对象

(2) 成员函数

成员函数	描述
void InitBackground(void)	加载游戏背景图片到位图 DC 中，设置游戏界面对话框的大小
virtual BOOL OnInitDialog()	对话框初始化函数，设置对话框的标题栏，并调用 InitBackground()函数
void OnPaint()	对话框 WM_PAINT 消息响应函数，将游戏背景图片绘制到对话框界面中
void InitElement(void)	初始化游戏元素和掩码
void UpdateMap(void)	更新游戏地图
void OnBnClickedBtnStart()	开始游戏按钮 BN_CLICKED 消息响应函数，开始游戏，生成游戏地图，并显示游戏地图
void UpdateWindow(void)	更新界面，从生成的游戏地图图结构中，取出顶点，根据顶点从游戏元素图片中，取出相对应的元素，显示在游戏地图中

5、CLLKDlg 类

CLLKDlg 类为主界面对话框类。在该类中添加一个“基本模式”按钮 BN_CLICKED 消息响应函数 CLLKDlg::OnBnClickedBtnBasic()。创建并显示游戏界面对话框，并控制主界面的隐藏和显示。

11.6.3 编码实现

导入“主界面”的解决方案，在此基础上进行迭代开发，实现步骤如下：

步骤一：添加游戏对话框资源。

步骤二：创建并显示游戏对话框。

步骤三：绘制游戏界面背景。

步骤四：游戏界面布局。

步骤五：生成顶点。

步骤六：生成图的边。

步骤七：显示游戏地图。

1、添加游戏对话框资源

在资源视图中，右键选择“Dialog”，在弹出菜单中选择“Insert Dialog”菜单项。即可添加一个对话框资源。

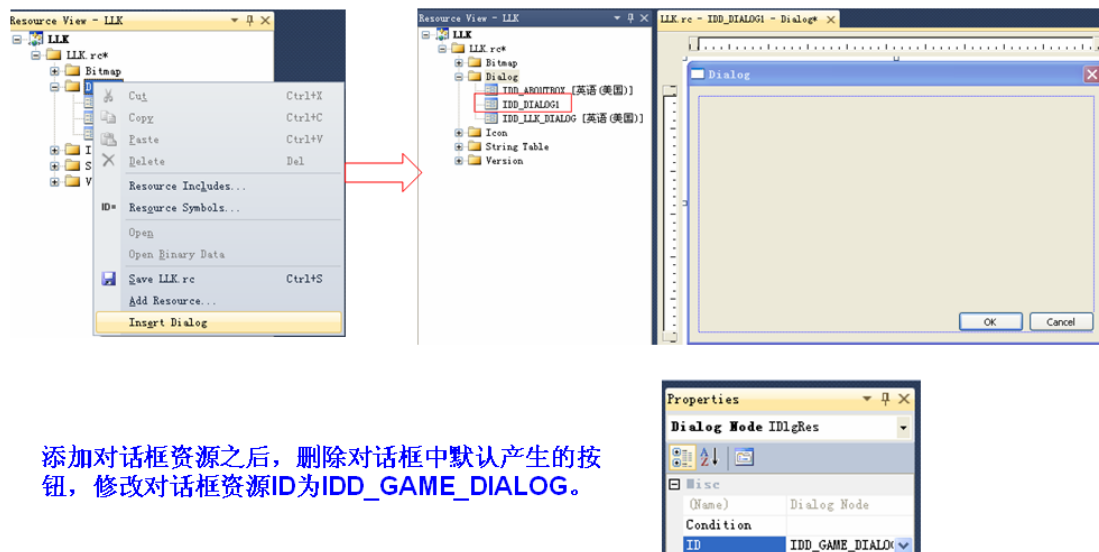


图 11-30 添加游戏对话框资源

2、创建并显示游戏对话框

(1) 添加游戏界面对话框类 CGameDlg。

1) 双击对话框资源，弹出“MFC Add Class Wizard”对话框。

2) 在对话框中输入对话框类的类名为“CGameDlg”。点击“Finish”，完成添加。

3) 添加完成后，在“Class View”可以看到添加 CGameDlg 类，在“Solution Explorer”可以看到 CGameDlg 类的源文件和头文件。

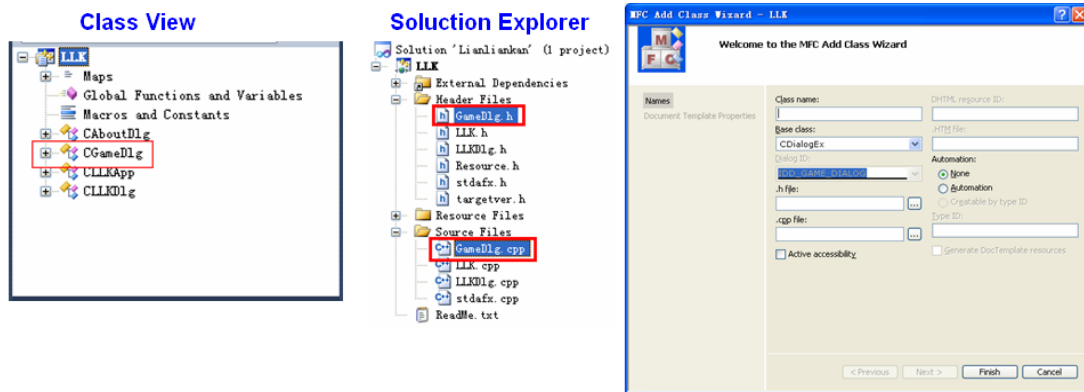


图 11-31 创建游戏对话框类

在 MFC 中，对话框资源和对话框类是通过对话框的资源 ID 进行关联的，每个对话框类都有一个对应的对话框资源。

(2) 创建并显示游戏对话框。

1) 使用“Class Wizard”，给主界面对话框中的“基本模式”按钮(IDC_BTN_BASIC)，添加 BN_CLICKED 消息响应函数 OnBnClickedBtnBasic()。

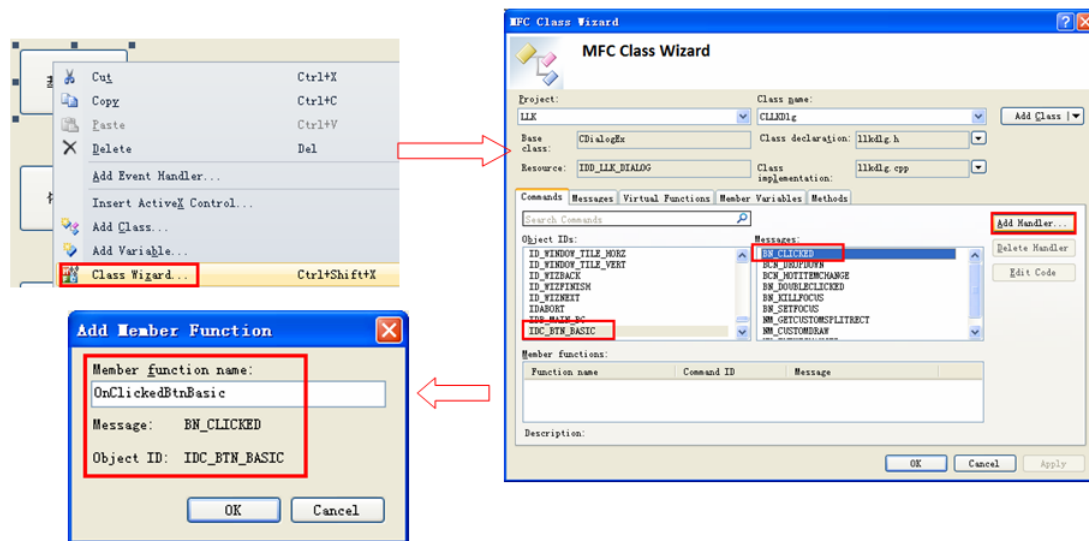


图 11-32 运行程序

2) 在 CLLKDlg::OnBnClickedBtnBasic()函数中，调用 CGameDlg::DoModel()函数创建并显示基本游戏对话框。

```
void CLLKDlg::OnBnClickedBtnBasic()
{
    CGameDlg dlg;
    dlg.DoModel();
}
```

3) 控制主界面的隐藏与显示

在 CLLKDlg::OnBnClickedBtnBasic()函数中，调用 CWnd::ShowWindow()函数，来控

制主界面的隐藏和显示。

3、绘制游戏界面背景

(1) 加载游戏界面背景图片

采用动态方式加载位图。

- 1) 在工程目录下新建一个“theme”文件夹，将游戏背景图片放到该文件夹中的“picture”文件夹中。

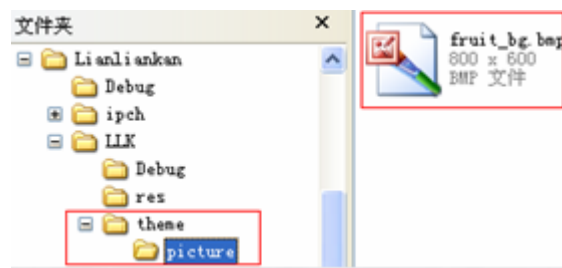


图 11-33 放置背景图片

- 2) 给 CGameDlg 类添加成员函数 void InitBackground(), 在该函数中调用 Windows API 函数 LoadImage()加载位图。

(2) 将图片选入位图内存。

在对话框初始化时，将位图保存到位图内存中。给 CGameDlg 类添加 OnInitDialog()函数，在 OnInitDialog()函数中，调用 InitBackground()函数。

- 1) 右键选择“CGameDlg”类，在弹出菜单中选择“Class Wizard”。
- 2) 在“Class Wizard”对话框中，选择“Virtual Functions”标签页，选择添加的函数为“OnInitDialog”。
- 3) 点击“Add Function”，完成 OnInitDialog()函数的添加。

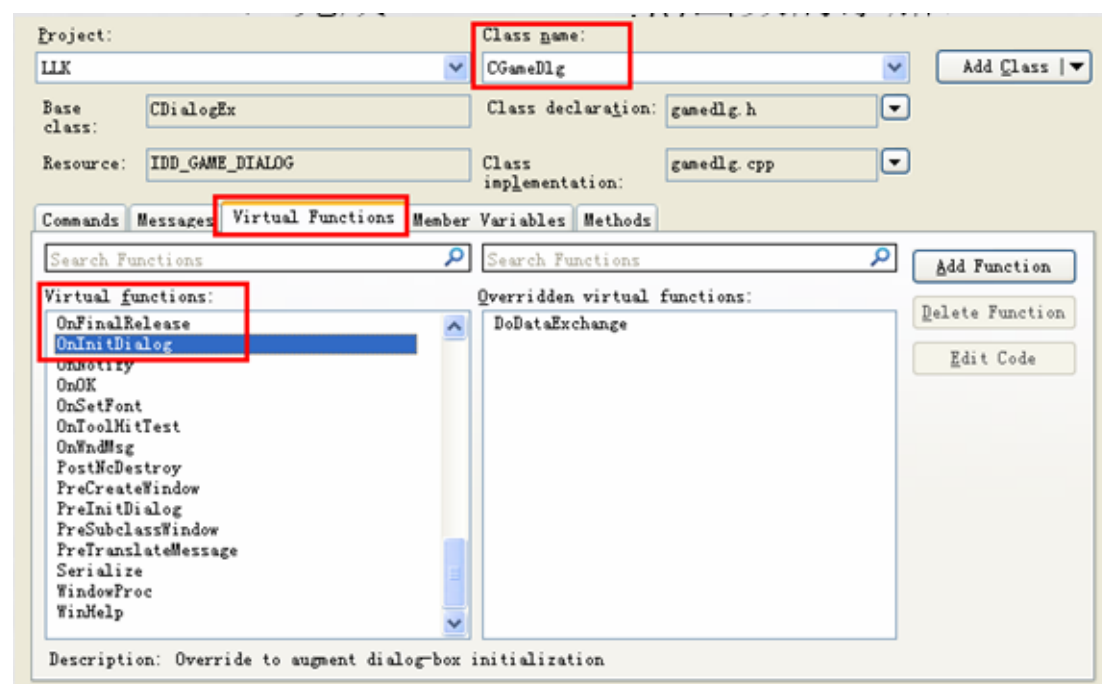


图 11-34 添加 OnInitDialog 函数

(3) 将图片从位图内存拷贝到视频内存。

给 CGameDlg 类添加 OnPaint()函数，在绘制界面时，从位图内存中拷贝位图到视频内存，进行显示。

1) 打开 CGameDlg 类的 Class Wizard。

2) 在 Class Wizard 中选择“Messages”标签页，选择“WM_PAINT”消息。点击“Add Handler”按钮，完成 WM_PAINT 消息响应函数 OnPaint()的添加。

3) 在 CGameDlg::OnPaint()函数中，调用 CPaintDC::BitBlt()函数，绘制游戏界面背景。

```
void CGameDlg::OnPaint()
{
    CPaintDC dc(this); // device context for painting

    // 绘制背景图片
    dc.BitBlt(0, 0, GAMEWND_WIDTH, GAMEWND_HEIGHT, &m_dcMem, 0, 0,
SRCCOPY);
}
```

(4) 添加 CGameDlg::UpdateWindow()函数，调整游戏窗口大小。

设置游戏界面客户区大小为 800*600，在 OnInitDialog()中调用 UpdateWindow()函数。

```
void CGameDlg::UpdateWindow(void)
{
    // 调整窗口大小
    CRect rtWin;
    CRect rtClient;
    this->GetWindowRect(rtWin);           // 获得窗口大小
    this->GetClientRect(rtClient);        // 获得客户区大小

    // 标题栏和外边框的大小
    int nSpanWidth = rtWin.Width() - rtClient.Width();
    int nSpanHeight = rtWin.Height() - rtClient.Height();

    // 设置窗口大小
    MoveWindow(0, 0, GAMEWND_WIDTH + nSpanWidth, GAMEWND_HEIGHT +
nSpanHeight);

    // 设置对话框显示是，在 windows 窗口正中央。
    CenterWindow();
}
```

4、游戏界面布局

(1) 设置游戏界面对话框标题

在 CGameDlg::OnInitDialog()函数中，设置对话框的标题为“欢乐连连看——基本模式”。

(2) 设置游戏界面对话框图标。

(3) 添加控件

在对话框中添加控件，并调整按钮在对话框中的位置。

5、生成顶点

(1) 添加 CGameControl、CGraph、CGameLogic 类。

(2) 在 CGameControl 类中，定义属性 CGraph m_graph，保存生成的游戏地图。

(3) 在 CGameControl::StartGame()函数中，调用 CGameLogic::InitMap()函数。

(4) 编写 CGameLogic::InitMap()函数，生成游戏地图。

1) 创建临时数组，用来保存地图的元素。

```
void CGameLogic::InitMap(CGraph &g)
{
    // 获取地图大小和花色
    int nRows = CGameControl::s_nRows;
    int nCols = CGameControl::s_nCols;
    int nPicNum = CGameControl::s_nPicNum;

    // 开辟内存区域
    int* pGameMap = new int[nRows * nCols];
    //.....
}
```

2) 初始有序地图

```
void CGameLogic::InitMap(CGraph &g)
{
    //.....
    // 多少花色
    if ((nRows * nCols) % (nPicNum * 2) != 0)
    {
        delete []pGameMap;
        throw new CGameException(_T("游戏花色与游戏地图大小不匹配! "));
    }
    int nRepeatNum = nRows * nCols / nPicNum;
    int nCount = 0;
    for(int i = 0; i < nPicNum; i++)
    {
        for(int j = 0; j < nRepeatNum; j++)
```

```

        {
            pGameMap[nCount++] = i;
        }
    }
    //.....
}

```

3) 打乱地图元素

```

void CGameLogic::InitMap(CGraph &g)
{
    //.....
    // 设置种子
    srand((int)time(NULL));
    // 随机任意交换两个数字
    int nVertexNum = nRows * nCols;
    for(int i = 0; i < nVertexNum; i++)
    {
        // 随机得到两个坐标
        int nIndex1 = rand() % nVertexNum;
        int nIndex2 = rand() % nVertexNum;
        // 交换两个数值
        int nTmp = pGameMap[nIndex1];
        pGameMap[nIndex1] = pGameMap[nIndex2];
        pGameMap[nIndex2] = nTmp;
    }
    //.....
}

```

(4) 将生成的地图数据，添加到 CGraph g 对象中。

```

void CGameLogic::InitMap(CGraph &g)
{
    .....
    for(int i = 0; i < nVertexNum; i++)
    {
        g.AddVertex(pGameMap[i]);
    }
    .....
}

```

(5) 在 CGraph 类中，将接收到的图片编号，保存到顶点数组中。

在 CGraph 类中添加 AddVertex() 函数，在 AddVertex() 函数中实现将接收到的图片编号，保存到顶点数组中。

```
int CGraph::AddVertex(int info)
{
    // 将顶点数据 info 添加到顶点数组中
    // 返回顶点数组中顶点个数
}
```

6、生成图的边

(1) 在 CGraph 类中，生成边。

添加 CGraph::AddArc()函数，根据顶点的索引，在图结构中，添加这两个索引的顶点的边。

```
void CGraph::AddArc(int nV1Index, int nV2Index)
{
    m_AdjMatrix[nV1Index][nV2Index] = true;
    m_AdjMatrix[nV2Index][nV1Index] = true;
}
```

(2) 在 CGameLogic::UpdateArc()函数中，实现生成图边数组。

UpdateArc()定义格式：UpdateArc(CGraph &g, int nRow, int nCol)

在开始游戏时，两个顶点有边的条件为：图片元素相同并且相邻。有下列四种情况：

- 1) 与该点左边的是否是相同图片
- 2) 与该点右边的是否是相同图片
- 3) 与该点正上方的是否是相同图片
- 4) 与该点正下方的是否是相同图片

```
void CGameLogic::UpdateArc(CGraph &g, int nRow, int nCol)
{
    //.....
    int nV1Index = nRow * nCols + nCol;
    if(nCol > 0) // 与左边的相连通
    {
        int nV2Index = nV1Index - 1;
    }

    if(nCol < nCols-1) //与右的相连通
    {
        int nV2Index = nV1Index + 1;
    }

    if(nRow > 0) //与正上方的相连通
    {
        int nV2Index = nV1Index - nCols;
    }

    if(nRow < nRows-1) //与正下方的相连通
    {

```

```
        int nV2Index = nV1Index + nCols;
    }
}
```

(3) 在 CGameLogic::InitMap()函数中，调用 UpdateArc()函数，根据生成的图的顶点生成图关系数组。

```
void CGameLogic::InitMap(CGraph &g)
{
    1: 游戏地图数组
    2: 将游戏地图数组中的元素保存到图 g 的顶点数组中
    3: 更新图关系数组
}
```

7、显示游戏地图

初始化图时，已经将游戏界面上显示图片的编号，按照显示的顺序，保存到图的顶点数组中。显示游戏地图时，需要从图的顶点数组中，一个个取出顶点，根据顶点的数值(图片编号)，选择图片元素，显示在游戏地图中。

(1) 在 CGraph 类中，添加 GetVertex()函数，根据索引，从顶点数组中取出相应的顶点。

```
int CGraph::GetVertex(int nIndex)
{
    return m_Vertices[nIndex];
}
```

(2) 在 CGameControl::GetElement()函数中，调用 CGraph::GetVertex()函数。

```
int CGameControl::GetElement(int nRow, int nCol)
{
    return m_Graph.GetVertex(nRow * CGameControl::s_nCols + nCol);
}
```

11.7 相邻同色消子

11.7.1 功能需求

如果选中的两张图片的图片元素相同，并且这两张图片在游戏地图中相邻，则进行消子，并更新图结构和游戏地图。图片满足消失条件，则瞬时显示连通路径，然后图片立即消失。

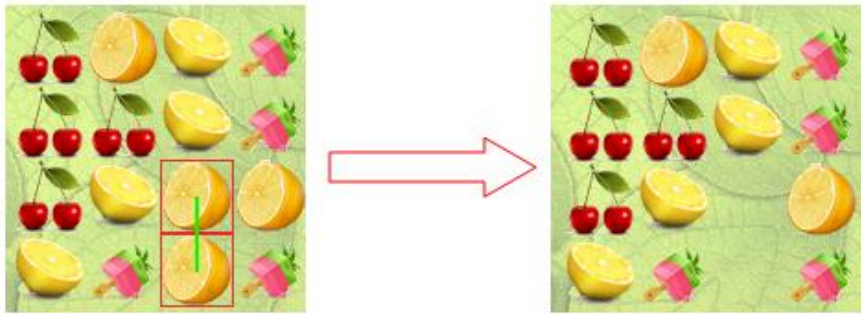


图 11-35 相邻同色消子

11.7.2 设计思路

在“开始游戏”的基础上进行迭代开发。

1、获得选中图片在游戏地图中的行号和列号

在“消子判断”迭代中，已经实现根据鼠标点击位置的坐标，计算出选中图片的行号 row1,row2 和列号 col1,col2。

2、获得选中图片在顶点数组中的索引号

根据选择的图片的行号和列表，计算选中的图片在顶点数组中 V 的索引号分别为 v1 和 v2。索引号 = 列号*游戏地图行数 + 行号。

3、类设计

(1) CGraph 类

数据结构类。

公有成员函数	描述
void UpdateVertex(int nIndex, int info)	将图顶点数组中索引号为 nIndex 的顶点的值更新为 info
bool GetArc(int nV1Index, int nV2Index)	获得两个顶点的弧信息
int GetVexnum(void)	获取顶点数组中的顶点数

(2) CGameLogic 类

CGameLogic 类为游戏业务逻辑类。

成员变量	描述
void Clear(CGraph &g, Vertex v1, Vertex v2)	清除顶点
bool IsLink(CGraph &g, Vertex v1, Vertex v2)	判断是否连通

(3) CGameControl 类

CGameControl 类为游戏控制类。

1) 数据成员

数据成员	描述
Vertex m_svSelFst	选中的第一个点
Vertex m_svSelSec	选中的第二个点

2) 成员函数

成员函数	描述
void SetFirstPoint(int nRow, int nCol)	设置第一个点
void SetSecPoint(int nRow, int nCol)	设置第二个点
bool Link(Vertex avPath[4], int &nVexnum)	消子判断

(4) 游戏对话框界面 CGameDlg 类

1) 成员变量

成员变量	描述
bool m_bFirstPoint	是否第一个点
bool m_bPlaying	游戏状态标识

2) 成员函数

成员函数	描述
void OnLButtonUp(UINT nFlags, CPoint point)	鼠标 WM_LBUTTONDOWN 消息响应函数, 选择两个图片, 进行连通判断
void DrawTipLine(Vertex asvPath[100], int nVexnum)	根据连接路径, 绘制连接线
void DrawTipFrame(int nRow, int nCol)	绘制提示框

11.7.3 编码实现

导入“开始游戏”的解决方案, 在此基础上进行迭代开发, 实现步骤如下:

步骤一: 添加鼠标事件。

步骤二: 选择图片。

步骤三: 获取选中的图片位置。

步骤四: 连通路径保存。

步骤五: 消子并更新游戏地图。

1、添加鼠标事件

(1) 鼠标点击事件

1) 用户使用鼠标点击窗口时, 当点击的位置在当前应用程序窗口所在区域, 操作系统会发消息通知当前的应用程序, 并且会把鼠标点击的坐标点和事件类型传入。

2) WM_LBUTTONDOWN 消息

当鼠标左键按钮被释放是响应。而 WM_LBUTTONDOWN 消息是在按下时响应。这两个消息都可以代表用户的点击事件。

3) WM_LBUTTONDOWN 消息响应函数

函数原型: void OnLButtonUp(UINT nFlags, CPoint point);

函数参数:

nFlags: 标识在鼠标点击的同时是否有功能键, 如: Ctrl、Shift 键。

point: 鼠标点击的坐标点, 以当前窗口客户区为坐标系。

(2) 添加鼠标点击事件

1) 选择 CGameDlg 类，打开类向导(ClassWizard)。

2) 给 CGameDlg 类添加 WM_LBUTTONDOWN 消息响应函数 OnLButtonDown()。

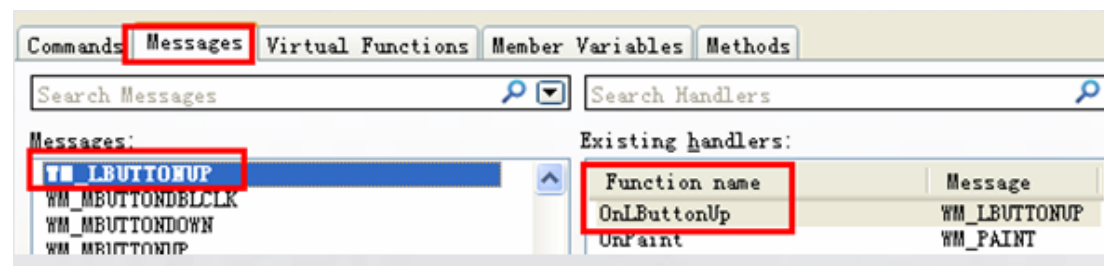


图 11-36 添加响应事件

2、选择图片

(1) 判断点击位置是否在游戏地图中

游戏矩形区域为 m_rtGameRect，点击位置的坐标必须在游戏矩形之内。

鼠标消息响应函数 OnLButtonDown()函数的参数 CPoint point 保存了鼠标点击位置的坐标。CPoint.x 表示横坐标，CPoint.y 表示纵坐标。

在 CGameDlg::OnLButtonDown()函数中判断点击位置是否在游戏地图中。

```
void CGameDlg::OnLButtonDown(UINT nFlags, CPoint point)
{
    // 判断鼠标点击的区域
    if(point.y < m_rtGameRect.top || point.y > m_rtGameRect.bottom
        || point.x < m_rtGameRect.left || point.x > m_rtGameRect.right)
    {
        return CDialogEx::OnLButtonDown(nFlags, point);
    }
}
```

(2) 计算鼠标点击位置的行号和列号

游戏地图在游戏界面对话框中的起始坐标为(50, 50)。假设点击位置的坐标为(x, y)。

1) 首先排除坐标(x, y)不在游戏区域的情况。

2) 根据(x, y)计算点击位置的行号和列号。

将 x>50 的区域按照图片的长度，分为 N 列，将 y>50 的区域按照图片的高度，分为 N 行。

点击位置的行号 row 和列号 col 分别为：row = (y - 50)/图片高度 和 col = (x - 50)/图片长度

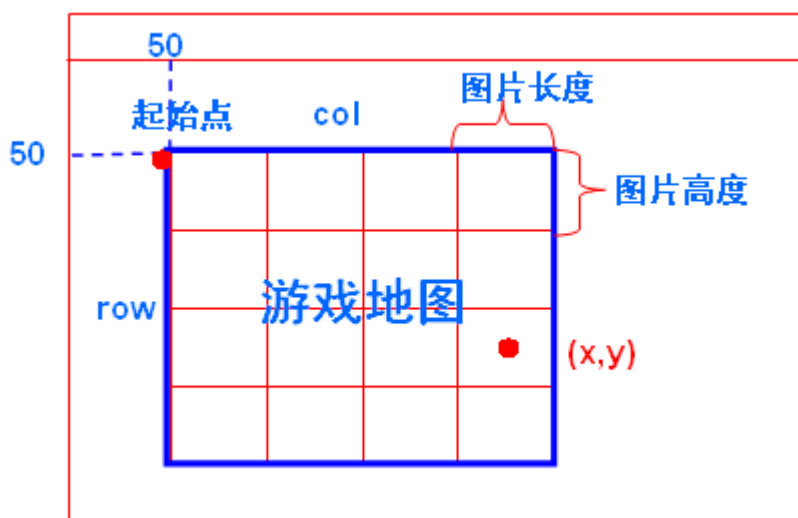


图 11-37 计算图片的行和列

根据点击位置的坐标、游戏地图中每个图片的大小，计算鼠标点击位置的行号和列号。

行号 = (点击位置 y 坐标 - 游戏地图的起始 y 坐标点)/元素的高度。

列号 = (点击位置 x 坐标 - 游戏地图的起始 x 坐标点)/元素的长度。

(3) 在鼠标选中的图中周围绘制矩形提示框。

在 CGameDlg 类中添加 DrawTipFrame()函数，绘制提示框。

当鼠标选中游戏地图中的一张图片时，在该图片的四周绘制矩形提示框。矩形框的颜色为 RGB(233, 43, 43)，大小和选中的图片大小一致。

1) 在 CGameDlg 类中，添加成员函数 DrawTipFrame()，根据选择的图片的行号和列号，在选择的图片周围绘制矩形提示框。

函数定义格式为：void DrawTipFrame(int nRow, int nCol)

2) 绘制矩形时，可以根据矩形的左上角坐标和右下角坐标进行绘制。

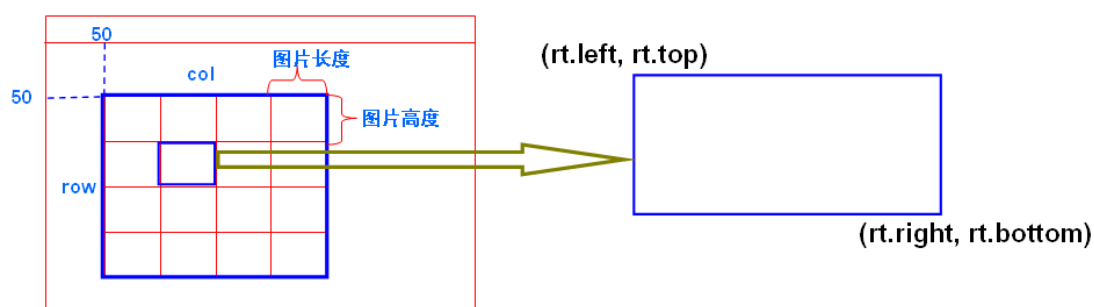


图 11-38 计算矩形框的坐标

3、获取选中图片位置

将游戏地图中选择的两张图片的位置，保存到了 CGameControl 类的属性 m_svSelFst 和 m_svSelSec 中。这两个点的数据(行号、列号)，通过这两个点来保存选择的两张图片的位置。

```
// 设置第一个点
void CGameControl::SetFirstPoint(int nRow, int nCol)
```

```

{
    m_svSelFst.col = nCol;
    m_svSelFst.row = nRow;
}

// 设置第二个点
void CGameControl::SetSecPoint(int nRow, int nCol)
{
    m_svSelSec.col = nCol;
    m_svSelSec.row = nRow;
}

```

4、连通路径保存

游戏的地图为 10 行 16 列，最长的路径可能经过所有的点，即 160 个元素。考虑到后面地图的元素个数会改变，最大可以有 160 个元素。取临时的顶点存储数组为 100 个。

(1) 修改 CGameDlg::OnLButtonUp()函数中，保存路径的数组大小为 100。

(2) 添加 CGameControl::Link()函数定义为如下格式：

```
bool Link(Vertex avPath[100], int &nVexnum)。
```

(3) 添加 CGameLogic 类中的属性 m_anPath[100]。

优化思路：此处的临时数组定义为 100，可以考虑进行适当的优化。如，只保存最近的三个点和拐点，其它的点都去掉。由于拐点不能超过 2 个，加上起始和结束点，共 4 个点，另外为了判断当前点是否为拐点，需要保存前面两个点，这样大概定义 8 个点即可。

5、消子并更新游戏地图

游戏地图的图片编号是根据图的顶点数据来生成的。更新图结构和更新游戏地图，只需要更新图结构，就能够更新游戏地图。

消子并更新图结构，只需要在 CGameLogic::Clear()函数中，更新图结构即可。

(1) 更新顶点信息

1) 在 CGameLogic::Clear()函数中，根据选择的点，计算该点在图的顶点数组中的索引号。

2) 调用 CGraph::UpdateVertex()函数，将该顶点的值设置为 BLANK(-1)。

(2) 更新边信息

更新边信息，直接在 CGameLogic::Clear()函数中，调用 UpdateArc()函数，更新就可以完成边的更新。

11.8 消子判断

11.8.1 功能需求

如果选中的两张图片能够通过一条或者多条直线进行消子，则显示连接路径，进行消子，并更新图结构和游戏地图。如果两张图片连接路径中超过 2 个拐点，则这两张图片不能消除。

11.8.2 设计思路

在“相邻同色消子”的基础上进行迭代开发。
进行游戏业务逻辑 CGameLogic 类设计。

1、数据成员

数据成员	描述
int m_anPath[100]	保存在进行连接判断时所经过的顶点
int m_nVexNum	顶点数
int m_nCorner	保存路径中的拐点数

2、成员函数

成员函数	描述
void PushVertex(int nV)	添加一个路径顶点
void PopVertex()	取出一个顶点
void UpdateArc(CGraph &g, int nRow, int nCol)	更新某一个顶点的弧信息
bool SearchPath(CGraph &g, int nV0, int nV1)	搜寻路径
bool IsExsit(int nVi)	判断顶点是否已在路径中存在
bool IsCornor(void)	判断拐点的有效性

11.8.3 编码实现

导入“相邻同色消子”的解决方案，在此基础上进行迭代开发，实现步骤如下：
步骤一：判断连通性。
步骤二：添加拐点限制。

1、判断连通性

- (1) 在 CGameLogic::SerachPath()函数，选择的两个顶点连通。
函数定义：bool SearchPath(CGraph &g, int nV0, int nV1)。
- (2) 修改 CGameLogic::IsLink()函数。
 - 1) 获取选择的两个顶点在顶点数组中的索引号。
 - 2) 将第一个点的索引号添加路径数组。
 - 3) 调用 CGameLogic::SerachPath()函数，判断这两个点在图中是否连通。

4) 如果不连通，从路径数组中取出第一个点的索引。

```
bool CGameLogic::IsLink(CGraph &g, Vertex v1, Vertex v2)
{
    // 获取顶点索引
    int nV1Index = v1.row * 4 + v1.col;
    int nV2Index = v2.row * 4_nCols + v2.col;

    // 压入第一个点

    // 搜寻两点之间的路径
    if(SearchPath(g, nV1Index, nV2Index) == true)
    {
        return true;
    }
    // 取出压入的顶点
    PopVertex();
    return false;
}
```

(3) 编译并运行程序。

(4) 编写 CGameLogic::SearchPath()函数，根据深度优先搜索算法判断是否连通。

- 1) 获取图中的顶点个数 nVexnum。
- 2) 遍历图中 V0 行，从 0 列到 nVexnum 列，有边的点 Vi。
- 3) 判断 Vi 在路径数组中是否存在。
 如果存在则寻找下一个有边的点。
- 4) 如果不存在，则将 Vi 放入路径数组。
- 5) 判断 Vi 是否和 V1 相等，如果相等，
 则 V0 和 V1 连通，返回 true。
- 6) 如果不相等，则重复 1)~5)。

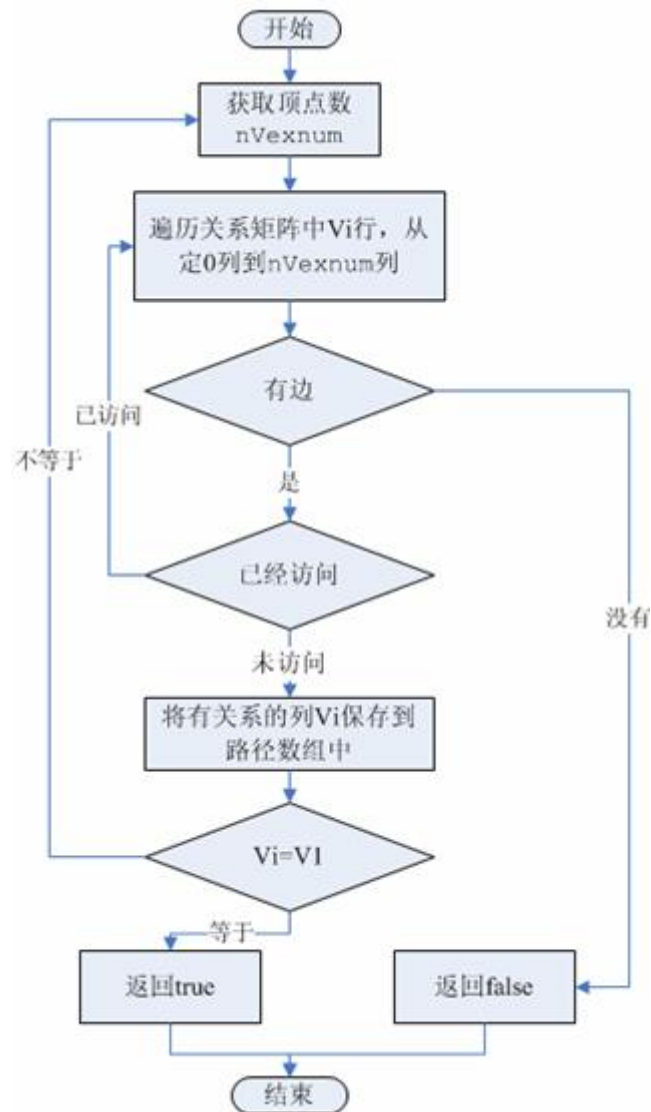


图 11-39 算法判断流程图

实现代码如下：

```

bool CGameLogic::SearchPath(CGraph &g, int nV0, int nV1)
{
    int nVexnum = g.GetVexnum();

    // 遍历图中 nV0 行，从 0 列到 Nvexnum 列，值为 true 的点
    for(int nVi = 0; nVi < nVexnum; nVi++)
    {
        if(g.GetArc(nV0, nVi) && !IsExsit(nVi))
        {
            // 压入当前顶点，假设为路径的一个有效顶点

            // 当中间顶点不是 nVi 时，继续搜寻下一个相邻且相连通的顶点
            if(nVi != nV1)
            {
                // 如果 nVi 是一个已消除的点，则判断(nVi, nVi)是否连通
            }
        }
    }
}
  
```



```

    }
    else // 如果(nV i== nVi),表示已经找到了一条连通路径, 则返回 true
        return true;
    }
//取出压入的顶点, 与 pushVertex()对应
    }
    }
    return false;
}

```

(5) 编写 CGameLogic::IsExsit()函数, 判断该顶点是否保存到路径数组中。

(6) 编译并运行程序。

发现如图显示的两个顶点也可以消除, 但是这是规则所不允许消除的, 如何处理?

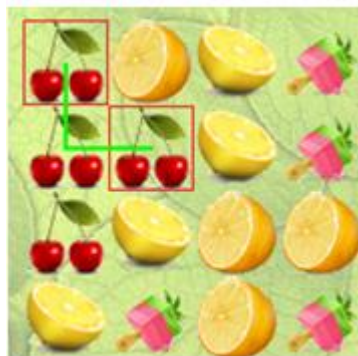


图 11-40 运行效果

2、添加拐角限制

在 CGameLogic 类中, 添加 IsCornor()函数, 判断 m_anPath 数组中连续的 3 个顶点是否有一个拐点。

函数定义格式如下: bool IsCornor()。

(1) 给 CGameLogic 类添加数据成员 int m_nCorner, 表示连接路径中的拐点数。

(2) 在 CGameLogic::PushVertex()函数中, 当向 m_anPath 中添加顶点后, 判断是否形成一个拐点, 如果是一个拐点, 则拐点数 m_nCorner 加 1。

(3) 在 CGameLogic::PopVertex()函数中, 当从 m_anPath 中取出顶点前, 判断是否是一个拐点, 如果是, 则拐点数 m_nCorner 减 1。

(4) 在 CGameLogic::SearchPath()函数中, 将 nVi 点添加到 m_anPath 后, 判断拐点数是否大于 2, 如果大于 2 则放弃该条路径, 从路径中取出 nVi 点。重新查找别的路径。

11.9 判断胜负

11.9.1 功能需求

1、介绍

在基本模式下如果将游戏地图中所有的图片都消除，则提示玩家获胜，并且可以重新开始新游戏。

2、输入

游戏地图中剩余图片数量。

3、处理

判断游戏地图中所有的图片是否都被消除。

4、输出

(1) 如果游戏地图中所有图片都消除了，则提示玩家获胜，并且可以通过开始游戏按钮重新开始新游戏。

(2) 如果没有都消除，玩家不能开始新游戏。直到所有图片都消除完。

11.9.2 设计思路

在“消子判断”的基础上进行迭代开发。

1、判断胜负的条件

界面上游戏地图中所有的图片都被消除。

2、判断胜负的功能实现

每次成功消除一对图片后，在 CGameLogic 中，判断 CGameControl 类保存游戏地图图结构 m_graph 中，顶点数组所有元素的值是否为空(-1)。为空表示该顶点已经被消除。

3、类设计

(1) CGraph 类

数据结构类。

成员函数	描述
void ClearGraph(void)	将图中顶点和边还原为初始状态

(2) CGameLogic 类

游戏业务逻辑类。

成员函数	描述
bool IsBlank(CGraph &g)	判断图 g 中所有的顶点是否为空，如果都为空，则返回 true；否则，返回 false

(3) CGameControl 类

游戏控制类。本次迭代需要在 CGameControl 类中增加一个成员函数 IsWin()。

成员函数	描述
bool IsWin(void)	判断是否获胜

11.9.3 编码实现

导入“消子判断”的解决方案，在此基础上进行迭代开发，实现步骤如下：

步骤一：判断胜负。

步骤二：控制开始游戏按钮状态。

步骤三：清理图结构。

1、判断胜负

判断胜负的规则为：游戏地图中所有的图片都被消除完，也就是图结构中的顶点数组中所有元素都为空(-1)。

(1) 逻辑层判断胜负

在 CGameLogic 类中，添加 IsBlank()函数。判断图中所有的顶点是否都为空。

IsBlank()函数定义为：bool IsBlank(CGraph &g)。

(2) 控制层判断胜负

在 CGameControl 类中，添加 IsWin()函数，调用 CGameLogic::IsBlank()函数，判断是否已经获胜。IsWin()函数定义为：bool IsWin(void)。

(3) 界面层判断胜负

在 CGameDlg::OnLButtonUp()函数中，调用 CGameControl::IsWin()函数，判断是否已经获胜。如果获胜，则提示用户获胜。

```
void CGameDlg::OnLButtonUp(UINT nFlags, CPoint point)
{
    //.....
    // 判断胜负
    if(bSuc && m_GameC.IsWin())
    {
        //.....
    }
}
m_bFirstPoint = !m_bFirstPoint;
}
```

2、控制开始游戏按钮状态

游戏开始后，到游戏获胜前，开始按钮的状态为禁用。直到游戏获胜之后，才可用。设置按钮的禁用状态，可以调 CWnd::EnableWindow()函数，参数为 FALSE 表示该对象不可用。参数为 TRUE 表示该对象可用。

(1) 给 CGameDlg 类添加数据成员 bool m_bPlaying，表示游戏是否正在进行。

(2) 在构造函数中初始化 m_bPlaying = false。

(3) 在 CGameDlg::OnBnClickedBtnStart()函数中, 开始游戏时, 将 m_bPlaying 赋值为 true。并将开始游戏按钮设置为禁用状态。

(4) 在 CGameDlg::OnLButtonUp()函数中, 首先判断游戏是否正在进行, 如果没有正在进行, 则不响应鼠标事件。然后在游戏获胜后将 m_bPlaying 赋值为 false, 最后将设置开始游戏按钮的状态为可用。

3、清理图结构

游戏获胜后, 图结构中所有的顶点都为空, 但是顶点间都还是有边的。所以需要将图结构还原, 使得在下次游戏开始时, 能够产生正确的图结构。

(1) 在 CGraph 类中, 添加成员函数 ClearGraph(), 调用 InitGraph()函数, 清理图结构。

(2) 在 CGameControl::IsWin()函数中, 游戏胜利后, 调用 CGraph::ClearGraph()函数, 清理图结构。

11.10 扩展功能

11.10.1 提示

在 CGameDlg 类中添加提示按钮的响应事件 CGameDlg::OnBnClickedBtnPrompt(), 调用 CGameControl 类的 Help() 函数, 查找可消子的图片对。若有一对元素可连通, 则在界面对应元素区绘制矩形框进行提示。1 秒后, 重绘界面, 矩形框消失。

在 CGameDlg 类中添加 DrawTipLine()函数, 绘制提示框。

在 CGameControl 类中添加 Help()函数, 先判断游戏是否为空, 不为空, 则调用 CGameLogic 类中的 SearchValidPath()函数来查找可消子的图片对。

在 CGameLogic 类中添加 SearchValidPath()函数, 搜寻有效可消子的路径, 算法实现的过程, 从左到右, 从上到下, 依次判断地图中同色元素是否可以连通。

11.10.2 重排

在 CGameDlg 类中添加重排按钮的响应事件 CGameDlg::OnBnClickedBtnReset(), 调用 CGameControl 类的 Reset() 函数进行重提成, 调用 UpdateMap()函数更新界面。

在 CGameControl 类中添加 Reset()函数, 调用 CGameLogic 类中的 ResetGraph()函数来对地图数据重排。

在 CGameLogic 类中添加 ResetGraph()函数, 随机任选地图中两个顶点, 将元素进行交换, 这样进行 100 次。

12 附录

12.1 C++编码规范

1、基本原则

- (1) C++命名规范基本遵循匈牙利命名。
- (2) 打算干什么，要简单，直接了当，代码精简，避免垃圾程序。
- (3) 尽量使用标准库函数。
- (4) 不要随意定义全局变量，尽量使用局部变量。
- (5) 仔细考虑逻辑分支，不能出现逻辑错误
- (6) 工程名、类名、成员名、函数名全部都不能用中文。
- (7) 不要使用 Goto 函数。
- (8) 注释行的数量不得少于程序行数量的 20%。

2、命名标准

(1) 变量命名

命名必须具有一定的实际意义，形式为 xAbcFgh，x 由变量类型确定，Abc、Fgh 表示连续意义字符串。

前缀	表示类型	例子
a	数组变量	aScore[50]
b	布尔变量	bFlag, bIsEnd
c	字符变量	cSex
i 或 n	整形变量	iNum, nNum
ui	无符号整形变量	uiStart, uiPos
sz 或 str	以 0 结束的字符串变量	szMyName, strMyName
p	指针变量	pszString, pMyDlg
h	句柄	hWnd, hPen, hDlg
m_	类的成员变量	m_xStart
C	类和结构	CDialo, CView, CMysdiApp, CRuntimeClass
ID*_	资源标识	ID_, IDD_, IDC_, IDB_, IDI_
g_	全局变量	g_nMsg, g_bFlag

局部变量中可采用如下几个通用变量：nTemp、nResult、index。

(2) 常量与宏命名

- 1) 常量和宏定义必须具有一定的实际意义。
- 2) 常量和宏定义在#include 和函数定义之间。
- 3) 常量和宏定义必须全部以大写字母来撰写，中间可根据意义的连续性用下划线连接，每一条定义的右侧必须有一简单的注释，说明其作用。

(3) 控件命名

控件命名时，遵循变量命名的基本规则，各类型控件的前缀如下：

前缀	类型	例子
btn	按钮	m_btnTest
stat	静态文本框	m_statMessage
edit	编辑框	m_editName
lst 或 lc	列表框	m_lstLog, m_lcLog
rd	单选框	m_rdBackgroud
cb	组合框	m_cbSex
ip	IP 控件	m_ipServer
dt	日期控件	m_dtStart
wnd	窗口	m_wndSetting

(4) 资源命名

资源名要求全部用大写，各资源命名如下所示：

- 1) 菜单:IDM_XX 或者 CM_XX
- 2) 位图:IDB_XX
- 3) 对话框:IDD_XX
- 4) 字符串:IDS_XX
- 5) 对话框下面控件： IDC_控件标签_描述，如 IDC_EDIT_YEAR，IDC_BUTTON_SUBMIT，IDC_COMBOX_CLASS

(5) 函数命名

第一个字母必须使用大写字母,要求用大小写字母组合规范函数命名,必要时可用下划线间隔,示例如下：

```
void PrintTrackData ();           // 函数的功能作用描述
void ImportantPoint (void);       // 函数的功能作用描述
void ShowChar (int , int);        // 函数的功能作用描述
```

(6) 类命名

类名以大写的“C”开头，以名词或名词性词组进行定义，单词首字母大写。如：CUserInfo

(7) 文件命名

- 1) 文件命名要具有实际意义。
- 2) 文件名只能包含字母，数字和下划线，不得使用其他的字母，超长的文件名应使用缩写方式减少文件名的长度，建议使用如下的缩写的规则：

规则一：缩写一般可以去掉无意字母以及不发音字母。

规则二：单词的首字母一般应该保留。

规则三：当一个单词必须编写成一个字母时，应该选用最有代表性的字母或首字母。
- 3) 当单词数量少，字母少时不需要缩写词。
- 4) 临时文件必须调用系统函数生成，禁止使用固定文件名，一般情况下，临时文件应在系统临时目录下（通过系统函数取到临时目录名）生成。

(8) 参数命名

参数命名遵循变量命名的规则。

3、注释规范

(1) 函数注释

在重要函数的声明之前，给出精炼的注释（不必牵扯太多的内部细节），让使用者能够快速获得足够的信息使用函数。尽量使用一两句话对其进行描述。

在函数的定义之前，要对函数的目的/功能、输入参数、输出参数、返回值进行说明。

格式如下：

```
/* *****  
[函数名称]          （必需）  
[函数功能]          （必需）  
[参    数]          （必需。标明各参数是输入参数还是输出参数。）  
[返 回 值]          （必需。解释返回值的意义。）  
***** */
```

(2) 类的注释

对类的注释，即在类的声明之前，对于类名、功能等进行精炼的注释。格式如下：

```
/* *****  
[类名]              （必需）  
[功能]              （必需）  
***** */
```

(3) 文件注释

在头文件、实现文件的首部，一定要有文件注释，用来介绍文件内容。注释格式要求如下：

```
/* *****  
[文件名]              （必需）  
[功能模块和目的]      （必需）  
***** */
```

(4) 代码注释

1) 除文件、函数等的注释允许使用“/**/”外，其他单行注释或语句体的注释必须使用“//”进行注释。

2) 常量定义时应有所说明。

3) 主要变量（结构、联合、类或对象）定义或引用时，除命名清晰外，还应对其含义进行注释。

4) 在典型算法前都应有注释。

5) 注释可以与语句在同一行，也可以在上行。

6) 空行和空白字符也是一种特殊注释,对于一个函数内的功能模块采用空行进行分割。

4、编码规范

1) 一行只包含一个语句。

2) 程序结构清晰，简单易懂，单个函数的程序行数建议不超过 100 行。

3) 对于变量的定义，尽量在方法的头部给出。

4) 在代码中垂直对齐左括号和右括号。例：

```
if ( x == 0 )
{
    .....
}
```

不允许以下情况：

```
if(x==0) {
    .....
}

if(x==0) {.....}
```

5) 在大多数运算符之前和之后使用空格，这样做时不会改变代码的意图却可以使代码容易阅读。

例：int j = i + k; 而不应写为：int j=i+k;

5、排版规范

对程序语句要按其逻辑进行水平缩进，以四个空格或一个 Tab 符为单位（建议使用空格），使同一逻辑层次上的代码在列上对齐。

12.2 MFC 应用程序介绍

1、MFC

MFC (Microsoft Foundation Class Library) 是一个微软公司提供的类库 (class libraries)，以 C++ 类的形式封装了 Windows API，并且包含一个应用程序框架，以减少应用程序开发人员的工作量。其中包含的类包含大量 Windows 句柄封装类和很多 Windows 的内建控件和组件的封装类。MFC 中的各种类结合起来构成一个应用程序框架，它的目的就是让程序员在此基础上建立 windows 下的应用程序，这是一种相对 SDK 来说更为简洁的方法。

MFC 应用程序类型包括对话框、单文档和多文档。

2、应用程序向导

与其他可视化开发工具一样，VS2010 提供了创建应用程序向导框架的向导 AppWizard 和相关的开发工具。

向导实质上是一个源代码生成器，利用应用程序向导可以快速创建各种风格的应用程序框架，自动生成程序通用的源代码，大大减轻了编写代码的工作量。

使用应用程序向导能够生成一个功能完整的新的应用程序，可以从中创建更为复杂的应用程序。

使用 MFC 应用程序向导创建的一个 MFC Dialog 应用程序，解决方案和工程文件夹如下：

(1) 解决方案文件夹

解决方案文件夹中，存放解决方案文件(.sln 文件)，和解决方案中的工程文件夹。一个

解决方案可以包含多个工程。

解决方案文件(.sln 文件), 包含当前解决方案中的工程信息, 存储解决方案的设置。
VS2010 中一个解决方案可以包含多个工程。



图 12-1 MFC 应用程序解决方案

(2) 工程文件夹

工程文件夹中存放工程文件(.vcxproj 文件)和 MFC AppWizard 创建的一个 Dialog 程序所产生的文件。

工程文件(.vcxproj 文件)包含当前工程的设置和工程所包含的文件等信息。

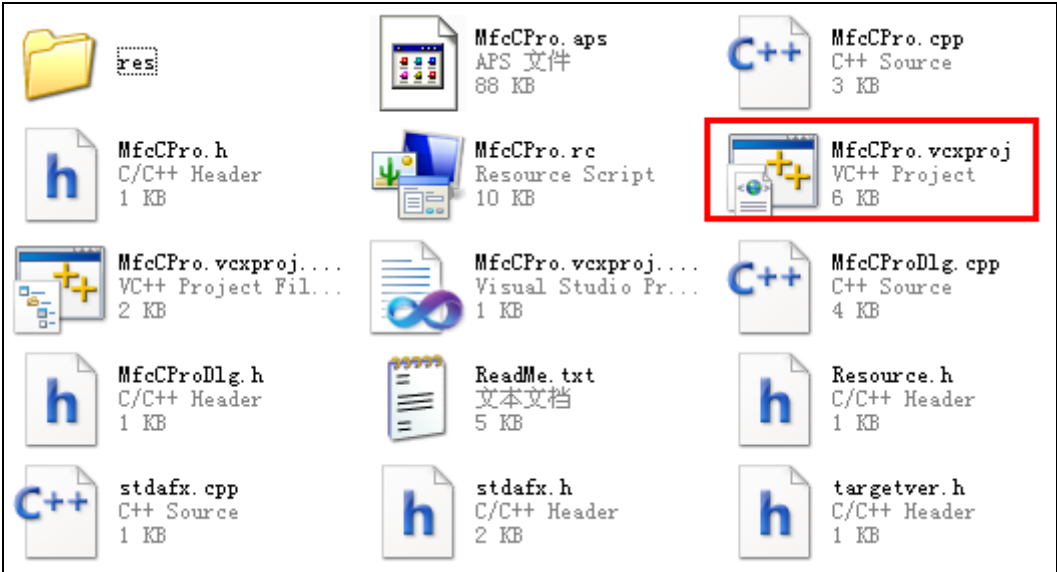


图 12-2 MFC Dialog 应用程序结构

3、MFC Dialog 应用程序的组成

使用应用程序向导, 创建一个 MFC Dialog 应用程序, 会产生下列两个主要的类, 这两个类就构成了 MFC Dialog 应用程序的组成。通过这两个类就能够启动一个 MFC Dialog 应用程序并显示应用程序窗口。

(1) 应用程序类 CMfcCProApp

CMfcCProApp 类是继承于 CWinApp 类的, CMfcCProApp 类的对象 theApp 表示当前应用程序。

CWinApp::InitInstance()函数: CMfcCProApp 类重载 CWinApp 的 InitInstance 函数作为当前应用程序的入口函数, 在该函数中实例化窗口对象并显示。

(2) 主界面对话框窗口类(CMfcCProDlg)

CMfcCProDlg 类继承与 CDialog 类, 该类与对话框窗口是对应的, 该类的对象表示当前的窗口。

CDialog::DoModal()函数: CMfcCProDlg 类继承于 CDialog 类, 通过调用该类的 DoModal()函数, 就可以显示对话框窗口。

CDialog::OnInitDialog()函数: CMfcCProDlg 类重载 CDialog::OnInitDialog()函数来初始

化窗口。

在 VS2010 中，可以通过以下三种方式，查看应用程序向导产生的内容。

(1) Solution Explorer

在“Solution Explorer”中可以看到创建的解决方案、解决方案中的工程、以及使用应用程序向导创建的文件。

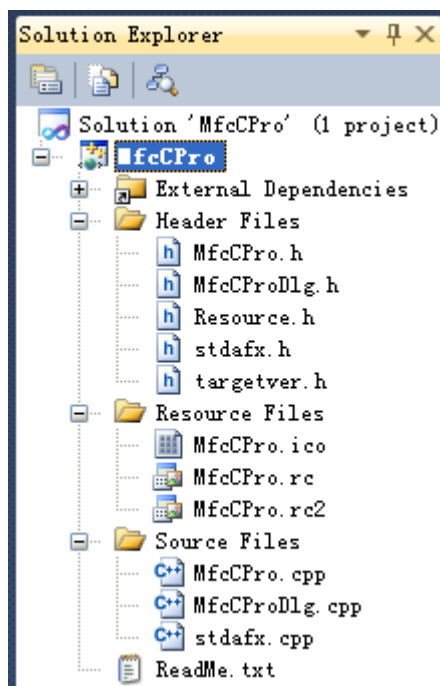


图 12-3 MFC Dialog 应用程序组成

(2) Class View

在“Class View”中，可以看到应用程序向导创建的应用程序类 CMfcCProApp 和对话框类 CMfcCProDlg。

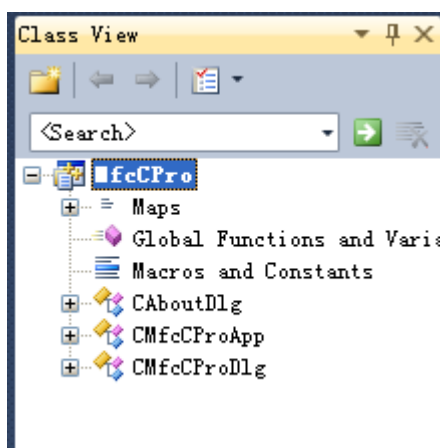


图 12-4 MFC 类视图

(3) Resource View

在“Resource View”中，可以看到对话框程序的资源。

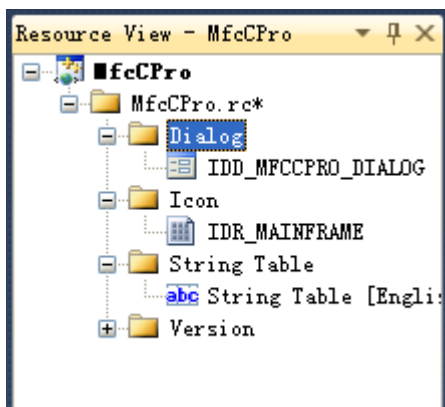


图 12-5 MFC 资源视图

4、MFC Dialog 应用程序运行过程

使用 Microsoft Visual Studio 2010 开发工具的应用程序向导(AppWizard)创建一个 MFC Dialog 程序，产生了这些文件和主要的两个类。为什么程序就能够启动并且显示对话框界面了呢？一个 MFC Dialog 应用程序又是如何运行起来的呢？

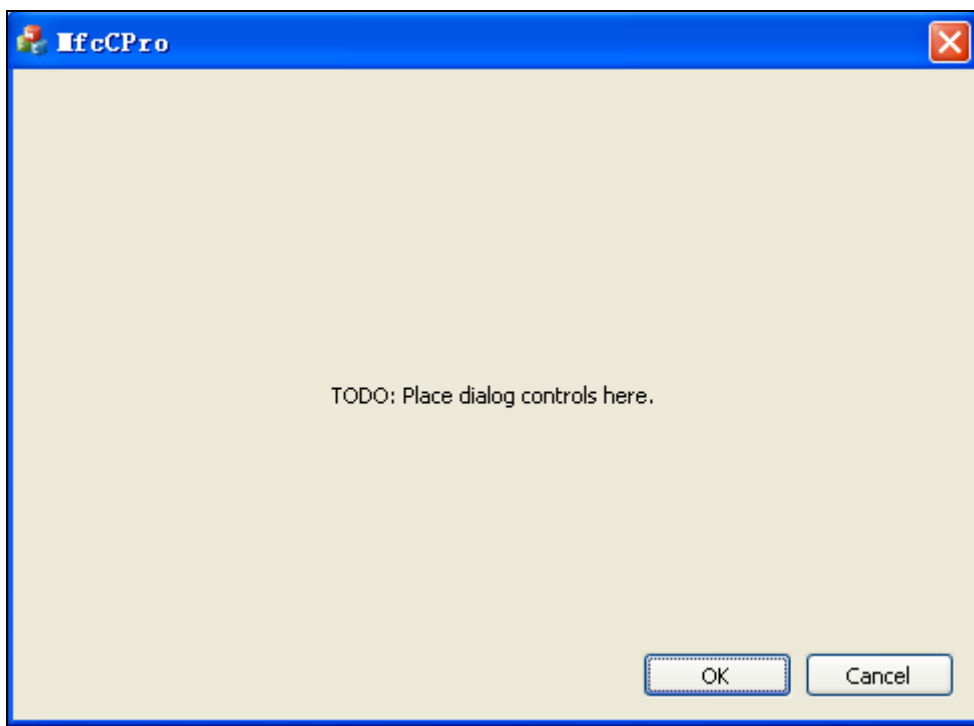


图 12-6 MFC Dialog 窗体

(1) 程序入口 InitInstance()

在控制台程序中必须包含一个 main 函数，程序才能正常运行，系统会自动调用程序中的 main 函数以运行程序。而 Win32（窗口）程序中，通常有一个 WinMain 函数，做为程序的入口函数，方能运行。而在 MFC 的对话框程序中，将 WinMain 函数进行了封装，而提供了一个 CWinApp 类来做为一个应用程序类，而任何一个程序中，至少有一个全局的应用程序类对象。

MFC 在 CWinApp 类中提供给编程人员 InitInstance 函数作为 MFC 程序的入口。它是一个虚函数，在 CWinApp 的派生类中会重载该函数，在调用虚函数 InitInstance 时实际调

用了 theApp 的 InitInstance 函数。

(2) 对话框程序的运行过程

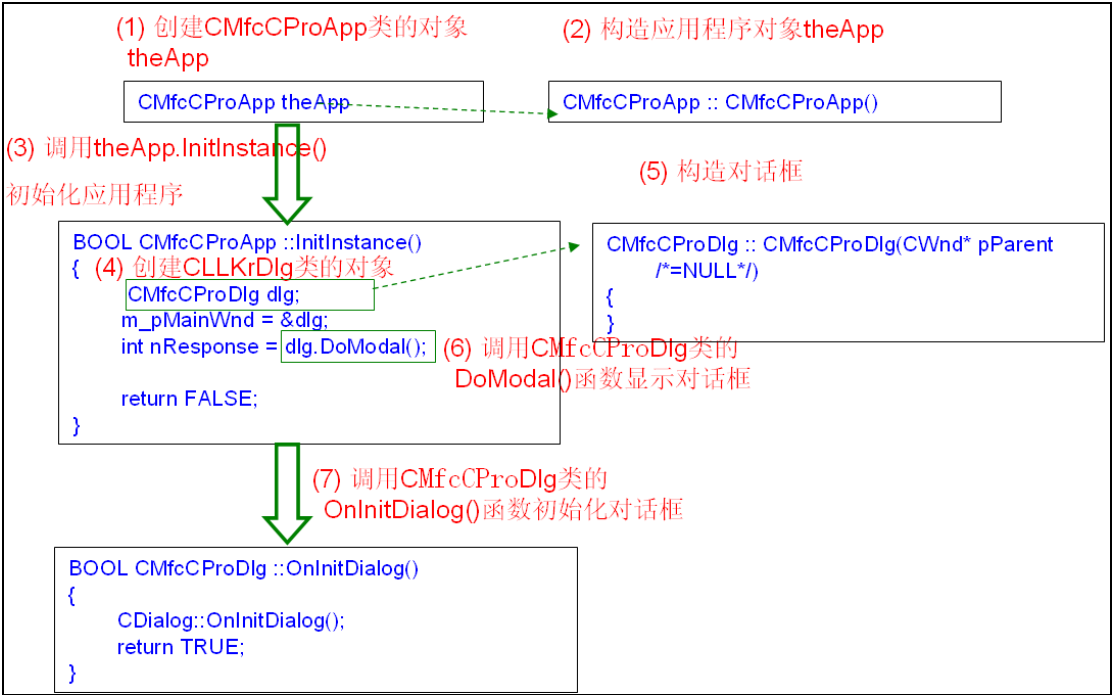


图 12-7 MFC Dialog 运行过程

这样，通过 MFC 框架很容易创建一个对话框应用程序，提高了开发的效率。通过 MFC 框架，很容易开发一个 C++应用程序。